

ECE 463/563

Fall `22

Instruction-Level Parallelism (ILP)
and Its Exploitation:
H&P Chapter 3

Prof. Eric Rotenberg

“issue”

- The term “issue” refers to *advancing an instruction to the execute stage when all of its source operands are ready*. We say, for example, “issue to a function unit”.
- In-order issue:
 - Instructions issue strictly in program order
 - A younger instruction cannot issue before an older instruction
- Out-of-order (OOO) issue:
 - Instructions may issue out of program order
 - Younger instructions may issue before older instructions

Outline

- From in-order issue to out-of-order issue:
 - Issue Queue (IQ)
- Speculation and register renaming:
 - Reorder Buffer (ROB)

Key for RISC ISA Pseudo-assembly

KEY:

```
add    rd, rs1, rs2    // RF[rd] = RF[rs1] + RF[rs2]
load   rd, #d(rs1)     // RF[rd] = MEM[ RF[rs1] + d ]
store  #d(rs1), rs2     // MEM[ RF[rs1] + d ] = RF[rs2]
```

Review of Data Hazards

- A *data hazard* exists when two dependent instructions are in the pipeline at the same time
 - Three types of data dependencies: true, anti-, output
- Read-after-write (RAW) hazard
 - Stems from a *true dependence*
 - add R1, R2, R3
 - add R4, R1, R5
- Write-after-read (WAR) hazard
 - Stems from an *anti-dependence*
 - add R1, R2, R3
 - add R2, R4, R5
- Write-after-write (WAW) hazard
 - Stems from an *output dependence*
 - add R1, R2, R3
 - add R1, R4, R5

From in-order to OOO

- Limitation of our simple 5-stage pipeline
 - Blocking Execute Stage
 - Single universal unpipelined ALU
 - EX stage blocks for a multi-cycle instruction
 - MEM stage
 - Cache-missed load or store in MEM stage blocks ALU-type instructions in EX stage
 - These are structural hazards
 - The blocking Execute Stage obscures key distinction between *in-order issue* and *OOO issue*
 - How data hazards are handled (RAW, WAR, and WAW)
 - Structural hazard supersedes data hazards

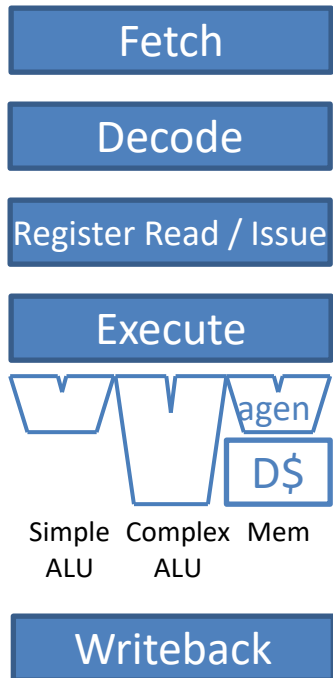
From in-order to OOO (cont.)

- A more aggressive *in-order scalar pipeline*
 - Non-blocking Execute Stage
 - Removes structural hazards
 - Highlight the nature of “in-order issue” policy
 - An instruction stalls if it has a RAW hazard with a preceding instruction
 - That’s o.k. (it must)
 - *All instructions after it also stall*
 - This is the “in-order issue” policy, and where the problem lies
 - Stepping-stone for OOO implementation

From in-order to OOO (cont.)

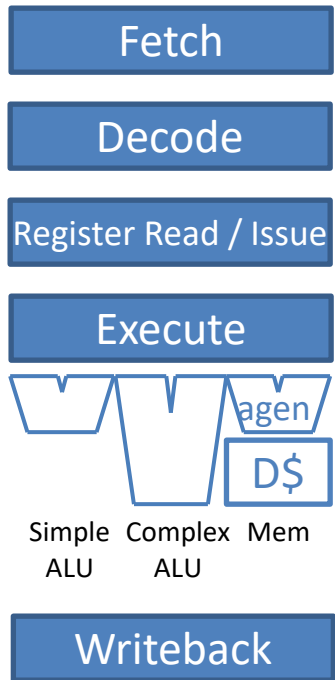
- *Out-of-order scalar pipeline*
 - An instruction stalls if it has a RAW hazard with a preceding instruction
 - That's o.k. (it must)
 - Independent instructions after it do *not* stall: they may issue out of program order
 - Two alternatives for handling WAR and WAW hazards:
 - Like in-order, stall the instruction and all instructions after it
 - Eliminate WAR and WAW hazards with *register renaming*

In-order Scalar Pipeline



In-order Scalar Pipeline

Assumptions:

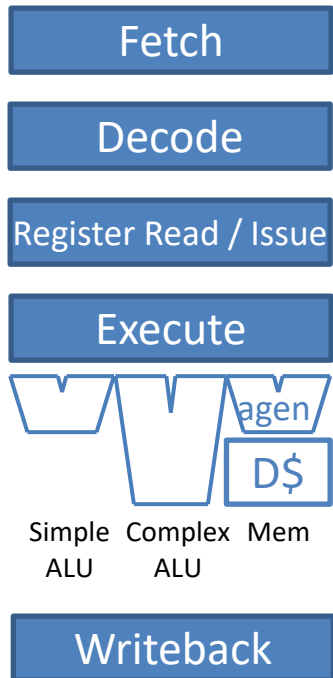


Scalar:

- fetch 1 instr/cycle
- decode 1 instr/cycle
- issue 1 instr/cycle to a function unit

In-order Scalar Pipeline

Assumptions:

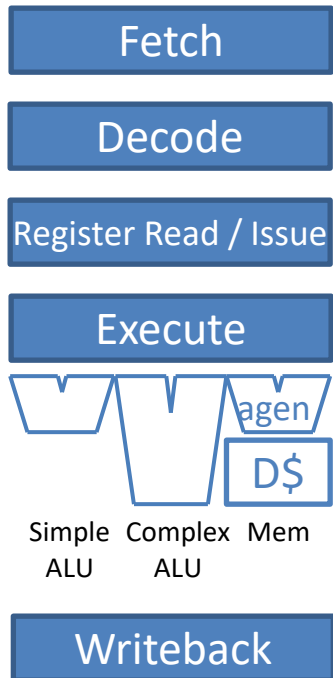


Execute stage:

- Contains multiple function units (FUs) to support different instruction classes.
- Multi-cycle function units are pipelined (e.g., Complex ALU, Mem).
- Therefore: may observe multiple instructions executing concurrently, yet only 1 new instruction may begin executing in a cycle (scalar issue).

In-order Scalar Pipeline

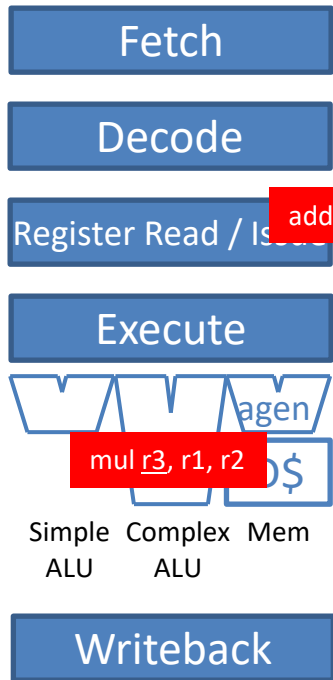
Assumptions:



Issue logic:

- RAW hazard:
Instruction stalls if its source register(s) are not ready.
- WAW hazard:
Non-blocking Execute Stage plus different FU latencies introduce out-of-order writeback. O.K. if writes are to different registers. Not O.K. if writes are to the same register.
Instruction stalls if its destination register is “busy”, i.e., conflicts with destination register of older instruction in Execute Stage.
- WAR hazard:
Not a problem in in-order pipelines. In-order issue ensures read by first instruction happens before write by second instruction.

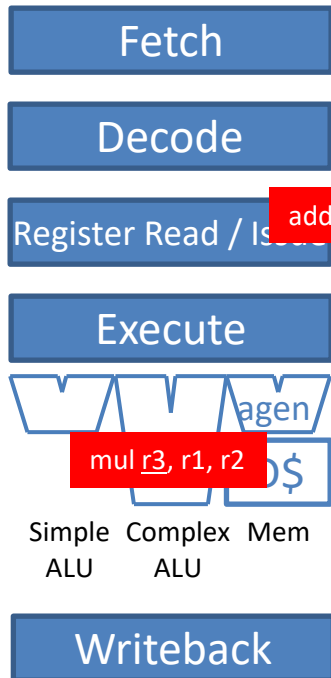
RAW example



Issue logic:

- RAW hazard:
Instruction stalls if its source register(s) are not ready.
- WAW hazard:
Non-blocking Execute Stage plus different FU latencies introduce out-of-order writeback. O.K. if writes are to different registers. Not O.K. if writes are to the same register.
Instruction stalls if its destination register is “busy”, i.e., conflicts with destination register of older instruction in Execute Stage.
- WAR hazard:
Not a problem in in-order pipelines. In-order issue ensures read by first instruction happens before write by second instruction.

WAW example



Issue logic:

- RAW hazard:
Instruction stalls if its source register(s) are not ready.
- WAW hazard:
Non-blocking Execute Stage plus different FU latencies introduce out-of-order writeback. O.K. if writes are to different registers. Not O.K. if writes are to the same register.
Instruction stalls if its destination register is “busy”, i.e., conflicts with destination register of older instruction in Execute Stage.
- WAR hazard:
Not a problem in in-order pipelines. In-order issue ensures read by first instruction happens before write by second instruction.

WAR example

Fetch

Decode add r3, r6, r7

Register Read / Issue add r5, r3, r4

Execute

Simple ALU mul r3, r1, r2 Complex ALU add r3, r6, r7 Mem add r5, r3, r4

Simple ALU Complex ALU Mem

Writeback

Issue logic:

- RAW hazard:
Instruction stalls if its source register(s) are not ready.
- WAW hazard:
Non-blocking Execute Stage plus different FU latencies introduce out-of-order writeback. O.K. if writes are to different registers. Not O.K. if writes are to the same register.
Instruction stalls if its destination register is “busy”, i.e., conflicts with destination register of older instruction in Execute Stage.
- WAR hazard:
Not a problem in in-order pipelines. In-order issue ensures read by first instruction happens before write by second instruction.

- Scenario 1

Scenario 1: load miss followed by independent instructions

i1: load r2, #0(r1)

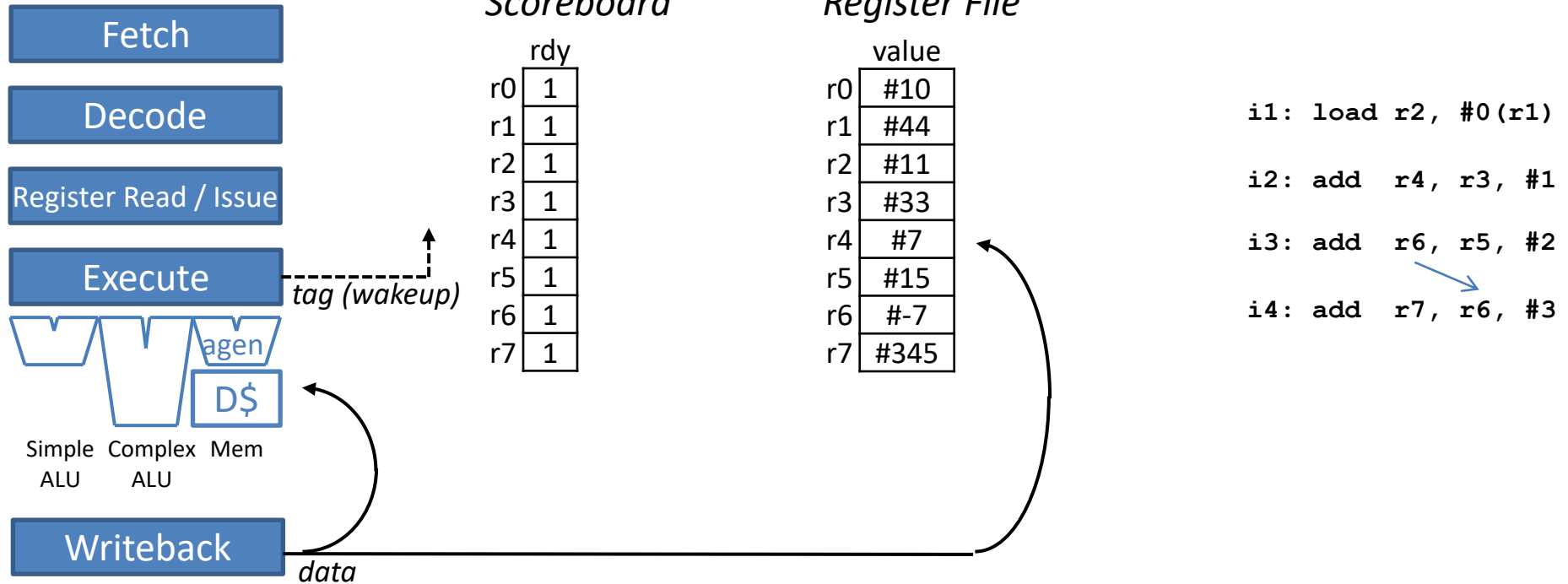
i2: add r4, r3, #1

i3: add r6, r5, #2

i4: add r7, r6, #3

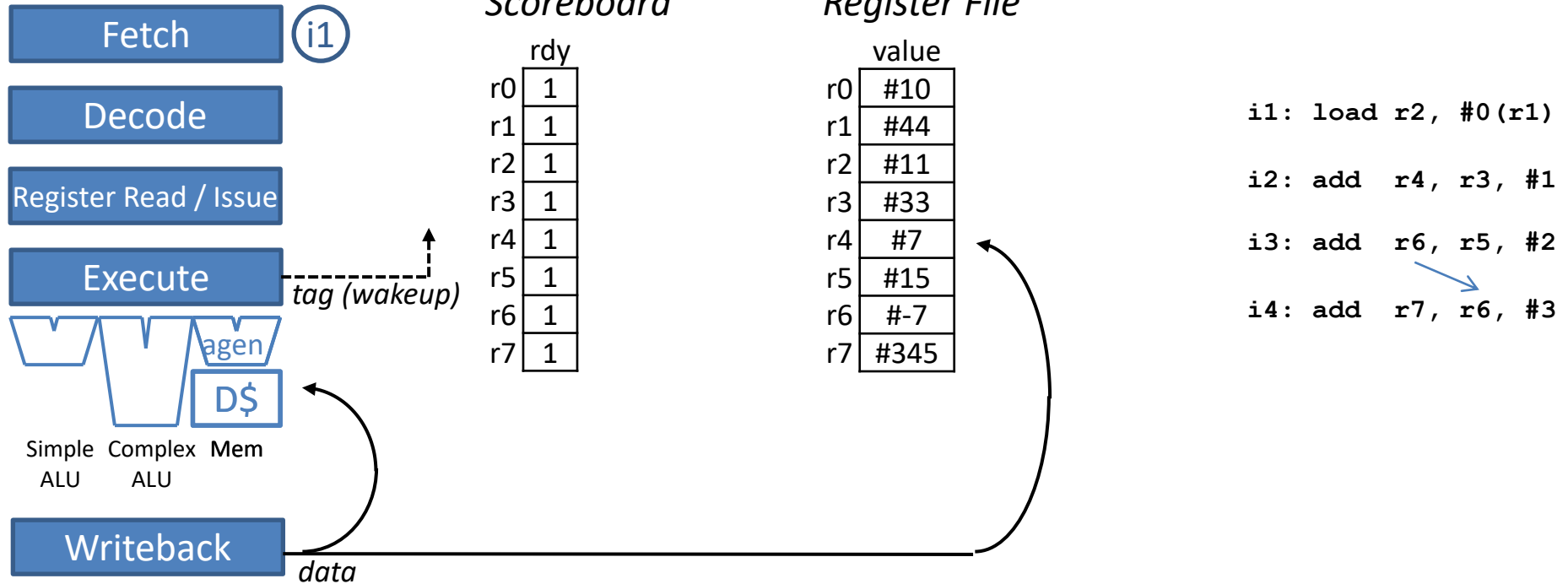


In-order Scalar Pipeline



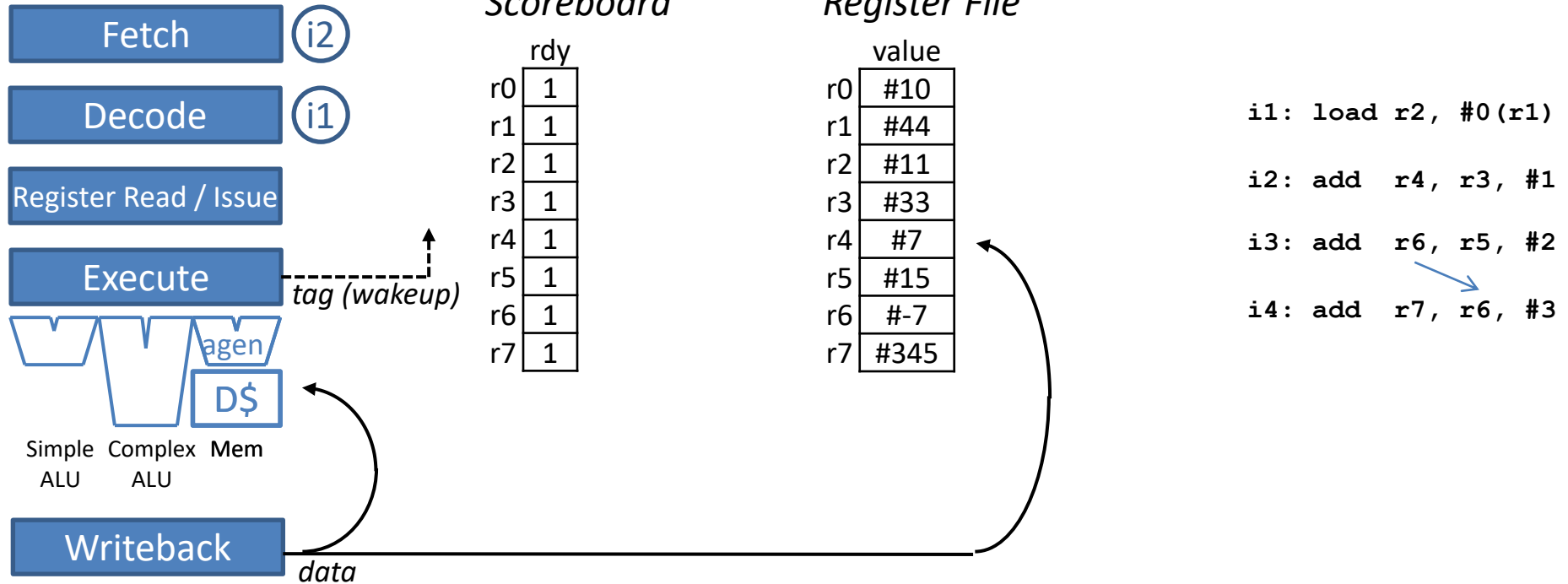
	1	2	3	4	5	6	7	8	9	10			
i1													
i2													
i3													
i4													

In-order Scalar Pipeline



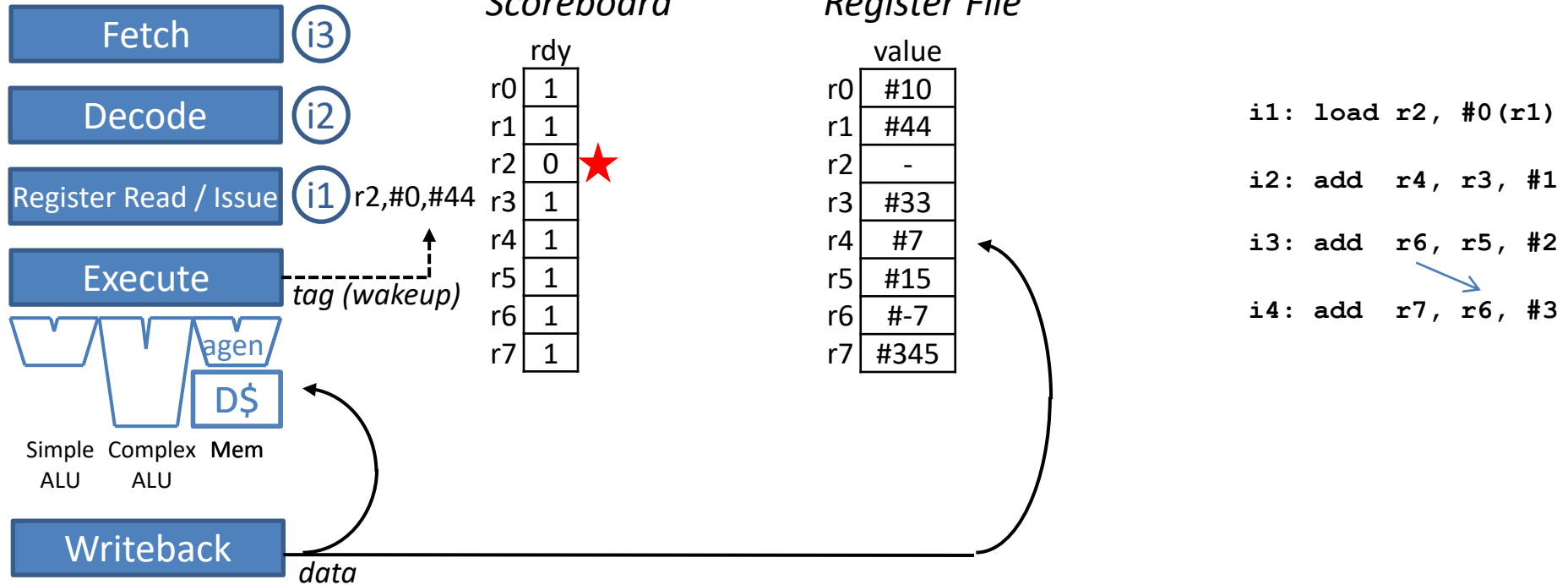
	1	2	3	4	5	6	7	8	9	10			
i1	FE												
i2													
i3													
i4													

In-order Scalar Pipeline



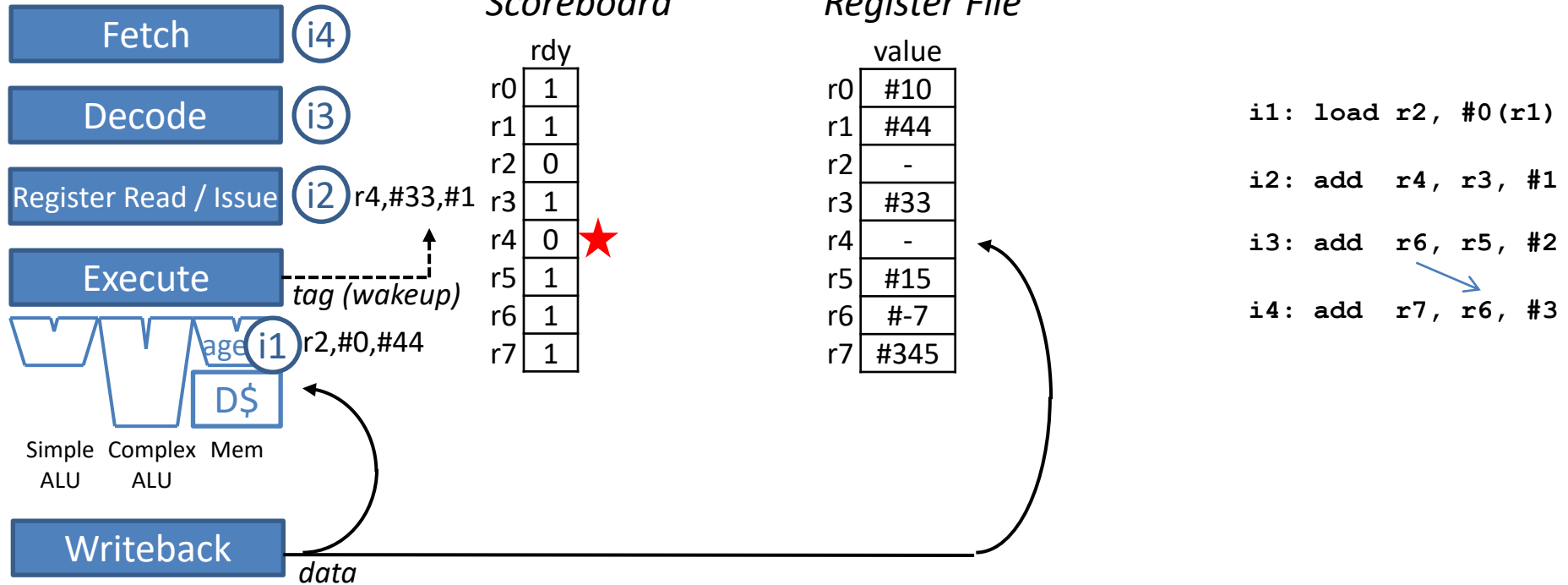
	1	2	3	4	5	6	7	8	9	10			
i1	FE	DE											
i2		FE											
i3													
i4													

In-order Scalar Pipeline



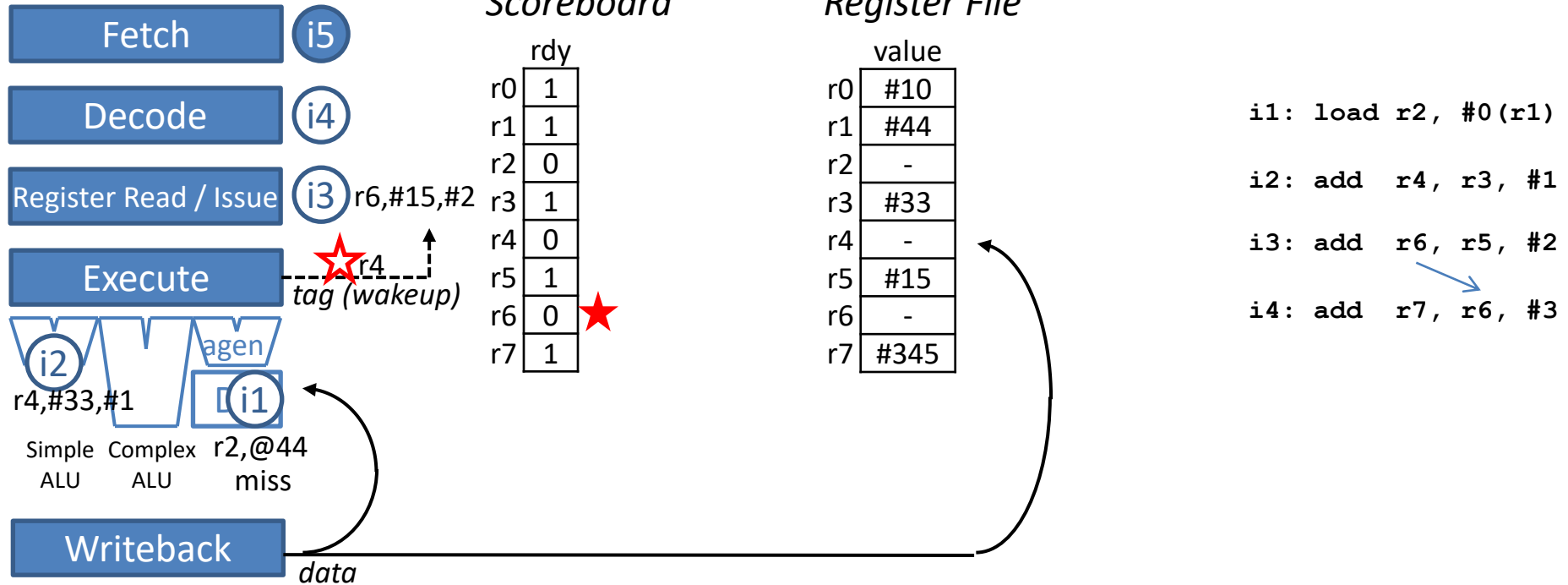
	1	2	3	4	5	6	7	8	9	10			
i1	FE	DE	RR										
i2		FE	DE										
i3			FE										
i4													

In-order Scalar Pipeline



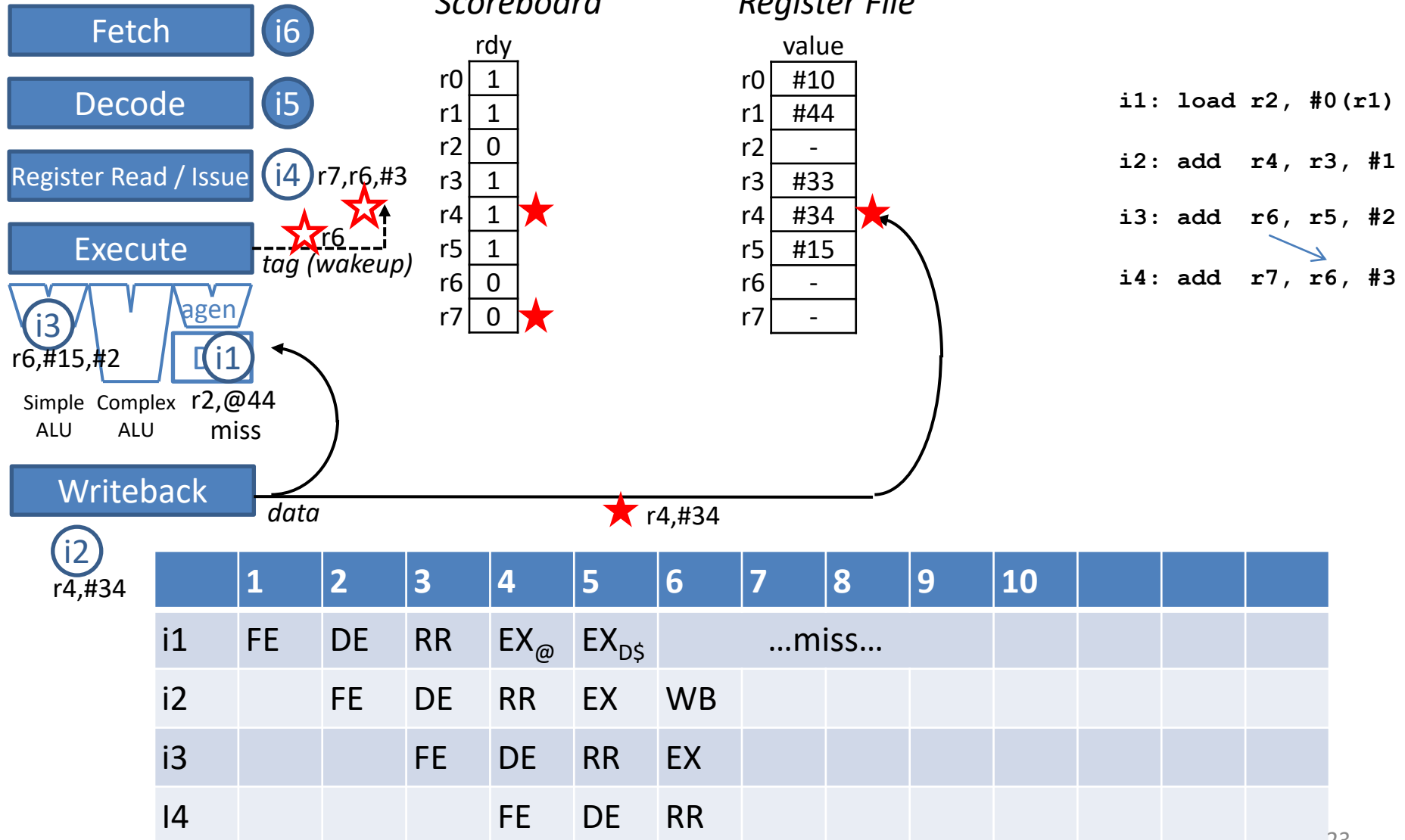
	1	2	3	4	5	6	7	8	9	10			
i1	FE	DE	RR	EX _@									
i2		FE	DE	RR									
i3			FE	DE									
i4				FE									

In-order Scalar Pipeline

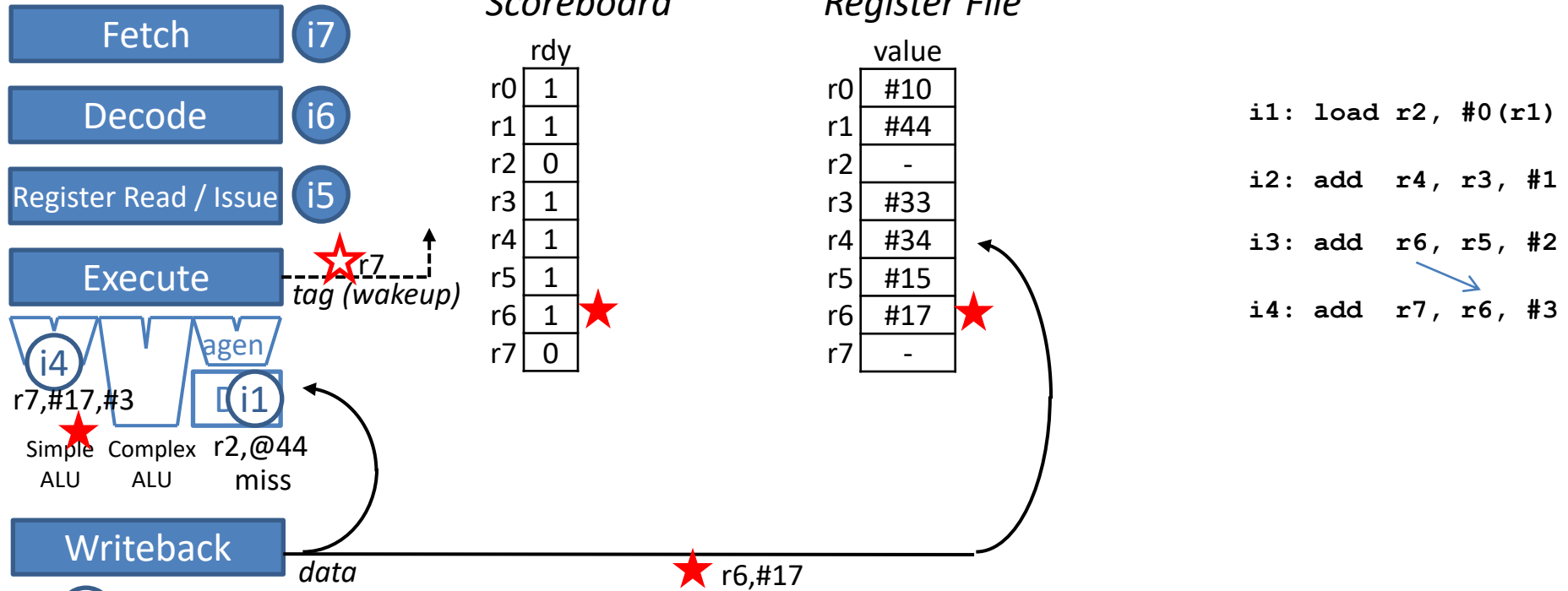


	1	2	3	4	5	6	7	8	9	10			
i1	FE	DE	RR	EX _@	EX _{D\$}	...miss...							
i2		FE	DE	RR	EX								
i3			FE	DE	RR								
i4				FE	DE								

In-order Scalar Pipeline

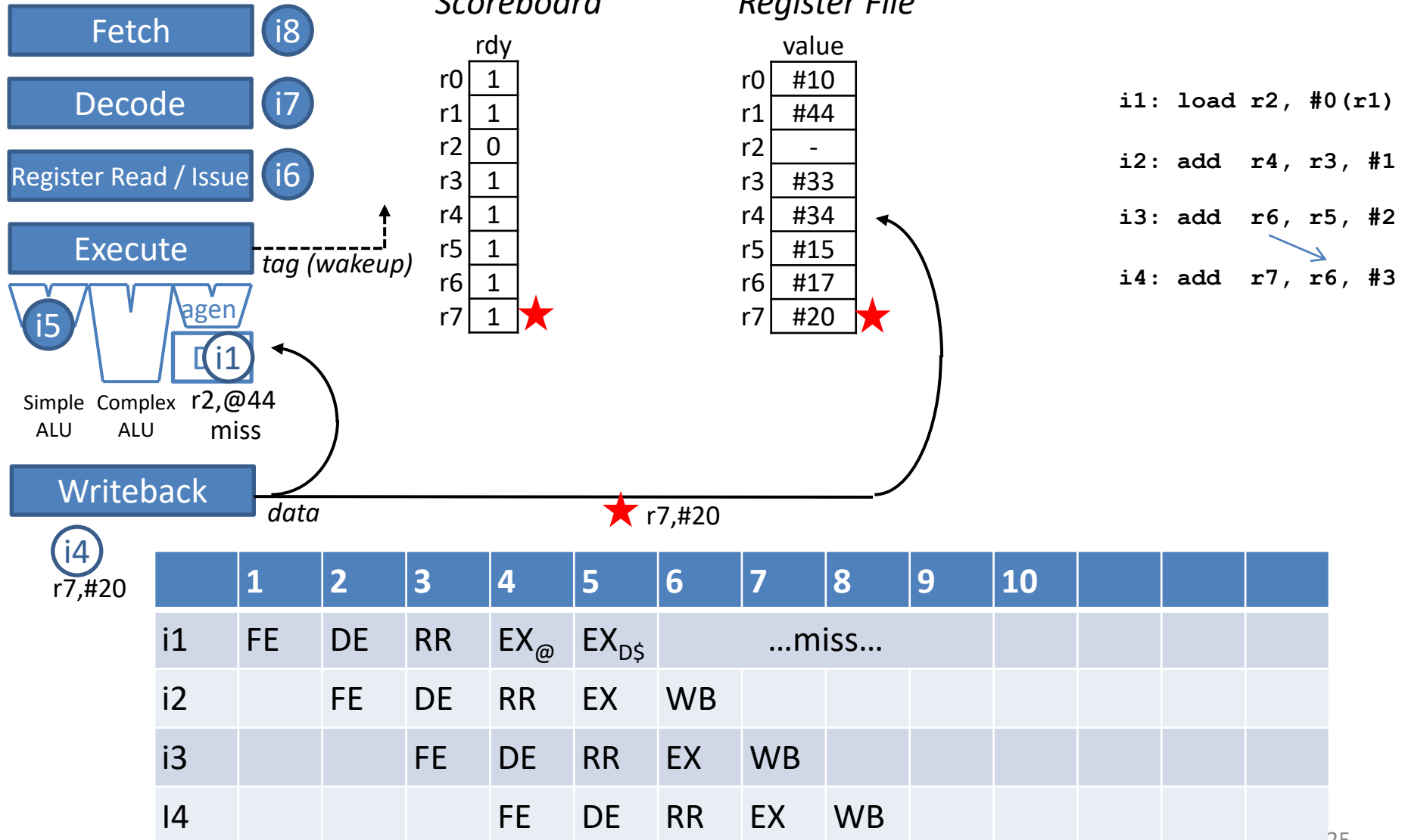


In-order Scalar Pipeline

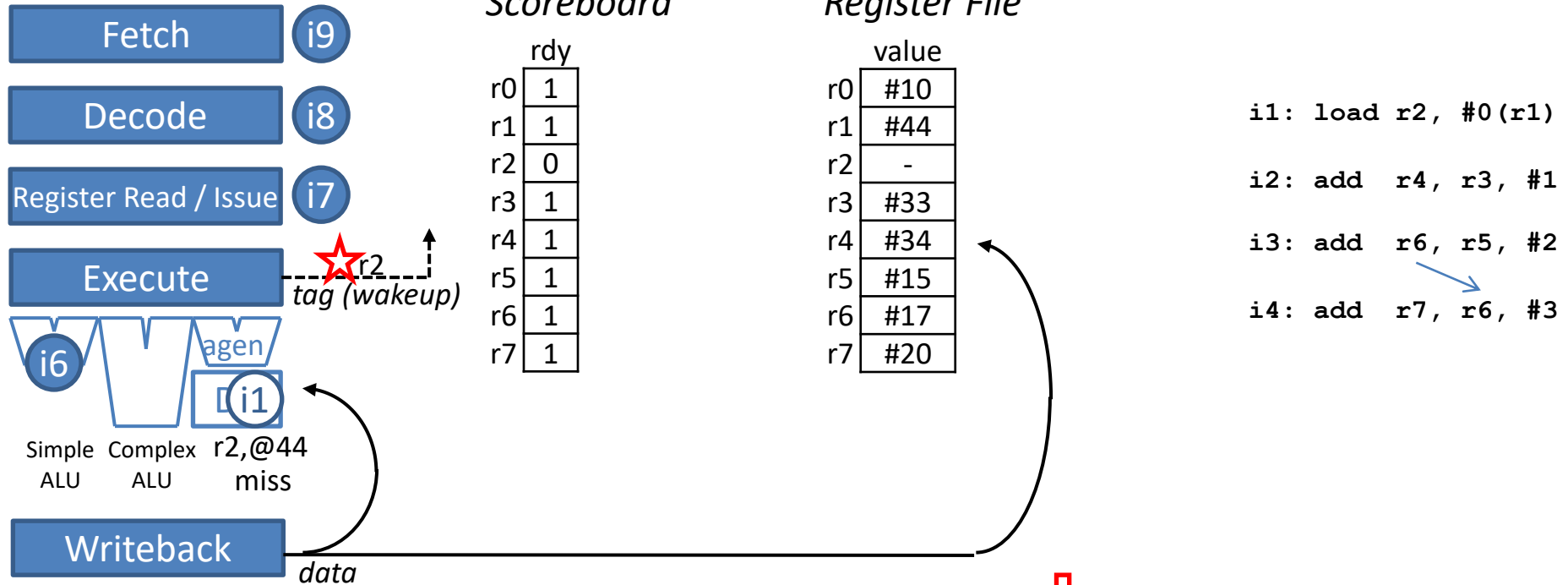


	1	2	3	4	5	6	7	8	9	10			
i1	FE	DE	RR	EX _@	EX _{D\$}	...miss...							
i2		FE	DE	RR	EX	WB							
i3			FE	DE	RR	EX	WB						
i4				FE	DE	RR	EX						

In-order Scalar Pipeline

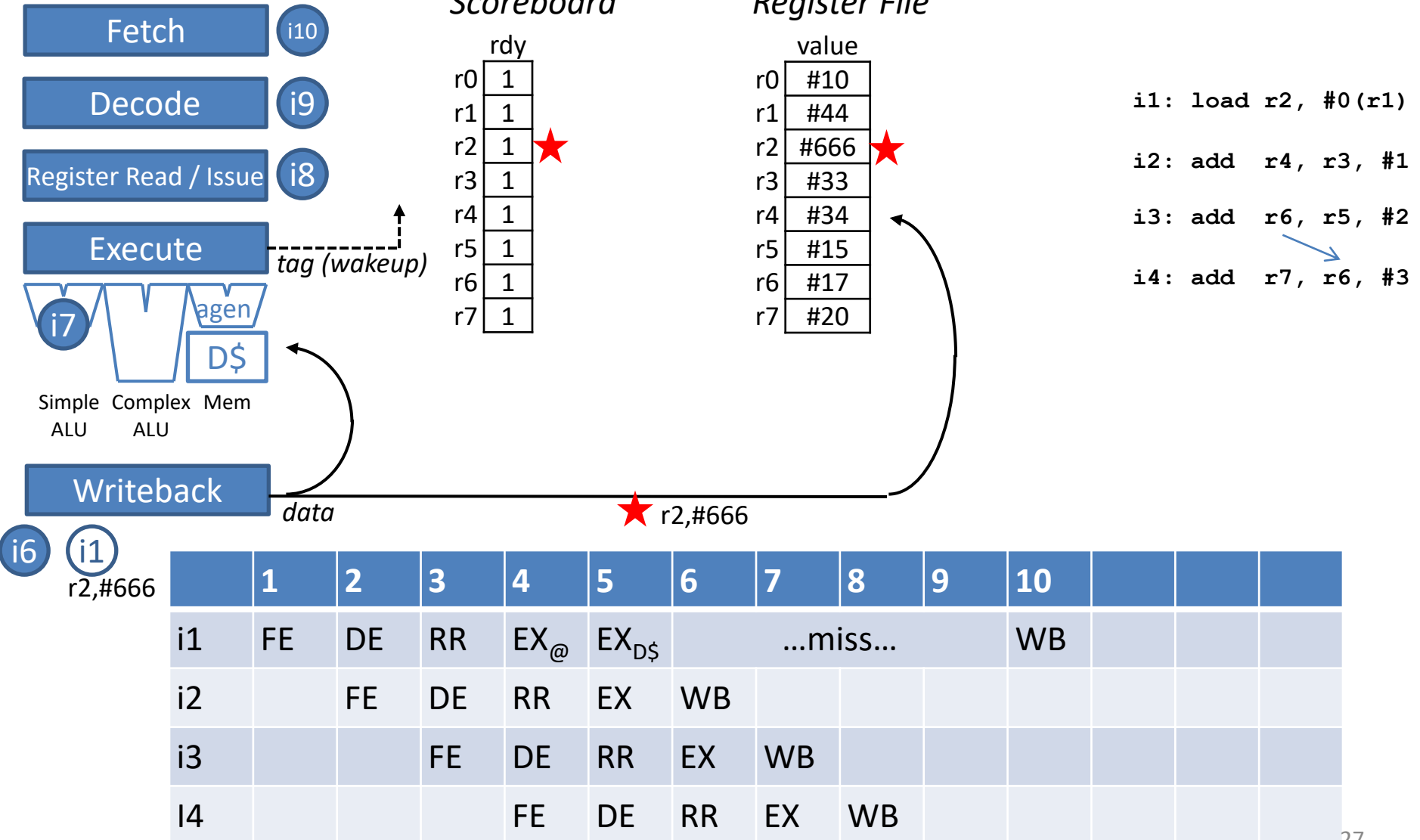


In-order Scalar Pipeline



	1	2	3	4	5	6	7	8	9	10			
i1	FE	DE	RR	EX _@	EX _{D\$}	...miss...							
i2		FE	DE	RR	EX	WB							
i3			FE	DE	RR	EX	WB						
i4				FE	DE	RR	EX	WB					

In-order Scalar Pipeline

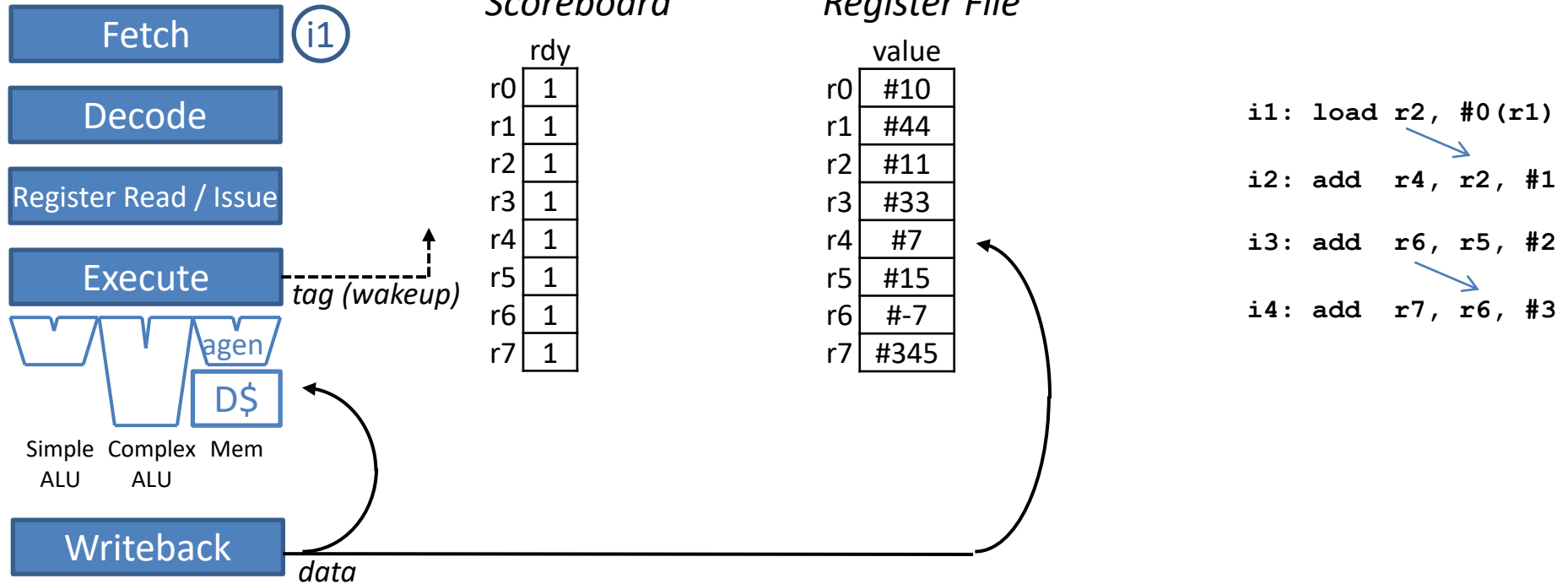


- Scenario 2

Scenario 2: load miss, followed by dependent instruction,
followed by independent instructions

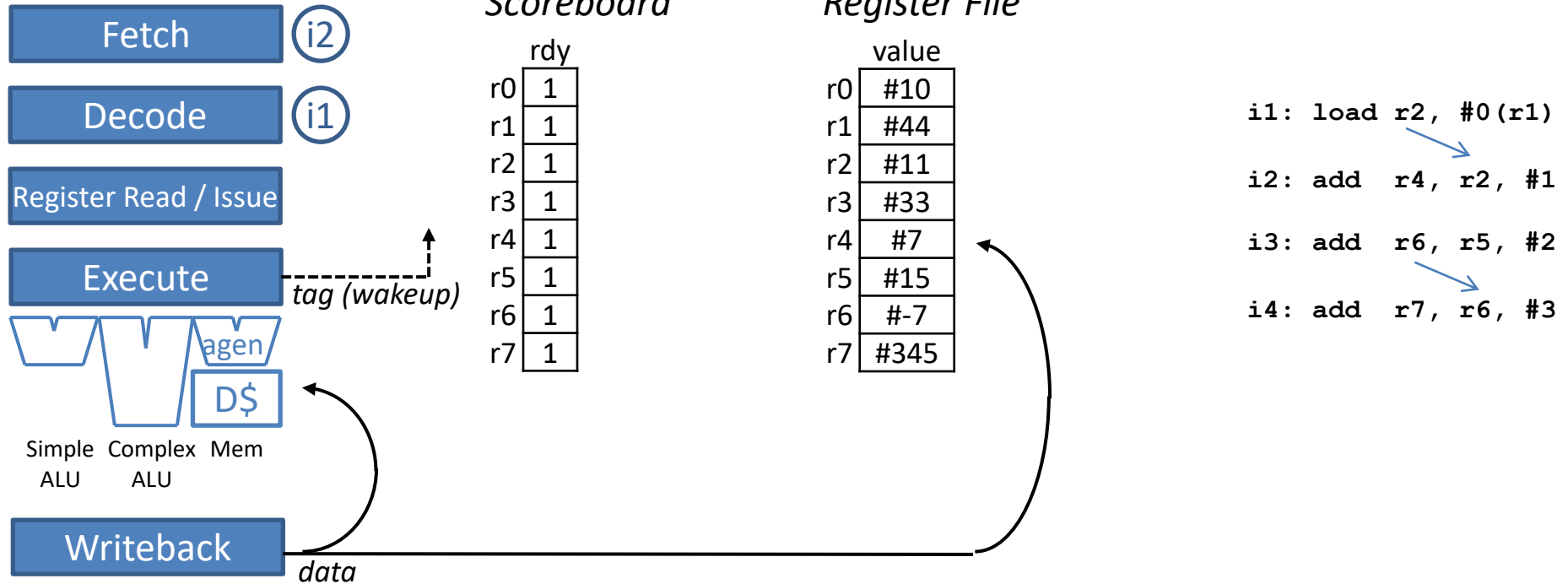
```
i1: load r2, #0(r1)
      ↓
i2: add  r4, r2, #1
      ↓
i3: add  r6, r5, #2
      ↓
i4: add  r7, r6, #3
```

In-order Scalar Pipeline



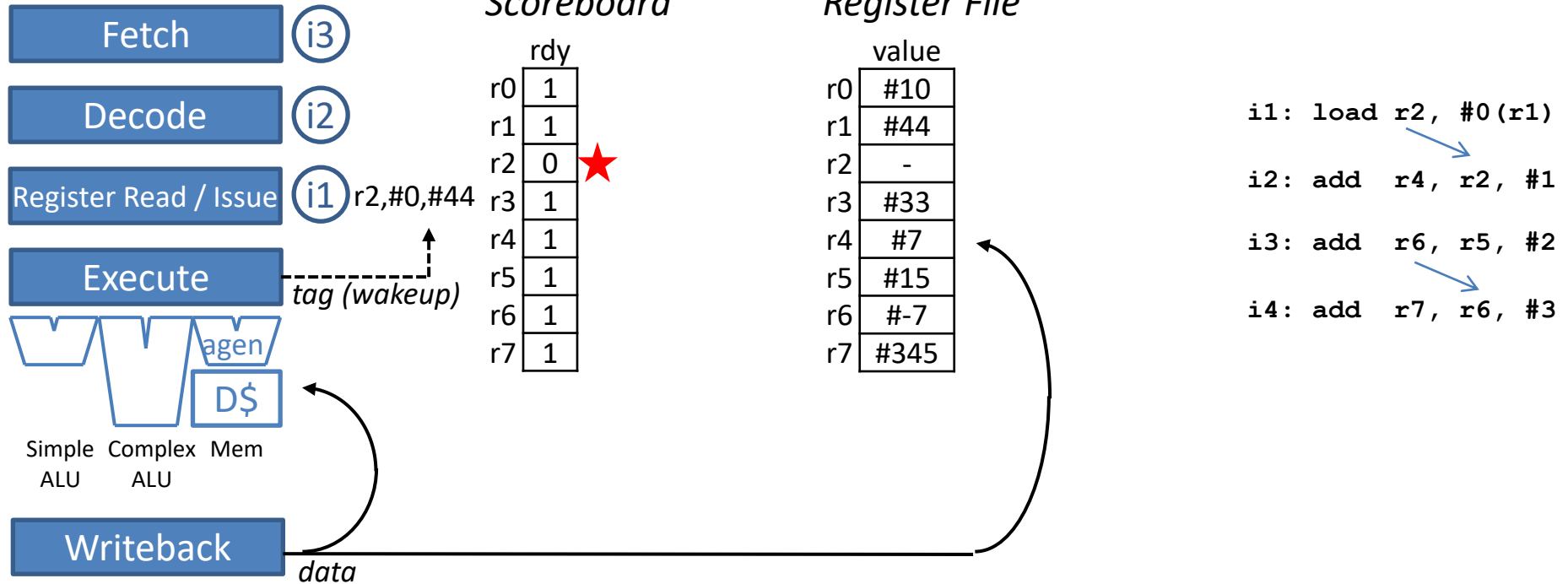
	1	2	3	4	5	6	7	8	9	10	11	12	13
i1	FE												
i2													
i3													
i4													

In-order Scalar Pipeline



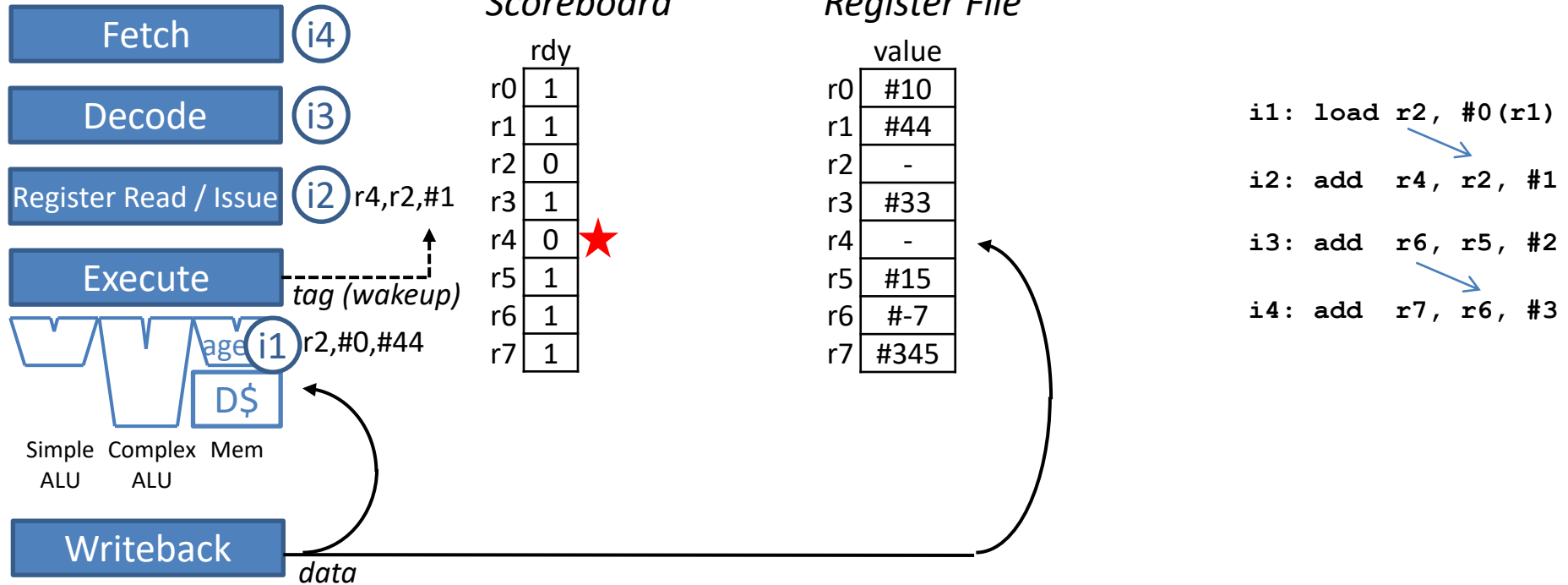
	1	2	3	4	5	6	7	8	9	10	11	12	13
i1	FE	DE											
i2		FE											
i3													
i4													

In-order Scalar Pipeline



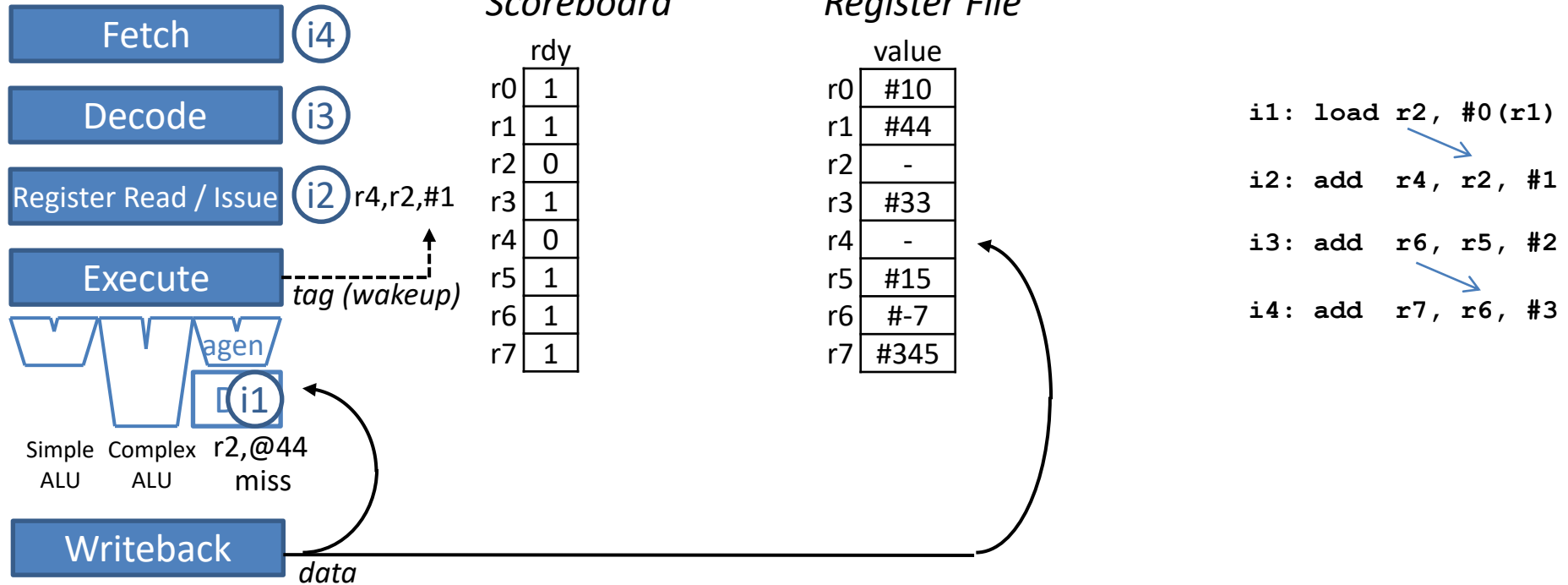
	1	2	3	4	5	6	7	8	9	10	11	12	13
i1	FE	DE	RR										
i2		FE	DE										
i3			FE										
i4													

In-order Scalar Pipeline



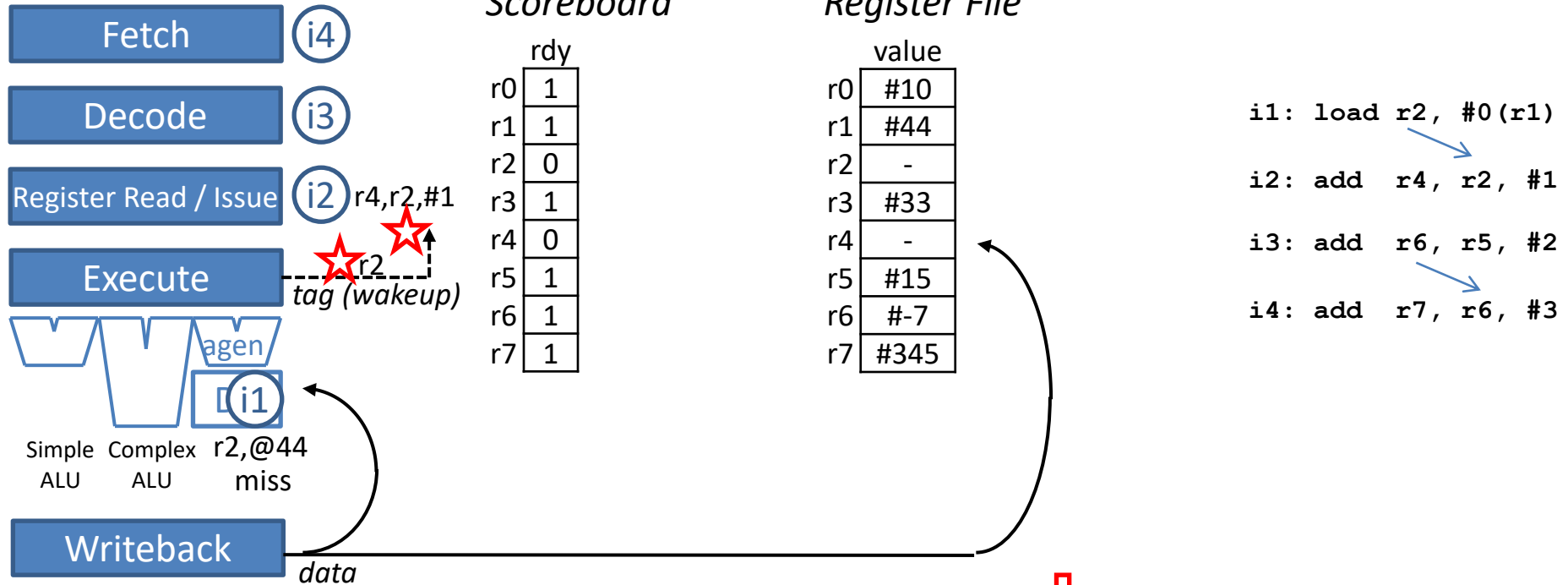
	1	2	3	4	5	6	7	8	9	10	11	12	13
i1	FE	DE	RR	EX _@									
i2		FE	DE	RR									
i3			FE	DE									
i4				FE									

In-order Scalar Pipeline



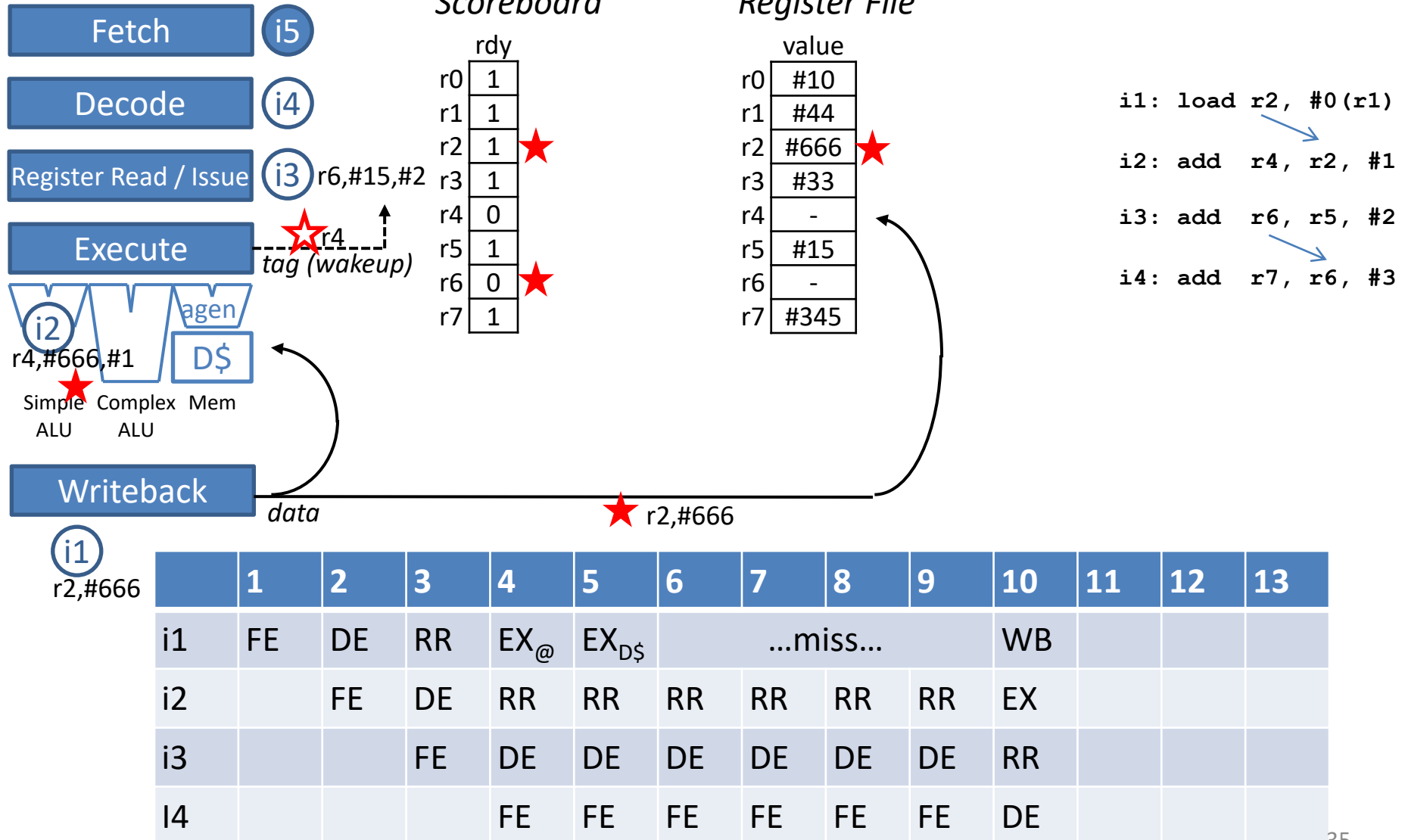
	1	2	3	4	5	6	7	8	9	10	11	12	13
i1	FE	DE	RR	EX _@	EX _{D\$}	...miss...							
i2		FE	DE	RR	RR								
i3			FE	DE	DE								
i4				FE	FE								

In-order Scalar Pipeline

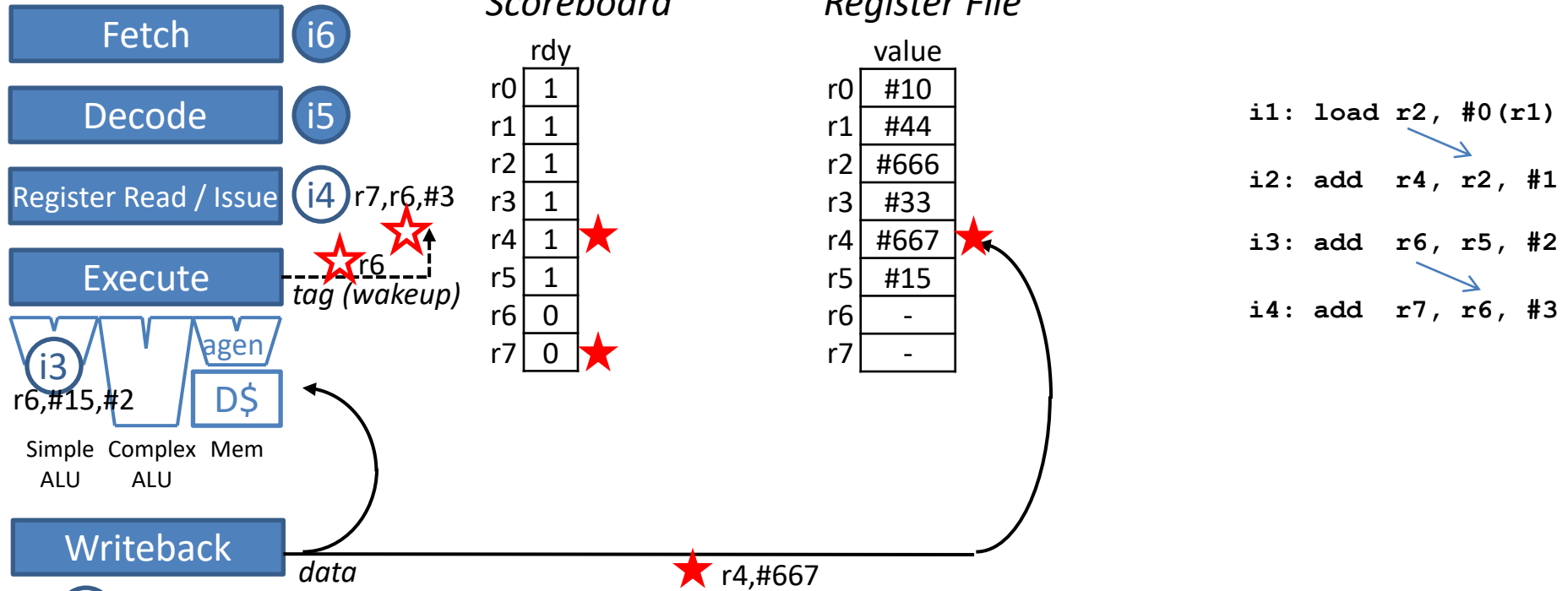


	1	2	3	4	5	6	7	8	9	10	11	12	13
i1	FE	DE	RR	EX _@	EX _{D\$}	...miss...							
i2		FE	DE	RR	RR	RR	RR	RR	RR				
i3			FE	DE	DE	DE	DE	DE	DE				
i4				FE	FE	FE	FE	FE	FE				

In-order Scalar Pipeline

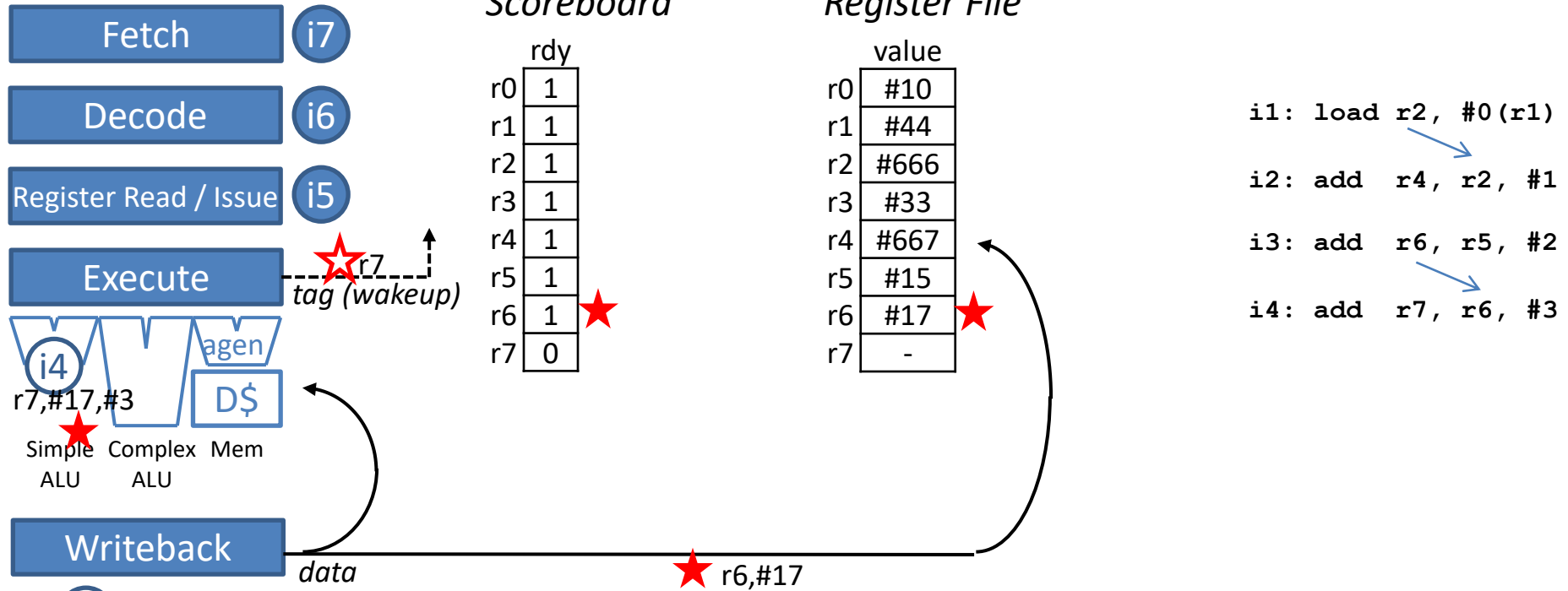


In-order Scalar Pipeline



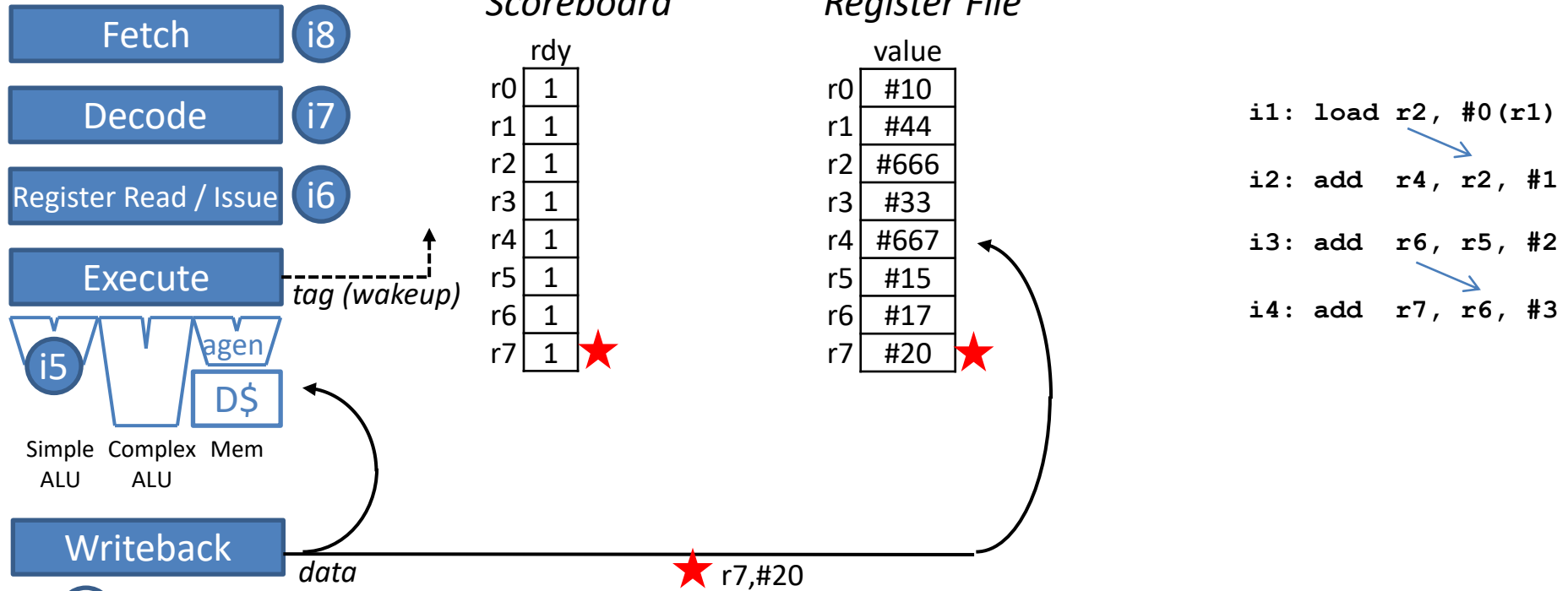
	1	2	3	4	5	6	7	8	9	10	11	12	13
i1	FE	DE	RR	EX _@	EX _{D\$}	...miss...				WB			
i2		FE	DE	RR	RR	RR	RR	RR	RR	EX	WB		
i3			FE	DE	DE	DE	DE	DE	DE	RR	EX		
i4				FE	FE	FE	FE	FE	FE	DE	RR		

In-order Scalar Pipeline



	1	2	3	4	5	6	7	8	9	10	11	12	13
i1	FE	DE	RR	EX _@	EX _{D\$}	...miss...				WB			
i2		FE	DE	RR	RR	RR	RR	RR	RR	EX	WB		
i3			FE	DE	DE	DE	DE	DE	DE	RR	EX	WB	
i4				FE	FE	FE	FE	FE	FE	DE	RR	EX	

In-order Scalar Pipeline

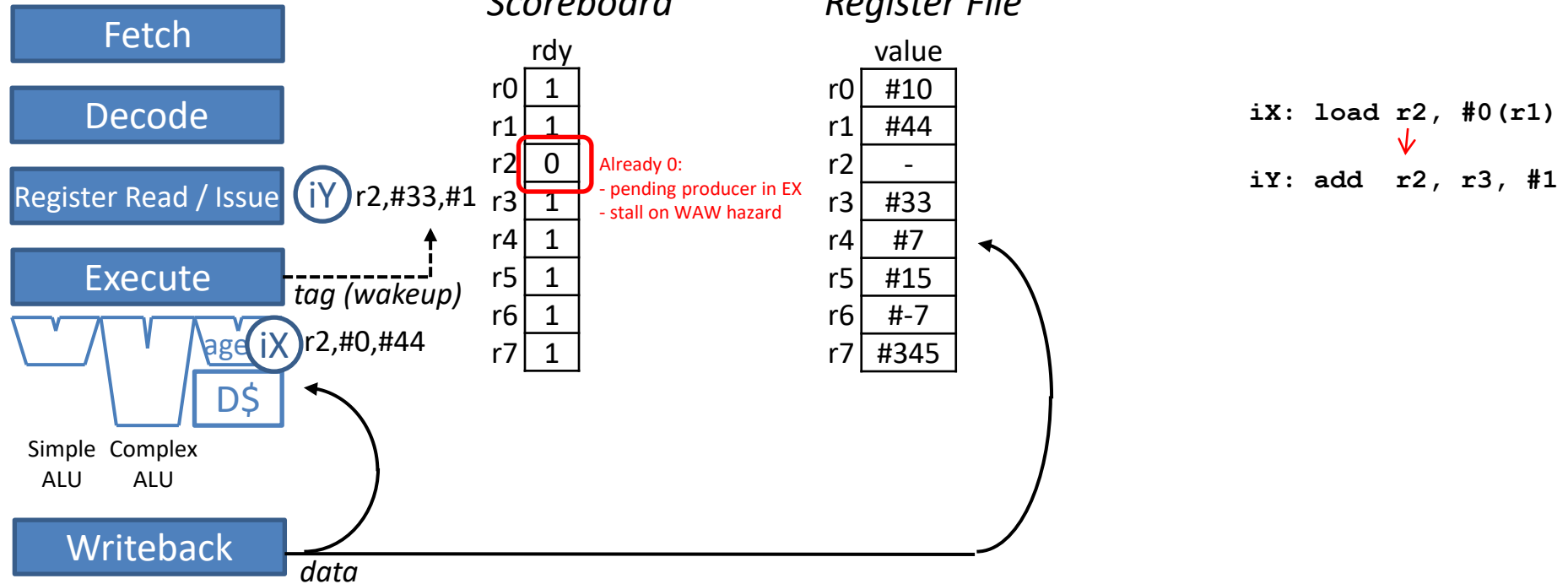


i4
r7, #20

	1	2	3	4	5	6	7	8	9	10	11	12	13
i1	FE	DE	RR	EX _@	EX _{D\$}	...miss...				WB			
i2		FE	DE	RR	RR	RR	RR	RR	RR	EX	WB		
i3			FE	DE	DE	DE	DE	DE	DE	RR	EX	WB	
i4				FE	FE	FE	FE	FE	FE	DE	RR	EX	WB

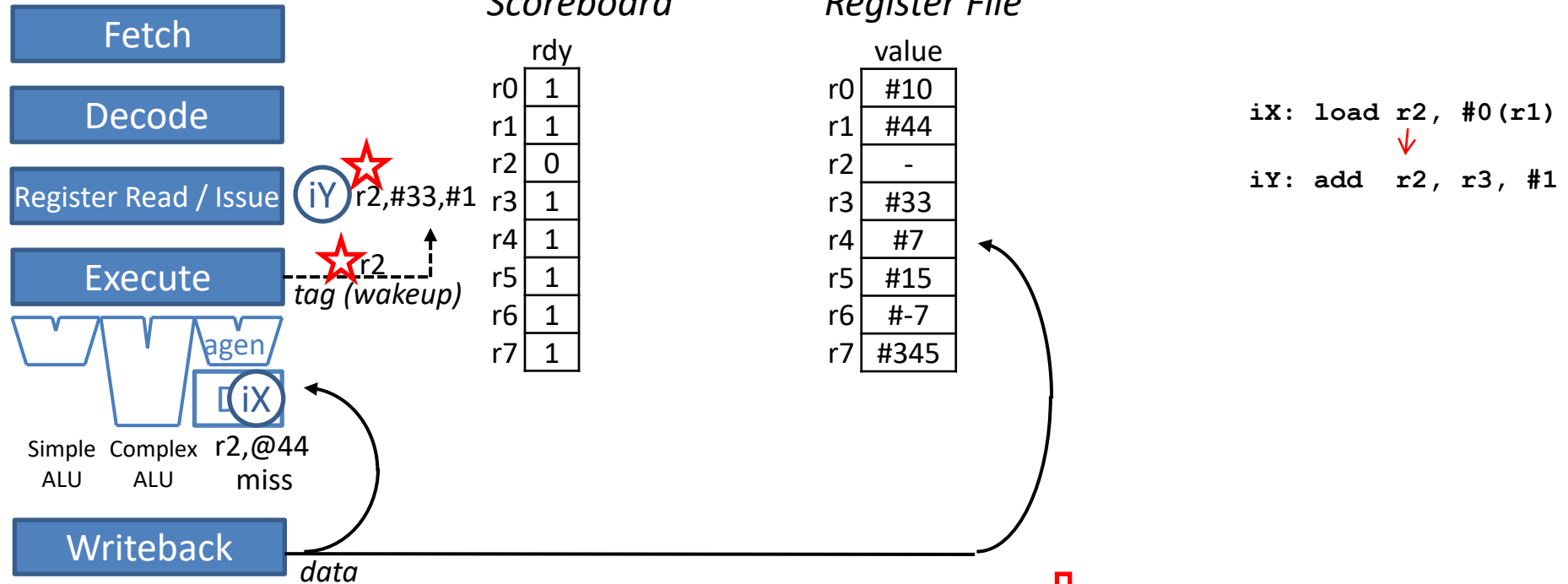
Begin aside: WAW hazard example

Aside: WAW hazard example (1)



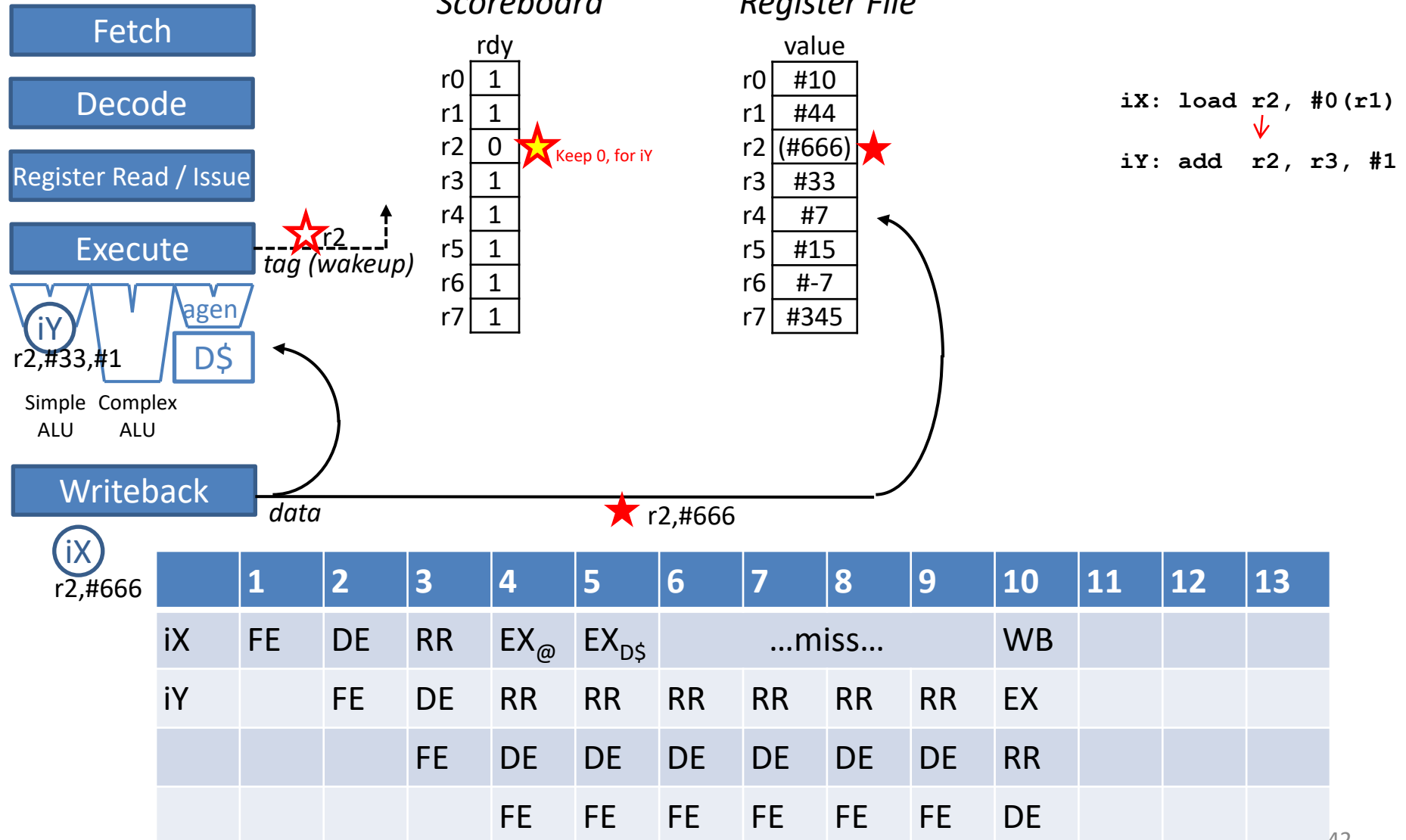
	1	2	3	4	5	6	7	8	9	10	11	12	13
iX	FE	DE	RR	EX _@									
iY		FE	DE	RR									
			FE	DE									
				FE									

Aside: WAW hazard example (2)

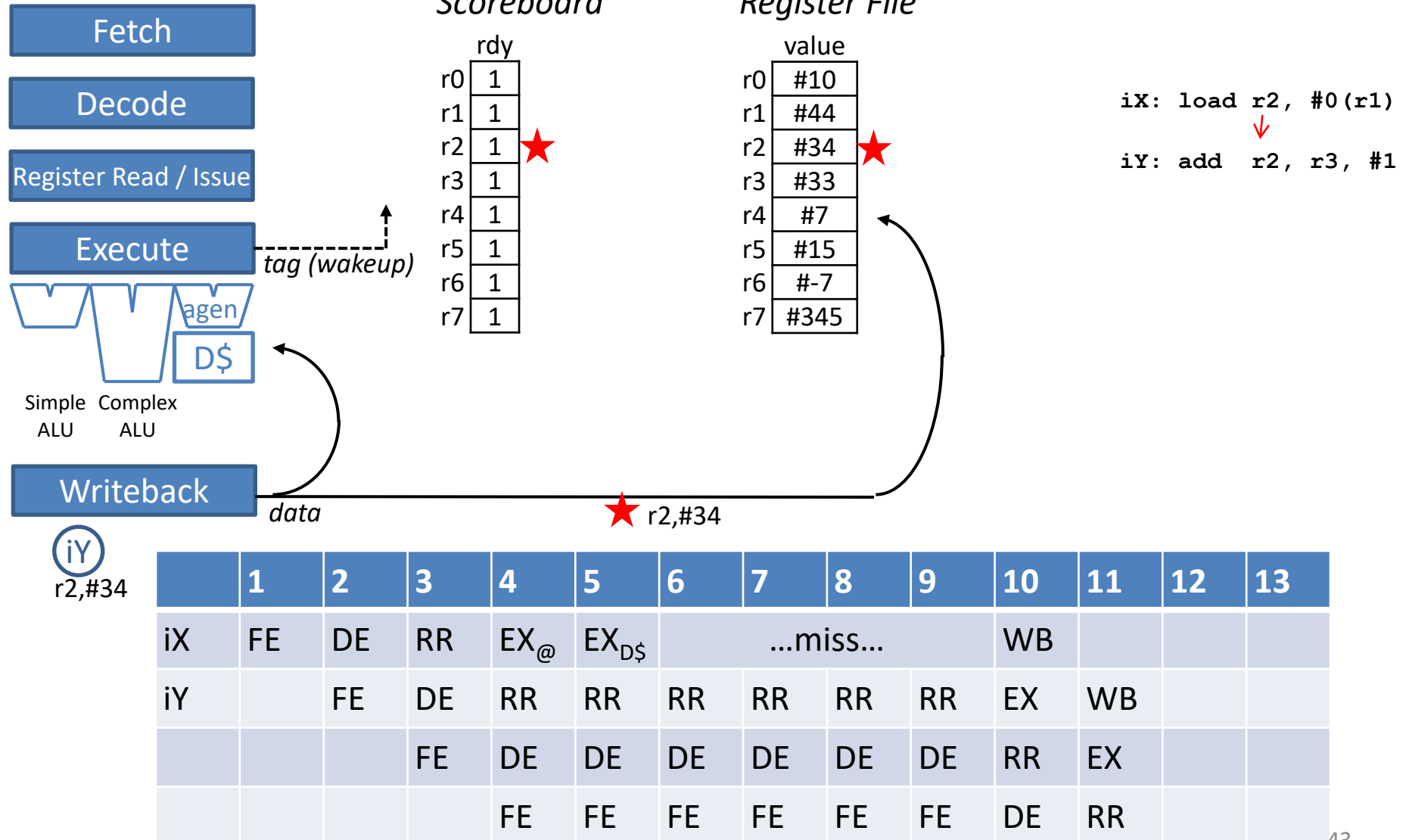


	1	2	3	4	5	6	7	8	9	10	11	12	13
iX	FE	DE	RR	EX _@	EX _{D\$}	...miss...							
iY		FE	DE	RR	RR	RR	RR	RR	RR				
			FE	DE	DE	DE	DE	DE	DE				
				FE	FE	FE	FE	FE	FE				

Aside: WAW hazard example (3)



Aside: WAW hazard example (4)



End aside: WAW hazard example

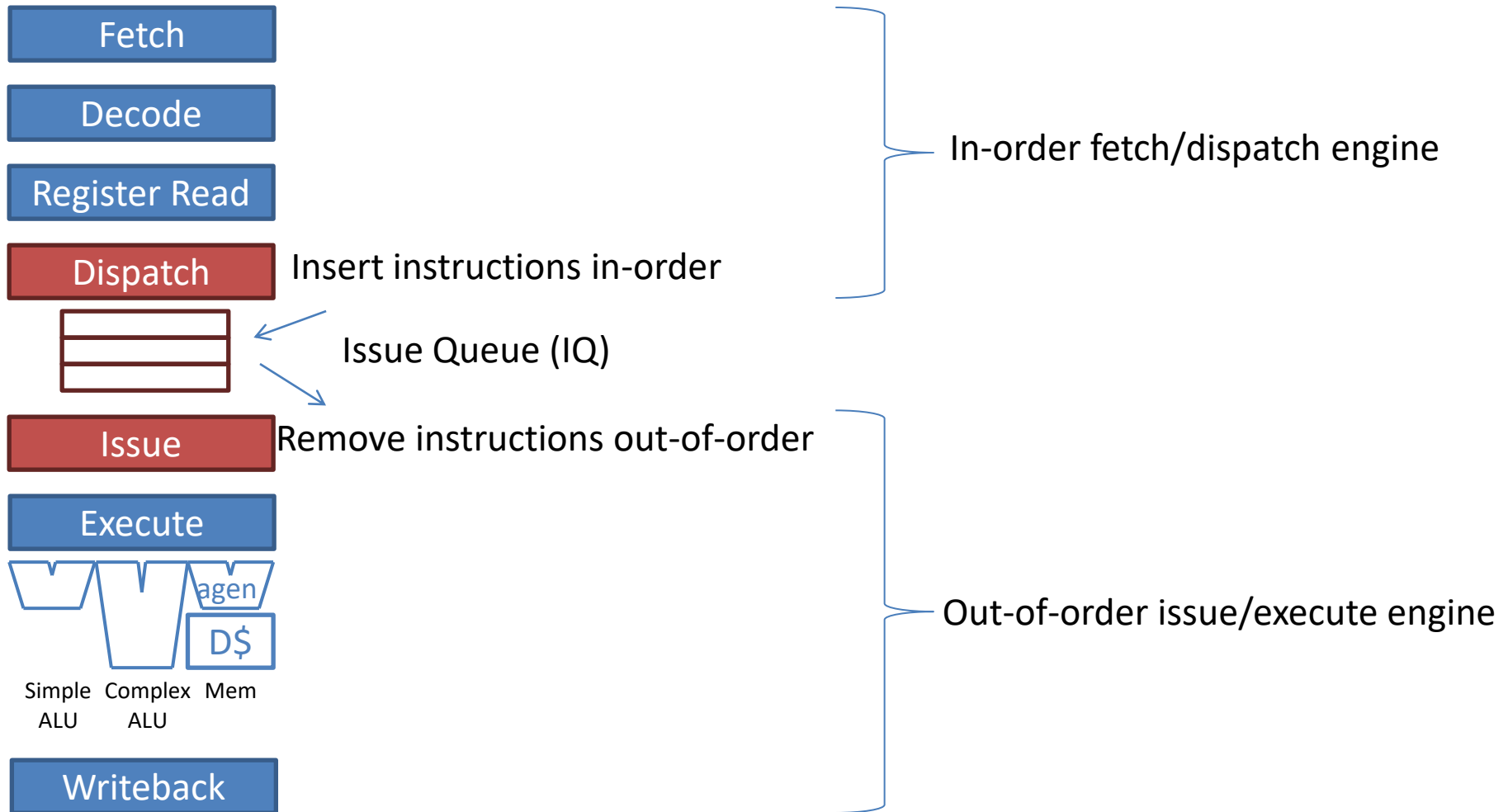
In-order Issue Bottleneck

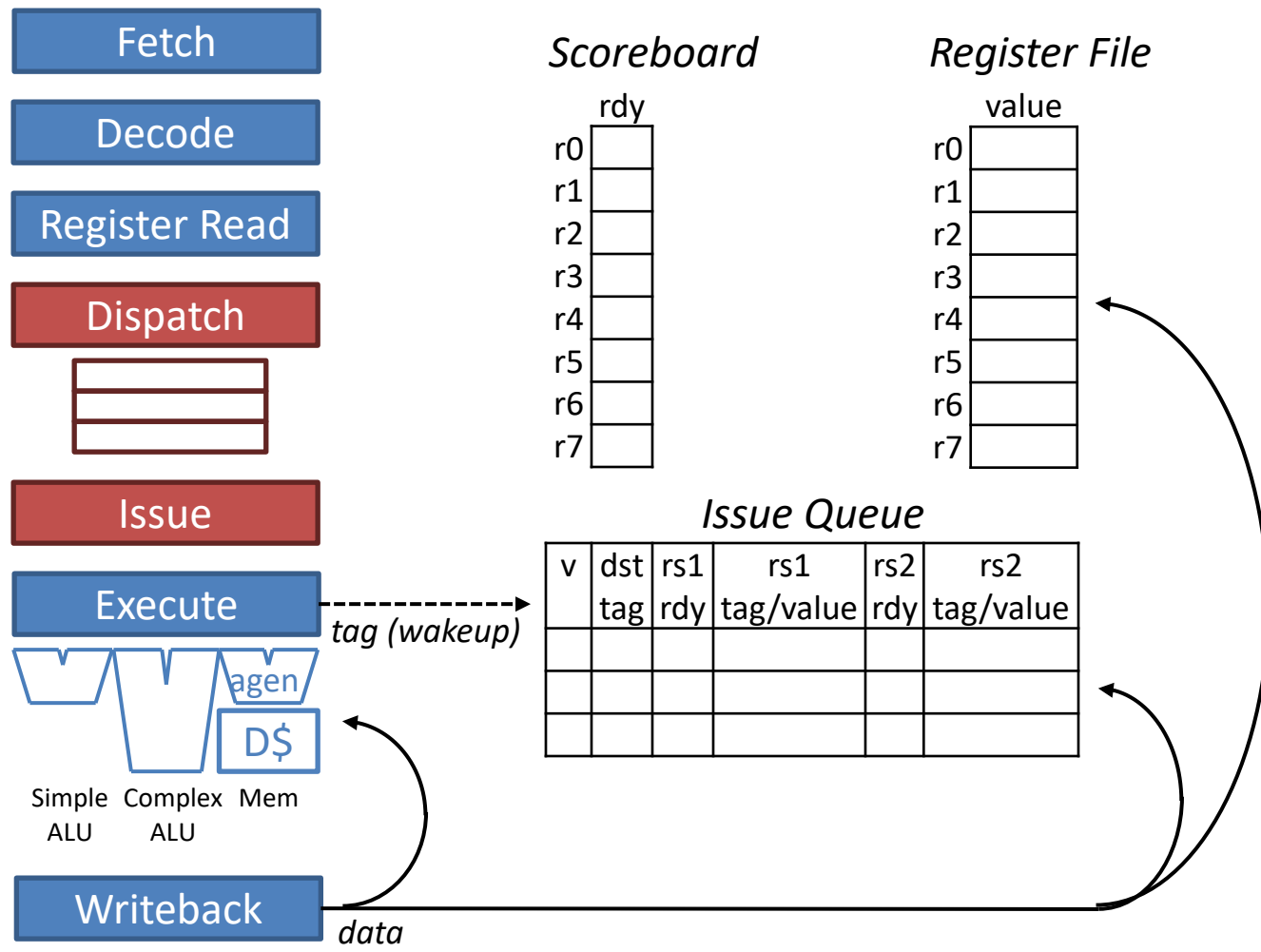
- *i2 must wait for i1*
 - *i2 depends on i1*
- *i3, i4 need not wait for i1, i2*
 - Independent
- Yet *i3, i4 also stall*
 - In-order issue translates to a structural hazard
 - RR stage (issue stage) blocked by the stalled *i2*
- OOO pipeline unblocks RR (issue) using a new instruction buffer for stalled data-dependent instructions
 - Many names in the literature
 - “reservation stations”, “issue buffer”, “issue queue”, “scheduler”, “scheduling window”

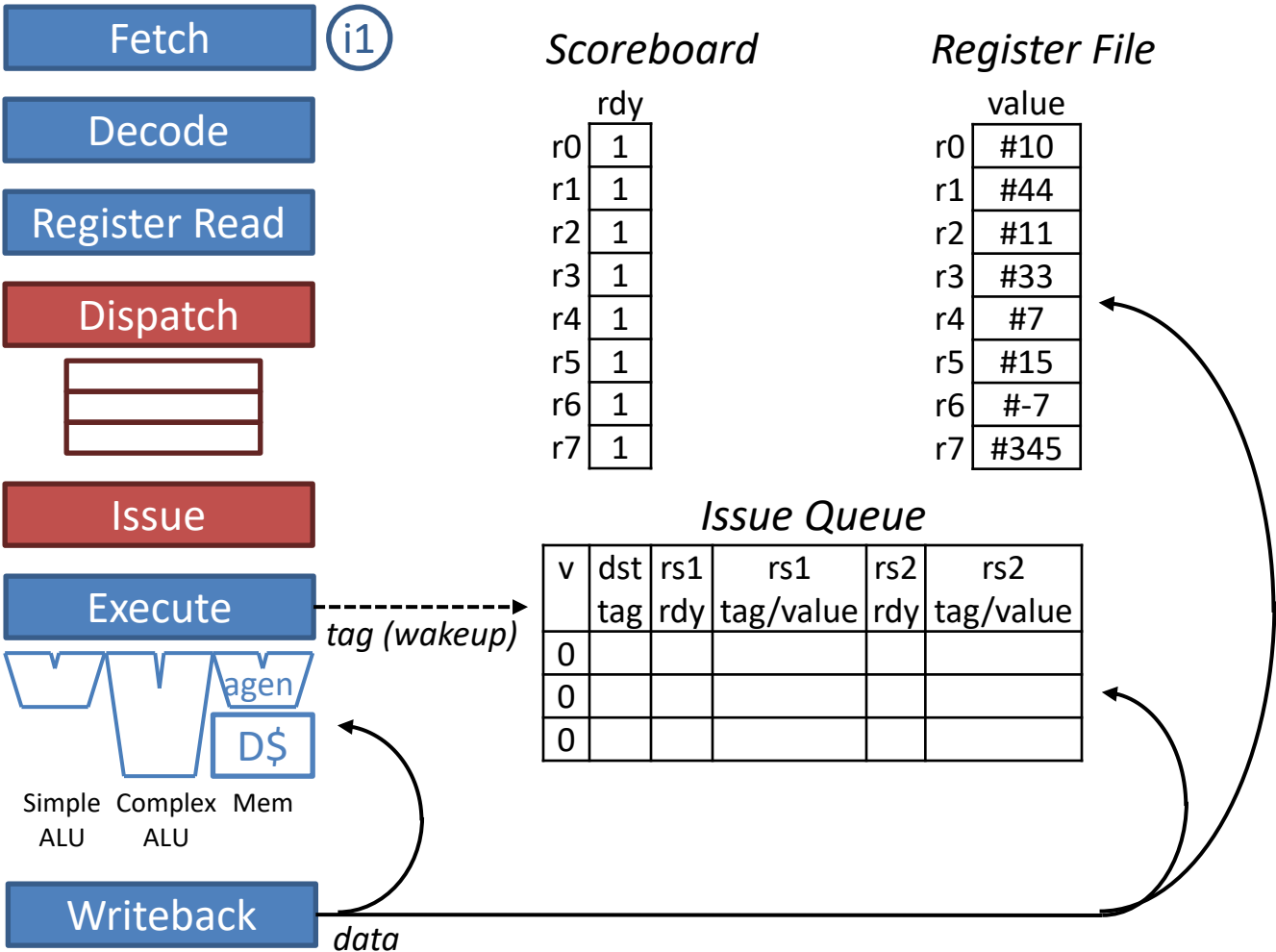
Issue Queue

- Stalled instructions do not impede instruction fetch
- Younger ready instructions issue and execute out-of-order with respect to older non-ready instructions
- Issue Queue opens up the pipeline to future independent instructions, to:
 - Tolerate long latencies (cache misses, floating-point)
 - Exploit instruction-level parallelism (ILP) (critical for superscalar)

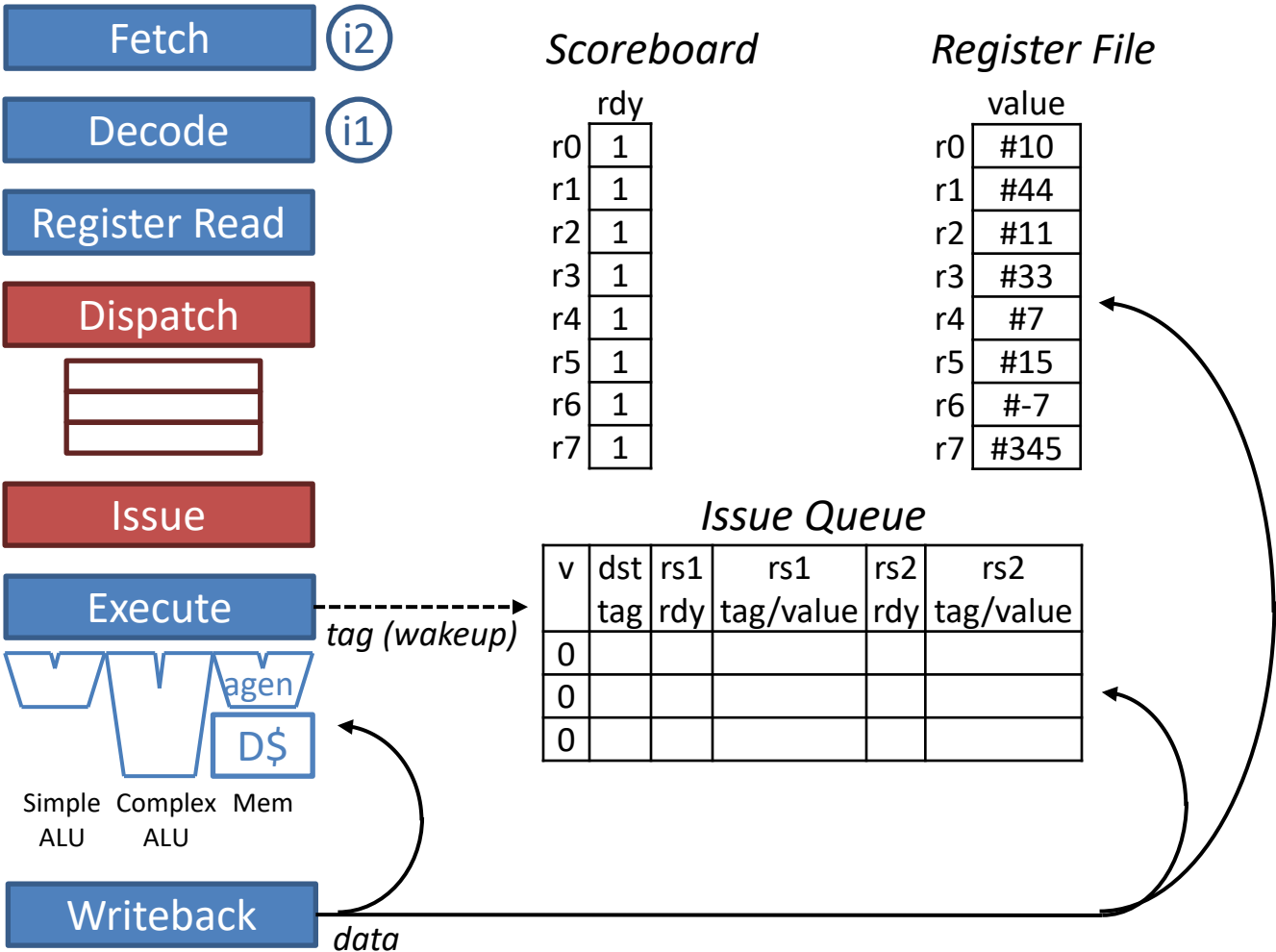
Out-of-Order Scalar Pipeline (vers. 1)



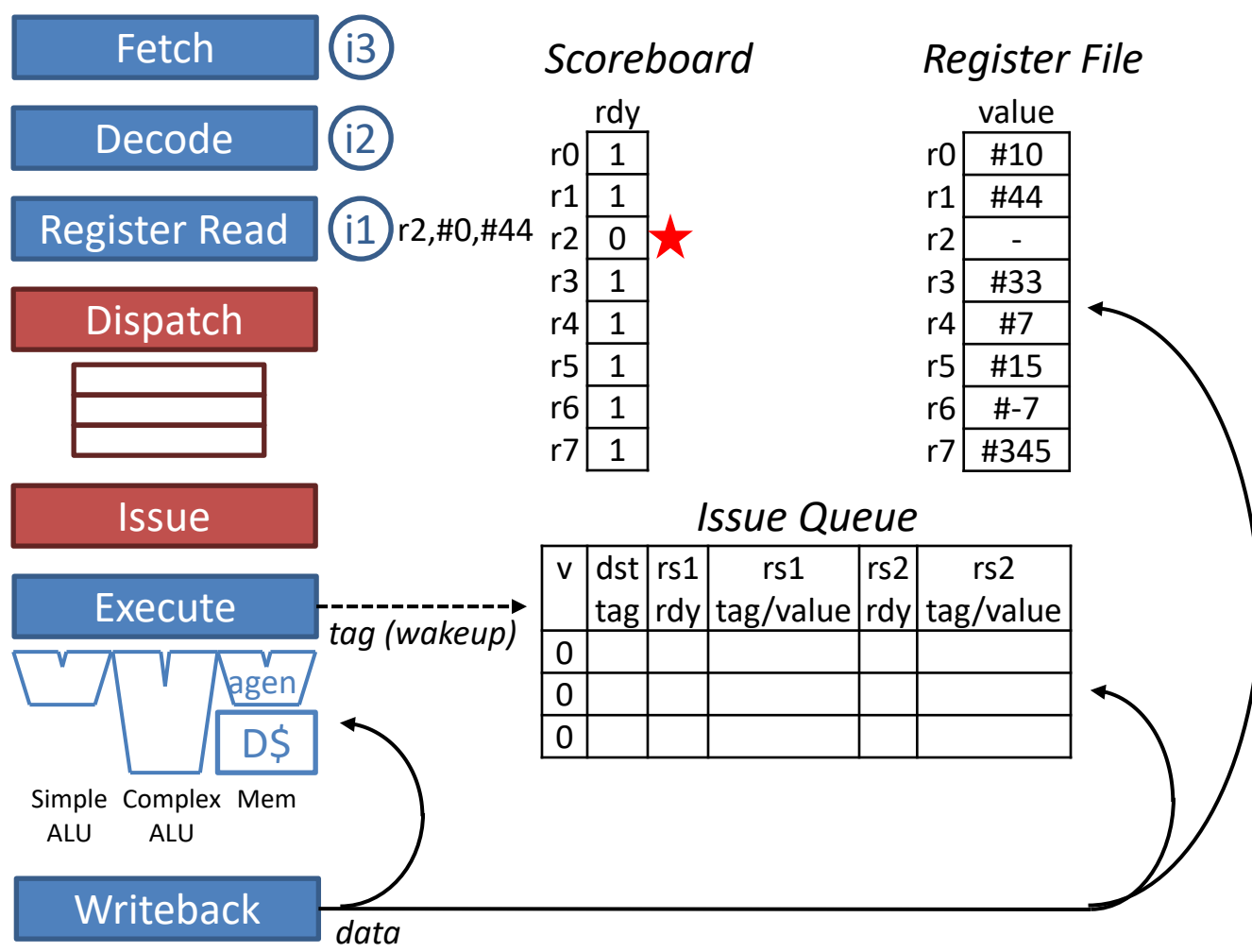




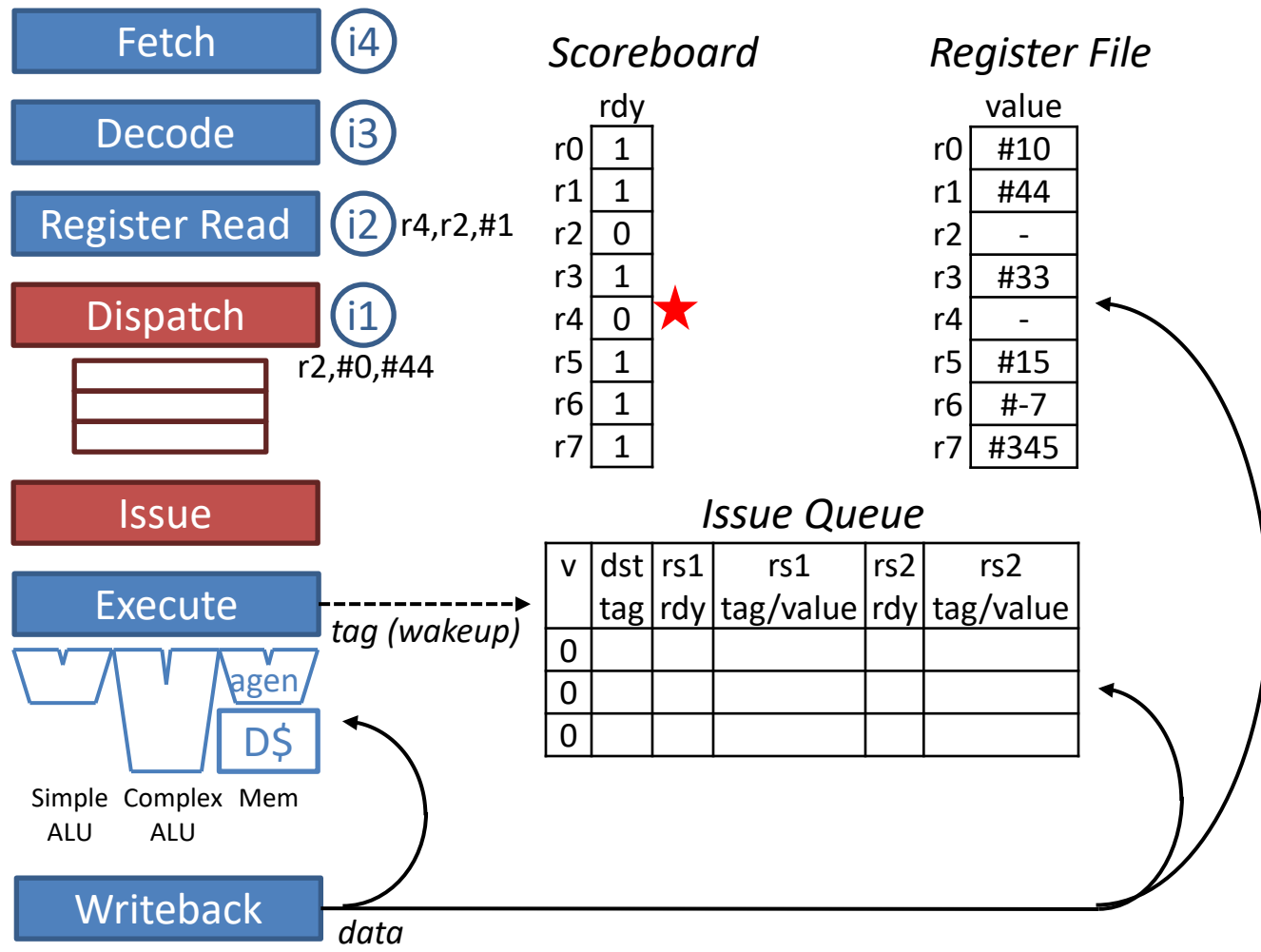
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE														
i2: add r4, r2, #1															
i3: add r6, r5, #2															
i4: add r7, r6, #3															



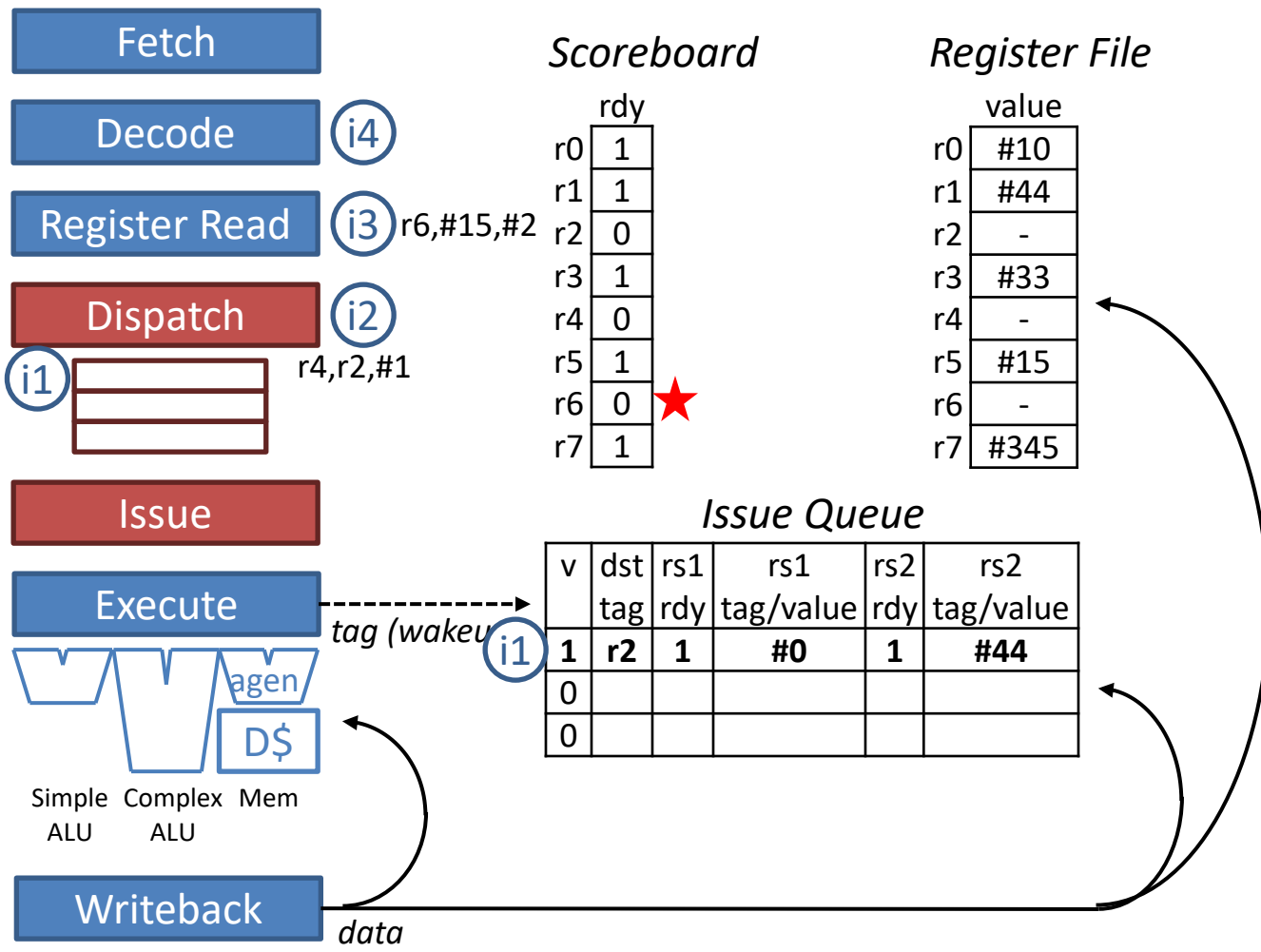
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE													
i2: add r4, r2, #1		FE													
i3: add r6, r5, #2															
i4: add r7, r6, #3															



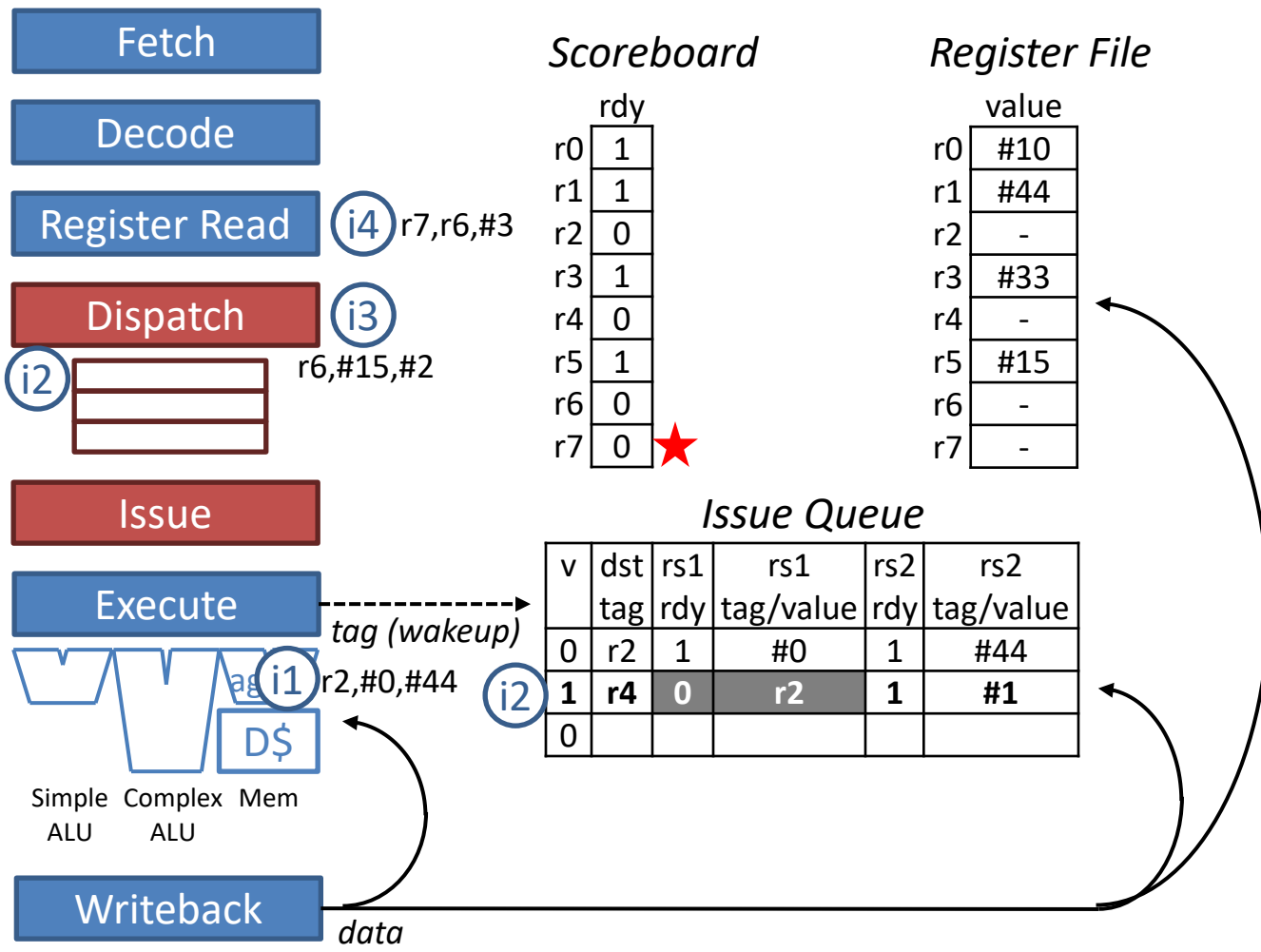
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR												
i2: add r4, r2, #1		FE	DE												
i3: add r6, r5, #2			FE												
i4: add r7, r6, #3															



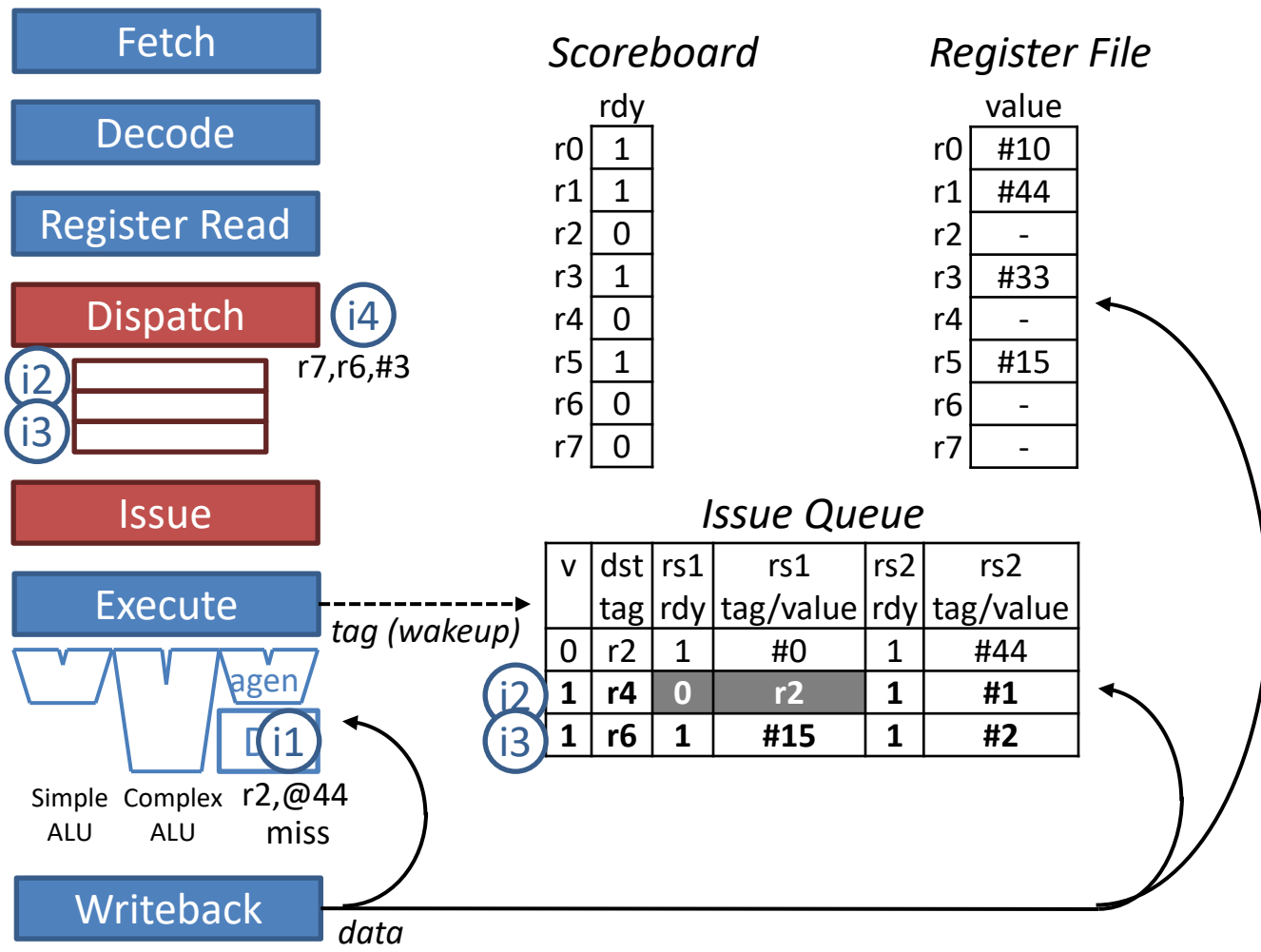
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI											
i2: add r4, r2, #1		FE	DE	RR											
i3: add r6, r5, #2			FE	DE											
i4: add r7, r6, #3				FE											



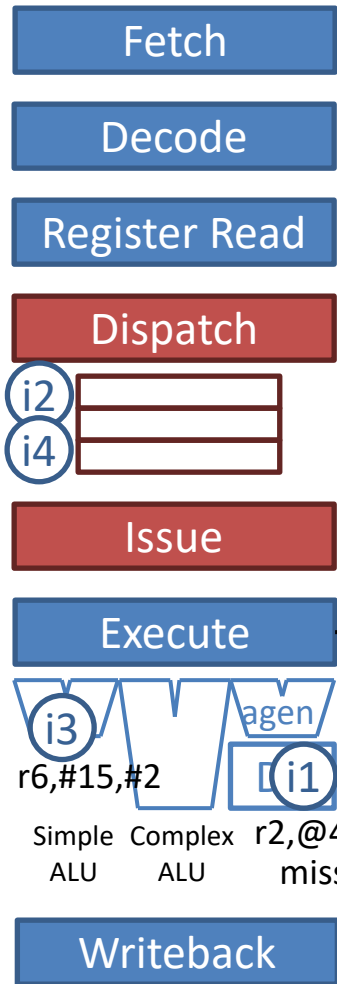
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS										
i2: add r4, r2, #1		FE	DE	RR	DI										
i3: add r6, r5, #2			FE	DE	RR										
i4: add r7, r6, #3				FE	DE										



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS	EX@									
i2: add r4, r2, #1		FE	DE	RR	DI	IS									
i3: add r6, r5, #2			FE	DE	RR	DI									
i4: add r7, r6, #3				FE	DE	RR									



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS	EX@	EXD\$	...miss...							
i2: add r4, r2, #1		FE	DE	RR	DI	IS	IS								
i3: add r6, r5, #2			FE	DE	RR	DI	IS								
i4: add r7, r6, #3				FE	DE	RR	DI								



Scoreboard

	rdy
r0	1
r1	1
r2	0
r3	1
r4	0
r5	1
r6	0
r7	0

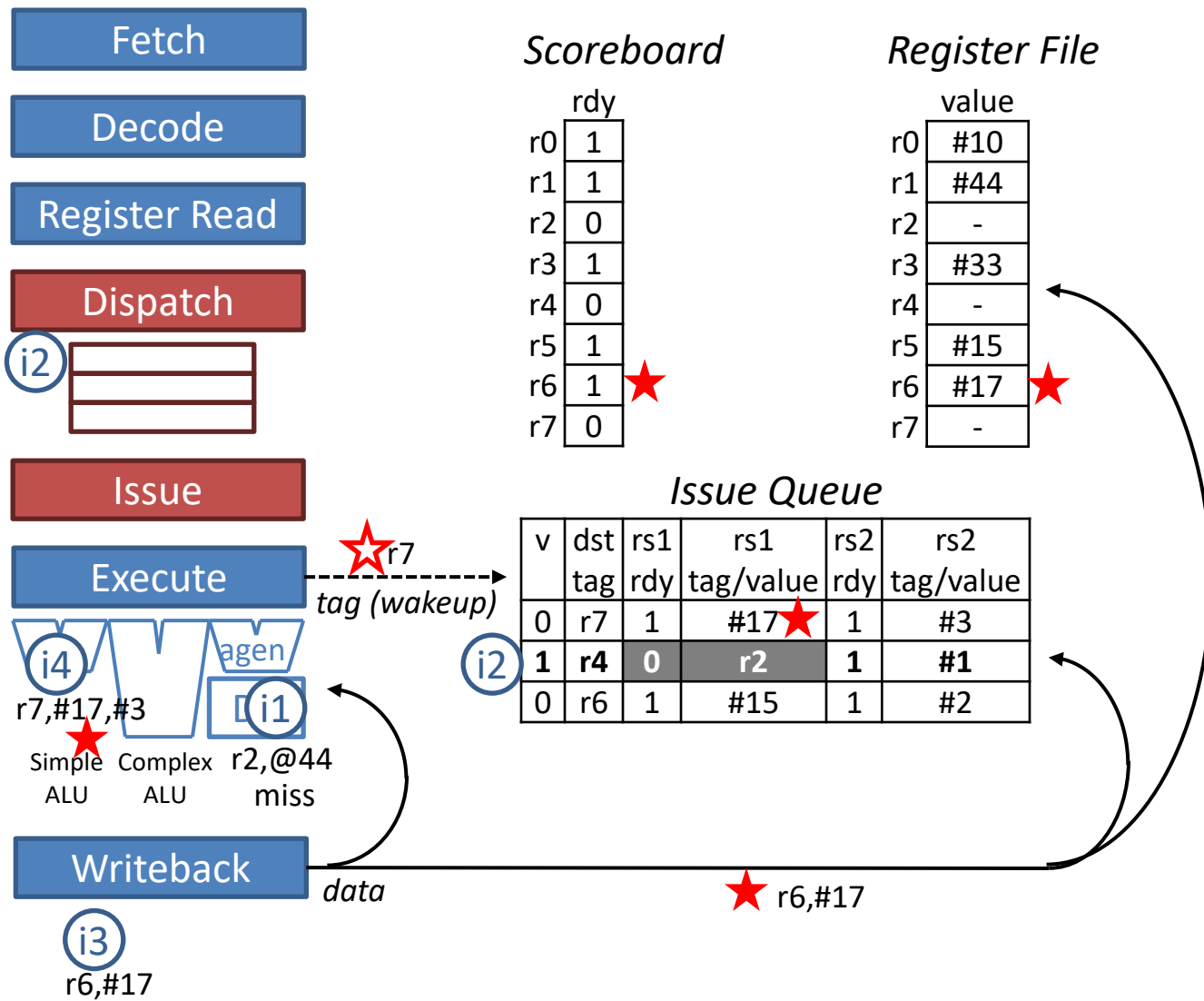
Register File

	value
r0	#10
r1	#44
r2	-
r3	#33
r4	-
r5	#15
r6	-
r7	-

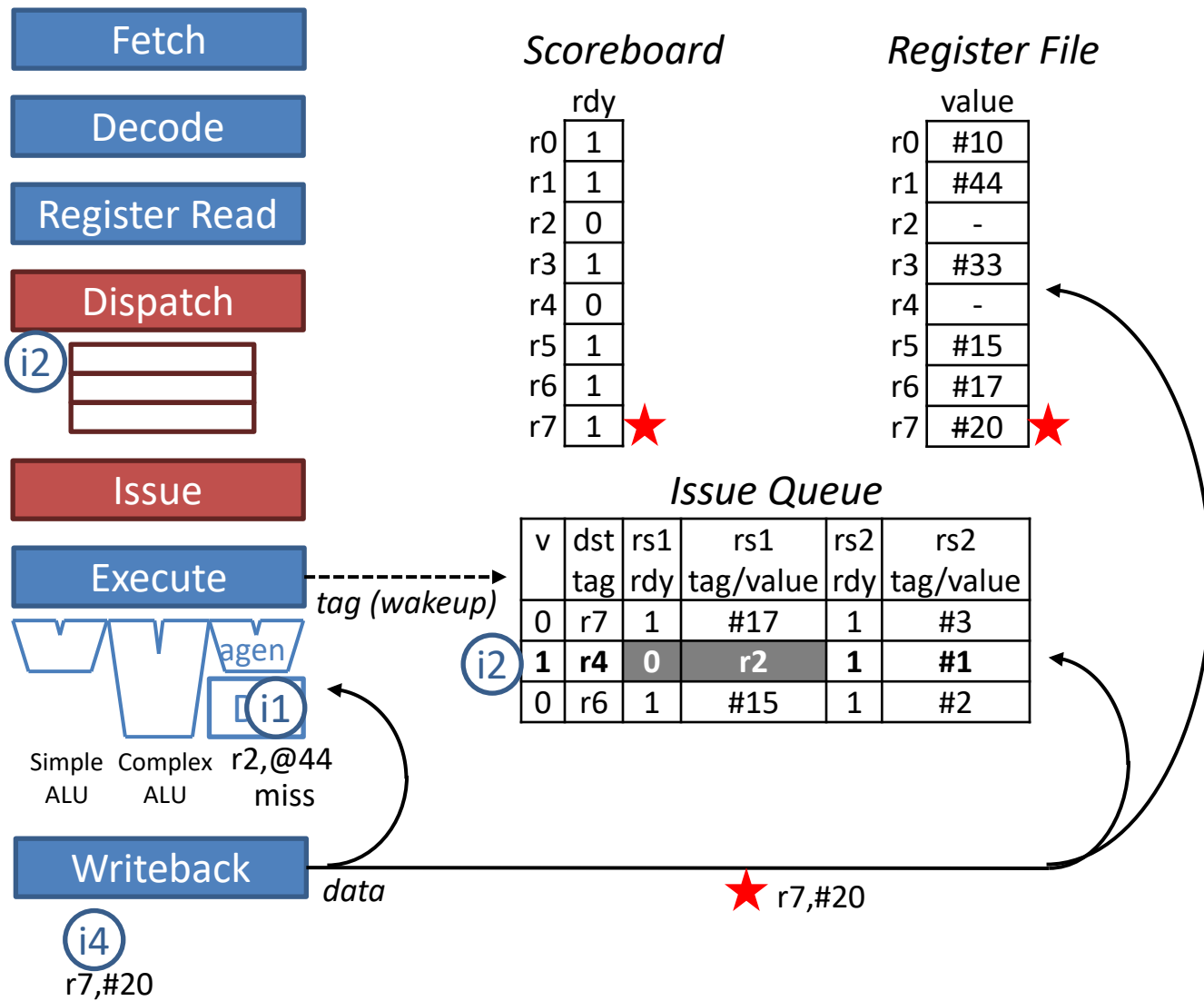
Issue Queue

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
<i>i4</i>	1	r7 0>1	<i>i4</i> r6	1	#3
<i>i2</i>	1	r4 0	r2	1	#1
0	r6	1	#15	1	#2

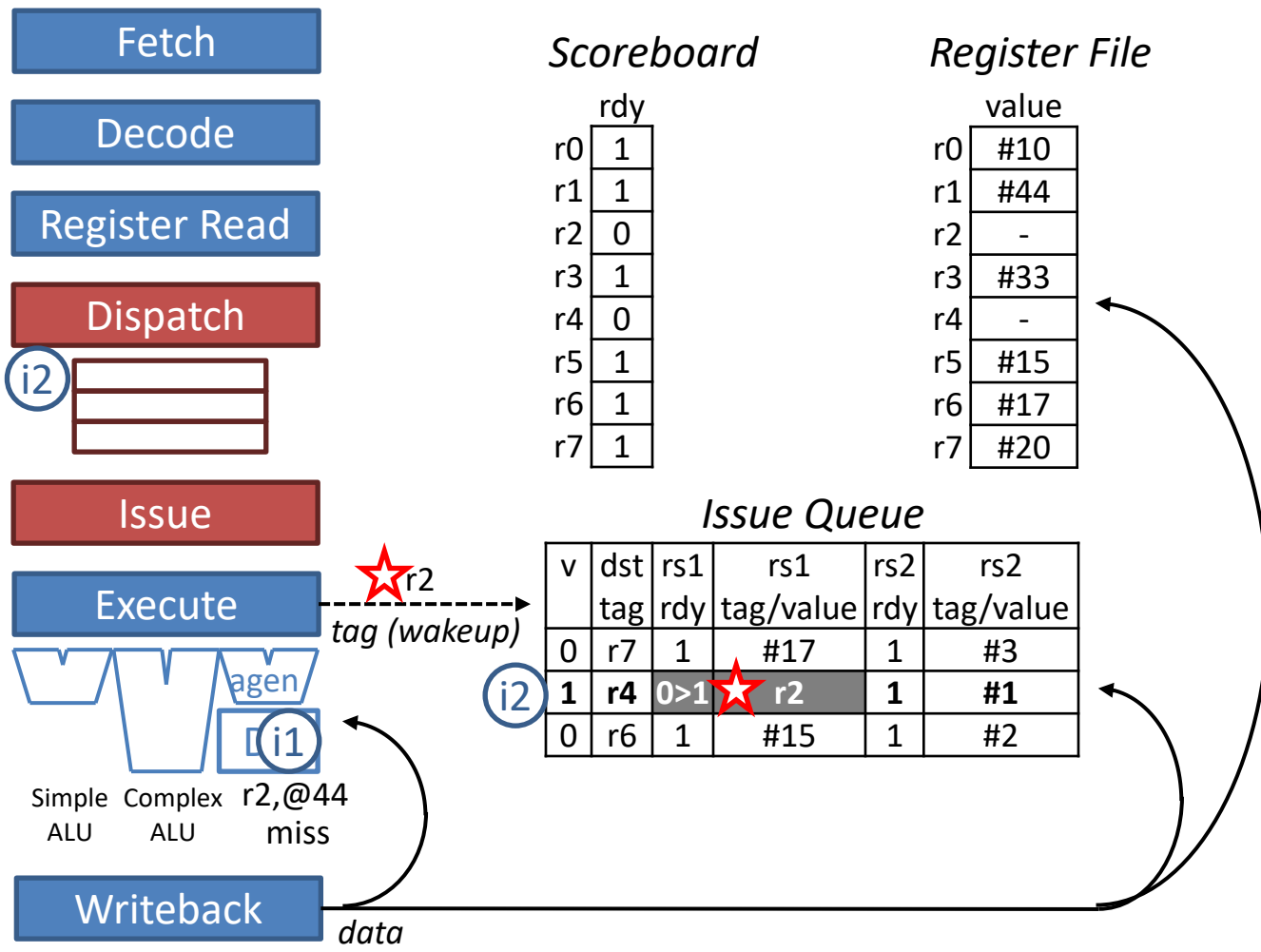
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS	EX _@	EX _{D\$}	...miss...							
i2: add r4, r2, #1		FE	DE	RR	DI	IS	IS	IS							
i3: add r6, r5, #2			FE	DE	RR	DI	IS	EX							
i4: add r7, r6, #3				FE	DE	RR	DI	IS							



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS	EX _@	EX _{D\$}	...miss...							
i2: add r4, r2, #1		FE	DE	RR	DI	IS	IS	IS	IS						
i3: add r6, r5, #2			FE	DE	RR	DI	IS	EX	WB						
i4: add r7, r6, #3				FE	DE	RR	DI	IS	EX						



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS	EX _@	EX _{D\$}	...miss...							
i2: add r4, r2, #1		FE	DE	RR	DI	IS	IS	IS	IS	IS					
i3: add r6, r5, #2			FE	DE	RR	DI	IS	EX	WB						
i4: add r7, r6, #3				FE	DE	RR	DI	IS	EX	WB					



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS	EX _@	EX _{D\$}	...miss...							
i2: add r4, r2, #1		FE	DE	RR	DI	IS	IS	IS	IS	IS	IS				
i3: add r6, r5, #2			FE	DE	RR	DI	IS	EX	WB						
i4: add r7, r6, #3				FE	DE	RR	DI	IS	EX	WB					

Fetch

Decode

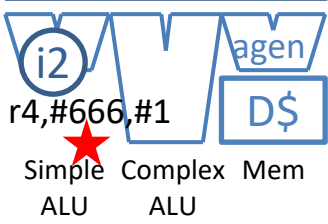
Register Read

Dispatch



Issue

Execute



Writeback

i1
r2,#666

Scoreboard

	rdy
r0	1
r1	1
r2	1
r3	1
r4	0
r5	1
r6	1
r7	1

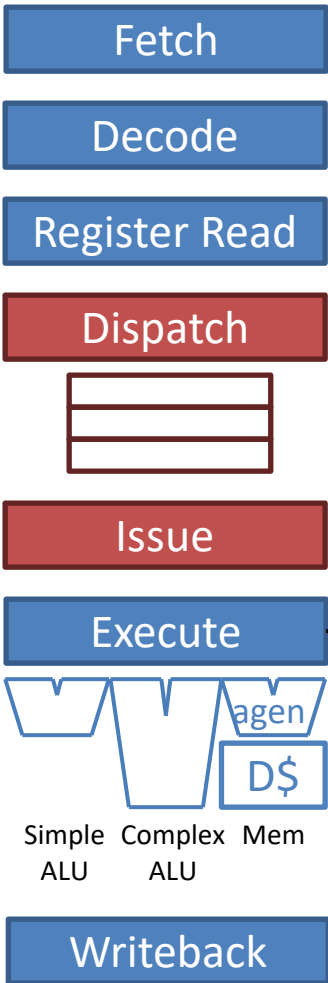
Register File

	value
r0	#10
r1	#44
r2	#666
r3	#33
r4	-
r5	#15
r6	#17
r7	#20

Issue Queue

v	dst	rs1	rs1	rs2	rs2
	tag	rdy	tag/value	rdy	tag/value
0	r7	1	#17	1	#3
0	r4	1	#666	1	#1
0	r6	1	#15	1	#2

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS	EX _@	EX _{D\$}	...miss...				WB			
i2: add r4, r2, #1		FE	DE	RR	DI	IS	IS	IS	IS	IS	IS	EX			
i3: add r6, r5, #2			FE	DE	RR	DI	IS	EX	WB						
i4: add r7, r6, #3				FE	DE	RR	DI	IS	EX	WB					



Scoreboard

	rdy
r0	1
r1	1
r2	1
r3	1
r4	1
r5	1
r6	1
r7	1

Register File

	value
r0	#10
r1	#44
r2	#666
r3	#33
r4	#667
r5	#15
r6	#17
r7	#20

Issue Queue

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	r7	1	#17	1	#3
0	r4	1	#666	1	#1
0	r6	1	#15	1	#2

tag (wakeup)

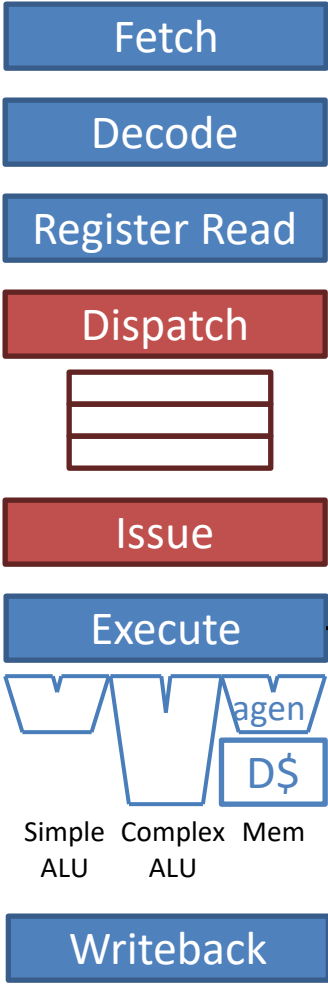
data

★ r4,#667

i2

r4,#667

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS	EX _@	EX _{D\$}	...miss...				WB			
i2: add r4, r2, #1		FE	DE	RR	DI	IS	IS	IS	IS	IS	IS	EX	WB		
i3: add r6, r5, #2			FE	DE	RR	DI	IS	EX	WB						
i4: add r7, r6, #3				FE	DE	RR	DI	IS	EX	WB					



Scoreboard

	rdy
r0	1
r1	1
r2	1
r3	1
r4	1
r5	1
r6	1
r7	1

Register File

	value
r0	#10
r1	#44
r2	#666
r3	#33
r4	#667
r5	#15
r6	#17
r7	#20

Issue Queue

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	r7	1	#17	1	#3
0	r4	1	#666	1	#1
0	r6	1	#15	1	#2

tag (wakeup)

data

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS	EX _@	EX _{D\$}	...miss...				WB			
i2: add r4, r2, #1		FE	DE	RR	DI	IS	IS	IS	IS	IS	IS	EX	WB		
i3: add r6, r5, #2			FE	DE	RR	DI	IS	EX	WB						
i4: add r7, r6, #3				FE	DE	RR	DI	IS	EX	WB					

Two Problems with Vers. 1

1. Can't recover from misspeculation
 - Younger instructions are *speculative* with respect to older instructions
 - There may be older predicted branches that haven't executed yet
 - Older instructions may raise exceptions, *e.g.*, TLB misses
 - Older load instructions may have executed speculatively with respect to prior unresolved stores (more later)
 - No way to undo the effects of younger instructions if an older mispredicted branch, *mispredicted load*, or exception is detected late

Fetch

Decode

Register Read

Dispatch



Issue

Execute



#666 != #0
branch to i7 x Mem
mispredict

Writeback



Scoreboard

	rdy
r0	1
r1	1
r2	1
r3	1
r4	1
r5	1
r6	1
r7	1

Register File

	value
r0	#10
r1	#44
r2	#666
r3	#33
r4	#7
r5	#15
r6	#17
r7	#20

Can't recover original values of r6, r7.

Issue Queue

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	r7	1	#17	1	#3
0	-	1	#666	1	#0
0	r6	1	#15	1	#2

tag (wakeup)

data

r2,#666

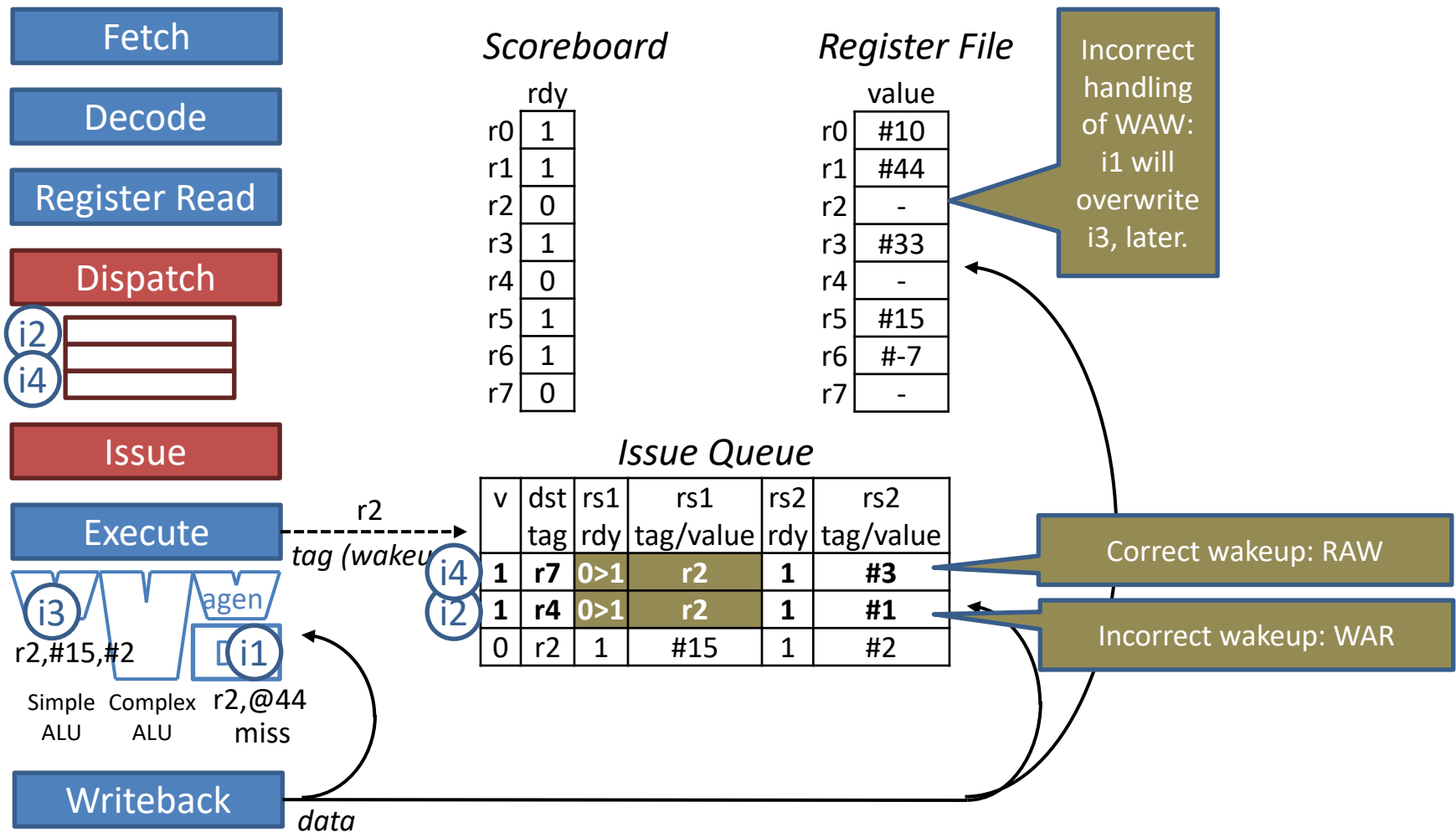
Misprediction detected!

Can't squash i3, i4: they are "long gone"

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS	EX _@	EX _{D\$}		...miss...			WB			
i2: bnez r2, i7		FE	DE	RR	DI	IS	IS	IS	IS	IS	IS	EX			
i3: add r6, r5, #2			FE	DE	RR	DI	IS	EX	WB						
i4: add r7, r6, #3				FE	DE	RR	DI	IS	EX	WB					

Two Problems with Vers. 1

2. Reverts to in-order when two producers have the same destination register (WAR and WAW hazards)
 - Must stall younger producer in Register Read stage until older producer executes
 - *Let's see what happens if we don't...*



What happens if we do not stall i3 in RR until i1 executes?

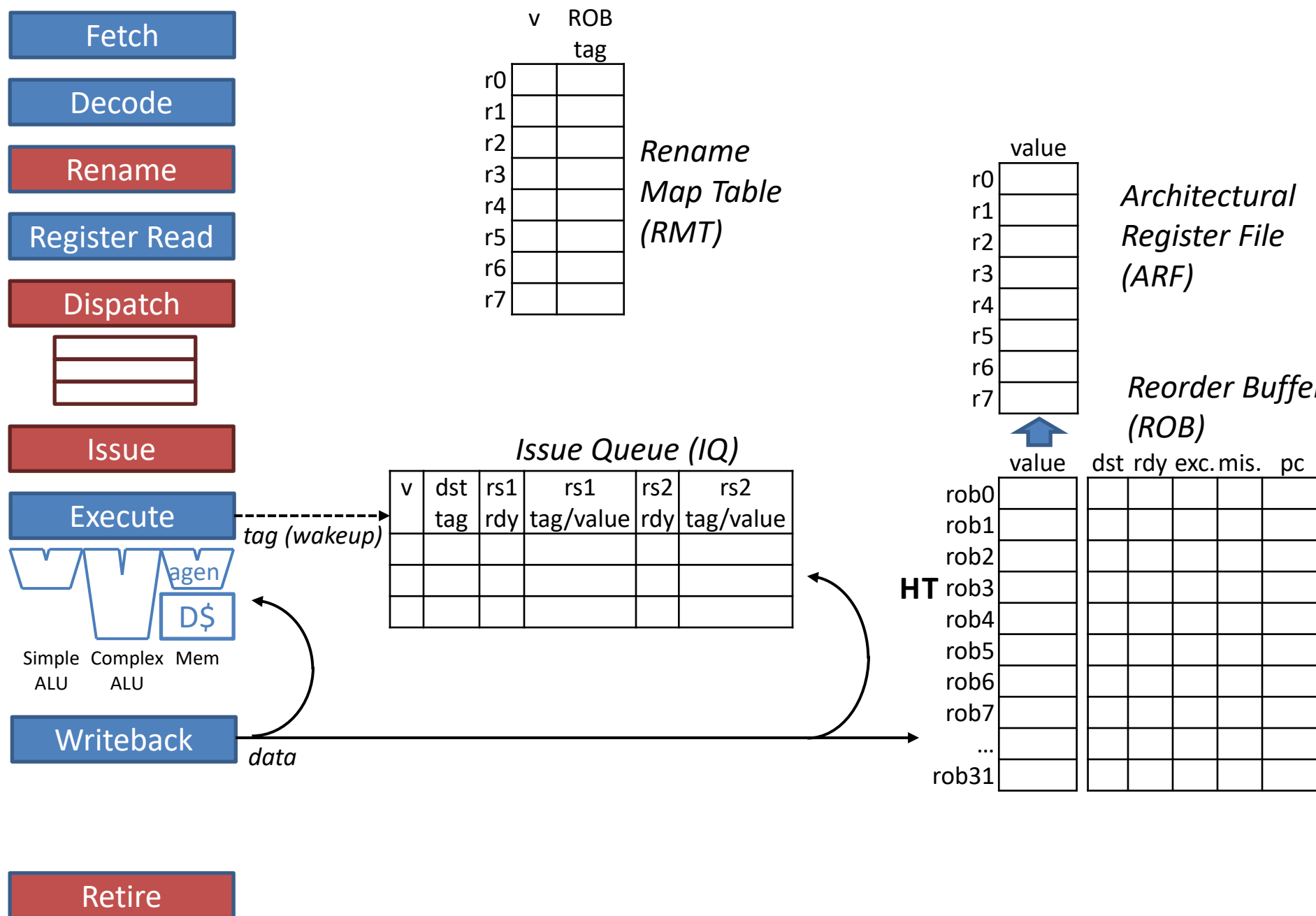
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i1: load r2, #0(r1)	FE	DE	RR	DI	IS	EX _@	EX _{D\$}	...miss...							
i2: add r4, r2, #1		FE	DE	RR	DI	IS	IS	IS							
i3: add r2 , r5, #2			FE	DE	RR	DI	IS	EX							
i4: add r7, r2 , #3				FE	DE	RR	DI	IS							

Solution: Reorder Buffer (ROB)

1. The ROB allows for OOO execution, while at the same time supporting recovery from mispredictions and exceptions
2. The ROB implements Register Renaming
 - Rename non-unique destination tags (architectural register specifiers) to unique destination tags (ROB tags)
 - Source tags renamed as well, unambiguously linking consumers to their producers
 - No in-order stall point due to WAR and WAW hazards, as they are eliminated post-renaming

Operation with ROB

- The “Register File” is replaced with an expanded set of registers split into two parts
 - **Architectural Register File (ARF)**: Contains values of architectural registers as if produced by an in-order pipeline. That is, contains committed (non-speculative) versions of architectural registers to which the pipeline may safely revert to if there is a misprediction or exception.
 - **Reorder Buffer (ROB)**: Contains speculative versions of architectural registers. There may be multiple speculative versions for a given architectural register.
- ROB is a circular FIFO with head and tail pointers
 - A list of oldest to youngest instructions in program order
 - Instruction at ROB Head is oldest instruction
 - Instruction at ROB Tail is youngest instruction
- New **Rename Stage** (after Decode and before Register Read)
 - The new instruction is allocated to the ROB entry pointed to by ROB Tail. This is also its unique “ROB tag”.
 - Source register specifiers are renamed to the expanded set of registers, the ARF+ROB. Renaming pinpoints the location of the value: ARF or ROB, and where in the ROB (ROB tag of producer). Thus, renaming unambiguously links consumers to their producers.
 - Destination register specifier is renamed to the instruction’s unique ROB tag.
 - **Rename Map Table (RMT)** contains the bookkeeping for renaming. (Intel calls it the Register Alias Table (RAT).)
- Register Read Stage
 - Obtain source value from ARF or ROB (using renamed source)
 - If renamed to ROB, ROB may indicate value not ready yet
 - Producer hasn’t executed yet
 - Keep renamed source as proxy for value
 - A consumer instruction obtains its source values from ARF, ROB, and/or bypass, depending on situation:
 - ARF: if producer of value has retired from ROB
 - ROB: if producer of value has executed but not yet retired from ROB
 - Bypass: if producer of value has not yet executed
- Writeback Stage
 - Instruction writes its speculative result OOO into ROB instead of ARF (at its ROB entry)
- New **Retire Stage** safely commits results from ROB to ARF in program order
- Misprediction/exception recovery
 - Offending instruction posts misprediction or exception bit in its ROB entry OOO
 - Wait until offending instruction reaches head of ROB (oldest unretired instruction)
 - Squash all instructions in pipeline and ROB, and restore RMT to be consistent with an empty pipeline



Fetch

i1

Decode

Rename

Register Read

Dispatch



Issue

Execute



Writeback

Retire

	v	ROB tag
r0	0	-
r1	0	-
r2	0	-
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	0	-

Rename Map Table (RMT)

	value
r0	#10
r1	#44
r2	#11
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Architectural Register File (ARF)

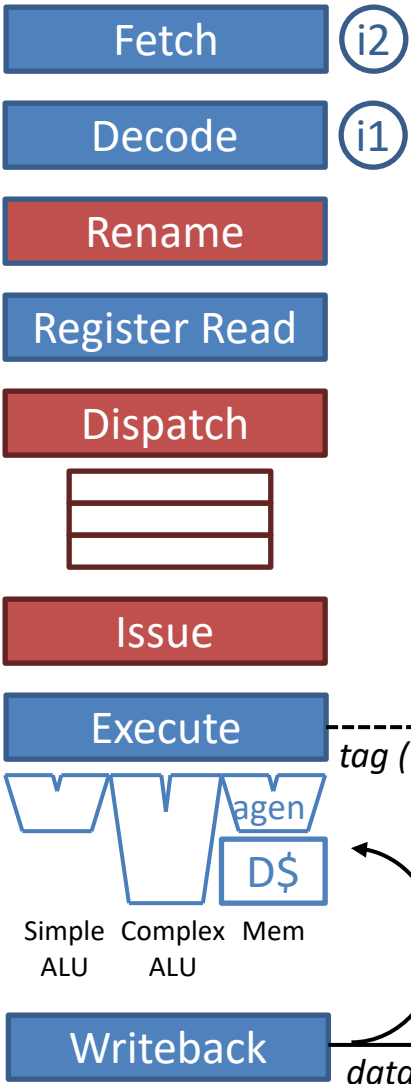
Reorder Buffer (ROB)

Issue Queue (IQ)					
v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0					
0					
0					

HT

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3						
rob4						
rob5						
rob6						
rob7						
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE																
i2: bnez r2, i7																	
i3: add r2, r5, #2																	
i4: add r7, r2, #3																	



Rename Map Table (RMT)

	v	ROB tag
r0	0	-
r1	0	-
r2	0	-
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	0	-

Architectural Register File (ARF)

	value
r0	#10
r1	#44
r2	#11
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Reorder Buffer (ROB)

Issue Queue (IQ)

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0					
0					
0					

HT

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3						
rob4						
rob5						
rob6						
rob7						
...						
rob31						

Retire

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE															
i2: bnez r2, i7		FE															
i3: add r2, r5, #2																	
i4: add r7, r2, #3																	

Fetch

i3

Decode

i2

Rename

i1

Register Read

Dispatch



Issue

Execute



Simple ALU Complex ALU Mem

Writeback

Retire

	v	ROB tag
r0	0	-
r1	0	-
r2	1	rob3
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	0	-

Rename Map Table (RMT)

rob3, #0, r1

Issue Queue (IQ)

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0					
0					
0					

tag (wakeup)

data

	value
r0	#10
r1	#44
r2	#11
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Architectural Register File (ARF)

Reorder Buffer (ROB)

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	-	r2	0	0	0	i1
rob4						
rob5						
rob6						
rob7						
...						
rob31						

HT

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN														
i2: bnez r2, i7		FE	DE														
i3: add r2, r5, #2			FE														
i4: add r7, r2, #3																	

Fetch

i4

Decode

i3

Rename

i2

Register Read

i1

Dispatch

Issue

Execute



Writeback

Retire

		v	ROB
			tag
r0		0	-
r1		0	-
r2		1	rob3
r3		0	-
r4		0	-
r5		0	-
r6		0	-
r7		0	-

Rename
Map Table
(RMT)

		value
r0		#10
r1		#44
r2		#11
r3		#33
r4		#7
r5		#15
r6		#-7
r7		#345

Architectural
Register File
(ARF)

Reorder Buffer
(ROB)

Issue Queue (IQ)					
v	dst	rs1	rs1	rs2	rs2
	tag	rdy	tag/value	rdy	tag/value
0					
0					
0					

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	-	r2	0	0	0	i1
rob4	-	-	0	0	0	i2
rob5						
rob6						
rob7						
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR													
i2: bnez r2, i7		FE	DE	RN													
i3: add r2, r5, #2			FE	DE													
i4: add r7, r2, #3				FE													

Fetch

Decode

Rename

Register Read

Dispatch

Issue

Execute

Writeback

Retire

i4

i3

i2

i1

rob5,r5,#2

rob4,rob3,#0

rob3,#0,#44

v		ROB
		tag
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	0	-

Rename
Map Table
(RMT)

		value
r0		#10
r1		#44
r2		#11
r3		#33
r4		#7
r5		#15
r6		#-7
r7		#345

Architectural
Register File
(ARF)

Reorder Buffer
(ROB)

Issue Queue (IQ)					
v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0					
0					
0					

tag (wakeup)

data

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	-	r2	0	0	0	i1
rob4	-	-	0	0	0	i2
rob5	-	r2	0	0	0	i3
rob6						
rob7						
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI												
i2: bnez r2, i7		FE	DE	RN	RR												
i3: add r2, r5, #2			FE	DE	RN												
i4: add r7, r2, #3				FE	DE												

Fetch

Decode

Rename

Register Read

Dispatch

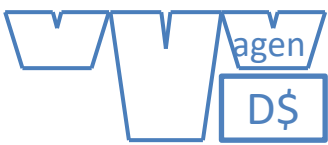
Issue

Execute

Writeback

Retire

i4 rob6,rob5,#3
i3 rob5,#15,#2
i2 rob4,rob3,#0



Simple ALU Complex ALU Mem

	v	ROB tag
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	1	rob6

Rename Map Table (RMT)

	value
r0	#10
r1	#44
r2	#11
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Architectural Register File (ARF)

Reorder Buffer (ROB)

Issue Queue (IQ)						
v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value	
1	rob3	1	#0	1	#44	
0						
0						

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
H rob3	-	r2	0	0	0	i1
rob4	-	-	0	0	0	i2
rob5	-	r2	0	0	0	i3
rob6	-	r7	0	0	0	i4
T rob7						
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS											
i2: bnez r2, i7		FE	DE	RN	RR	DI											
i3: add r2, r5, #2			FE	DE	RN	RR											
i4: add r7, r2, #3				FE	DE	RN											

Fetch

Decode

Rename

Register Read

Dispatch

Issue

Execute

Writeback

Retire

	v	ROB tag
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	1	rob6

Rename Map Table (RMT)

	value
r0	#10
r1	#44
r2	#11
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Architectural Register File (ARF)

Reorder Buffer (ROB)

	v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
	0	rob3	1	#0	1	#44
	1	rob4	0	rob3	1	#0
	0					

Issue Queue (IQ)

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
H rob3	-	r2	0	0	0	i1
rob4	-	-	0	0	0	i2
rob5	-	r2	0	0	0	i3
rob6	-	r7	0	0	0	i4
T rob7						
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX@										
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS										
i3: add r2, r5, #2			FE	DE	RN	RR	DI										
i4: add r7, r2, #3				FE	DE	RN	RR										

Fetch

Decode

Rename

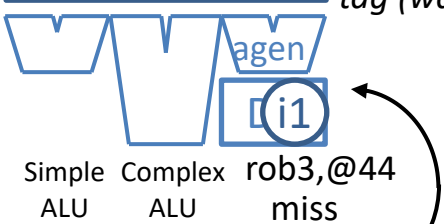
Register Read

Dispatch



Issue

Execute



Writeback

Retire

	v	ROB tag
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	1	rob6

Rename Map Table (RMT)

	value
r0	#10
r1	#44
r2	#11
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Architectural Register File (ARF)

Reorder Buffer (ROB)

Issue Queue (IQ)

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	rob3	1	#0	1	#44
1	rob4	0	rob3	1	#0
1	rob5	1	#15	1	#2

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
H rob3	-	r2	0	0	0	i1
rob4	-	-	0	0	0	i2
rob5	-	r2	0	0	0	i3
rob6	-	r7	0	0	0	i4
T rob7						
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX_@	EX_D\$		...miss...							
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS									
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS									
i4: add r7, r2, #3				FE	DE	RN	RR	DI									

Fetch

i5 add r4, r2, r7

Decode

Rename

Register Read

Dispatch

i2
i4

Issue

Execute

i3
i1
Simple ALU
Complex ALU
agen
rob5, #15, #2
rob3, @44 miss

Writeback

Retire

	v	ROB tag
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	1	rob6

Rename Map Table (RMT)

	value
r0	#10
r1	#44
r2	#11
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Architectural Register File (ARF)

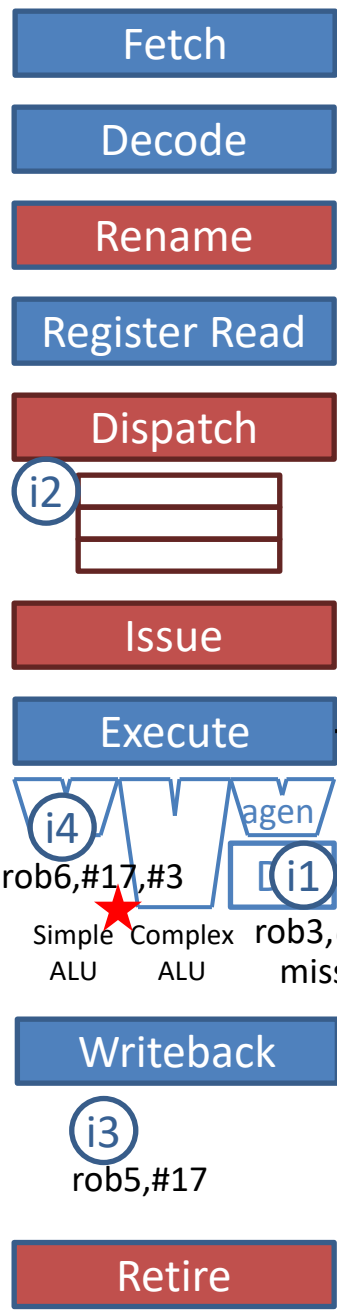
Reorder Buffer (ROB)

Issue Queue (IQ)

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
i4	1	rob6	0>1	1	#3
i2	1	rob4	0	1	#0
	0	rob5	1	1	#2

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
H rob3	-	r2	0	0	0	i1
rob4	-	-	0	0	0	i2
rob5	-	r2	0	0	0	i3
rob6	-	r7	0	0	0	i4
T rob7						
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX@	EXD\$		...miss...							
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS								
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX								
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS								



Rename Map Table (RMT)

	v	ROB tag
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	1	rob6

Architectural Register File (ARF)

	value
r0	#10
r1	#44
r2	#11
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Reorder Buffer (ROB)

Issue Queue (IQ)

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	rob6	1	#17 ★	1	#3
i2 1	★ rob4	0	rob3	1	#0
0	rob5	1	#15	1	#2

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
H rob3	-	r2	0	0	0	i1
rob4	-	-	0	0	0	i2
★ rob5	#17	r2	1	0	0	i3
rob6	-	r7	0	0	0	i4
T rob7						
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX _@	EX _{D\$}		...miss...							
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS							
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB							
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX							

Fetch

Decode

Rename

Register Read

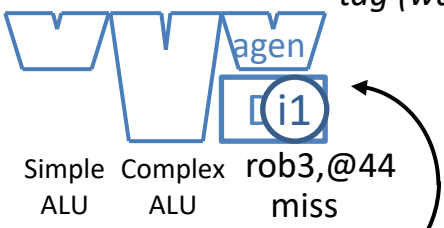
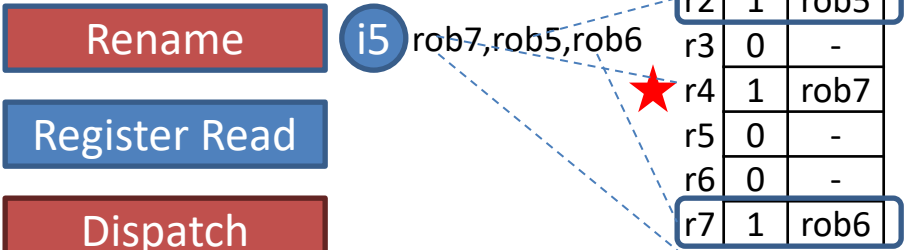
Dispatch

Issue

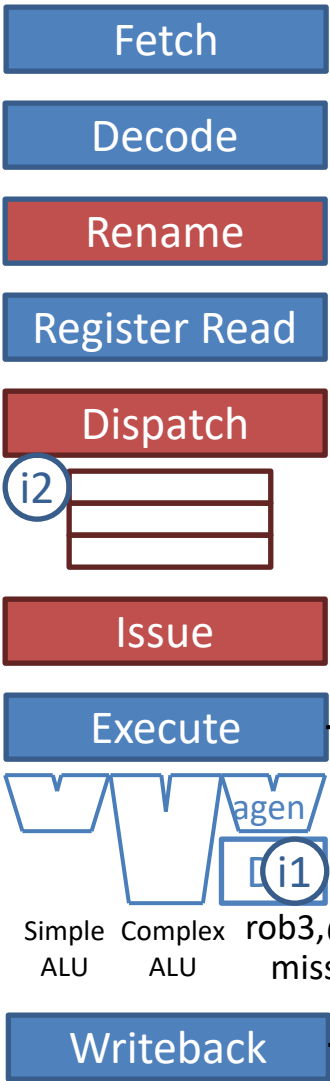
Execute

Writeback

Retire



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX@	EXD\$		...miss...							
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS						
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT						
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB						



	v	ROB tag
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	1	rob7
r5	0	-
r6	0	-
r7	1	rob6

Rename Map Table (RMT)

	value
r0	#10
r1	#44
r2	#11
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

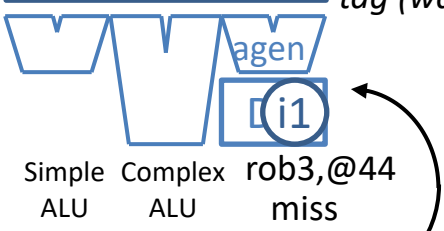
Architectural Register File (ARF)

Reorder Buffer (ROB)

Issue Queue (IQ)

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	rob6	1	#17	1	#3
1	rob4	0	#15	1	#0
0	rob5	1	#15	1	#2

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	-	r2	0	0	0	i1
rob4	-	-	0	0	0	i2
rob5	#17	r2	1	0	0	i3
rob6	#20	r7	1	0	0	i4
rob7	-	r4	0	0	0	i5
...						
rob31						



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX _@	EX _D		...miss...							
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS	IS					
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT	RT					
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB	RT					

Fetch

Decode

Rename

Register Read

Dispatch

i5 rob7,#17,#20

Issue

Execute

i2
rob4,#0,#0
Simple Complex
#0!=#0 is false, not-taken, no misp.
D\$

Writeback

i1
rob3,#0

Retire

i3 i4

	v	ROB tag
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	1	rob7
r5	0	-
r6	0	-
r7	1	rob6

Rename Map Table (RMT)

	value
r0	#10
r1	#44
r2	#11
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Architectural Register File (ARF)

Reorder Buffer (ROB)

Issue Queue (IQ)					
v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	rob6	1	#17	1	#3
0	rob4	1	#0	1	#0
0	rob5	1	#15	1	#2

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	#0	r2	1	0	0	i1
rob4	-	-	0	0	0	i2
rob5	#17	r2	1	0	0	i3
rob6	#20	r7	1	0	0	i4
rob7	-	r4	0	0	0	i5
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX@	EXD\$		...miss...			WB				
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS	IS	EX				
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT				
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB	RT	RT				

Fetch

Decode

Rename

Register Read

Dispatch

i5

Issue

Execute

Simple ALU
Complex ALU
agen
D\$

Writeback

i2
rob4, no misp.

Retire

i1 i3 i4

	v	ROB tag
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	1	rob7
r5	0	-
r6	0	-
r7	1	rob6

Don't reset:
rob5 != rob3

Rename
Map Table
(RMT)

	value
r0	#10
r1	#44
r2	#0
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

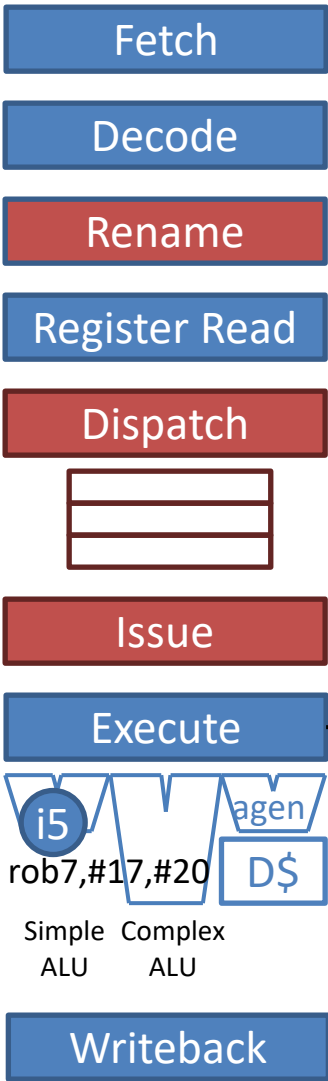
Architectural
Register File
(ARF)

Reorder Buffer
(ROB)

	v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
i5	1	rob7	1	#17	1	#20
	0	rob4	1	#0	1	#0
	0	rob5	1	#15	1	#2

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	#0	r2	1	0	0	i1
rob4	-	-	1	0	0	i2
rob5	#17	r2	1	0	0	i3
rob6	#20	r7	1	0	0	i4
rob7	-	r4	0	0	0	i5
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX@	EXD\$		...miss...			WB	RT			
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS	IS	EX	WB			
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT			
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT			



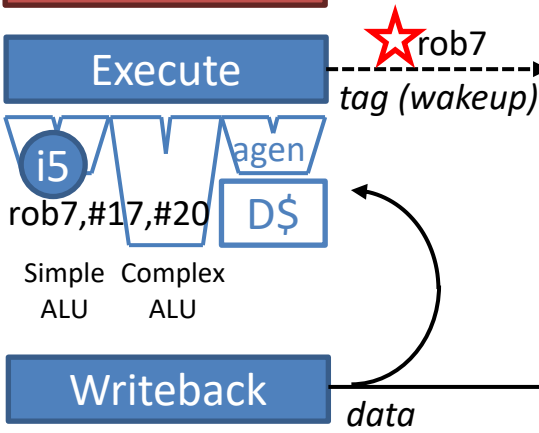
Rename Map Table (RMT)

	v	ROB tag
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	1	rob7
r5	0	-
r6	0	-
r7	1	rob6

Architectural Register File (ARF)

	value
r0	#10
r1	#44
r2	#0
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Reorder Buffer (ROB)



Issue Queue (IQ)

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	rob7	1	#17	1	#20
0	rob4	1	#0	1	#0
0	rob5	1	#15	1	#2

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	#0	r2	1	0	0	i1
rob4	-	-	1	0	0	i2
rob5	#17	r2	1	0	0	i3
rob6	#20	r7	1	0	0	i4
rob7	-	r4	0	0	0	i5
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX@	EXD\$		...miss...			WB	RT			
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS	IS	EX	WB	RT		
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT	RT		
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT		

Fetch

Decode

Rename

Register Read

Dispatch



Issue

Execute



Simple ALU Complex ALU

Writeback

i5
rob7,#37

Retire

i3 i4

		v	ROB tag
r0		0	-
r1		0	-
r2	★	0	rob5
r3		0	-
r4		1	rob7
r5		0	-
r6		0	-
r7		1	rob6

Reset: rob5 == rob5

Rename
Map Table
(RMT)

		value
r0		#10
r1		#44
r2	★	#17
r3		#33
r4		#7
r5		#15
r6		#-7
r7		#345

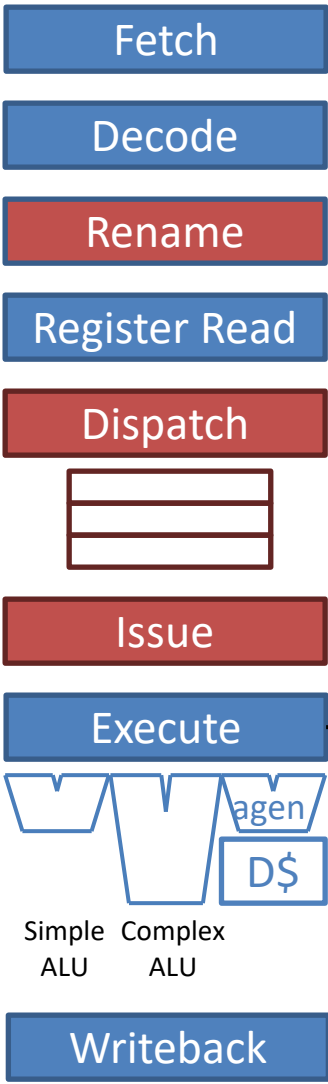
Architectural
Register File
(ARF)

Reorder Buffer
(ROB)

Issue Queue (IQ)					
v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	rob7	1	#17	1	#20
0	rob4	1	#0	1	#0
0	rob5	1	#15	1	#2

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	#0	r2	1	0	0	i1
rob4	-	-	1	0	0	i2
rob5	#17	r2	1	0	0	i3
rob6	#20	r7	1	0	0	i4
rob7	#37	r4	1	0	0	i5
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX@	EXD\$		...miss...			WB	RT			
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS	IS	EX	WB	RT		
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT	RT	RT	
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT	RT	



Reset: rob7 == rob7

Rename Map Table (RMT)

	v	ROB tag
r0	0	-
r1	0	-
r2	0	-
r3	0	-
r4	0	rob7
r5	0	-
r6	0	-
r7	0	-

Architectural Register File (ARF)

	value
r0	#10
r1	#44
r2	#17
r3	#33
r4	#37
r5	#15
r6	#-7
r7	#20

Reorder Buffer (ROB)

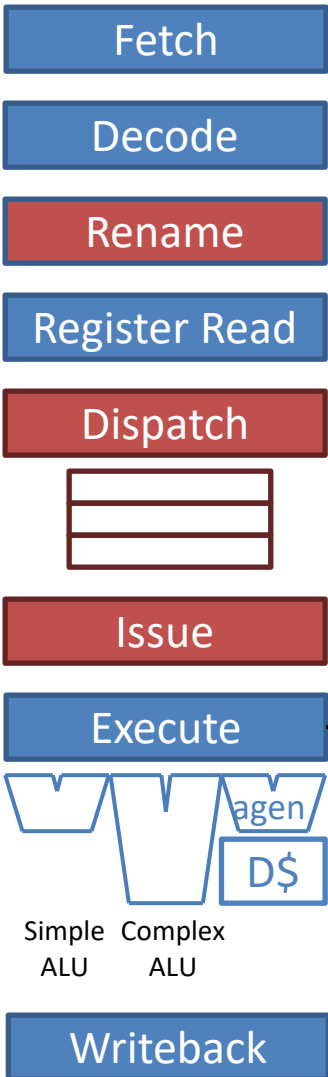
Issue Queue (IQ)

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	rob7	1	#17	1	#20
0	rob4	1	#0	1	#0
0	rob5	1	#15	1	#2

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	#0	r2	1	0	0	i1
rob4	-	-	1	0	0	i2
rob5	#17	r2	1	0	0	i3
rob6	#20	r7	1	0	0	i4
rob7	#37	r4	1	0	0	i5
rob31						



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX@	EXD\$		...miss...			WB	RT			
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS	IS	EX	WB	RT		
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT	RT	RT	
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT	RT	RT



	v	ROB tag
r0	0	-
r1	0	-
r2	0	-
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	0	-

Rename Map Table (RMT)

	value
r0	#10
r1	#44
r2	#17
r3	#33
r4	#37
r5	#15
r6	#-7
r7	#20

Architectural Register File (ARF)

Reorder Buffer (ROB)

Issue Queue (IQ)					
v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	rob7	1	#17	1	#20
0	rob4	1	#0	1	#0
0	rob5	1	#15	1	#2

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	#0	r2	1	0	0	i1
rob4	-	-	1	0	0	i2
rob5	#17	r2	1	0	0	i3
rob6	#20	r7	1	0	0	i4
rob7	#37	r4	1	0	0	i5
rob31						



	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX _@	EX _{D\$}		...miss...			WB	RT			
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS	IS	EX	WB	RT		
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT	RT	RT	
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT	RT	RT

Recovery Scenario

- Previous scenario
 - ROB did not impede OOO performance
 - Fetch didn't stall despite D\$ miss (tolerated D\$ miss), the same as without ROB.
 - Thus, in-order retirement did not impede OOO, speculative execution.
 - On the other hand, ROB was not ultimately called upon for recovery
 - Hindsight is perfect
 - Note: did leverage ROB for register renaming
- Revise the scenario: mispredicted branch
 - i1 (load instruction) gets the value #666 instead of #0

Fetch

Decode

Rename

Register Read

Dispatch

Issue

Execute

Writeback

Retire

	v	ROB
	tag	
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	1	rob7
r5	0	-
r6	0	-
r7	1	rob6

Rename
Map Table
(RMT)

	value
r0	#10
r1	#44
r2	#11
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Architectural
Register File
(ARF)

Reorder Buffer
(ROB)

i5 rob7,#17,#20

Issue Queue (IQ)

v	dst	rs1	rs1	rs2	rs2
	tag	rdy	tag/value	rdy	tag/value
0	rob6	1	#17	1	#3
0	rob4	1	#666	1	#0
0	rob5	1	#15	1	#2

tag (wakeup)



Simple Complex
#666!=#0 is true, taken, misp.!

data

★ rob3,#666

i1
rob3,#666

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
★ rob3	#666	r2	1	0	0	i1
rob4	-	-	0	0	0	i2
rob5	#17	r2	1	0	0	i3
rob6	#20	r7	1	0	0	i4
rob7	-	r4	0	0	0	i5
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX@	EX_D\$		...miss...			WB				
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS	IS	EX				
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT				
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB	RT	RT				

Fetch

Decode

Rename

Register Read

Dispatch



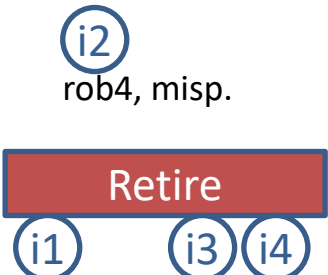
Issue

Execute



Writeback

Retire



	v	ROB tag
r0	0	-
r1	0	-
r2	1	rob5
r3	0	-
r4	1	rob7
r5	0	-
r6	0	-
r7	1	rob6

Don't reset:
rob5 != rob3

Rename
Map Table
(RMT)

	value
r0	#10
r1	#44
r2	#666
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Architectural
Register File
(ARF)

Reorder Buffer
(ROB)

Issue Queue (IQ)					
v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
1	rob7	1	#17	1	#20
0	rob4	1	#666	1	#0
0	rob5	1	#15	1	#2

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	#666	r2	1	0	0	i1
rob4	-	-	1	0	1	i2
rob5	#17	r2	1	0	0	i3
rob6	#20	r7	1	0	0	i4
rob7	-	r4	0	0	0	i5
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX _@	EX _{D\$}		...miss...			WB	RT			
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS	IS	EX	WB			
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT			
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT			

Fetch

Decode

Rename

Register Read

Dispatch



Issue

Execute



Simple ALU Complex ALU

Writeback

Retire

i2

★ Recover RMT by flash-clearing all valid bits.

	v	ROB tag
r0	0	-
r1	0	-
r2	0	-
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	0	-

Rename Map Table (RMT)

	value
r0	#10
r1	#44
r2	#666
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Architectural Register File (ARF)

Reorder Buffer (ROB)

★ Squash ROB by setting T=H.

Issue Queue (IQ)

v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	rob7	1	#17	1	#20
0	rob4	1	#666	1	#0
0	rob5	1	#15	1	#2

tag (wakeup)

data

★ Squash all pipeline stages. (i5) They contain only younger instructions when branch reaches ROB head.

HT

	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	#666	r2	1	0	0	i1
rob4	-	-	1	0	1	i2
rob5	#17	r2	1	0	0	i3
rob6	#20	r7	1	0	0	i4
rob7	-	r4	0	0	0	i5
...						
rob31						

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX@	EXD\$		...miss...			WB	RT			
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS	IS	EX	WB	RT		
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT	RT		
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT		

Fetch i7

Decode

Rename

Register Read

Dispatch



Issue

Execute



Simple ALU
Complex ALU

Writeback

Retire

	v	ROB tag
r0	0	-
r1	0	-
r2	0	-
r3	0	-
r4	0	-
r5	0	-
r6	0	-
r7	0	-

Rename
Map Table
(RMT)

	value
r0	#10
r1	#44
r2	#666
r3	#33
r4	#7
r5	#15
r6	#-7
r7	#345

Architectural
Register File
(ARF)

Reorder Buffer
(ROB)

Issue Queue (IQ)					
v	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
0	rob7	1	#17	1	#20
0	rob4	1	#666	1	#0
0	rob5	1	#15	1	#2

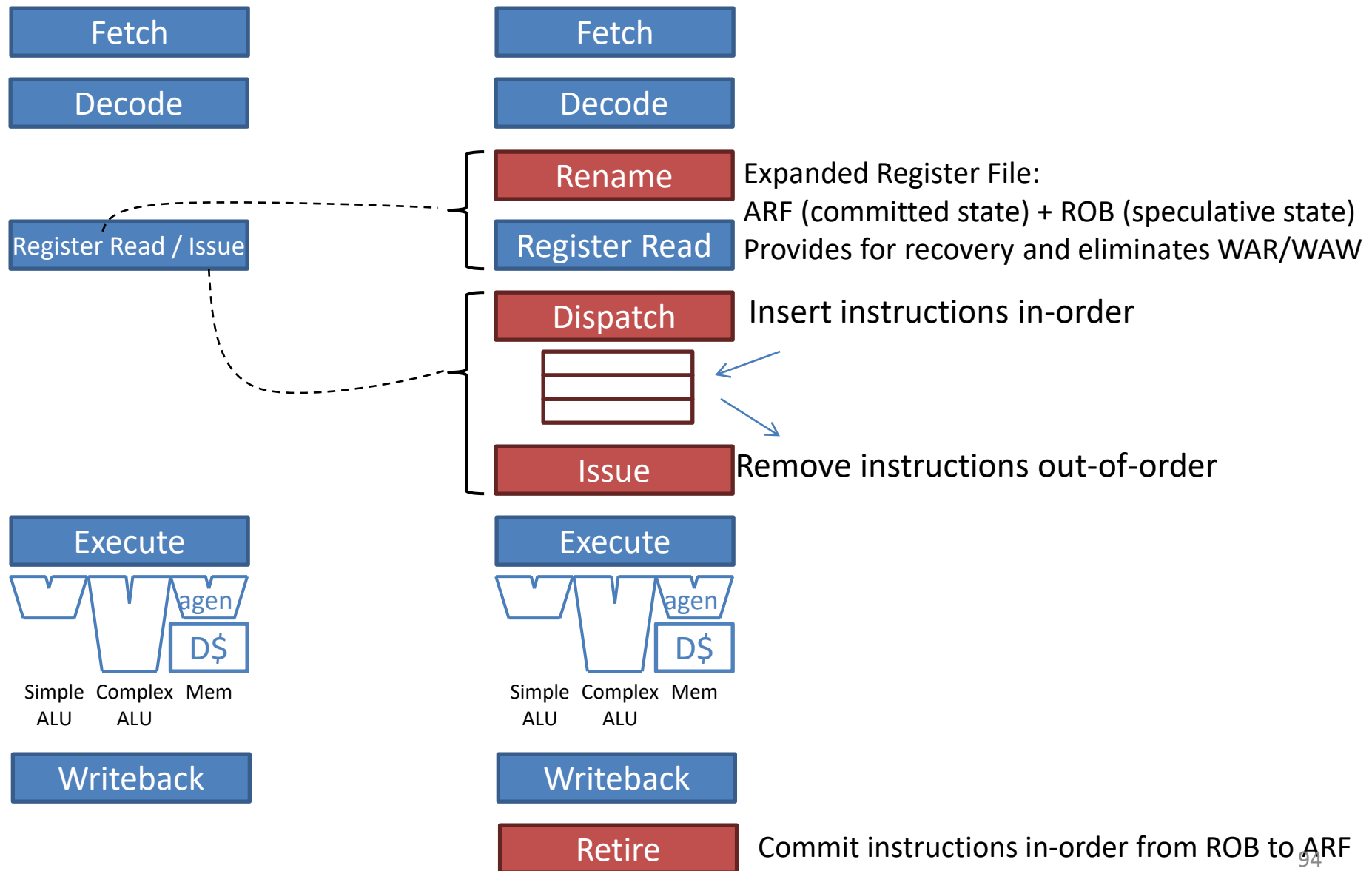
	value	dst	rdy	exc.	mis.	pc
rob0						
rob1						
rob2						
rob3	#666	r2	1	0	0	i1
rob4	-	-	1	0	1	i2
rob5	#17	r2	1	0	0	i3
rob6	#20	r7	1	0	0	i4
rob7	-	r4	0	0	0	i5

HT

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
i1: load r2, #0(r1)	FE	DE	RN	RR	DI	IS	EX _@	EX _{D\$}		...miss...			WB	RT			
i2: bnez r2, i7		FE	DE	RN	RR	DI	IS	IS	IS	IS	IS	IS	EX	WB	RT		
i3: add r2, r5, #2			FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT	RT		
i4: add r7, r2, #3				FE	DE	RN	RR	DI	IS	EX	WB	RT	RT	RT	RT		
i5: add r4, r2, r7									FE	DE	RN	RR	DI	IS	EX		
i7:																	FE

(note: could have fetched more instr. here)

From In-order to Out-of-Order



ECE 463/563

Fall `22

Additional ILP Topics

Prof. Eric Rotenberg

Additional ILP topics

- Interrupts
 - Types
 - Exception is akin to a mispredicted branch
 - ROB supports exception recovery
- Faster recovery of mispredicted branches
 - Checkpointing and restoring the RMT
- Handling memory dependencies
 - Store Queue (SQ) and Load Queue (LQ)
- Going from scalar to superscalar
 - Examples of superscalar complexity

Additional ILP topics

- Interrupts
 - Types
 - Exception is akin to a mispredicted branch
 - ROB supports exception recovery
- Faster recovery of mispredicted branches
 - Checkpointing and restoring the RMT
- Handling memory dependencies
 - Store Queue (SQ) and Load Queue (LQ)
- Going from scalar to superscalar
 - Examples of superscalar complexity

Interrupts

- Interrupt
 - Event that requires temporarily stopping program execution to service the event
- Synchronous vs. asynchronous interrupts
 - Synchronous: Instruction in the program causes the interrupt
 - Asynchronous: External request causes the interrupt

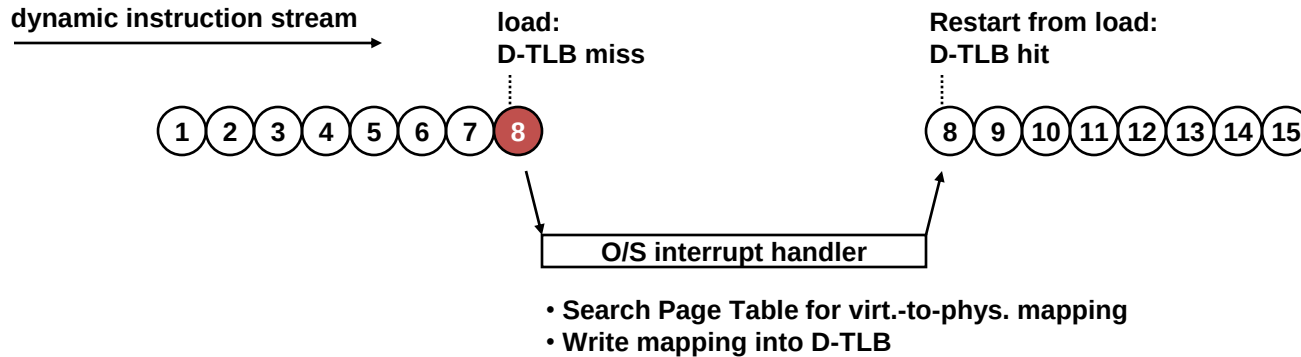
Three types of interrupts

- External interrupts
 - Asynchronous
 - Examples
 - I/O device request
 - Timer interrupt
 - Power failure interrupt
- Exceptions
 - Synchronous
 - Examples
 - Invalid opcode
 - Arithmetic overflow, divide-by-0, *etc.*
 - TLB miss exception (if software page table walk)
 - TLB miss followed by page fault (if hardware page table walk)
- Operating system (O/S) calls
 - Synchronous
 - Initiated via an explicit instruction in ISA, *e.g.*, “syscall” or “trap”

Handling interrupts

- Stop program execution
 - Synchronous: Stop at the instruction causing the interrupt
 - Asynchronous: Stop at an arbitrary instruction, as soon as convenient
- Service the interrupt
 - Transfer control to O/S interrupt handler
 - Depending on the interrupt, the program may be:
 - resumed immediately following the interrupt handler, OR
 - context-switched out and resumed later
- Resume program from where we left off

Example: TLB Miss



Note:

In this example and the example on the next slide, the ISA specifies that a TLB miss is an exception because the page table is walked by software, not hardware. See Slide 18 of [virtual-memory-and-TLB.pptx](#) for “software page table walker”.

Example: TLB Miss + Page Fault

dynamic instruction stream
→

load:
D-TLB miss



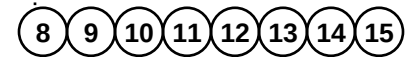
O/S interrupt handler

work on other program

Disk interrupts
processor:
done with
DMA transfer

O/S interrupt handler

Restart from load:
D-TLB hit

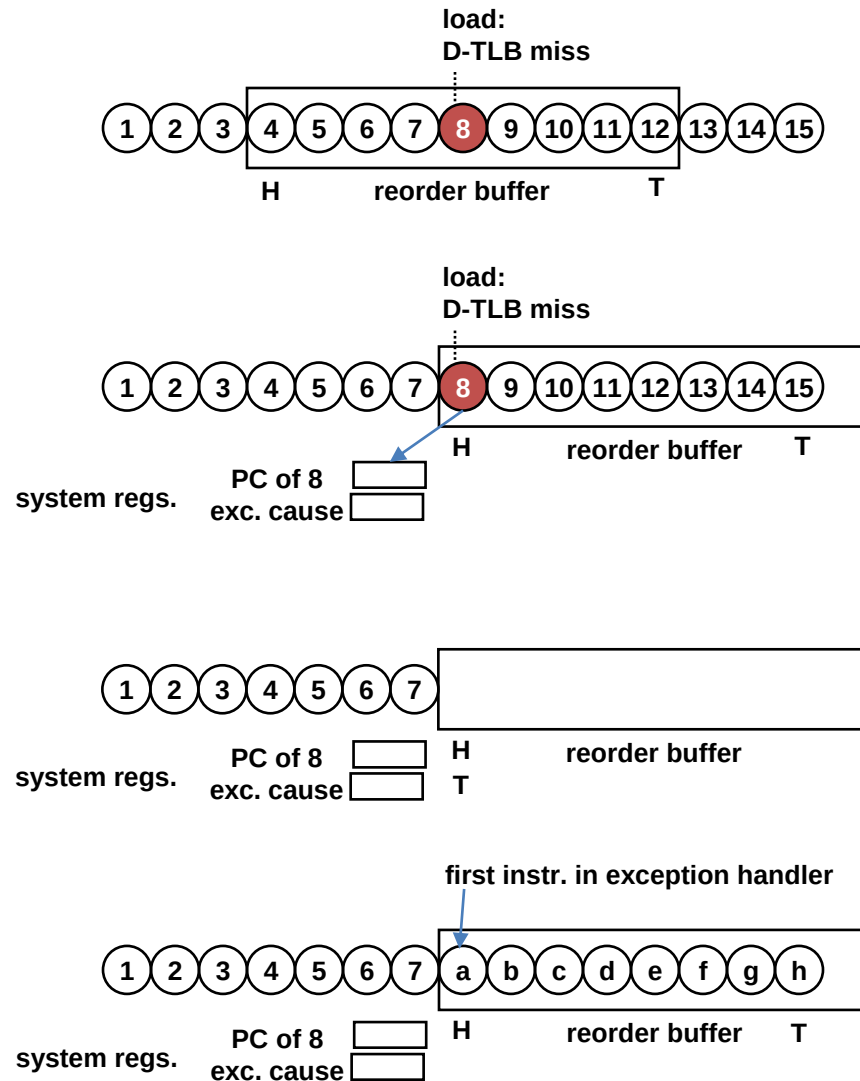


- Search Page Table for virt.-to-phys. mapping
- Discover “page fault”:
 - * Page is on disk
 - * Initiate page swap between disk and DRAM
 - * Takes a few milliseconds in the background
- Context-switch to keep processor busy in the meantime:
 - (1) Save arch. registers of current program to memory
 - (2) Restore arch. registers of some other suspended program (resume it)
- Context-switch to resume original program:
 - (1) Save arch. registers of other program to memory
 - (2) Restore arch. registers of original program (resume it)
- Write mapping into D-TLB

Impact of exceptions from a microarchitecture standpoint

- Offending instruction is akin to a mispredicted branch
 - Fetch unit predicted the next PC as usual
 - When the exception is detected later, we realize that the next PC after the offending instruction should be that of the exception handler
- ROB supports recovery for this unexpected control-flow transfer
 - Offending instruction posts exception flag and exception cause in its ROB entry
 - Wait until ROB head (H) reaches the offending instruction
 - Save PC of offending instruction and exception cause (both are available in its ROB entry) in ISA-defined system registers
 - Squash the pipeline (ROB T=H, squash frontend and backend pipeline stages, flash-clear RMT valid bits)
 - Set Fetch Unit's PC to that of the generic exception handler (this PC is either pre-determined by the ISA or in an ISA-defined system register)

Exception recovery via ROB



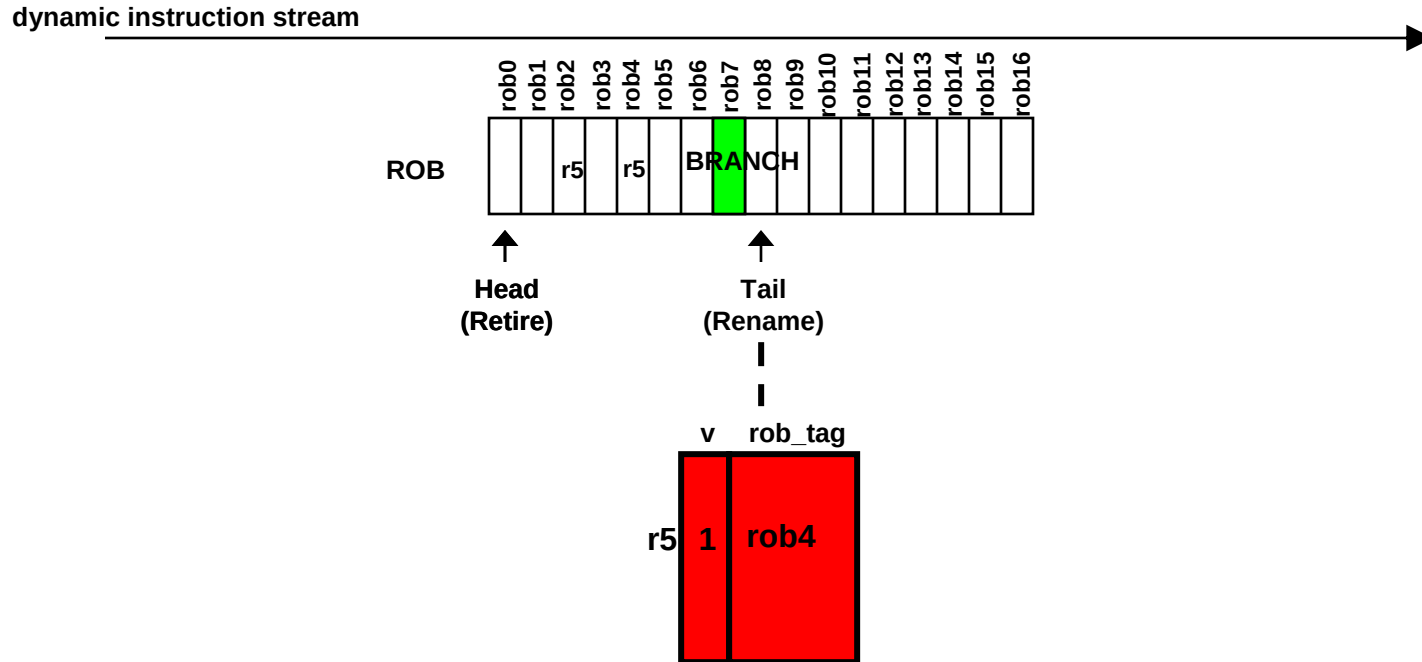
Additional ILP topics

- Interrupts
 - Types
 - Exception is akin to a mispredicted branch
 - ROB supports exception recovery
- **Faster recovery of mispredicted branches**
 - Checkpointing and restoring the RMT
- Handling memory dependencies
 - Store Queue (SQ) and Load Queue (LQ)
- Going from scalar to superscalar
 - Examples of superscalar complexity

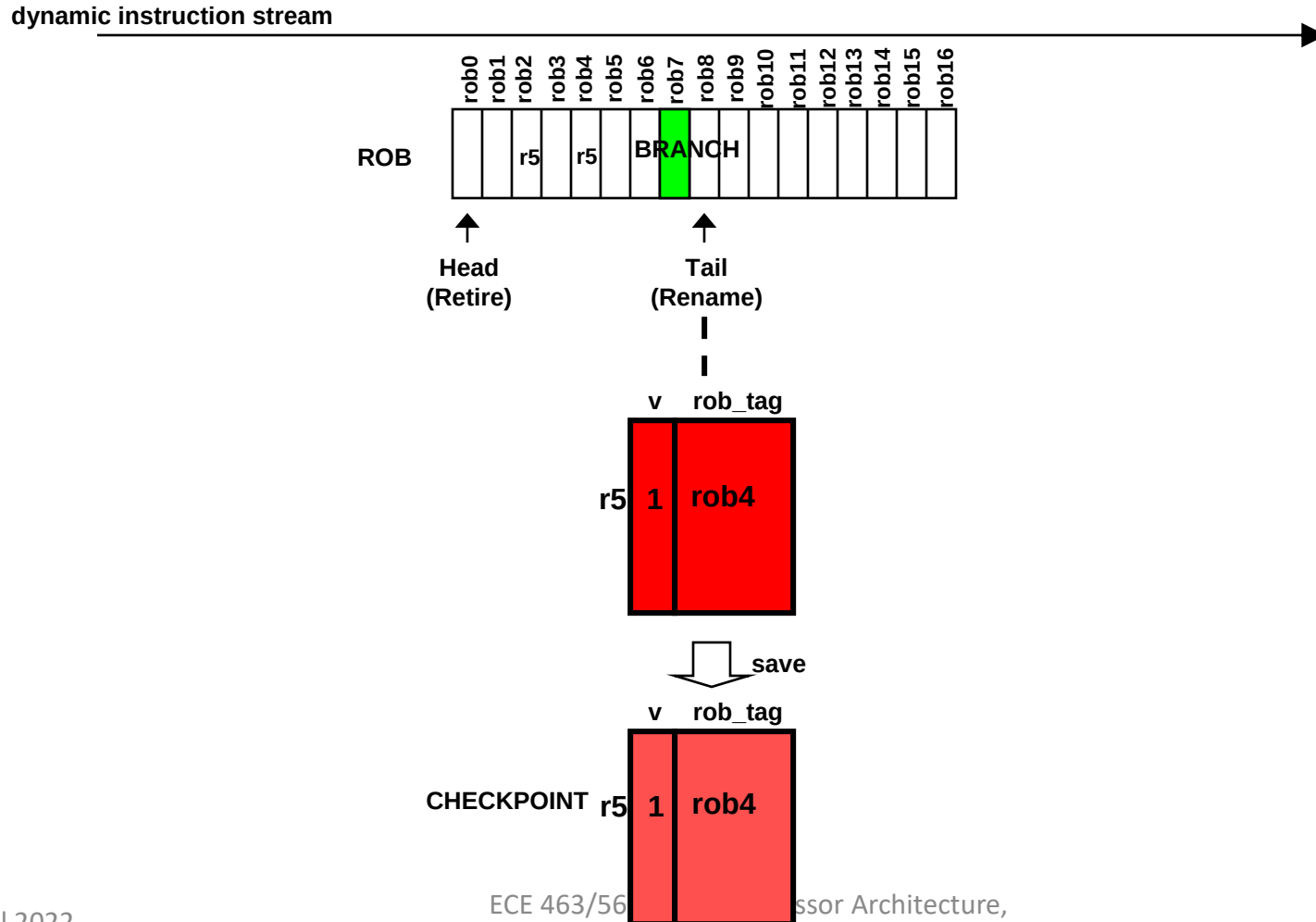
Branch misprediction recovery alternatives

1. Wait until mispredicted branch reaches head of ROB
 - + Simple recovery mechanism
 - ROB: Tail = Head (after retiring the branch)
 - Pipeline: squash *all* instructions in pipeline (all instructions are younger than the branch)
 - RMT: flash-clear all valid bits
 - Delaying recovery until retirement increases the misprediction penalty
2. Immediately initiate recovery, while branch is in middle of ROB
 - + Minimizes the misprediction penalty
 - Complex recovery mechanism
 - ROB: Tail = entry after branch's ROB entry (still simple)
 - Pipeline: *Selectively squash* only those instructions in the pipeline that are younger than the branch and not those that are older than the branch.
 - RMT: Must restore RMT to what it was just after renaming the branch. One approach is to checkpoint the RMT after every branch. Restore RMT to the corresponding checkpoint when a mispredicted branch is detected. (Note that retirement unit updates not only RMT but also RMT-checkpoints.)

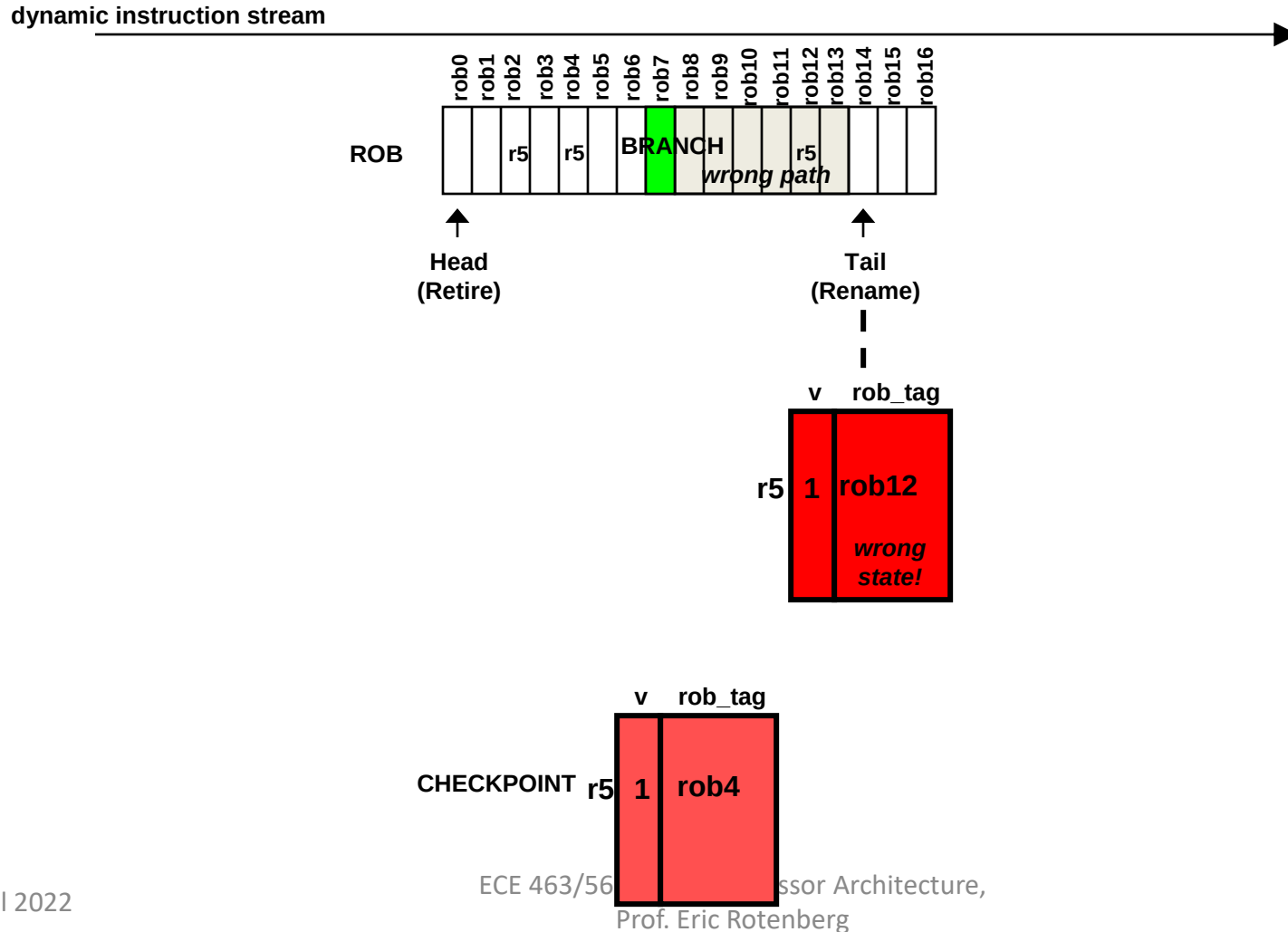
RMT Checkpoint & Recovery (1)



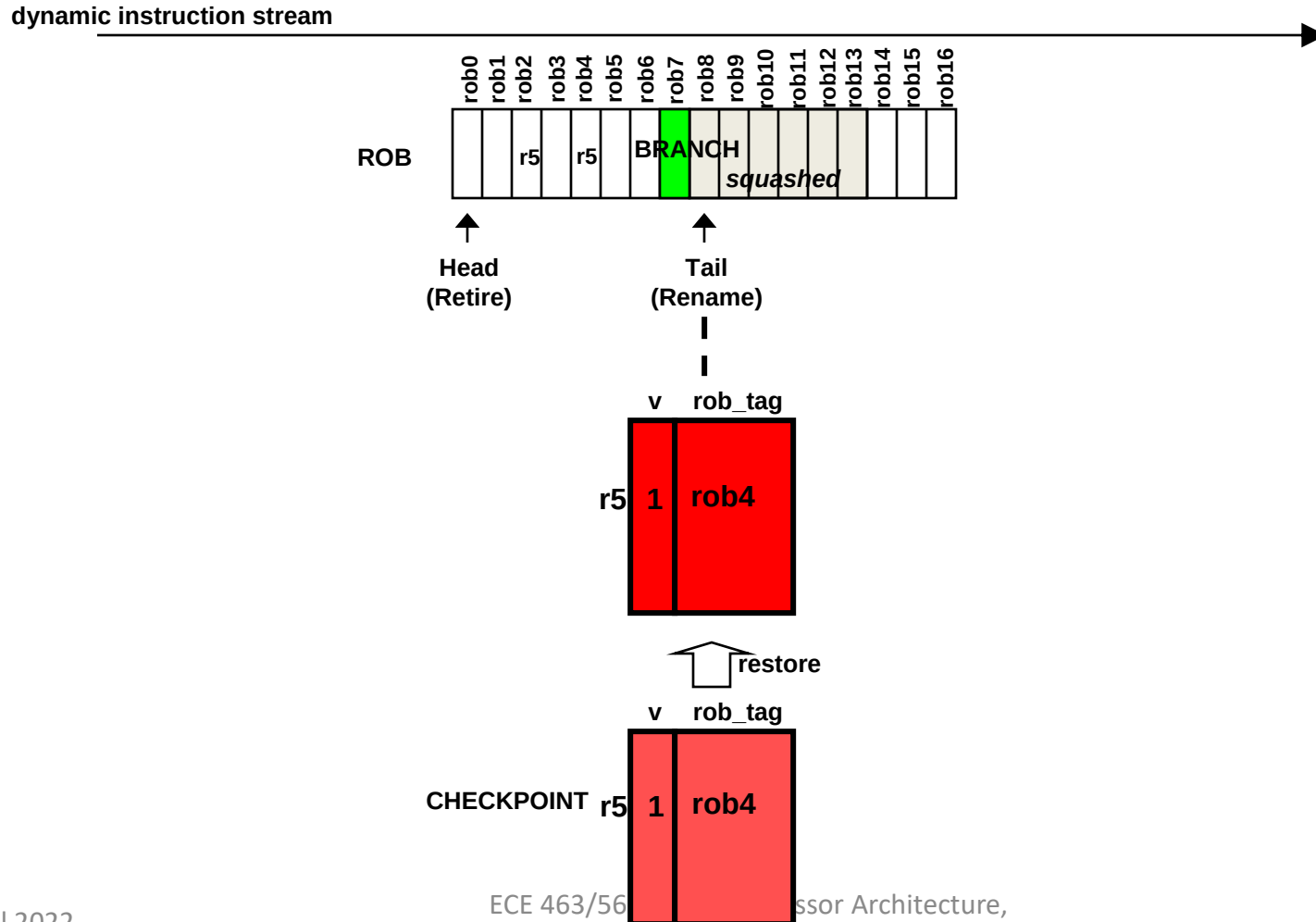
Checkpoint & Recovery (2)



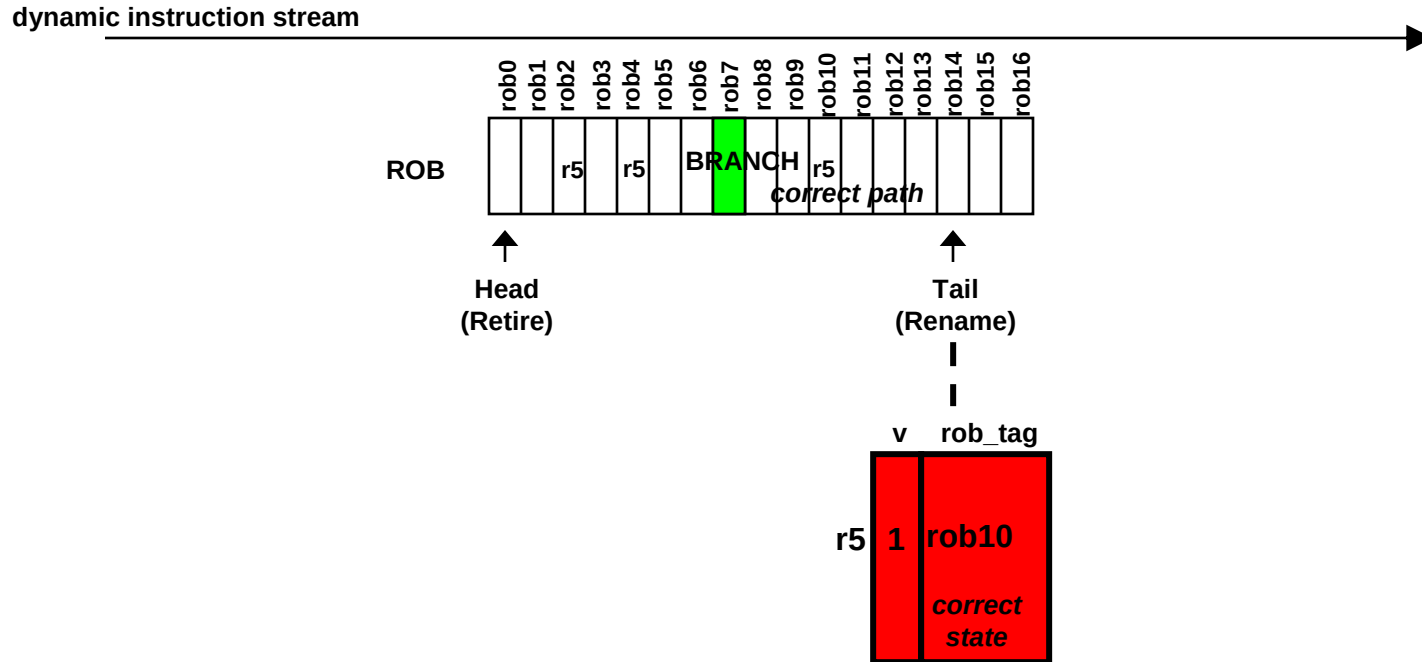
Checkpoint & Recovery (3)



Checkpoint & Recovery (4)



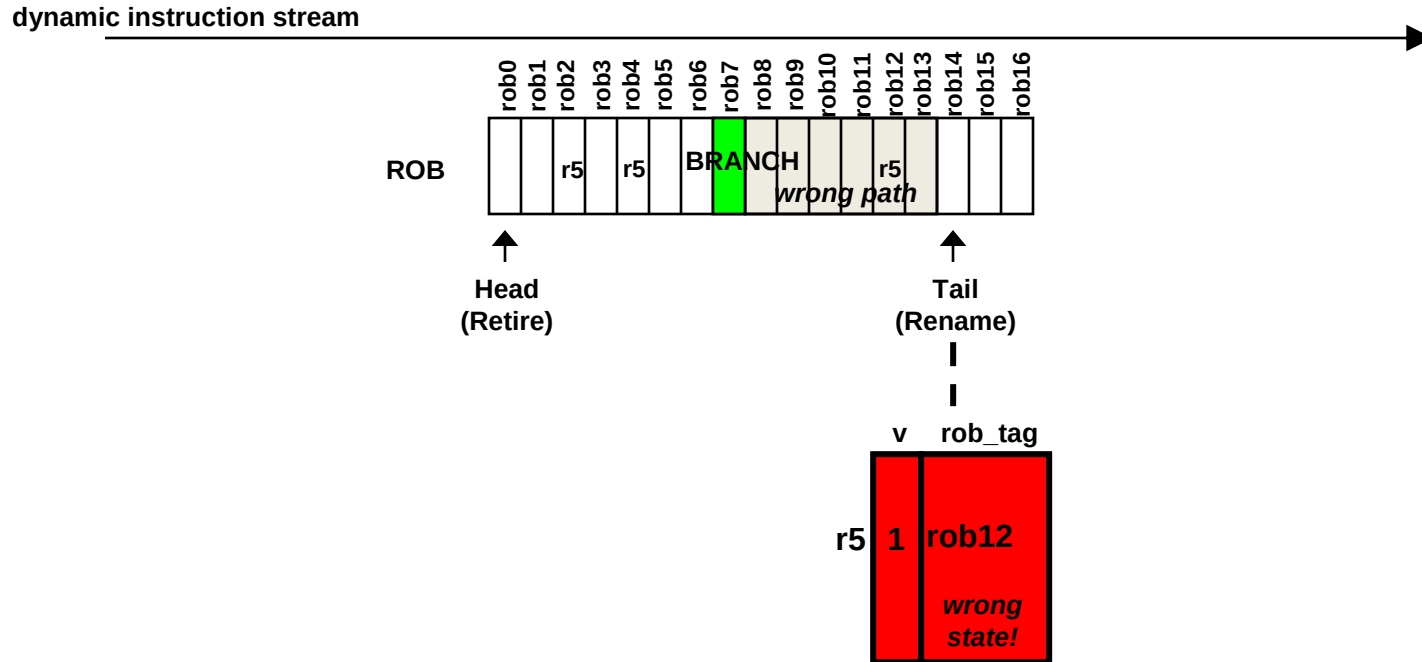
Checkpoint & Recovery (5)



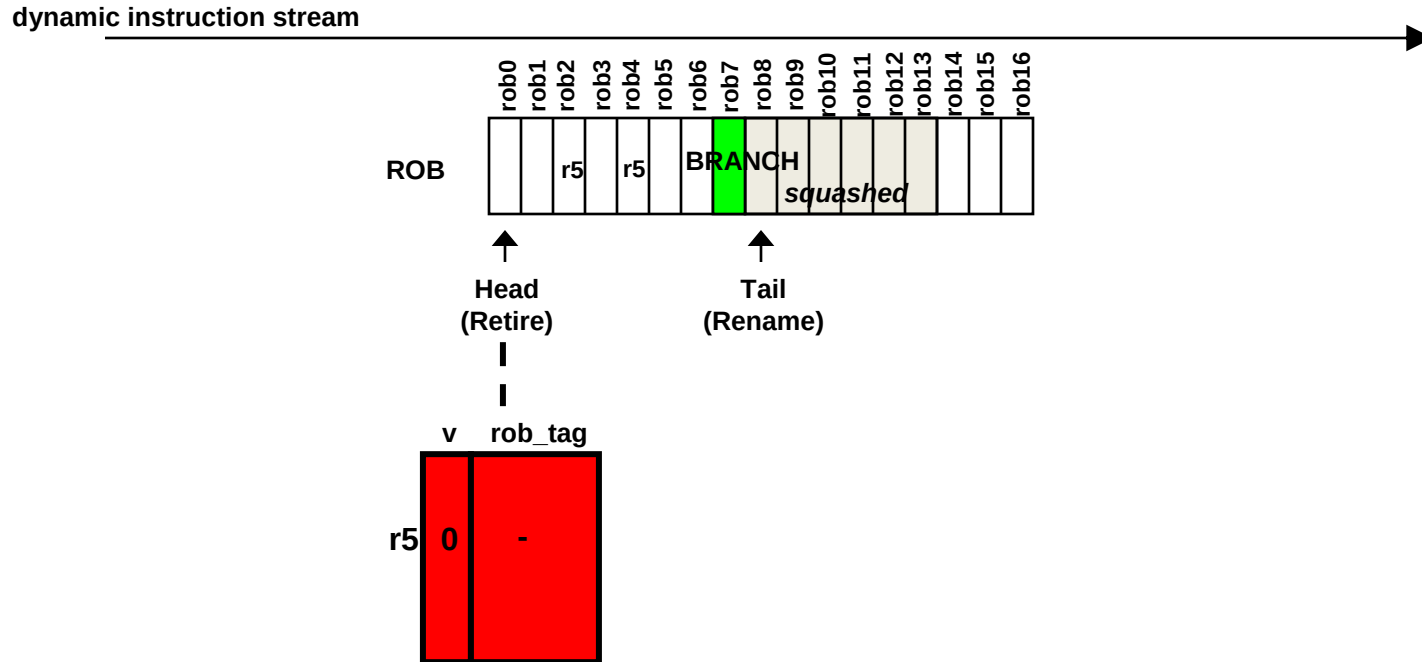
What about exception recovery?

- There's nothing functionally wrong with initiating exception recovery immediately, while the offending instruction is in the middle of the ROB
 - It is even ok to speculatively transfer control to the exception handler
 - If there is an older mispredicted branch or an older exception, this offending instruction and its control-transfer to the handler will be squashed
- The problem is that checkpoints are resource intensive and we can't checkpoint the RMT at every instruction that could post an exception
- There are alternate methods for repairing the RMT that leverage the ROB, for example:
 - Flash-clear the RMT valid bits. This rolls it back to the ROB head, which is too far back.
 - Temporarily stall the rename and retire stages.
 - Serially restore RMT mappings by walking ROB entries from the ROB head to the offending instruction (ROB is used as a “redo log” to redo updates to the RMT).
 - See ECE 721 for more on this!

Using ROB to serially repair RMT (1)

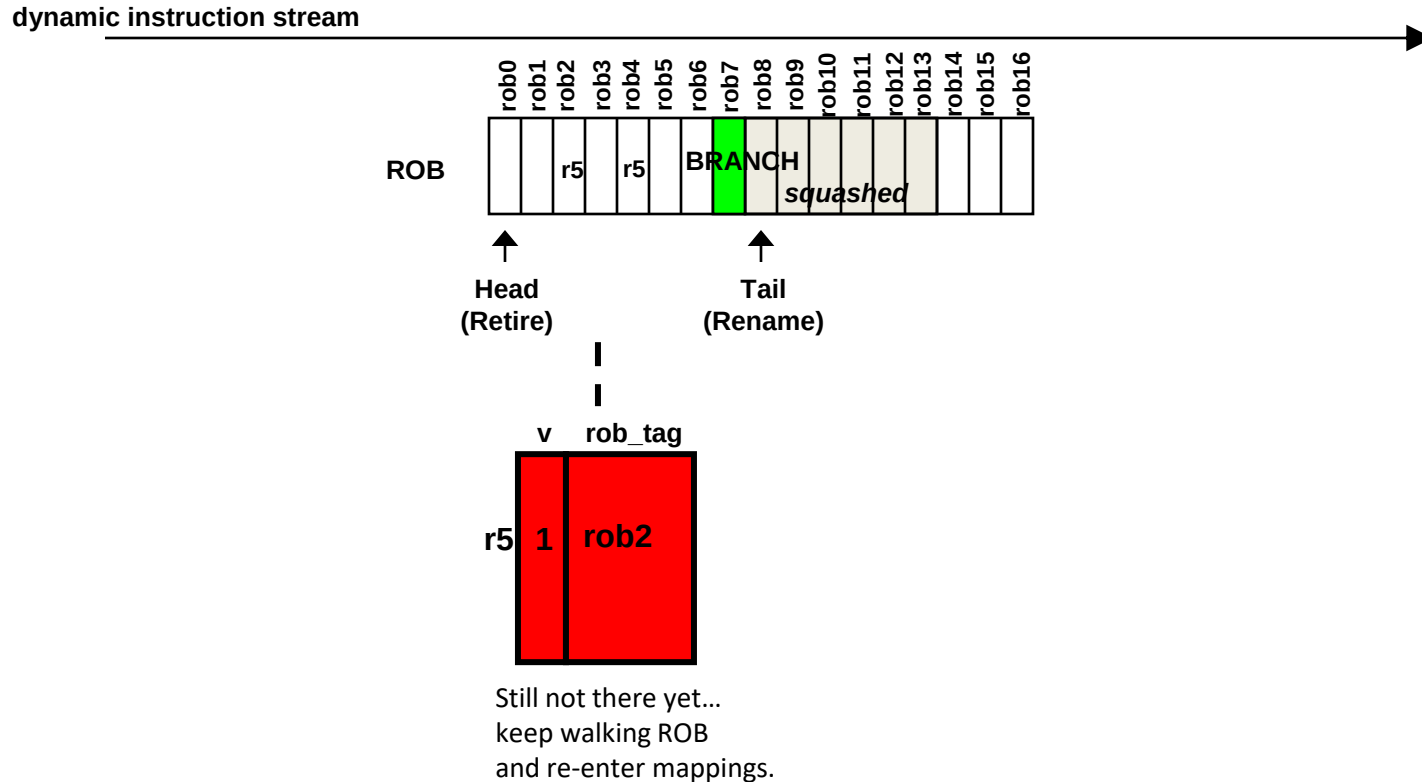


Using ROB to serially repair RMT (2)

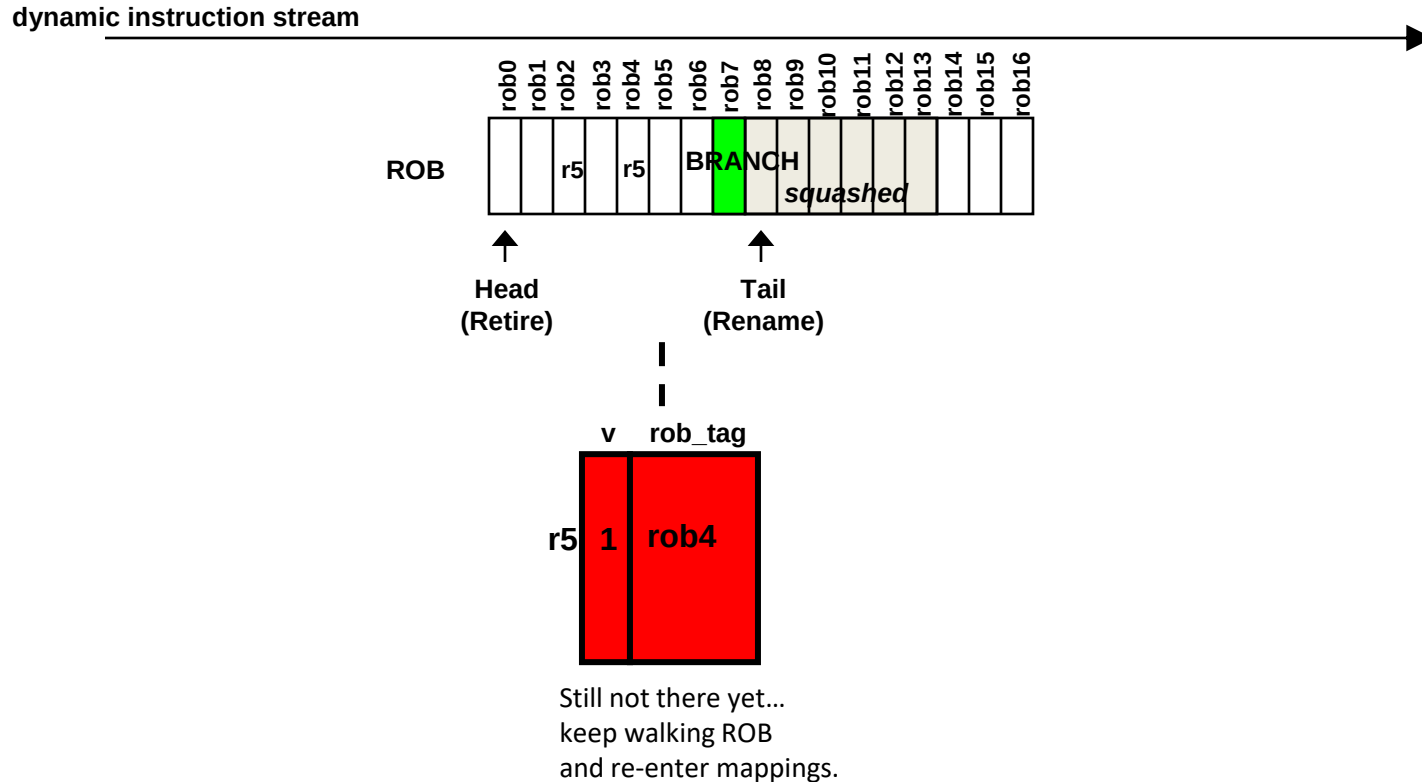


Flash-clear all valid bits ...
rolls back too far.
Start walking each ROB entry to
re-enter mappings into RMT.

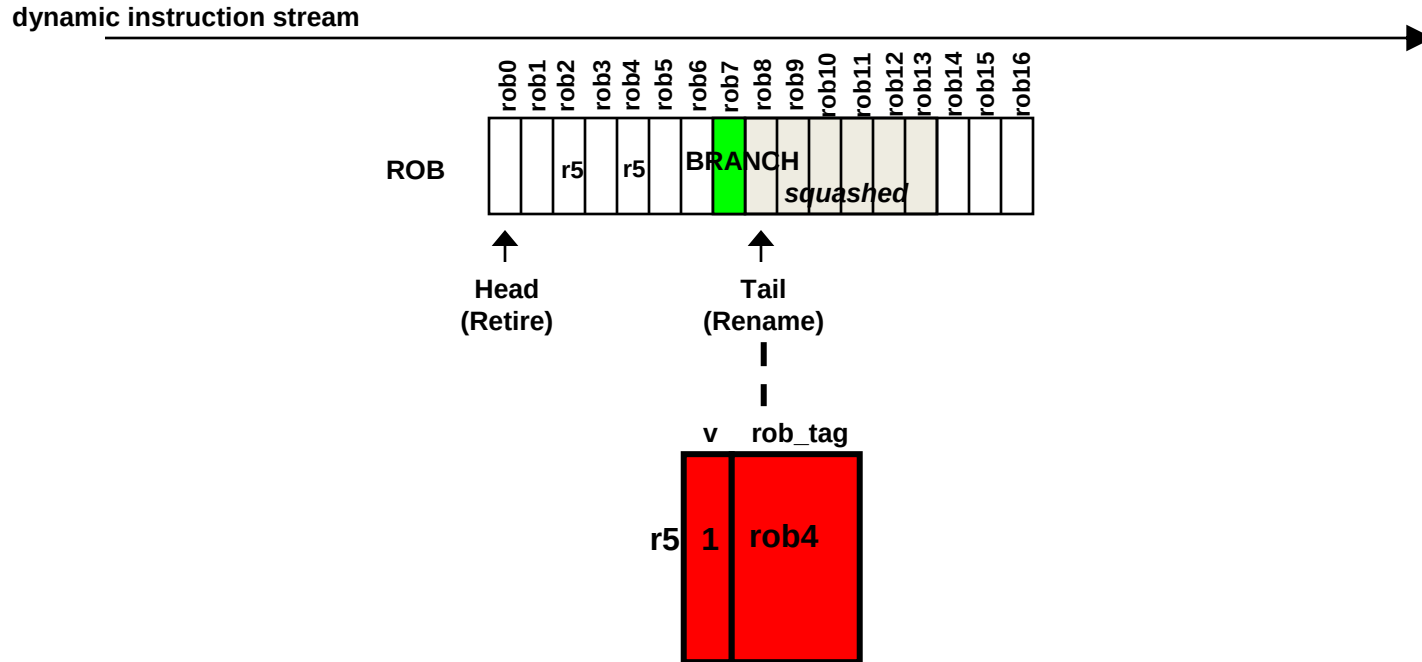
Using ROB to serially repair RMT (3)



Using ROB to serially repair RMT (4)



Using ROB to serially repair RMT (5)



Done repairing the RMT.

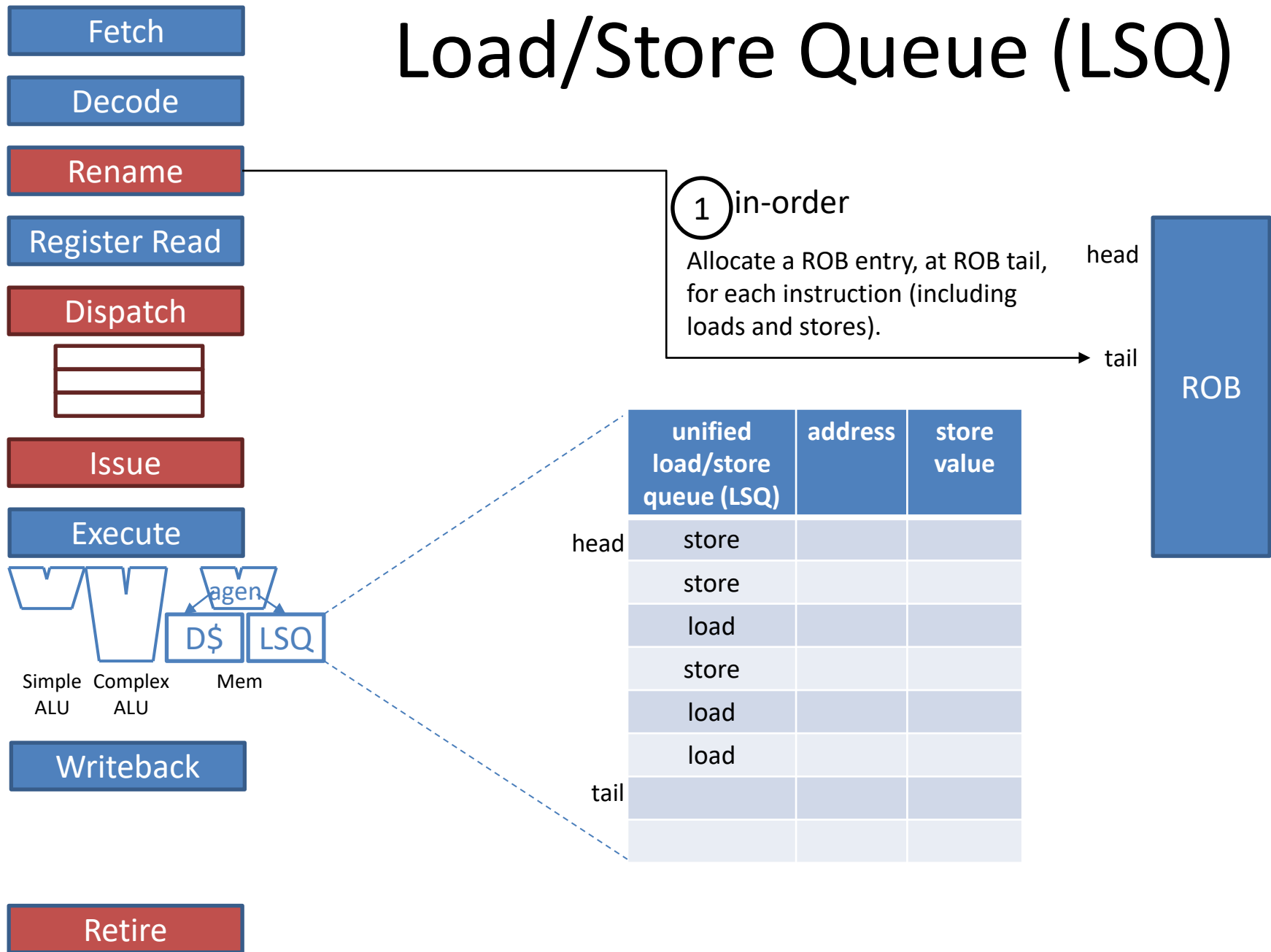
It again reflects having renamed all instructions up to and including the branch instruction.

Repair strategy works for other offending instructions, such as exceptions and other kinds of mispredictions.

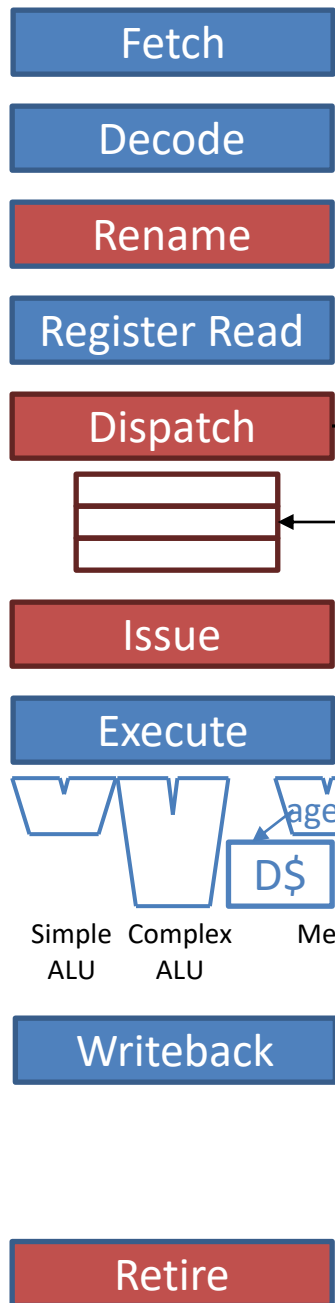
Additional ILP topics

- Interrupts
 - Types
 - Exception is akin to a mispredicted branch
 - ROB supports exception recovery
- Faster recovery of mispredicted branches
 - Checkpointing and restoring the RMT
- Handling memory dependencies
 - Load/Store Queue (LSQ)
 - Separate Store Queue (SQ) and Load Queue (LQ)
- Going from scalar to superscalar
 - Examples of superscalar complexity

Load/Store Queue (LSQ)



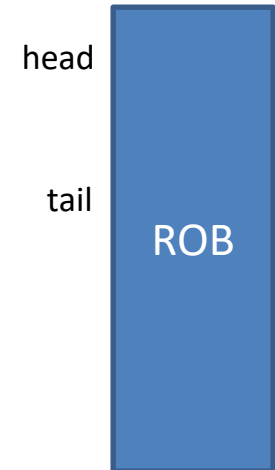
Load/Store Queue (LSQ)



Allocate a free IQ entry for each instruction (including loads and stores).

Allocate an LSQ entry, at the LSQ tail, for each load or store instruction.

	unified load/store queue (LSQ)	address	store value
head	store		
	store		
	load		
	store		
	load		
	load		
tail			



Load/Store Queue (LSQ)

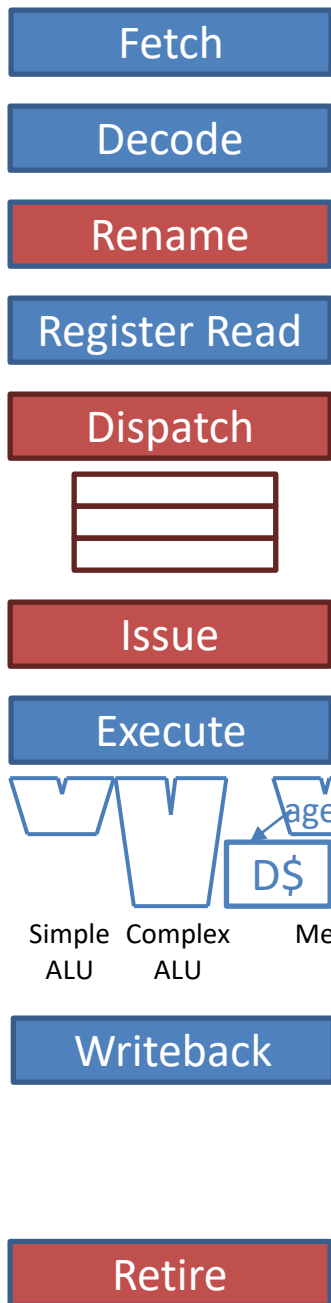
After a load or store issues from the IQ (i.e., after register dependencies are resolved), its address is computed in the AGEN unit.

Load:

- (1) Deposit address into its LSQ entry.
- (2) Search both the D\$ and older stores in the LSQ (details to follow).

Store:

- (1) Deposit address and value into its LSQ entry.
- (2) Search younger loads for mispredicted loads (details to follow).



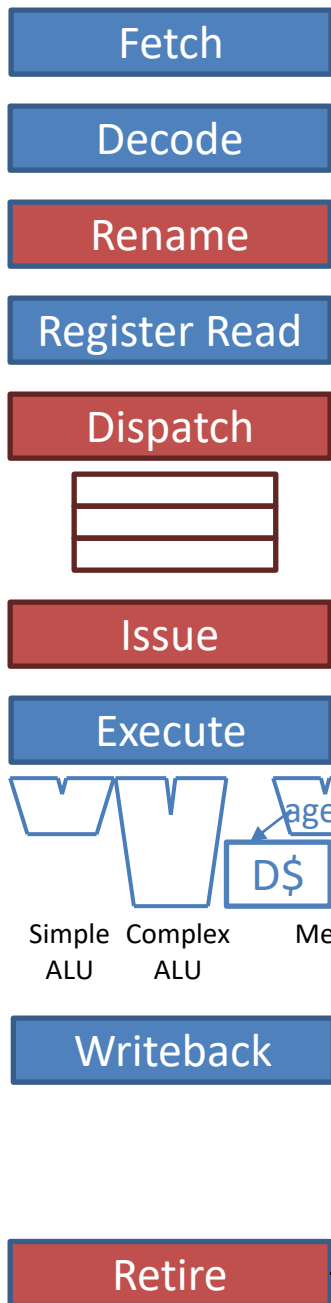
	unified load/store queue (LSQ)	address	store value
head	store		
	store		
	load		
	store		
	load		
	load		
tail			

head

tail

ROB

Load/Store Queue (LSQ)



When the Retire stage retires a load or store from the ROB head, signal the LSQ to pop the same load or store from the LSQ head.

Load: If ROB misp. flag is set, squash (including mispredicted load) and restart fetching from the load (details to follow).

Store: Commit store to the D\$ using its address and value available in its LSQ entry (details to follow).

unified load/store queue (LSQ)	address	store value
store		
store		
load		
store		
load		
load		

head

tail

store

ROB

4 in-order

Principle #1:

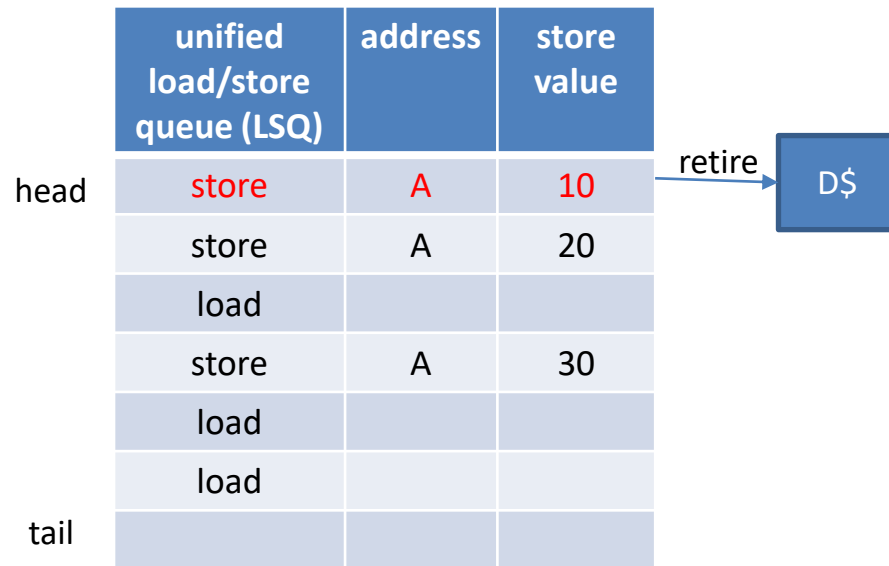
Retire stores in-order and non-speculatively

- LSQ holds all in-flight stores (dispatched but not yet retired) in their original program order
- Retire a store from the LSQ to D\$ only when the ROB signals that the store is at ROB head and is completed
 - Stores to same address retire in-order: correctly handle WAW hazards through memory
 - Each store retires non-speculatively: all prior instructions have retired so there are no prior unresolved mispredictions or exceptions

	unified load/store queue (LSQ)	address	store value
head	store		
	store		
	load		
	store		
	load		
	load		
tail			

	unified load/store queue (LSQ)	address	store value
head	store		
	store		
	load		
	store	A	30
	load		
	load		
tail			

	unified load/store queue (LSQ)	address	store value
head	store		
	store	A	20
	load		
	store	A	30
	load		
tail	load		

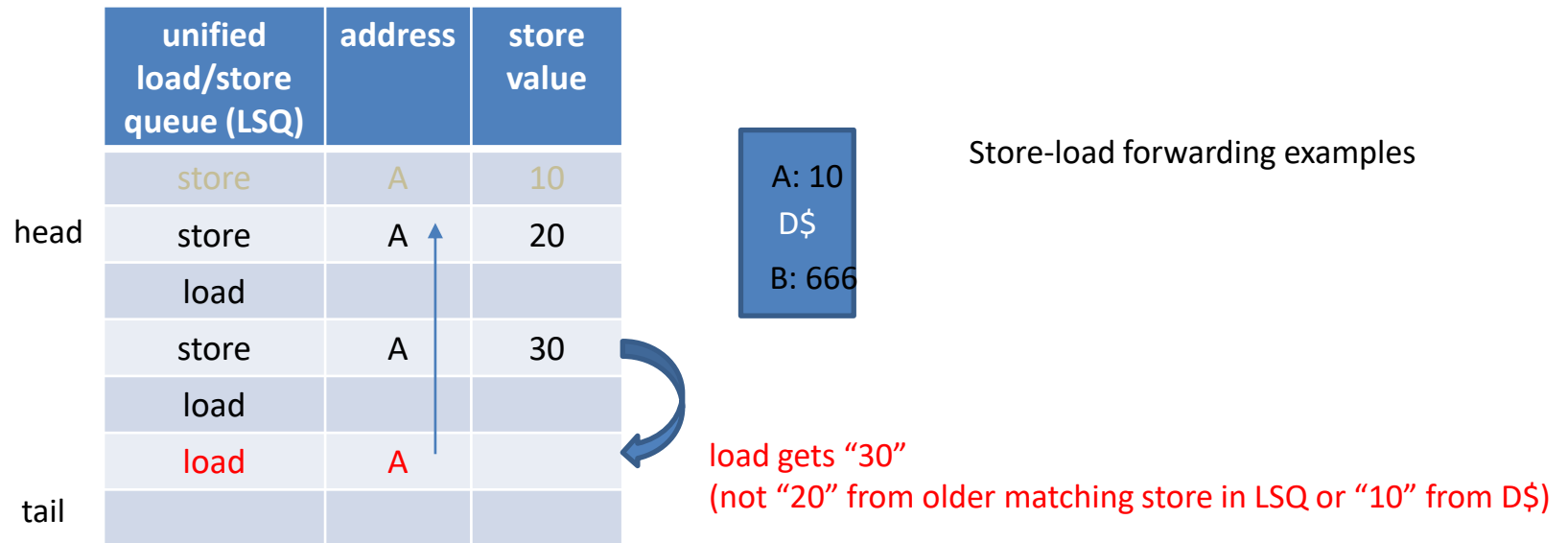


In-order retirement from ROB and LSQ:

1. Correct handling of WAW through memory.
2. Correct recovery to precise memory state (mispredictions, exceptions).

Principle #2:

A load searches both the LSQ and D\$ to get the most recent value corresponding to its address



	unified load/store queue (LSQ)	address	store value
head	store	A	10
	store	A	20
	load		
	store	A	30
	load	B	
tail	load	A	

A: 10
 D\$: 666
 B: 666

load gets "666" from D\$ (no LSQ hit)

load received "30"

Principle #3:

Need a load execution policy to handle unknown prior store addresses

- Loads and stores issue from IQ 000 based on their source register readiness
- A load's address may be generated before prior stores' addresses
- This load sees that there are prior stores in the LSQ, but whether or not it conflicts with these prior stores is unknown
- Two options:
 - Stall the load similar to a cache miss (even if D\$ has the block): “memory disambiguation stall”
 - Speculatively execute the load based on a prediction that these prior stores do not conflict

	unified load/store queue (LSQ)	address	store value
head	store	A	10
	store	A	20
	load		
	store	(?)	
	load	B	
tail	load		

A: 10
D\$
B: 666

load speculatively gets "666" from D\$ (no LSQ hit)

	unified load/store queue (LSQ)	address	store value
head	store	A	10
	store	A	20
	load		
	store	A	30
tail	load	B	
	load		

A: 10
D\$
B: 666

load speculatively got “666” from D\$ (no LSQ hit at the time)

Older store, which executes late, searches LSQ for speculatively executed younger loads.

No conflict detected so do not post a misprediction. Speculative “load B” confirmed OK.

	unified load/store queue (LSQ)	address	store value
head	store	A	10
	store	A	20
	load		
	store	(?)	
	load		
tail	load	A	

load speculatively gets 20

A: 10
D\$
B: 666

	unified load/store queue (LSQ)	address	store value
head	store	A	10
	store	A	20
	load		
	store	A	30
tail	load		
	load	A	

A: 10
D\$
B: 666

load speculatively received "20"  mispredicted load detected (got 20 instead of 30)

Older store, which executes late, searches LSQ for speculatively executed younger loads.

Conflict detected (late) so post a misprediction in the mispredicted load's ROB entry.

Separate SQ and LQ

- Typically, the LSQ is implemented *physically* with a separate Store Queue (SQ) and Load Queue (LQ)
 - The effect is the same as having a unified LSQ
 - More efficient hardware design (smaller CAMs / associative searches)
 - Loads only need to search the SQ (not other loads) for store-load forwarding
 - Stores only need to search the LQ (not other stores) for detecting mispredicted loads
 - How it works
 - A load knows where it logically exists among stores in the SQ, even though it doesn't literally have a SQ entry. So it knows which stores are before it.
 - A store knows where it logically exists among loads in the LQ, even though it doesn't literally have a LQ entry. So it knows which loads are after it.

Load/Store Execution Lane

- AGEN unit for computing load and store addresses
- Three structures
 - L1 D\$ (and L1 D-TLB)
 - Store Queue (SQ): contains all active stores in program order
 - Stores are speculative until they reach head of ROB
 - SQ commits stores to D\$ non-speculatively and in-order
 - Loads search SQ for store values on which they depend
 - Load Queue (LQ): contains all active loads in program order
 - Loads may execute out-of-order with respect to prior stores
 - Executed load gets wrong value if it depends on an older store that hasn't executed yet
 - Stores search LQ for mispredicted loads

Store Instruction

- Dispatch Stage (in-order)
 - Store is allocated the SQ entry at tail of SQ (its SQ_index)
 - Store also notes the current LQ tail, so it knows which loads are after it in program order (its LQ_index)
- Execute Stage (out-of-order)
 - AGEN: Generate store's address
 - Write SQ: Write store's address and value into its SQ entry (at its SQ_index)
 - Read LQ: Use store's address and LQ_index to search LQ for mispredicted loads: loads after the store in program order (between its LQ_index and LQ_tail), with the same address as the store, which already executed.
- Retire Stage (in-order)
 - When a store reaches the head of the ROB, ROB signals SQ to commit its oldest store (SQ_head) to the D\$

Load Instruction

- Dispatch Stage (in-order)
 - Load is allocated the LQ entry at tail of LQ (its LQ_index)
 - Load also notes the current SQ tail, so it knows which stores are before it in program order (its SQ_index)
- Execute Stage (out-of-order)
 - AGEN: Generate load's address
 - Write LQ: Write load's address into its LQ entry (at its LQ_index)
 - Read SQ and D\$: Use load's address and SQ_index to search SQ for best estimate (some stores' addresses still unknown) of producer store: nearest store before the load in program order (between SQ_head and its SQ_index), with the same address as the load. If SQ hit, use store value, else use D\$ value.
- Retire Stage (in-order)
 - When a load reaches the head of the ROB:
 - Signal LQ to remove its oldest load (LQ_head)
 - If load's misprediction bit is set in ROB, initiate misprediction recovery. Fetch unit is redirected to PC of load so that the load re-executes, this time correctly since it is oldest instruction in pipeline (all prior stores have committed to D\$).

Speculative Load Handling: A Rich Design Space

- A load is speculative if there are prior unknown store addresses (in general)
- Four dimensions of speculative load handling
 - Memory Dependence Predictor
 - Store-load synchronization strategy
 - Load misprediction recovery strategy
 - Impact of store execution (split stores vs. no split stores)

ECE 721 covers store and load handling in depth.

Additional ILP topics

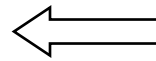
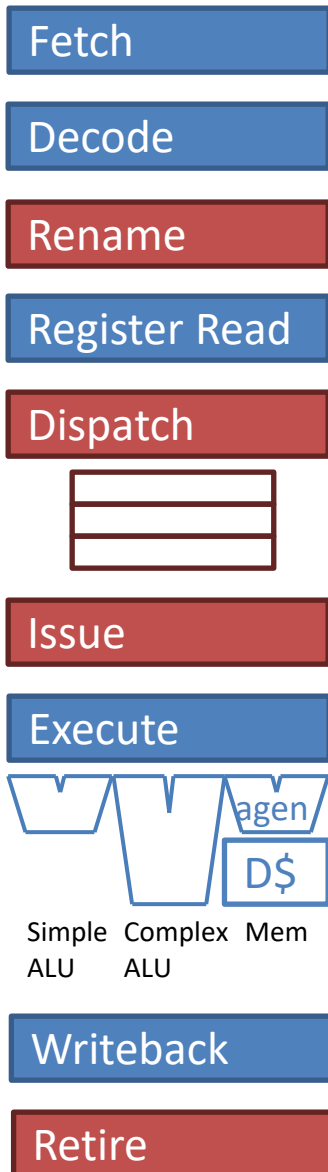
- Interrupts
 - Types
 - Exception is akin to a mispredicted branch
 - ROB supports exception recovery
- Faster recovery of mispredicted branches
 - Checkpointing and restoring the RMT
- Handling memory dependencies
 - Store Queue (SQ) and Load Queue (LQ)
- Going from scalar to superscalar
 - Examples of superscalar complexity

Superscalar processing

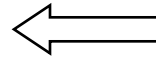
- scalar = 1 instruction/cycle
- superscalar: N instructions/cycle
 - Increase width of each stage of the pipeline

FE	DE	RN	RR	DI	IS	EX	WB	RT			
FE	DE	RN	RR	DI	IS	EX	WB	RT			
	FE	DE	RN	RR	DI	IS	EX	WB	RT		
	FE	DE	RN	RR	DI	IS	EX	WB	RT		
		FE	DE	RN	RR	DI	IS	EX	WB	RT	
		FE	DE	RN	RR	DI	IS	EX	WB	RT	

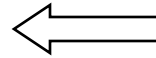
Superscalar Complexity



- Achieving peak fetch width requires interleaved I\$ plus fetch bundle formation logic.
- Predict multiple branches per cycle.
- Predicted-taken branches cause lower-than-peak fetch width.
- I\$ / branch predictor performance critical for superscalar: larger I\$/predictor is slower.



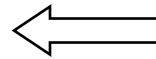
- Highly-ported RMT. N-way superscalar: 2N read ports and N write ports.
- Must handle dependencies within rename bundle: cross-check logic + RMT bypass muxes.



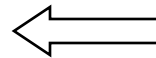
- Highly-ported register file (ARF+ROB). N-way superscalar: 2N read ports and N write ports.



- N-way superscalar: (1) must locate up to N free IQ entries, (2) N write ports into IQ.



- Exposing more instruction-level parallelism requires larger IQ.
- Issue logic more complex. N-way superscalar:
(1) N wakeup ports, (2) select and issue up to N ready instructions (N arbiters, N read ports).



- Bypass network is very complex. Each execution lane forwards its value to every other execution lane. N-way superscalar: (1) N bypasses, (2) bypasses are long wires (must span all execution lanes), (3) each lane has an {N+1}:1 MUX at bypass sinks to select a forwarded value from any lane or the instruction's value from the IQ.

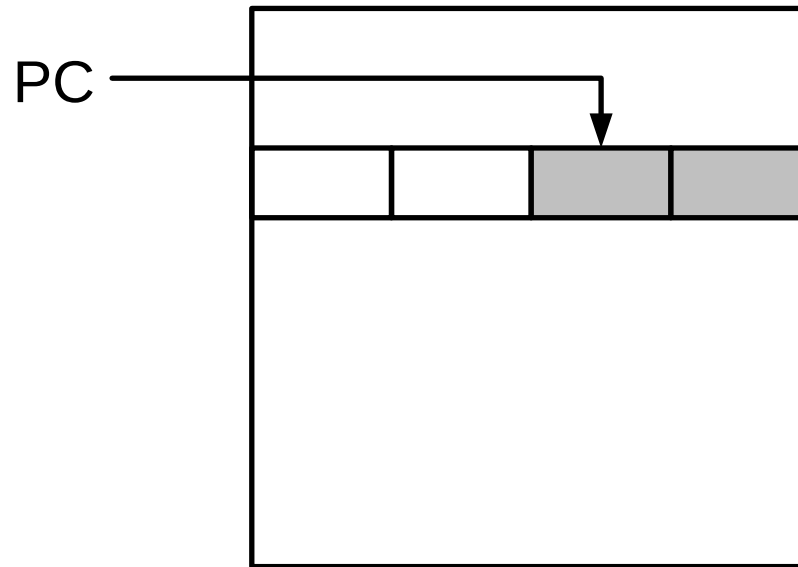
Navigating the IPC/frequency Trade-off

- Naïvely increasing superscalar complexity can increase cycle time, hence, decrease frequency
- How to navigate the IPC/frequency trade-off
 - Microarchitect must balance IPC and frequency to achieve best overall performance
 - Heroic physical design so that you can “have your cake (high IPC) and eat it too (high frequency)”
 - Surgical pipelining of critical paths
 - Custom cells vs. standard cells
 - Expend more power for speed where profitable

Case study of superscalar complexity: fetch multiple instructions in parallel

- Suppose I\$ block size = 16 bytes = four 4-byte instr.
- Suppose we want a superscalar processor with a fetch width of four instructions
- Conventional I\$ can supply 1 block at most
 - Good: get 4 instr. if fetch bundle is aligned at a block boundary.
 - Bad: get fewer than 4 instr. if it is not aligned.
- A taken branch's target can land the fetch PC in the middle of a block, *i.e.*, the next fetch bundle starts in the middle of a block

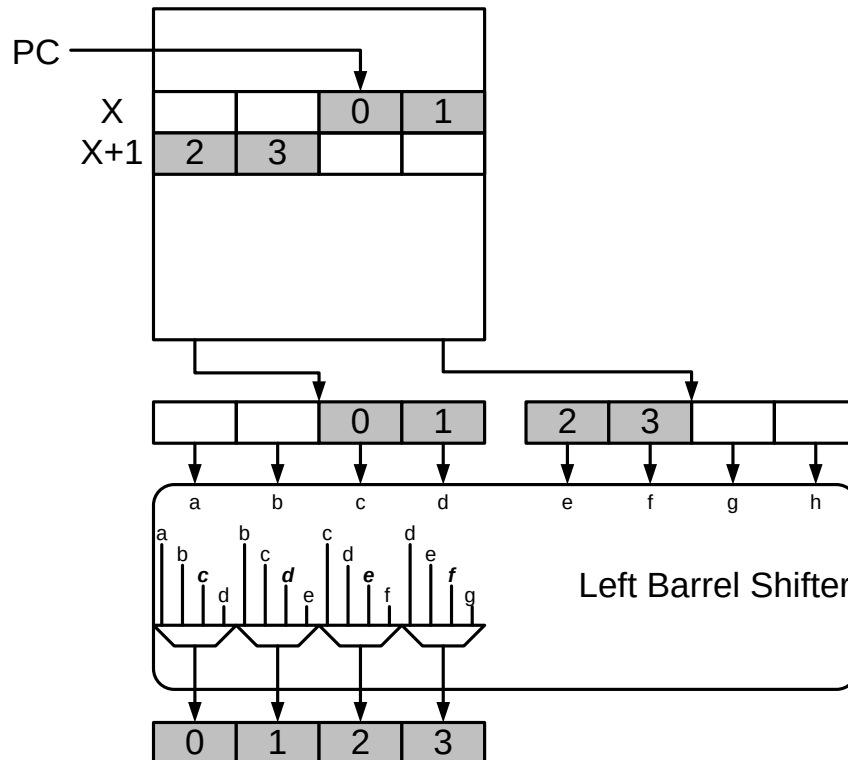
I\$ misalignment



- Fetch 1 aligned cache block / cycle
- Unaligned PC: Fetch useless instructions before 1st instr. and not enough instructions after

Dual-Ported I\$

- Read two blocks to get full bandwidth
- Two read ports are expensive

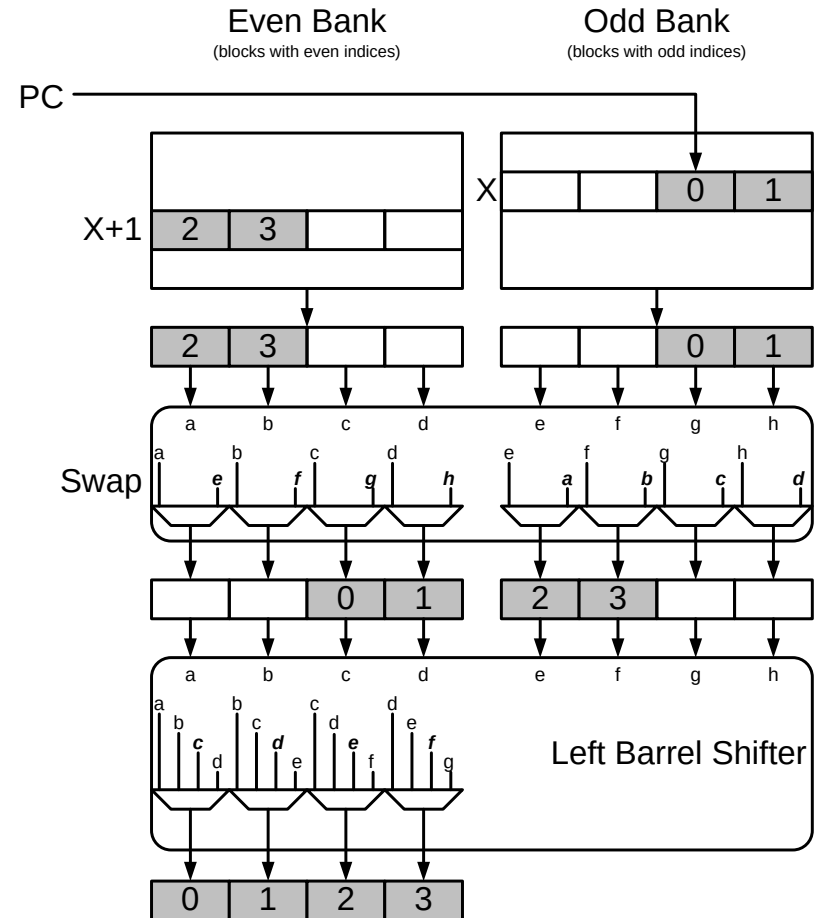
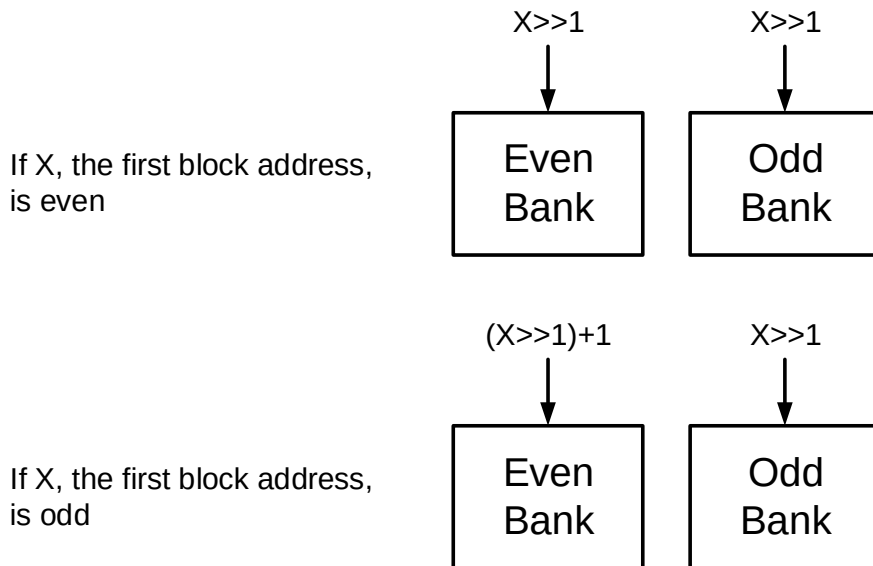


Inexpensive dual-porting: interleaving

- Realization
 - Don't need to read out any two blocks
 - Just need to read out two consecutive blocks
 - Split cache into two banks: one bank holds blocks with even block addresses, other holds blocks with odd block addresses
 - This is called 2-way interleaving or banking
 - Guarantees full fetch width each cycle despite misaligned PC (assuming no predicted-taken branch terminates the fetch bundle early)

Interleaved I\$

- Divide cache into two banks
 - First bank has even blocks
 - Second bank has odd blocks
 - Called an *interleaved cache* or *banked cache*



ECE 463/563

Fall `18

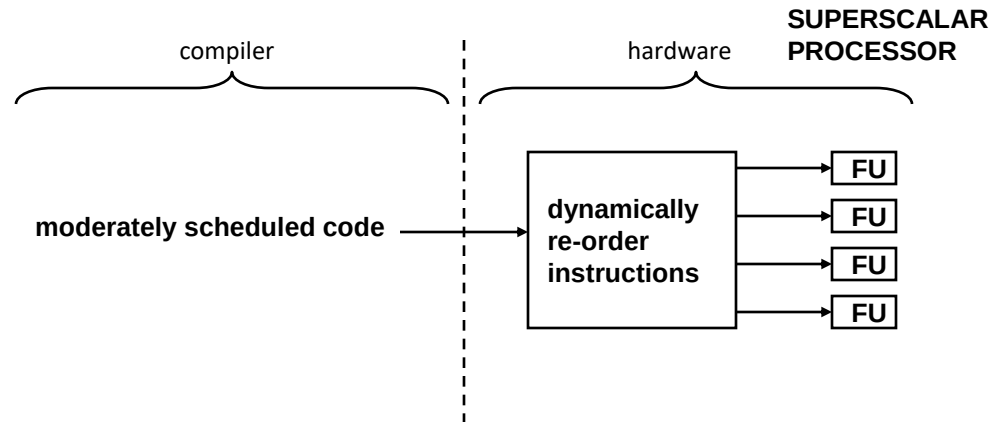
Additional ILP topic #5: VLIW

Also: ISA topics

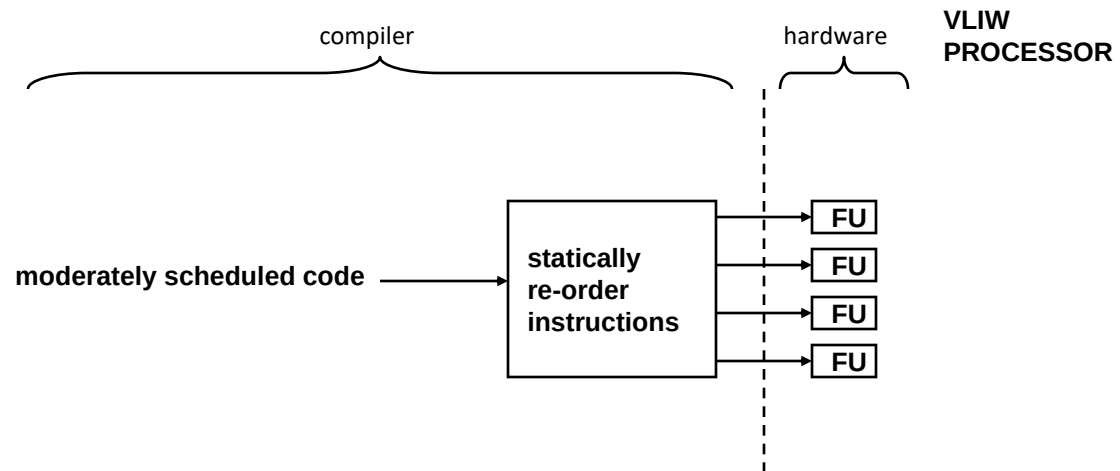
Prof. Eric Rotenberg

Static / Dynamic Scheduling

- Dynamic



- Static



VLIW challenge #1: branches

- Branches limit static scheduling window
- “Basic block” (compiler and architecture term)
 - Formal definition: A code fragment with a single-entry and single-exit
 - Practical definition (a little different): code fragment ending in a branch
- Local scheduling, limited to within a single basic block, does not expose enough ILP
- How to form larger scheduling regions across multiple branches
 - Loop unrolling
 - Unroll loop body N times for larger loop body
 - Also reduces number of dynamic branches (lower IC)
 - Software pipelining (modulo scheduling)
 - Schedule dependent instructions from the same original iteration, across sw pipelined iterations
 - This puts independent instructions from different original iterations in the same sw pipelined iteration
 - Like loop unrolling, without literal unrolling
 - Trace scheduling: speculate a path and branch to fix-up code if misp.
 - Predication: execute both paths of branch, commit one path

Predication

- ISA role
 - Provide predicate registers
 - Provide predicate-setting instructions (*e.g.*, compare)
 - Some (partial predication) or all (full predication) opcodes can be guarded with predicates
- Compiler role
 - Replace branch with predicate computation
 - Guard alternate paths with <predicate> and <!predicate>
- Hardware role
 - Execute all predicated code, *i.e.*, both paths after a branch
 - Do not commit either path until predicate is known
 - Conditionally commit or squash, depending on predicate

Source code

```
if (x > 0)
    y += z;
else
    y -= z;
```

Assembly code

```
A: blte r1,#0,D
B: add r2,r2,r3
C: jump E
D: sub r2,r2,r3
E:
```

Predicated assembly code (branch-free)

```
A: cgt p1,r1,#0    // p1 = (r1 > 0)
B: add r2,r2,r3,p1  // (p1 ? r2=r2+r3 : NOP)
C: sub r2,r2,r3,!p1 // (!p1 ? r2=r2-r3 : NOP)
```

VLIW challenge #2: registers

- Static scheduling window is limited by number of architectural registers
- Just as OOO needs more register renaming registers (ROB) for a larger dynamic scheduling window

Scenario #1: Only r1-r3 available.

```
load r1, A
add  r2, r2, r1
load r1, B          // anti-dependency with add instr. through r1
add  r3, r3, r1
```

Scenario #2: r1-r4 available.

```
load r1, A
add  r2, r2, r1
load r4, B
add  r3, r3, r4
```

vliw bundle 1	load r1, A	load r4, B
vliw bundle 2	add r2,r2,r1	add r3,r3,r4

VLIW challenge #3: cache misses

- Uncertain, disparate latencies are hard to statically schedule
- Hence OOO in general-purpose high-perf. processors

What is an ISA?

- Instruction Set Architecture (ISA) is a specification of the hardware-software interface
- ISA defines:
 - STATE OF THE PROGRAM:
 - Registers
 - General-purpose registers, for example:
 - » Integer registers
 - » Floating-point registers
 - » SIMD / Vector registers
 - Special-purpose registers, for example:
 - » Program counter (PC)
 - » Condition code registers
 - » System control registers
 - » TLB
 - » Other...
 - Memory
 - WHAT INSTRUCTIONS DO: Which state they use, which state they update, and how
 - HOW INSTRUCTIONS ARE REPRESENTED: instr. formats, bit encodings

Why is the ISA important?

- Choices affect performance, cost, and power, often in complex ways. We'll use classic CISC vs. RISC distinctions to demonstrate how choices affect:
 - memory cost for instructions
 - performance factors: IC, CPI, and CT

CISC vs. RISC

- CISC: Complex Instruction Set Computing
 - Many, diverse instructions
 - Individual instructions encapsulate a lot of work
 - E.g., complex addressing modes, exotic instructions, etc.
 - Variable-length instruction encodings
 - *Memory-memory or register-memory* architecture
 - Arithmetic instructions can have both register and memory operands
 - The implication is that an arithmetic instruction is more than an ALU operation. It has one or more implied load/store operations to load operands from memory / store operands to memory.
- RISC: Reduced Instruction Set Computing
 - Fewer, less diverse instructions
 - Instructions are primitive (convey piecemeal amount of work)
 - Fixed-length instruction encodings
 - *Load-store* architecture
 - Arithmetic instructions can have only register operands
 - Memory only accessed via explicit load and store instructions

CISC vs. RISC (cont.)

- Memory cost for instructions
 - CISC binaries generally more compact than RISC binaries due to:
 - Variable-length instruction encoding
 - More work conveyed by a single instruction
- Performance factors (IC, CPI, CT)
 - CISC expresses same amount of work with fewer dynamic instructions
 - ↓ IC
 - RISC lends itself to more efficient, higher performance pipelines (CISC has workarounds, *e.g.*, x86 micro-ops: see next slide)
 - Fixed-length instruction encodings => Easier to decode multiple instructions in parallel since you know where each instruction in the decode bundle starts and ends, in advance.
 - Simple instructions, uniformity (just a few major classes of instructions) => Efficient pipeline.
 - ↓ CPI, CT -or- lower cost and power for same CPI, CT

CISC vs. RISC (cont.)

- How to design efficient pipelines for CISC ISAs
 - Micro-ops
 - In decode stage, crack CISC instructions into one or more RISC-like micro-operations
 - All subsequent pipeline stages are designed for an internal ISA that looks RISC
 - Examples: x86 micro-ops in Intel and AMD processors
 - Binary translation
 - Program binary is first translated from one ISA to another, and the translated binary is what is run
 - Static binary translation: translate once, use this binary over and over
 - Dynamic binary translation: translate each time the program is run, either all at once at the beginning or incrementally as the program runs

Outline of Miscellaneous ISA Topics

- Small issues you need to be aware of
 - Alignment
 - Endian-ness
- Expressing parallelism in ISAs

Load and Store Instructions

- Different load and store instructions for different data sizes
 - load byte / store byte (1 byte)
 - load halfword / store halfword (2 bytes)
 - load word / store word (4 bytes)
 - load doubleword / store doubleword (8 bytes)
- Anything larger than a byte introduces the issue of “aligned” versus “unaligned” accesses

```
if (load/store address is an integer multiple of the data size)
    access is "aligned"
else
    access is "unaligned"
```

aligned halfword accesses

memory (bytes) memory (bytes) memory (bytes) memory (bytes)

0	0	0	0
1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	5	5	5
6	6	6	6
7	7	7	7

unaligned halfword accesses

memory (bytes) memory (bytes) memory (bytes)

0	0	0
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7

aligned word accesses

memory (bytes) memory (bytes)

0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7

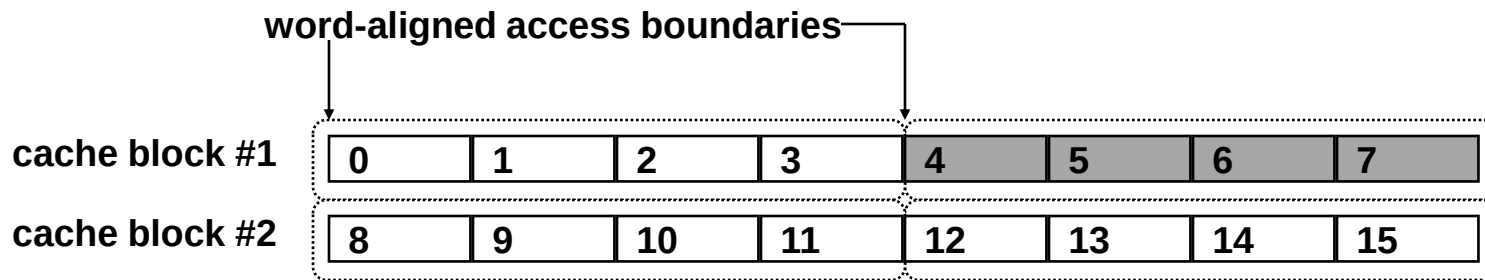
unaligned word accesses

memory (bytes) memory (bytes) memory (bytes)

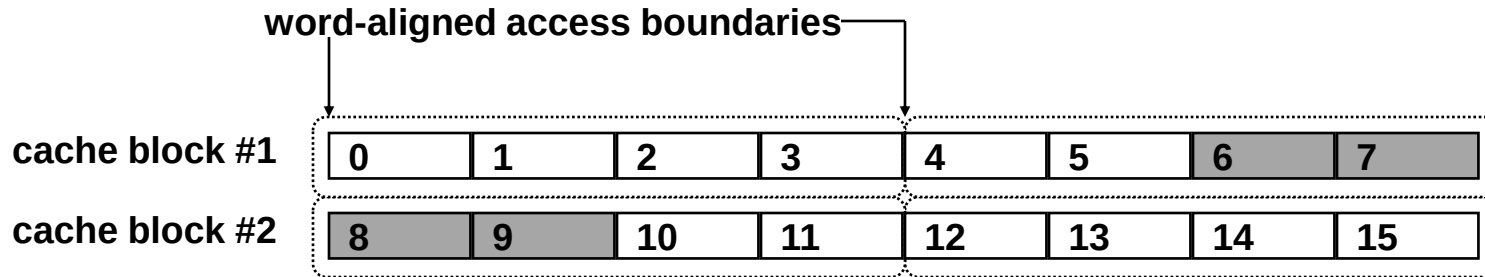
0	0	0
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7

Impact of Unaligned Accesses on Hardware Complexity

Aligned Word Access



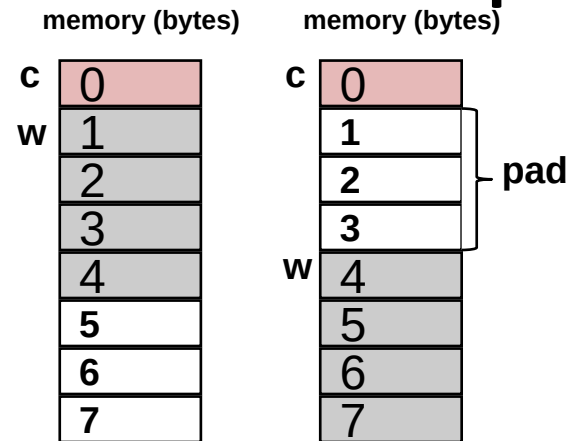
Unaligned Word Access



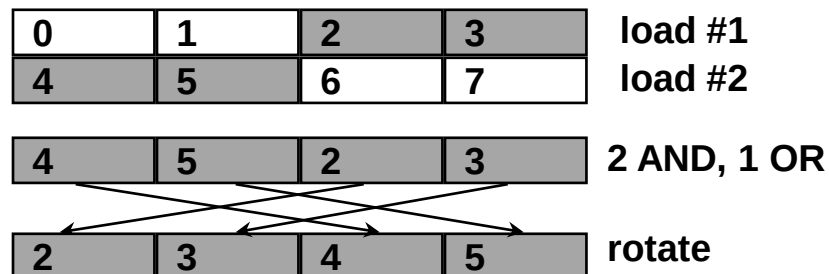
How software can help

- Pad data structures

```
struct {
    char c;
    int w;
}
```



- Emulate unaligned access with multiple instructions
 - 2 word-aligned load instructions
 - The unaligned word spans two aligned words
 - 2 AND instructions to mask unused bytes in the two aligned words
 - 1 OR instruction to merge the useful bytes from the two aligned words
 - 1 rotate instruction to reorder the bytes



Alignment Policy: Example of ISA affecting hardware/software tradeoffs

ISA alignment policy	Impact on hardware	Impact on software
Unaligned accesses allowed	More complex: must support reading two blocks instead of just one ↑ CPI, ↑ energy, ↑ h/w cost	Less complex: no constraints on data placement (no padding), no extra instructions for unaligned access
Unaligned accesses forbidden	Less complex: always read just one block (detect unaligned accesses and raise alignment exception if encountered)	More complex: constraints on data placement (padding), extra instructions for unaligned access ↑ IC, ↑ energy -or- ↑ memory waste (padding)

Endian-ness

- Consider the integer value 0xDEADBEEF
 - 4-byte value
 - When 4-byte value is stored in memory, where is the most-significant byte (MSB)?
 - Choice is arbitrary, and two camps evolved.

Big-endian (e.g., IBM PowerPC)

**Byte address 0 is the “big end”:
it contains the MSB**

Memory (bytes)	
address	data
0	DE
1	AD
2	BE
3	EF
4	
5	
6	
7	

Little-endian (e.g., x86)

**Byte address 0 is the “little end”:
it contains the LSB**

Memory (bytes)	
address	data
0	EF
1	BE
2	AD
3	DE
4	
5	
6	
7	

Expressing Parallelism in the ISA

- Most commercial ISAs are “Von Neumann architecture”
 - Very sequential, doesn’t express parallelism
 - Superscalar Processors expend significant effort uncovering inherent instruction-level parallelism
- Unconventional ISAs express parallelism explicitly
 - SIMD and Vector: Express *data-level parallelism*
 - Most commercial ISAs (x86, ARM, MIPS, Power) now incorporate SIMD/vector as ISA extensions for multimedia and scientific computing
 - VLIW: Express *instruction-level parallelism*
 - Commercial success stories: some general-purpose processors (e.g., Intel’s IA64), many digital signal processors (e.g., Texas Instruments DSPs)
 - Dataflow: Eliminate notion of a single program counter. Instruction sequencing is data-driven: producer instructions point to consumer instructions.

ECE 463/563

Fall `21

Performance and Cost

Prof. Eric Rotenberg

Outline of topics

- Static instructions vs. dynamic instructions
- CPU time equation
- Influence of programmer, compiler, microarchitecture, circuit design, and technology on CPU time
- Comparing performance of two processors
 - What we mean by “n times faster”
- Benchmarks
 - Choice of benchmarks
 - Summarizing performance (arithmetic, harmonic, and geometric means)
- Speedup
- Amdahl’s Law
- Cost
 - Area
 - Power

Static vs. Dynamic Instructions

```
#include <stdio.h>
#include <inttypes.h>

#define HUGE 1000000

int main(int argc, char *argv[]) {
    uint64_t a[HUGE];
    uint64_t sum = 0;

    // I know, a[] uninitialized... just a demo.
    for (uint64_t i = 0; i < HUGE; i++)
        sum += a[i];

    printf("sum = %lu\n", sum);

    return(0);
}
```

Loop has 4 static instructions.
Dynamically, at run-time, the loop executes 1 million times.

static instructions = 4
dynamic instructions = 4 million

Dynamic instruction count influences number of cycles.

a5: starts with address of first element of array a[] (&a[0])
a3: contains address just after last element of array a[] (&a[1000000])
a1: contains on-going sum, initialized to 0

```
1a538: ld      a4,0(a5)      // a4 = a[i]
1a53c: addi     a5,a5,8         // increment address to point to next element of a[]
1a540: add      a1,a1,a4        // sum += a[i]
1a544: bne      a5,a3,1a538 <main+0x34> // branch to top of loop if not after last element of a[]
```

CPU time equation

CPU time = time to execute a program on CPU

cycles = number of clock cycles to execute a program

Cycle Time (CT) = clock period = $1 / (\text{clock frequency})$

$$\text{CPU time} = (\# \text{ cycles}) \times (\text{CT})$$

Instruction Count (IC) = number of instructions executed

Cycles-per-Instruction (CPI) = $(\# \text{ cycles}) / (\text{IC}) \rightarrow (\# \text{ cycles}) = (\text{IC}) \times (\text{CPI})$

$$\text{CPU time} = (\text{IC}) \times (\text{CPI}) \times (\text{CT})$$

$$\text{CPU time} = \text{IC} \times \text{CPI} \times \text{CT}$$

Influence on CPU time

- Programmer influence (IC, CPI)
 - Algorithm affects IC
 - Algorithm affects temporal and spatial locality. *E.g.*, tiling -> lower D\$ miss rates -> fewer cycles per load -> lower CPI
- Compiler influence (IC, CPI)
 - Compiler optimizations affect IC
 - Compiler affects temporal and spatial locality. *E.g.*, tiling -> lower D\$ miss rates -> fewer cycles per load -> lower CPI
 - Instruction scheduling aims to increase instruction-level parallelism (ILP) -> increase IPC -> decrease CPI
- Microarchitecture influence (CPI, CT)
 - Pipeline optimizations aim to increase instruction-level parallelism (ILP) (the number of concurrently executing instructions and the extent of their overlapped execution) -> increase IPC -> decrease CPI
 - Pipeline optimizations may increase CT due to increased logic complexity
 - Deeper pipelining aims to decrease CT
- Circuit design influence (CT)
 - Faster circuits aim to decrease CT
- Technology influence (CT)
 - Faster transistors and wires aim to decrease CT



(e.g., pipelining, data bypassing, branch prediction, caches, dynamic scheduling, superscalar, etc.)

Comparing performance of two processors

- Run benchmark program on both processors
 - Measure CPU time
- When we say “Computer X is n times faster than Computer Y”, it means:
 - $n = \text{Time}(Y) / \text{Time}(X)$

Benchmarks

- Benchmark
 - Test program
 - Measure time it takes for processor to execute it
- Why use benchmarks
 - Processor designer: Evaluate performance impact of proposed mechanisms, enhancements, *etc.*
 - Run benchmark on processor without enhancement
 - Run benchmark on processor with enhancement
 - Observe speedup
 - Customer: Compare performance of different computers
 - Run benchmark on computer A
 - Run benchmark on computer B
 - Observe which one takes less time to run benchmark

Benchmarking Challenges

1. Choice of benchmark

- Which benchmark is a good testcase?
 - Good means representative of real usage
 - Benchmarking pitfall:
 - Observe big speedup on benchmark due to gizmo
 - Conclude gizmo is good idea
 - Gizmo doesn't speedup applications actually run by users, maybe even slows them down (and consumes power, increases cost, *etc.*)
- One benchmark is probably not representative of all usage scenarios. Use benchmark suite (collection of benchmarks targeting a certain computing market).
 - SPEC CPU: PCs, laptops, smart phones (application processors)
 - SPEC WEB: web servers
 - TPC: database servers
 - EEMBC: embedded systems

Benchmarking Challenges (cont.)

2. Summarizing performance of benchmark suite as a whole

— Processor designer:

- A proposed microarch. technique may speedup some benchmarks and slow down others
- Or, it may give big speedup on a few benchmarks and no effect on most benchmarks
- Should the proposed microarch. technique be used?

— Customer:

- Some benchmarks run faster on Computer A and some run faster on Computer B
- Which computer should customer buy?

Summarizing performance

	Computer A	Computer B	Computer C
Benchmark Program P1 (sec)	1	10	20
Benchmark Program P2 (sec)	1000	100	20
Total time (sec)	1001	110	40

- A is 10 x faster than B for P1
- B is 10 x faster than A for P2
- A is 20 x faster than C for P1
- C is 50 x faster than A for P2
- *etc.*
- Total execution time gives clearest picture:
 - B is $1001/110 = 9.1$ x faster than A for both programs
 - C is 25 x faster than A for both programs
 - C is 2.75 x faster than B for both programs
 - Which would you buy? (Answer: C is fastest, overall)
- *Arithmetic mean* of times is good too (A:500.5, B:55, C:20)

$$time = \sum_{i=1}^N time_i$$

$$\overline{time} = \frac{\sum_{i=1}^N time_i}{N}$$

Applying weights

- Some benchmarks may be more valuable than others (relative importance, frequency of use, *etc.*)
- Use weighted time or weighted arithmetic mean

$$time = \sum_{i=1}^N w_i \cdot time_i \qquad \overline{time} = \frac{\sum_{i=1}^N w_i \cdot time_i}{\sum_{i=1}^N w_i}$$

Definitions

metric acronym	description	unit
IC	“dynamic instruction count” <i>i.e.</i> , # instructions executed at run-time (different from “static instruction count”, which is # compiled instr. in the program binary)	instr.
CPI	“cycles-per-instruction” $CPI = 1/IPC$	cycles/instr.
IPC	“instructions-per-cycle” $IPC = 1/CPI$	instr./cycle
CT	“cycle time”, a.k.a., “clock period” $CT = 1/f$	s/cycle
f	“clock frequency” or “frequency” $f = 1/CT$	cycles/s (Hz)
IPS	“instructions-per-second” $IPS = IPC \cdot f$	instr./s

On the use of IPC and IPS

$$\begin{aligned}\text{CPU time} &= \text{IC} \times \text{CPI} \times \text{CT} \\ &= \text{IC} \times (1/\text{IPC}) \times (1/f) \\ &= \text{IC} / (\text{IPC} \times f) \\ &= \text{IC} / \text{IPS}\end{aligned}$$

- Time is the only true measure of performance
- When is it valid to compare computers based on IPC alone?
 - Only if IC and CT are the same

$$\text{speedup}_{B \text{ wrt } A} = \frac{T_A}{T_B} = \frac{\text{IC} \cdot \text{CPI}_A \cdot \text{CT}}{\text{IC} \cdot \text{CPI}_B \cdot \text{CT}} = \frac{\text{CPI}_A}{\text{CPI}_B} = \frac{\text{IPC}_B}{\text{IPC}_A}$$

- When is it valid to compare computers based on IPS alone?
 - Only if IC is the same

$$\text{speedup}_{B \text{ wrt } A} = \frac{T_A}{T_B} = \frac{\text{IC}/\text{IPS}_A}{\text{IC}/\text{IPS}_B} = \frac{\text{IPS}_B}{\text{IPS}_A}$$

On the proper use of means for summarizing metrics of a benchmark suite

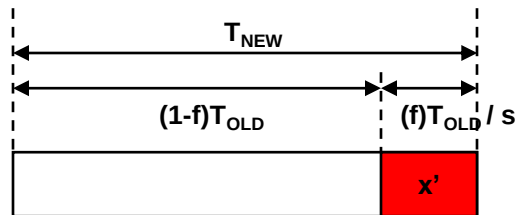
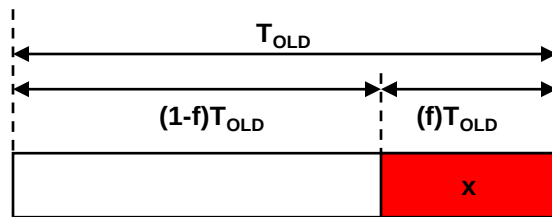
What	Example	Proper mean	Formula	Formula with all weights 1
A quantity	time (s), cycles, CPI, energy (J)	Arithmetic mean	$\overline{time} = \frac{\sum_{i=1}^N w_i \cdot time_i}{\sum_{i=1}^N w_i}$	$\overline{time} = \frac{\sum_{i=1}^N time_i}{N}$
A rate (quantity per unit time)	IPS (1/s), IPC (1/cycle), power (J/s)	Harmonic mean	$\overline{IPC} = \frac{\sum_{i=1}^N w_i}{\sum_{i=1}^N w_i \cdot \left(\frac{1}{IPC_i}\right)}$	$\overline{IPC} = \frac{N}{\sum_{i=1}^N \left(\frac{1}{IPC_i}\right)}$
A ratio (unitless)	Speedup w.r.t. a reference computer	Geometric mean		$\overline{speedup} = \sqrt[N]{\prod_{i=1}^N speedup_i}$

Speedup

- Enhance a processor with some new mechanism
- $\text{speedup} = \text{Time}_{\text{OLD}} / \text{Time}_{\text{NEW}}$

Amdahl's law

- Performance improvement ("*speedup*") is limited by the part you cannot improve.



$$0 \leq f \leq 1$$

$$(1 - f)T_{OLD} + (f)T_{OLD} = (1 - f + f)T_{OLD} = T_{OLD}$$

speedup fraction f by a factor of s

$$S = \frac{x}{x'} = \frac{f \cdot T_{old}}{x'} \quad x' = \frac{f \cdot T_{old}}{s}$$

$$S_{overall} = \frac{T_{OLD}}{T_{NEW}} = \frac{T_{OLD}}{(1 - f)T_{OLD} + \frac{f \cdot T_{OLD}}{s}} = \frac{1}{(1 - f) + \frac{f}{s}}$$

$$S_{overall} = \frac{1}{(1 - f) + \frac{f}{s}}$$

Amdahl's law: Example #1

- Suppose a novel microarchitecture technique provides a speedup of 1000 (pretty amazing), but only applies 80% of the time to a given benchmark. What is the overall speedup for this benchmark?

- $s=1000, f=0.8$

- $S_{overall} = \frac{1}{(1-f) + \frac{f}{s}} = \frac{1}{(1-0.8) + \frac{0.8}{1000}} = \frac{1}{.2008} = 4.98$

- What if $s = \infty$? $S_{overall} = \frac{1}{0.2} = 5.00$ ←

- What if $s = 100$? $S_{overall} = \frac{1}{0.208} = 4.81$

First interpretation of Amdahl's Law: Overall speedup is limited by the part you cannot improve.
(20% of time is not reduced, so you are limited to an overall speedup of no more than 5.)

Amdahl's law: Example #2

- Which is better:
 - Invention A: gives a speedup of 10 when it applies (give the inventor a Nobel Prize!), and it applies 5% of the time for a benchmark of interest.
 - $S_{overall} = \frac{1}{(1-f)+\frac{f}{s}} = \frac{1}{(1-0.05)+\frac{0.05}{10}} = \frac{1}{0.955} = 1.047$ (even for $s = \infty$: 1.053)
 - Invention B: gives a speedup of 1.10 when it applies (how mundane!), and it applies 95% of the time for a benchmark of interest.
 - $S_{overall} = \frac{1}{(1-f)+\frac{f}{s}} = \frac{1}{(1-0.95)+\frac{0.95}{1.10}} = \frac{1}{0.914} = 1.095$ (close to 1.10)
 - The boring invention is better!

Second interpretation of Amdahl's Law: The common case matters most.
(It was better to do an incremental idea that applied frequently.)

Cost of a Chip

100s of engineers:

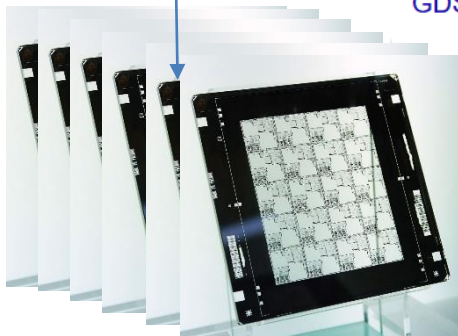
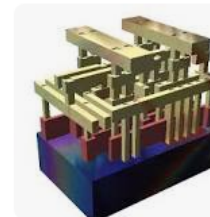
- Microarchitecture
- Performance modeling
- RTL design
- Design-for-test
- Verification (functional, timing, phys. design)
- Physical design (RTL to layout)

Layout in GDSII:
Patterns for many layers.
These patterns guide
lithographic fabrication of
the layers of the chip.

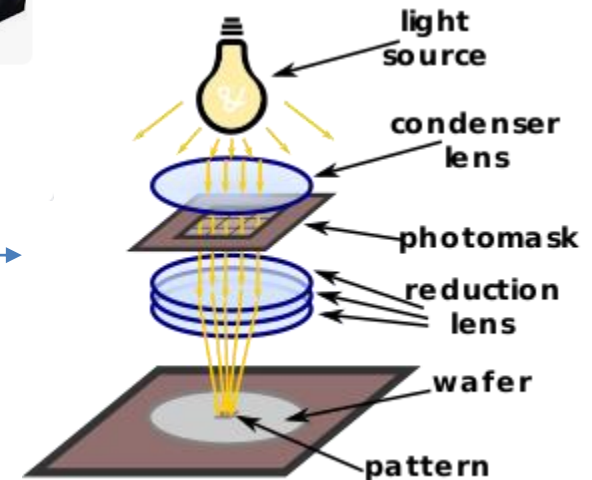
GDSII stream format, common acronym **GDSII**, is a database file format which is the de facto industry standard for data exchange of integrated circuit or **IC layout** artwork. It is a binary file format representing planar geometric shapes, text labels, and other information about the **layout** in hierarchical form.

[en.wikipedia.org > wiki > GDSII](https://en.wikipedia.org/wiki/GDSII)

[GDSII - Wikipedia](https://en.wikipedia.org/wiki/GDSII)



Masks: quartz plates with patterns on them. A mask for one or several fabrication steps.



Cost of a Chip (cont.)

- Designing a chip means *designing and manufacturing a tool (masks) to fabricate that chip*
- NRE cost of the tool *(NRE cost: non-recurring engineering cost, i.e., one-time cost)*
 - e.g., 5 years of salaries and benefits of 100s of engineers
 - Cost of the masks themselves (expensive, too)
- Example:
 - Suppose the NRE cost of the tool is \$10,000,000 (includes 5 years of design effort and cost of mask set).
 - Suppose we fabricate only 1 chip. What is the cost to the consumer of that 1 chip? Answer: \$10,000,000!
 - So we have to fabricate many chips to bring the cost per chip down.
- Cost has a time factor to it
 - A given CPU design has an expiration date, which is the time at which the next generation of CPU is going to come out.
 - We have until this expiration date to mass-produce chips for the current generation of CPU.
 - This is why “yield” from each silicon wafer is a critical factor in the cost of a chip. Yield is how many working chips we get out of each wafer, which affects how many working chips are produced in, say, a week.

Cost of a Chip (cont.)

Example:

cost of tool = \$10,000,000 (5 years design effort + cost of masks)

lifetime of current CPU generation = 2 years

wafers processed per week = 50 wafers/week

2 years of useful manufacturing lifetime



yield example for chip design 1: 10 working chips from each wafer



$$\text{cost/chip for chip design 1} = \frac{\$10,000,000}{10 \frac{\text{chips}}{\text{wafer}} \times 50 \frac{\text{wafers}}{\text{week}} \times 52 \frac{\text{weeks}}{\text{year}} \times 2 \text{ years}} = \frac{\$10,000,000}{52,000 \text{ chips}} = \$192$$

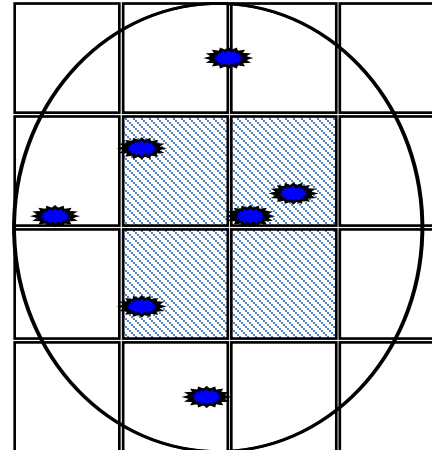
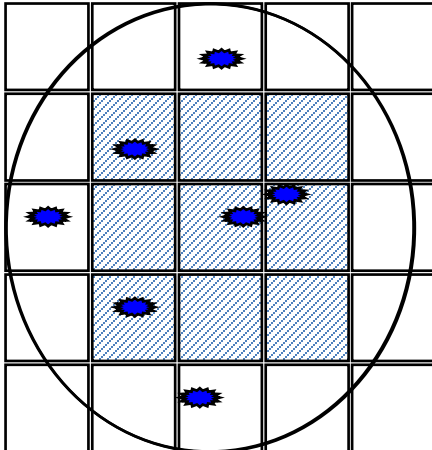
yield example for chip design 2: 5 working chips from each wafer (1/2 the yield of above)



$$\text{cost/chip for chip design 2} = \frac{\$10,000,000}{5 \frac{\text{chips}}{\text{wafer}} \times 50 \frac{\text{wafers}}{\text{week}} \times 52 \frac{\text{weeks}}{\text{year}} \times 2 \text{ years}} = \frac{\$10,000,000}{26,000 \text{ chips}} = \$384$$

Cost of a Chip (cont.)

- Cost is exponential with die area $\text{cost} \propto (\text{die area})^4$
 - Cost depends on *yield*: average number of working die from each wafer
 - Yield is very sensitive to die area
- Two effects as die area increases:
 1. Fewer die per wafer.
 2. Lower percentage yield among die for the same defect pattern.



Other costs: Energy and Power

- Energy
 - Energy is a *quantity*
 - Why we care
 - Battery-powered devices: Battery contains finite amount of charge (Q), hence, finite amount of energy ($E=QV$)
 - Plugged-in devices: Utility bill
- Power
 - Power is a *rate*
 - Power is the rate at which energy is consumed
 - $P = E / \text{time}$
 - Why we care
 - Sustained power
 - Higher sustained power results in higher temperature
 - Cooling technology limits the sustained power of the chip, called the *thermal design power (TDP)*
 - This means power has become a performance limiter in the semiconductor industry
 - Microarchitects need to be inventive to increase performance without exceeding TDP
 - Instantaneous power
 - Inductive noise problem, $\Delta v = L(di/dt)$
 - Spike in current draw can cause a transient fluctuation in V_{dd} , unreliable operation

CMOS Dynamic Power

$$P_{dyn} = \alpha \cdot C \cdot V^2 \cdot f$$

- See ECE 546 (or lower level circuits courses?) for formal derivation. Here's a naïve derivation:
 - Energy consumed in 1 processor cycle: $E = QV = \alpha CV^2$
 - Multiply by frequency to convert to a rate
- α = switching activity factor (fraction of devices switching each cycle, on average)
 - Number between 0 and 1
- C = total capacitance of all devices on chip
- V = supply voltage
- f = clock frequency (rate of switching)

CMOS Static Power

$$P_{static} = V \cdot I_{leakage}$$

- Static power
 - Power consumed even when there is no switching activity
 - In CMOS, this is due to leakage currents in MOSFETs that are supposedly turned off (cut-off region)
- CMOS technology scaling
 - Lowering V_t with each technology generation
 - Increase # transistors being switched (C) +
Increase clock frequency (f) =
Too much dynamic power !
$$P_{dyn} = \alpha \cdot C \cdot V^2 \cdot f$$
 - Lower V_{dd} (supply voltage) to help dynamic power
 - Lowering V_{dd} without also lowering V_t slows down transistors (see ECE 546, etc.)
 - So must also lower V_t
 - But lowering V_t exponentially increases leakage current
 - Whereas 10 years ago most power was dynamic, now as much as half of chip power may be static
 - Circuit and microarchitectural tricks are being used to keep static power in check

Energy

E : *energy*

t : *time*

$$P = \frac{E}{t}$$

$$E = P \cdot t$$

Remarks

1. Consider which power-related metric applies in a particular scenario

What am I concerned with? (scenario)		Relevant metric	Comment
Battery lifetime		energy	Important for battery-operated devices (smart phones, tablets, and other mobile devices).
Utility cost		energy	Important for large data centers (e.g., Google, cloud computing, etc.).
Reliability	inductive noise	instantaneous power	Current spikes cause Vdd fluctuation ($\Delta v = L di/dt$) which can cause faulty operation. Power supply takes time to recover.
	TDP	sustained power	Running too hot for the cooling technology causes overheating of chip which may lead to failure. TDP has become a performance-limiter in the computing industry and has contributed to frequency stagnating.

2. Consider dynamic and static energy in your design decisions

- How much will enhancement increase performance?
- How much will enhancement increase dynamic energy?
- How much will enhancement increase *area* (more devices which leak), hence, static energy?
- How much additional energy are you willing to pay for the performance increase?

Efficiency

- A processor enhancement *will* increase power consumption
- Is this necessarily a bad thing?
 - Recall that power is the rate at which energy is consumed
 - $P = E/t$
- What higher power could mean:
 - IDEAL: Same energy consumption, less time
 - The higher power is due to consuming same energy in less time
 - The performance enhancement came at the price of no extra energy consumption.
This is fantastic.
 - NON-IDEAL: Higher energy consumption, less time
 - The higher power is due to consuming more energy in less time
 - The performance enhancement came at the price of extra energy consumption.
This is more typical, and the goal of the processor designer is to minimize the extra energy cost paid for the higher performance.

ECE 463/563

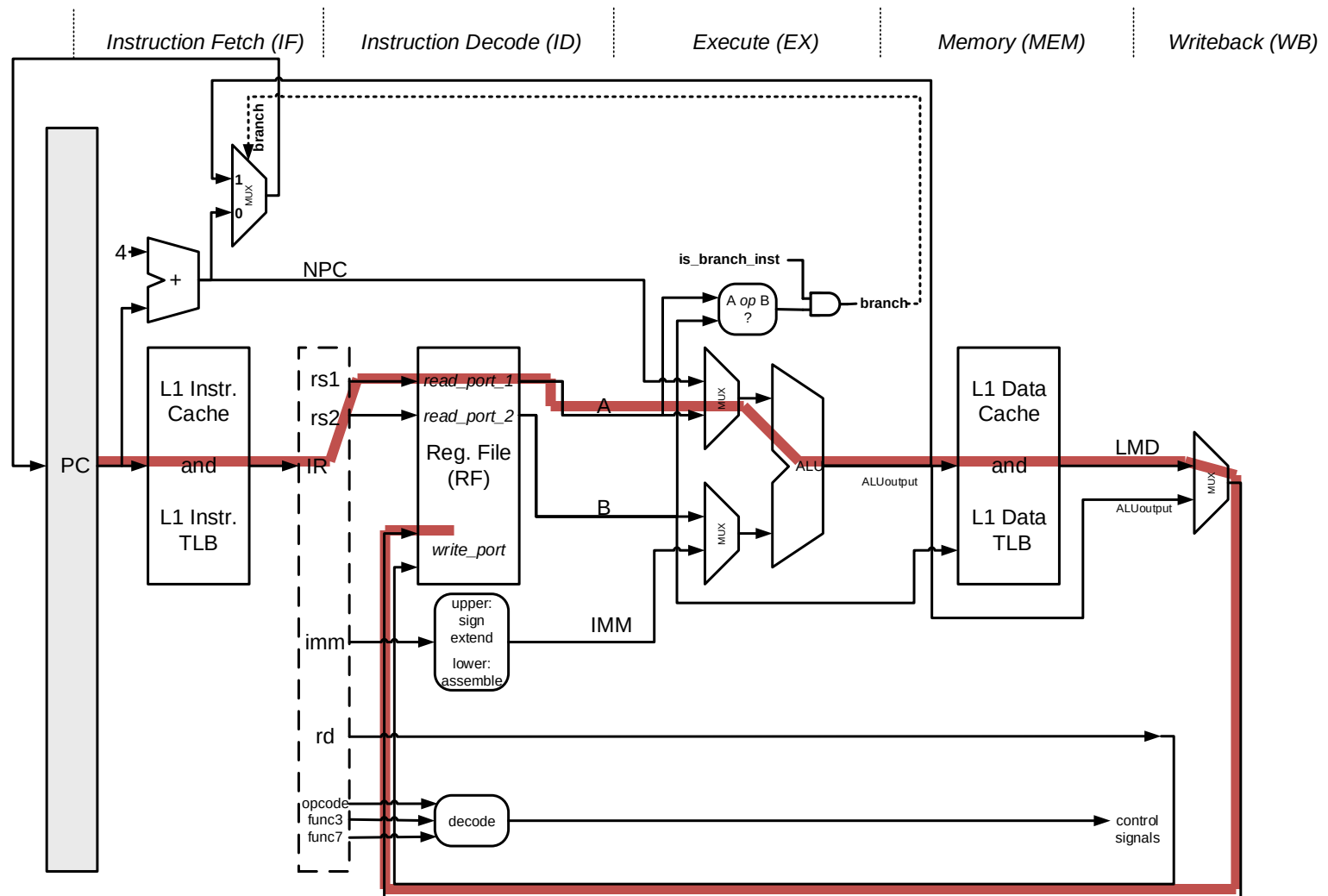
Fall `20

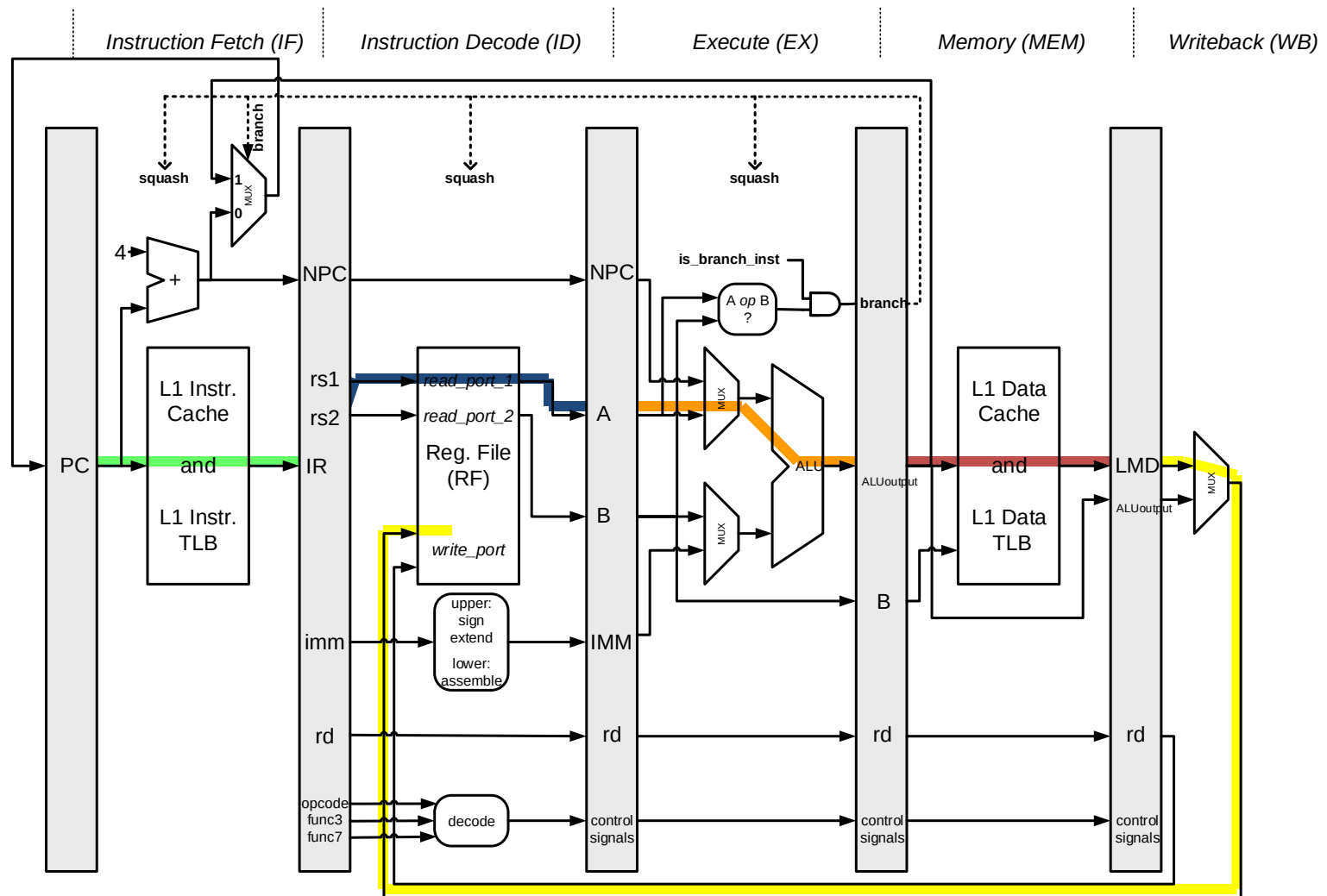
Pipelining:
critical path,
pipeline hazards

Prof. Eric Rotenberg

Critical Path

- Critical path
 - Longest delay (logic and wire delay) between any two flip-flops (Q output to D input)
 - Dictates the minimum cycle time (clock period), hence, maximum clock frequency
- Going from unpipelined datapath to pipelined datapath reduces cycle time
- Deeper pipelining reduces cycle time even further





Example #1

- Unpipelined datapath:
 - Critical path = 50 ns
 - THEREFORE: $CT \geq 50 \text{ ns}$ ($\text{freq} \leq 20 \text{ MHz}$)
- Pipelined datapath:
 - “Local” critical path of IF stage: 10 ns
 - “Local” critical path of ID stage: 10 ns
 - “Local” critical path of EX stage: 10 ns
 - “Local” critical path of MEM stage: 10 ns
 - “Local” critical path of WB stage: 10 ns
 - (Notice $10+10+10+10+10 = 50$)
 - THEREFORE: $CT \geq 10 \text{ ns}$ ($\text{freq} \leq 100 \text{ MHz}$)
- Somewhat idealistic
 - Stages are perfectly *balanced* in this example!

Example #2

- Unpipelined datapath:
 - Critical path = 50 ns
 - THEREFORE: $CT \geq 50 \text{ ns}$ ($\text{freq} \leq 20 \text{ MHz}$)
- Pipelined datapath:
 - “Local” critical path of IF stage: 10 ns
 - “Local” critical path of ID stage: 12 ns
 - “Local” critical path of EX stage: 8 ns
 - “Local” critical path of MEM stage: 13 ns
 - “Local” critical path of WB stage: 7 ns
 - (Notice $10+12+8+13+7 = 50$)
 - THEREFORE: $CT \geq 13 \text{ ns}$ ($\text{freq} \leq 77 \text{ MHz}$)
- STILL somewhat idealistic
 - Ignores delay of intervening pipeline registers!
 - Latches/flip-flops have delay: overhead of pipelining

Example #3

- Unpipelined datapath:
 - Critical path = 50 ns
 - THEREFORE: $CT \geq 50 \text{ ns}$ ($\text{freq} \leq 20 \text{ MHz}$)
- Pipelined datapath:
 - Assume worst-case flip-flop delay is 1 ns
 - “Local” critical path of IF stage: 11 ns
 - “Local” critical path of ID stage: 13 ns
 - “Local” critical path of EX stage: 9 ns
 - “Local” critical path of MEM stage: 14 ns
 - “Local” critical path of WB stage: 8 ns
 - (Notice $11+13+9+14+8 = 55$)
 - THEREFORE: $CT \geq 14 \text{ ns}$ ($\text{freq} \leq 71 \text{ MHz}$)

instr.



→ cycle

	0	1	2
<i>i</i>	IF ID EX MEM WB		
<i>i+1</i>		IF ID EX MEM WB	
<i>i+2</i>			IF ID EX MEM WB

Unpipelined datapath:

- Long cycle time (CT)
- CPI = 1
- Example: CT=50ns

$$TPI = CPI * CT$$

$$= 1 * 50$$

$$= 50ns / instr.$$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
<i>i</i>	IF	ID	EX	MEM	WB										
<i>i+1</i>						IF	ID	EX	MEM	WB					
<i>i+2</i>											IF	ID	EX	MEM	WB

Pipelined datapath,
but no instr. overlap:

- Short cycle time (CT)
- CPI = 5
- Example: CT=10ns

$$TPI = CPI * CT$$

$$= 5 * 10$$

$$= 50ns / instr.$$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
<i>i</i>	IF	ID	EX	MEM	WB										
<i>i+1</i>		IF	ID	EX	MEM	WB									
<i>i+2</i>			IF	ID	EX	MEM	WB								

Pipelined datapath,
instr. overlap:

- Short cycle time (CT)
- CPI = 1 (*pipelining*)
- Example: CT=10ns

$$TPI = CPI * CT$$

$$= 1 * 10$$

$$= 10ns / instr.$$

Summary: Pipelined RISC-V

- Isolate each of IF, ID, EX, MEM, WB with latches or flip-flops (“pipeline registers”)
- When instruction i is in WB, $i+1$ is in MEM, *etc.*
- Graphically:

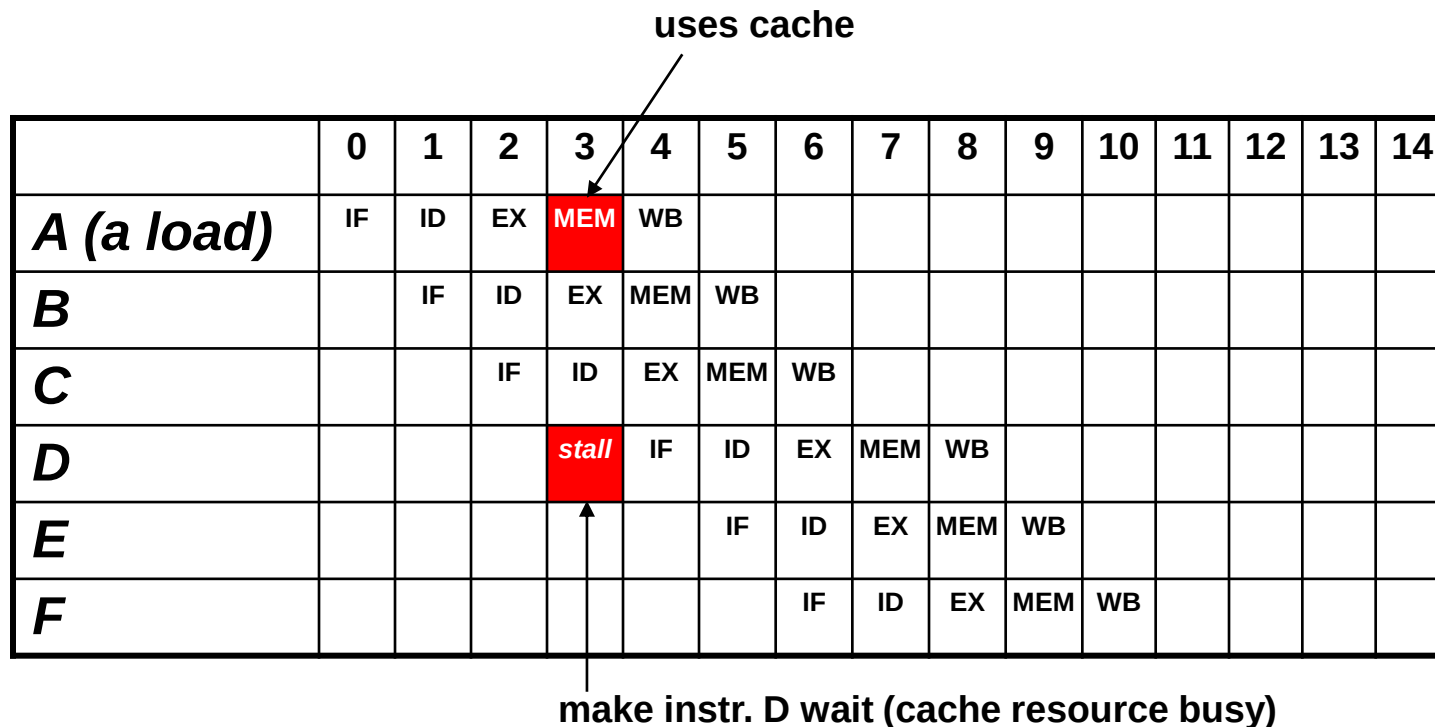
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
i	IF	ID	EX	MEM	WB										
$i+1$		IF	ID	EX	MEM	WB									
$i+2$			IF	ID	EX	MEM	WB								
$i+3$				IF	ID	EX	MEM	WB							
$i+4$					IF	ID	EX	MEM	WB						
$i+5$						IF	ID	EX	MEM	WB					

Pipeline Hazards

- A hazard reduces the performance of the pipeline
- Three kinds:
 - *Structural hazards*: Not enough hardware resources for all combinations of concurrent instructions
 - *Control hazards*: Mispredicted branches incur a misprediction penalty
 - *Data hazards*: Dependencies between instructions prevent their overlapped execution

Structural Hazards

- Uncommon for our pipeline example
- Consider a pipeline with a *unified data+instruction cache*:



Control Hazards

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
A	IF	ID	EX	MEM	WB										
B		IF	ID	EX	-	-									
C			IF	ID	-	-	-								
D				IF	-	-	-	-							
X				squash redirect	IF	ID	EX	MEM	WB						
Y						IF	ID	EX	MEM	WB					

A: bne r1,r2,X
B: ...
C: ...
D: ...
E: ...
 ...
X: ...
Y: ...

Suppose $r1 \neq r2$
 (branch is *taken*)

- $PC = PC + 4$ (sequential) not always correct
- Taken branch incurs 3-cycle penalty

ECE 463/563

Fall `20

pipelining:
data hazards

Prof. Eric Rotenberg

Data Hazards

- read-after-write (RAW) hazard

add r1, r2, r3

add r4, r1, r5

- write-after-read (WAR) hazard

add r1, r2, r3

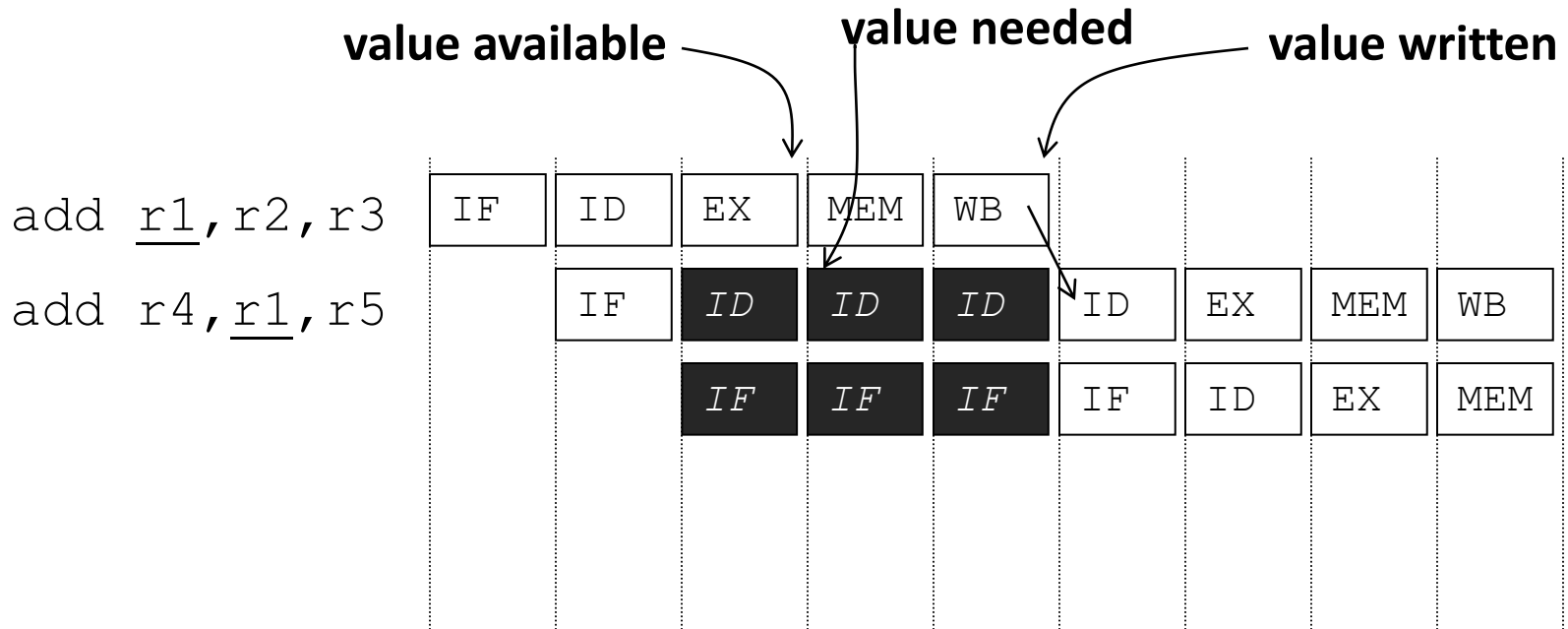
add r2, r4, r5

- write-after-write (WAW) hazard

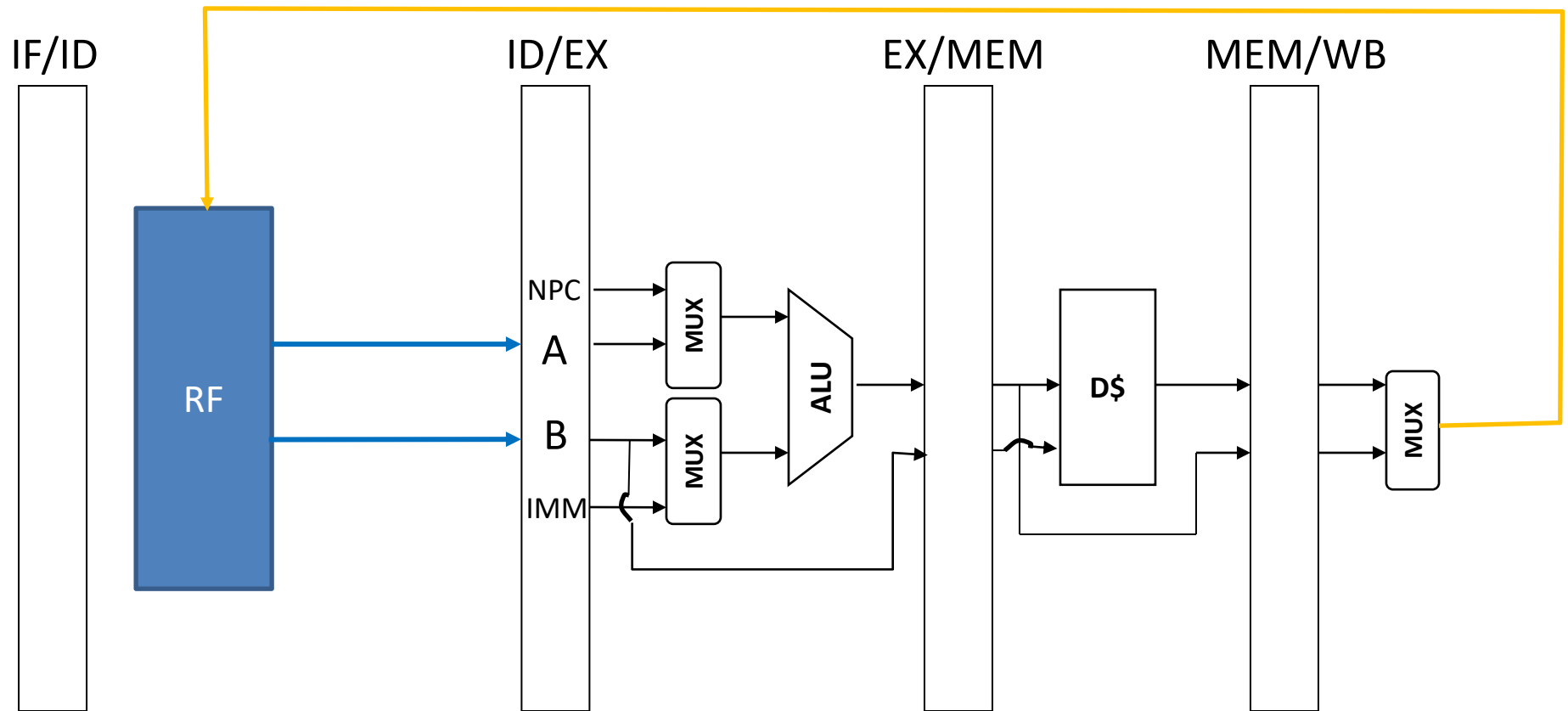
add r1, r2, r3

add r1, r4, r5

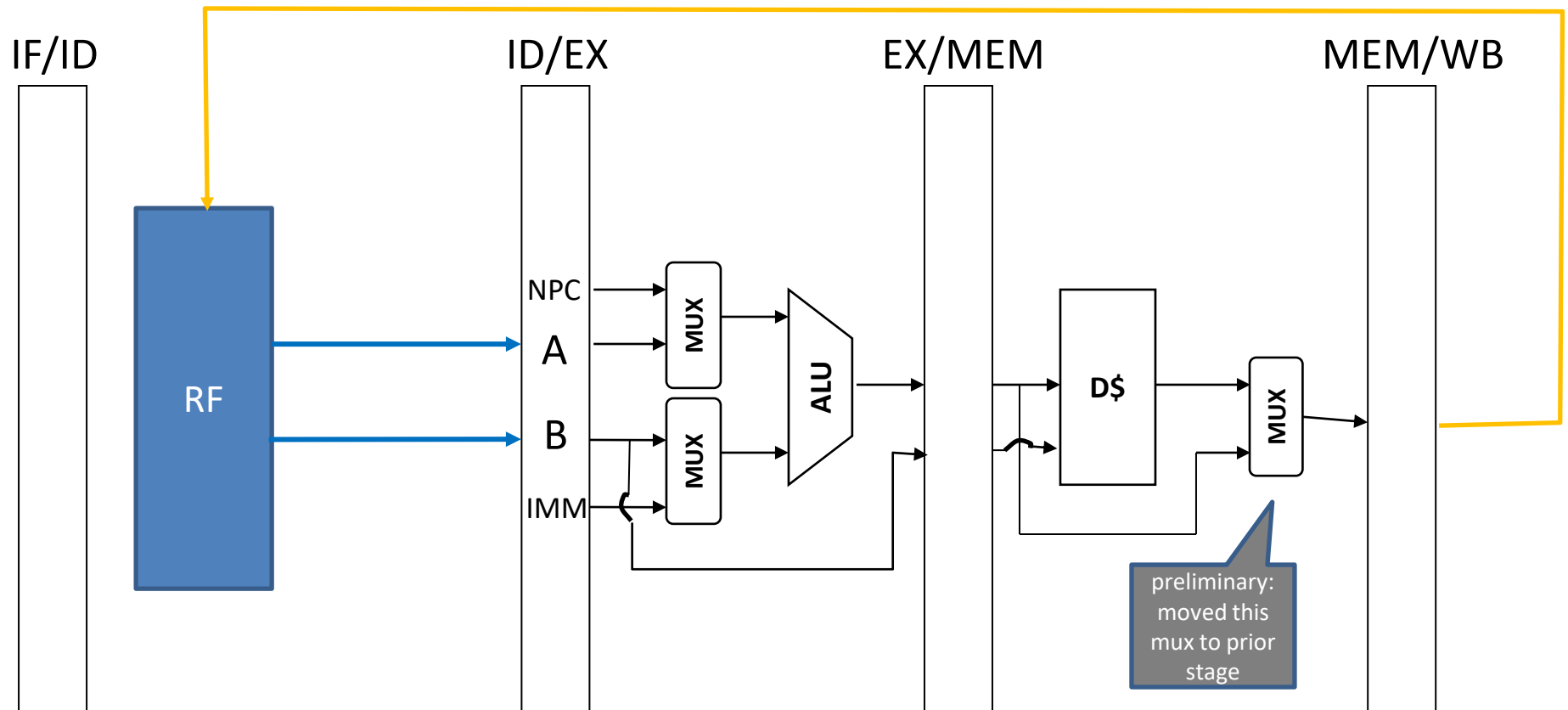
RAW Hazard



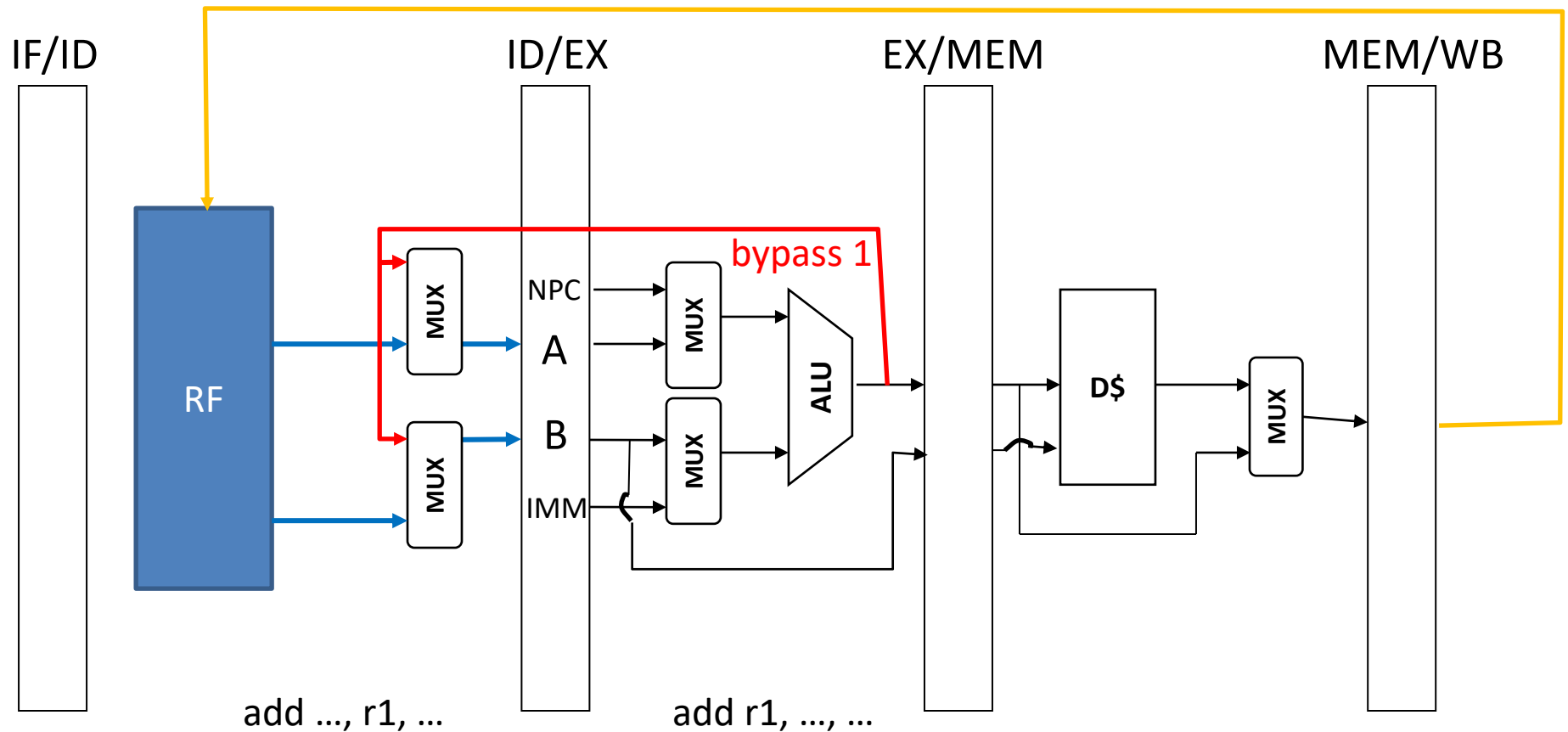
Data forwarding (bypasses)



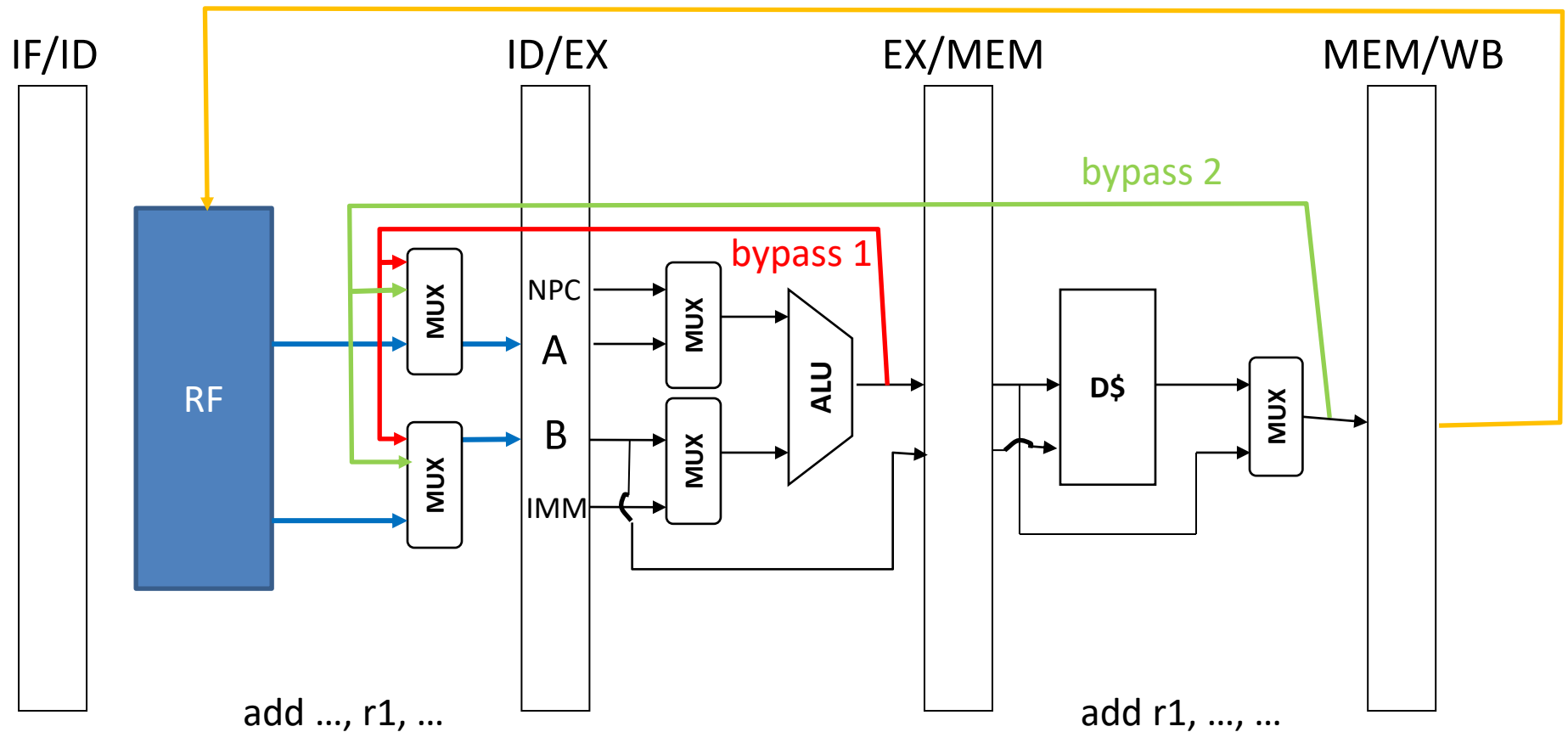
Data forwarding (bypasses)



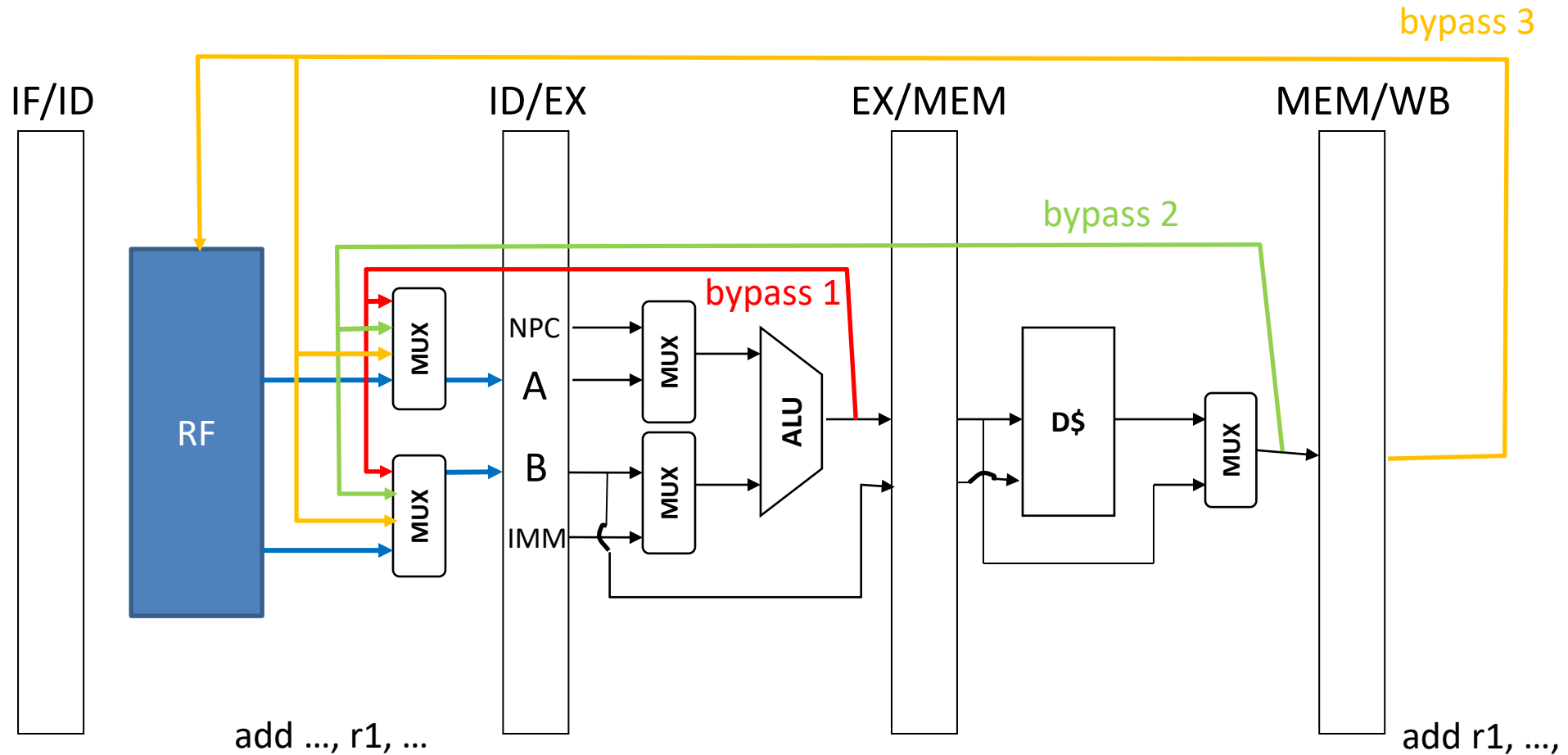
Data forwarding (bypasses)



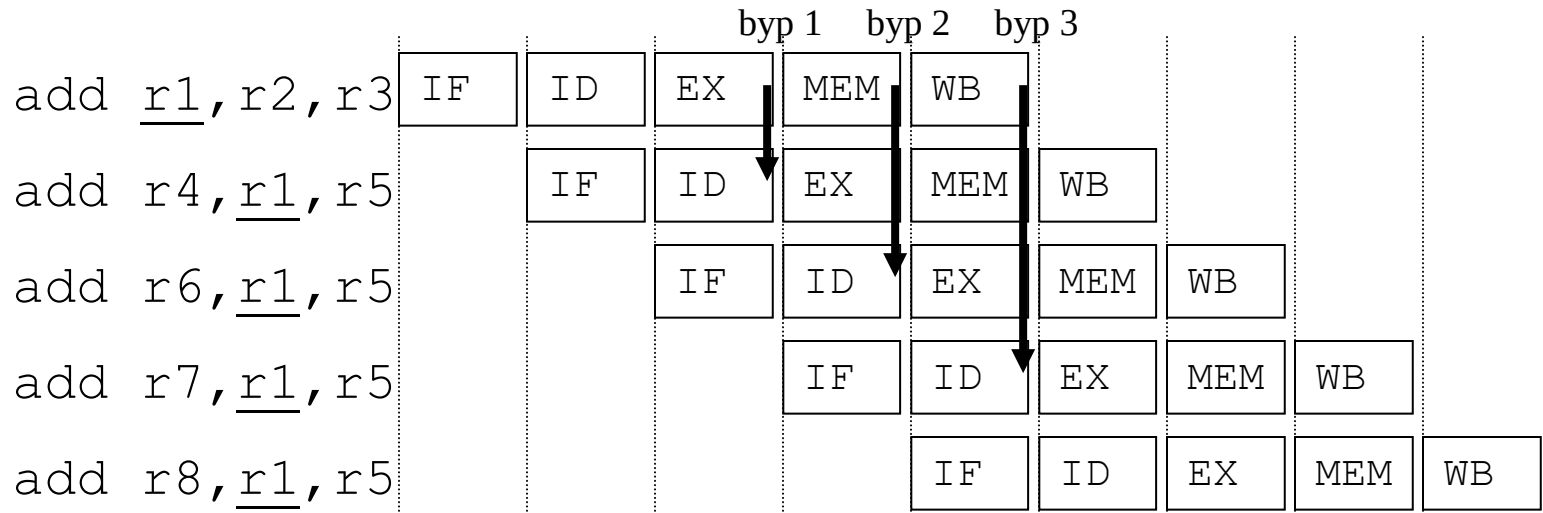
Data forwarding (bypasses)



Data forwarding (bypasses)

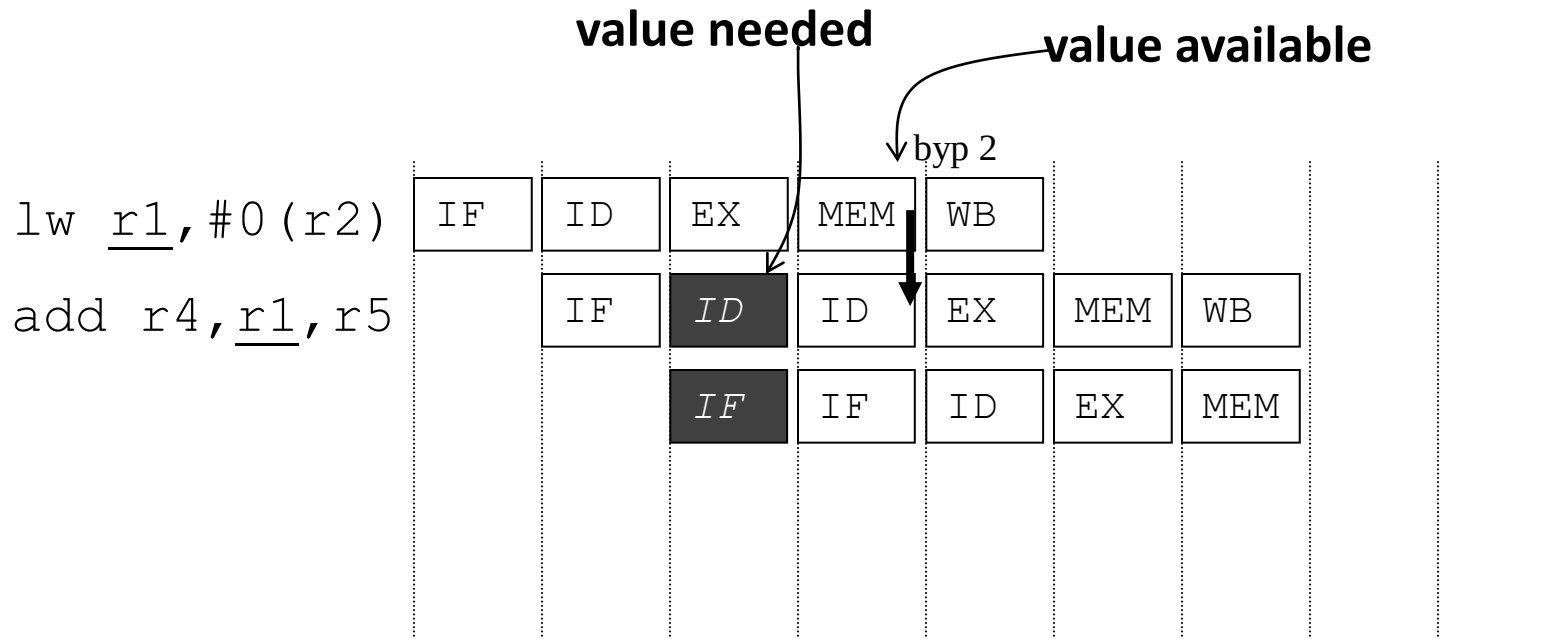


Data forwarding (cont.)



Stalls due to RAW hazards

- With bypasses
 - No stall if producer is ALU type instruction
 - Unavoidable 1-cycle stall if producer is a load instruction (cache hit) AND its dependent instruction is right behind it. This is because the dependent instruction must wait for BYPASS 2 for the load value.
 - Note: If an instruction takes multiple cycles in EX (*e.g.*, complex ALU operation) or multiple cycles in MEM (load/store cache miss), all instructions behind the long-latency instruction stall whether or not they depend on the data. The stall is attributable to a structural hazard caused by the current pipeline design (EX and MEM stages are “blocking” for multi-cycle instructions).



WAR and WAW Hazards

- WAR hazard
 - A: ADD R1, R2, R3
 - B: ADD R2, R4, R5
 - Problem if B writes R2 before A reads R2
 - A will get the wrong value of R2 (that of B)
 - Doesn't happen in most in-order pipelines (like ours)
 - Happens in out-of-order pipelines
 - A may be stalled and B may be ready to execute out-of-order and write R2
- WAW hazard
 - A: ADD R1, R2, R3
 - B: ADD R1, R4, R5
 - Problem if B writes R1 before A writes R1
 - Later instructions will get the wrong value of R1 (that of A)
 - Doesn't happen in most in-order pipelines (like ours)
 - Happens in out-of-order pipelines
 - A may be stalled and B may be ready to execute out-of-order and write R1 first

WAR and WAW Hazards (cont.)

- Options for handling WAR and WAW hazards
 - Stall the later instruction to defer its register write, OR
 - Register renaming (*see next major topic: ILP*)
- Either way, compiler can limit the occurrence of WAR and WAW hazards by limiting the reuse of destination register specifiers among nearby instructions

Types of program dependencies

- True dependence (pure dependence, flow dependence)
 - ADD R1, R2, R3
 - ADD R4, R1, R5
 - Causes RAW hazard if the two instructions are in the pipeline concurrently
- Anti-dependence
 - ADD R1, R2, R3
 - ADD R2, R4, R5
 - Causes WAR hazard if the two instructions are in the pipeline concurrently
- Output dependence
 - ADD R1, R2, R3
 - ADD R1, R4, R5
 - Causes WAW hazard if the two instructions are in the pipeline concurrently

Program dependencies (cont.)

- A *true dependence* is called that, because there is truly a data dependency between the two instructions
 - Producer-consumer relationship
 - Second instruction needs the data produced by the first
- Anti-dependence and output dependence are sometimes clubbed together as *false dependence* because there isn't actually a data dependency between the two instructions, just a register conflict

ECE 463/563

Fall `20

Pipelining:
dynamic branch prediction

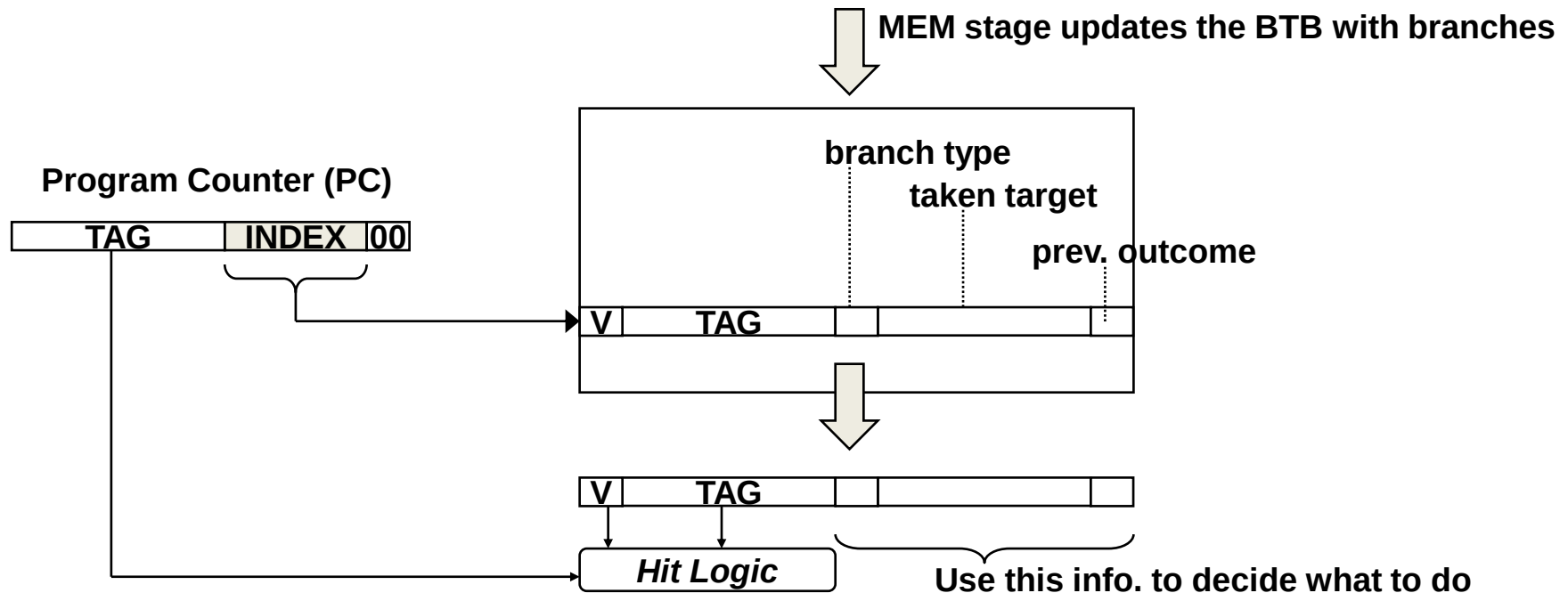
Prof. Eric Rotenberg

Introducing: Branch Prediction

- Current policy in the IF stage
 - Sequential fetch only ($PC = PC + 4$), until redirected by a taken branch in the MEM (branch completion) stage
 - Fetch unit *implicitly* predicts that there is not a branch or, if there is one, that it is not-taken
 - Good: no penalty for not-taken branches
 - Bad: 3-cycle penalty for all taken branches
- Solution
 - Explicit branch prediction in IF stage
 - Predict three things in IF stage:
 - Is the instruction that is being fetched a branch?
 - If so, what is its taken target?
 - Is the branch taken or not-taken?
 - No penalty when prediction is correct
 - 3-cycle penalty when prediction is incorrect (“branch misprediction”)

Branch Target Buffer (BTB)

- Branch Target Buffer (BTB)
 - Predicts branches in the IF stage
 - PC-indexed cache that contains information about previously seen branches
 - May be direct-mapped, set-associative, or fully associative



Training the BTB (“update”)

- Each instruction carries with it a flag that indicates whether it hit or missed in the BTB during IF stage
- When a conditional branch reaches the branch completion stage (MEM in our case), the BTB is updated with information about the branch
 - If the branch is not already in the BTB (*i.e.*, it missed during the IF stage, as indicated by the flag above), then allocate the branch into the BTB:
 - Valid bit and tag (upper bits of PC for identifying the branch)
 - The branch’s type
 - The branch’s taken target
 - The branch’s outcome (taken or not-taken)
 - If the branch is already in the BTB (*i.e.*, it hit during the IF stage, as indicated by the flag above), then only need to update the branch’s “prev. outcome” field and only if it differs:
 - The branch’s outcome (taken or not-taken)

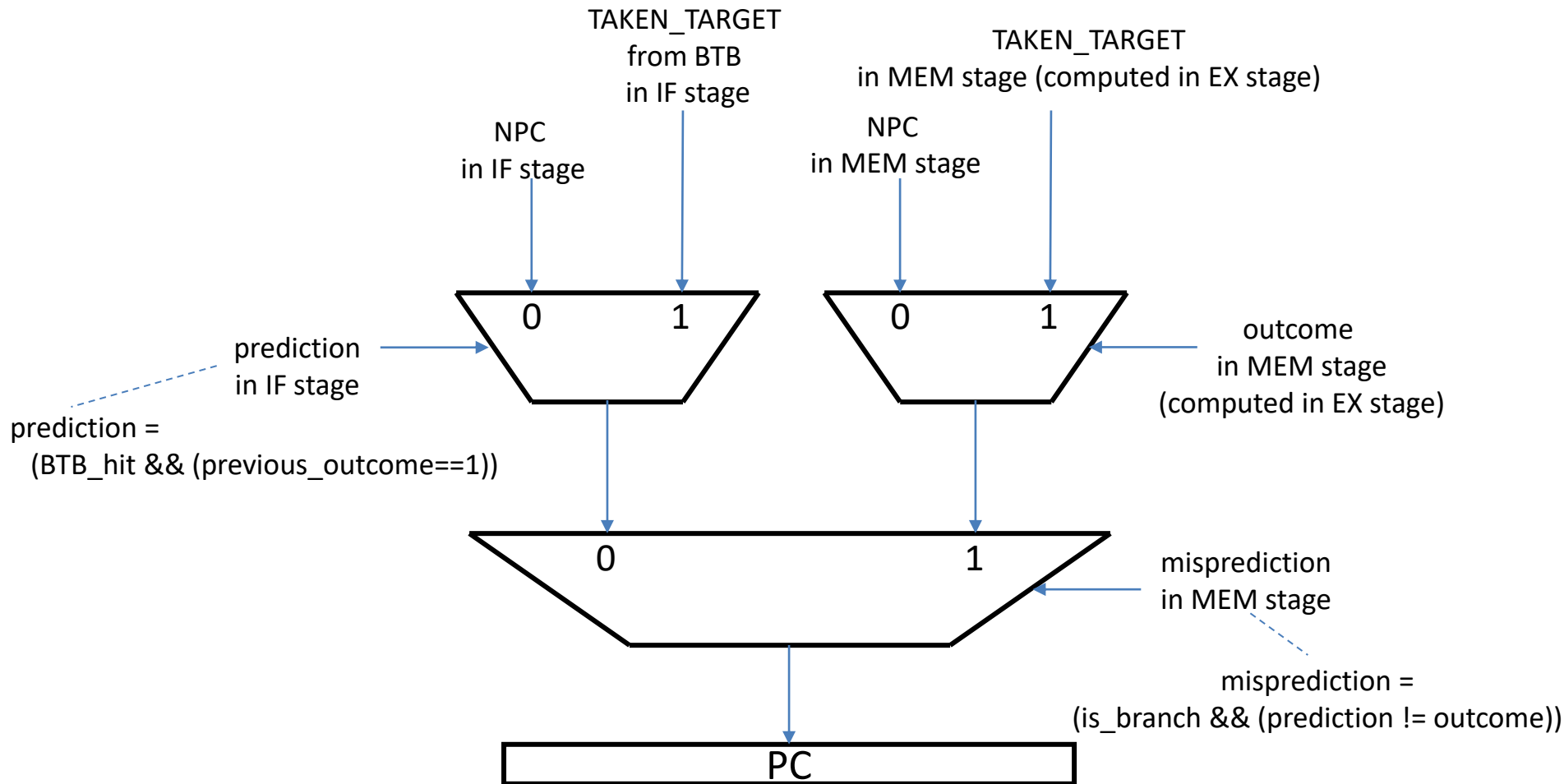
Using the BTB (“predict”)

- IF stage looks up the current PC in the BTB
 - BTB Hit:
 - The instruction being fetched is definitely a branch
 - Use previous outcome to predict the branch’s direction
 - If previously not-taken, then predict not-taken this time
 - If previously taken, then predict taken this time
 - If predicted taken, we have the taken target from the BTB
 - BTB Miss:
 - Assume the instruction being fetched is either not a branch at all or a not-taken branch, hence, predict not-taken.

Detecting and recovering from mispredictions

- Taken/not-taken prediction made in the IF stage flows with the instruction down the pipeline
 - BTB miss: Prediction is not-taken.
 - BTB hit: Prediction is either not-taken or taken, depending on the “previous outcome” field of the BTB entry.
- When a branch reaches the MEM stage (branch completion stage), it compares its prediction against its actual outcome
 - If its prediction does not match its actual outcome, then a misprediction is detected and recovery is initiated: squash younger instructions in IF, ID, and EX, and redirect the fetch unit to the correct PC.
 - If prediction was ‘not-taken’ (used NPC when the branch was predicted in IF) and actual outcome is ‘taken’, redirect the PC to taken target.
 - If prediction was ‘taken’ (used taken target from BTB when the branch was predicted in IF) and actual outcome is ‘not-taken’, redirect the PC to NPC.

Next-PC MUX



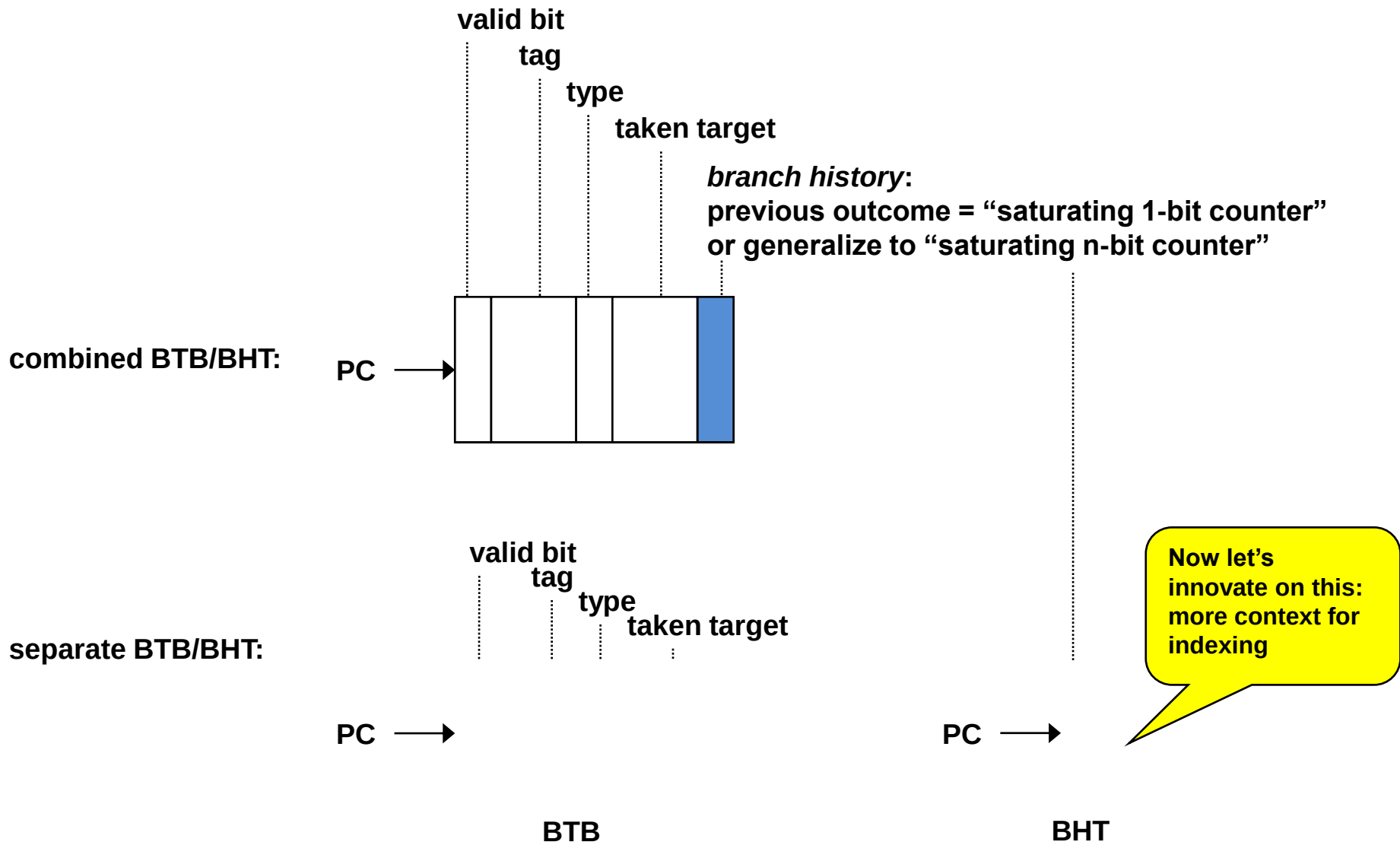
Predicting direction of conditional branches (taken vs. not-taken)

- It's fairly easy to (1) know that you have a branch and (2) know its taken target
 - This is a matter of caching branches in the BTB
 - More branches can be cached by increasing BTB capacity: this is merely a resource issue and a cycle time issue, not a fundamental accuracy issue
- It's much harder to (3) know which direction to take (taken or not-taken?)
 - Why is this part hard?
 - This is not just a capacity issue. Even if all static branches fit in the BTB, the directions of dynamic instances of these branches will still be mispredicted.
 - All you have is past history, and history is not a perfect indicator of the future
 - Predicting the direction of conditional branches has been the subject of much research, in academia and industry
- Two approaches:
 - Hardware branch prediction (“dynamic branch prediction”)
 - Software branch prediction (“static branch prediction”)
 - Heuristics
 - Profiling

Branch History Table (BHT)

- Recall the BTB has a 1-bit field which is the branch's "previous outcome"
 - More generally referred to as "branch history" since it reflects the branch's most recent history
 - Can be thought of as a "saturating 1-bit counter"
 - Increment if branch was taken, but saturate at max value of 1
 - Decrement if branch was not-taken, but saturate at min value of 0
- Let's consider some modifications to branch history
 - Generalize to "saturating n-bit counter"
 - Better accuracy
 - Move the branch history out of the BTB into a separate table called a Branch History Table (BHT)
 - Gives us freedom to develop better taken/not-taken predictors that use other context for indexing (besides just the PC)
 - Better accuracy

Combined vs. Separate BTB / BHT



1-bit counter

- How 1-bit counter works
 - Set counter = 1 if branch was *taken*, 0 if branch was *not-taken*
 - At IF stage, check the 1-bit counter of the branch:
 - if counter = 1 then predict *taken*
 - else predict *not-taken*
 - Essentially predicts the branch will do the same thing it did the last time
 - Problems:
 - Some branches don't do what they did the last time!
 - Need more sophisticated predictor

Example

```
void some_function() {
    for (i = 0; i < 10; i++) {
        ...
        ...
    }
}
```



```
r1 = 0, r2 = 10
LOOP: ...
...
addi r1, r1, #1
bne  r1, r2, LOOP
```

```
int main() {
    while (1)
        some_function();
}
```

Mispredicts fall-through of loop AND first instance

bne r1, r2, LOOP

Actual Outcome	T	T	T	T	T	T	T	T	T	T	N	T	T	T	T	T	T	T	T	N	T
1-bit counter	0	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	0	1
Prediction	N	T	T	T	T	T	T	T	T	T	T	N	T	T	T	T	T	T	T	T	N

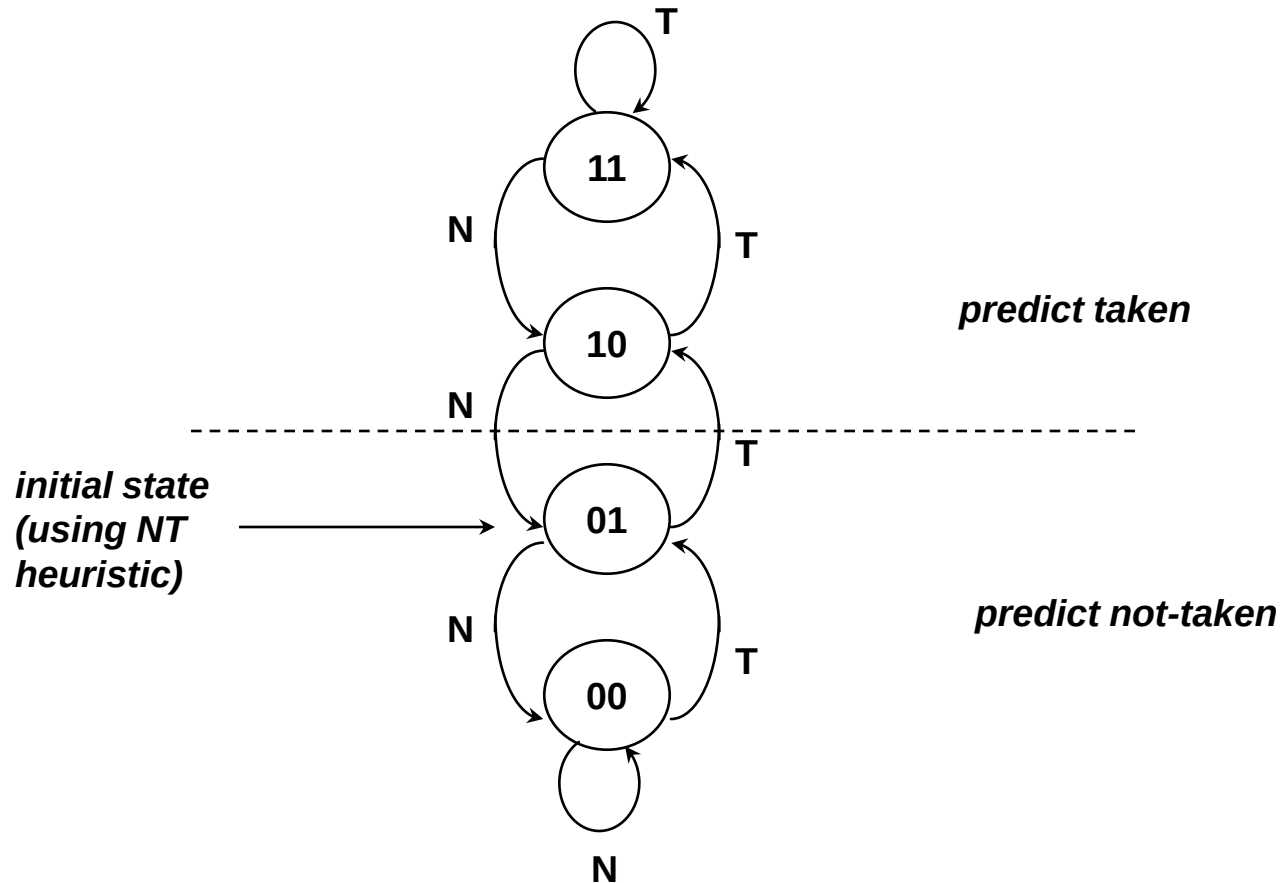
2 mispredicted branches for every 10 branches:
20% misprediction rate (80% accuracy)

Problem with 1-bit counter

- Changes mind too quickly
 - Some branches are highly biased taken or not-taken, with a few isolated changes
 - Perhaps shouldn't change prediction after a single change
- Consider the previous loop example
 - After mispredicting the not-taken branch that exits the loop, the 1-bit counter again mispredicts the first instance of the branch the next time the loop is visited
 - Exiting the loop causes two mispredictions instead of just one
 - Exiting the loop (single not-taken instance) is not the norm, so should predict taken for the first instance of the branch the next time the loop is visited

Smith 2-bit counter

- Replace prediction bit with 2-bit counter:



Weak vs. Strong States

- Middle states are referred to as “weak states”
 - 01: “weakly not-taken”
 - 10: “weakly taken”
- Saturated states are referred to as “strong states”
 - 00: “strongly not-taken”
 - 11: “strongly taken”
- The distinction reflects the fact that:
 - From a strong state, it takes two mispredictions to change prediction
 - From a weak state, it takes only one misprediction to change prediction
- Makes sense to initialize counters to a weak state
 - Don’t have any training yet
 - Want to train quickly
 - Quicker to change prediction when in weak state

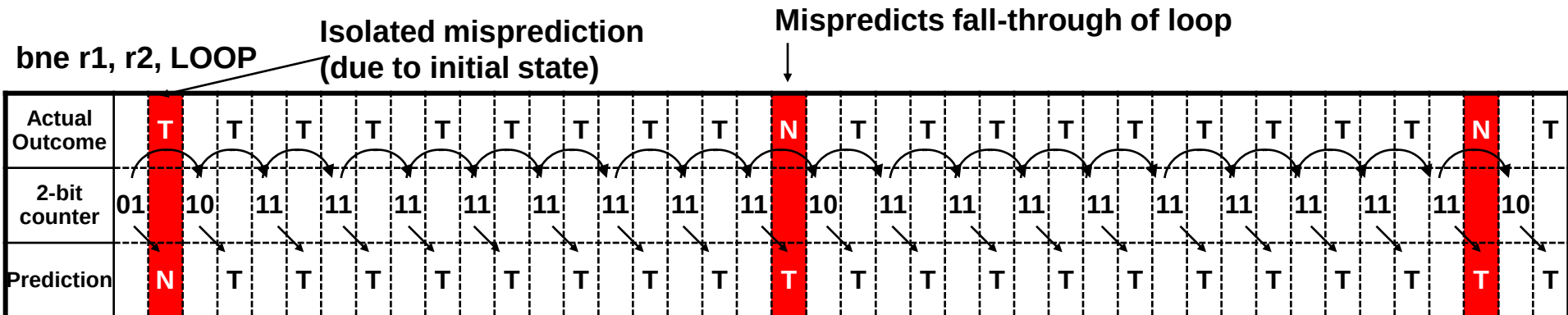
Revisit Example with 2-bit counter

```
void some_function() {
    for (i = 0; i < 10; i++) {
        ...
        ...
    }
}
```



```
r1 = 0, r2 = 10
LOOP: ...
...
addi r1, r1, #1
bne r1, r2, LOOP
```

```
int main() {
    while (1)
        some_function();
}
```



1 mispredicted branch for every 10 branches:
10% misprediction rate (90% accuracy)

Smith n-bit counter

- In general, can use n-bit counter
 - Potential problems with $n > 2$
 - Smith called it “inertia”

Another Example: Toggle Branch

- Suppose initial state = 10 (weakly taken)

Actual Outcome	T	N	T	N	T	N	T	N	T	N
2-bit counter	10	11	10	11	10	11	10	11	10	11
Prediction	T	T	T	T	T	T	T	T	T	T

5 mispredicted branches for every 10 branches: 50% misprediction rate (50% accuracy)
(Not so bad for an unbiased branch.)

- Suppose initial state = 01 (weakly not-taken)

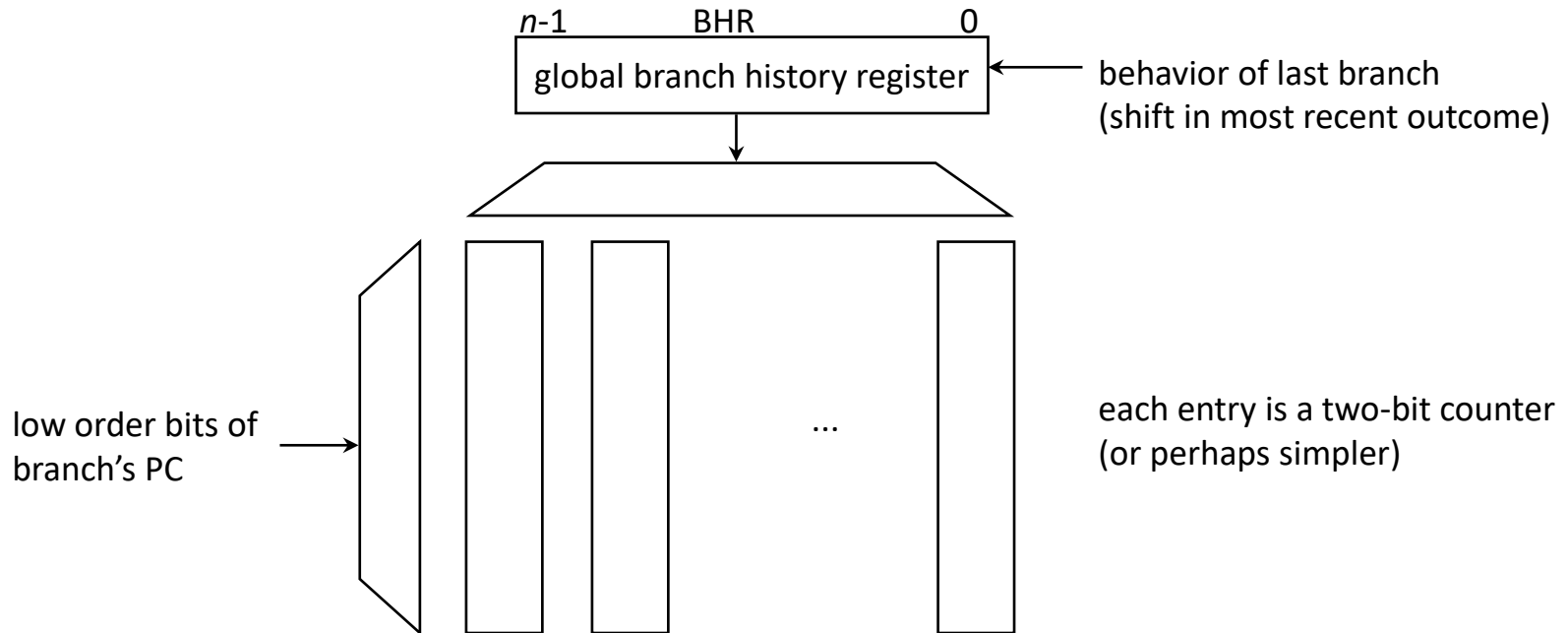
Actual Outcome	T	N	T	N	T	N	T	N	T	N
2-bit counter	01	10	01	10	01	10	01	10	01	10
Prediction	N	T	N	T	N	T	N	T	N	T

10 mispredicted branches for every 10 branches: 100% misprediction rate (0% accuracy)
(Pretty bad.)

Next Leap in Accuracy

- First wave of branch prediction, circa 1980
 - Smith counter
- Second wave of branch prediction, circa 1990
 - Yeh & Patt; Pan *et al.*; McFarling
 - *Global branch history*
 - Exploit correlation among different branches
 - Exploit recurring patterns in the global branch history (all branches)
 - *Local branch history*
 - Exploit recurring patterns in local branch histories (individual branches)

gselect branch predictor



dual toggle-branch example

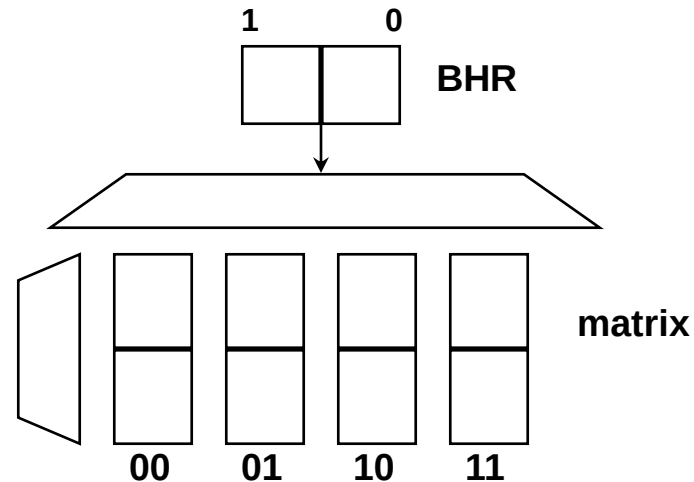
```
r1=0, r2=0  
A: beq r1,r2,D  
B: ...  
C: ...  
D: beq r1,r2,F  
E: ...  
F: xori r1,r1,#1  
G: jump A
```

PC	A	D	A	D	A	D	A	D
outcome	T	T	N	N	T	T	N	N

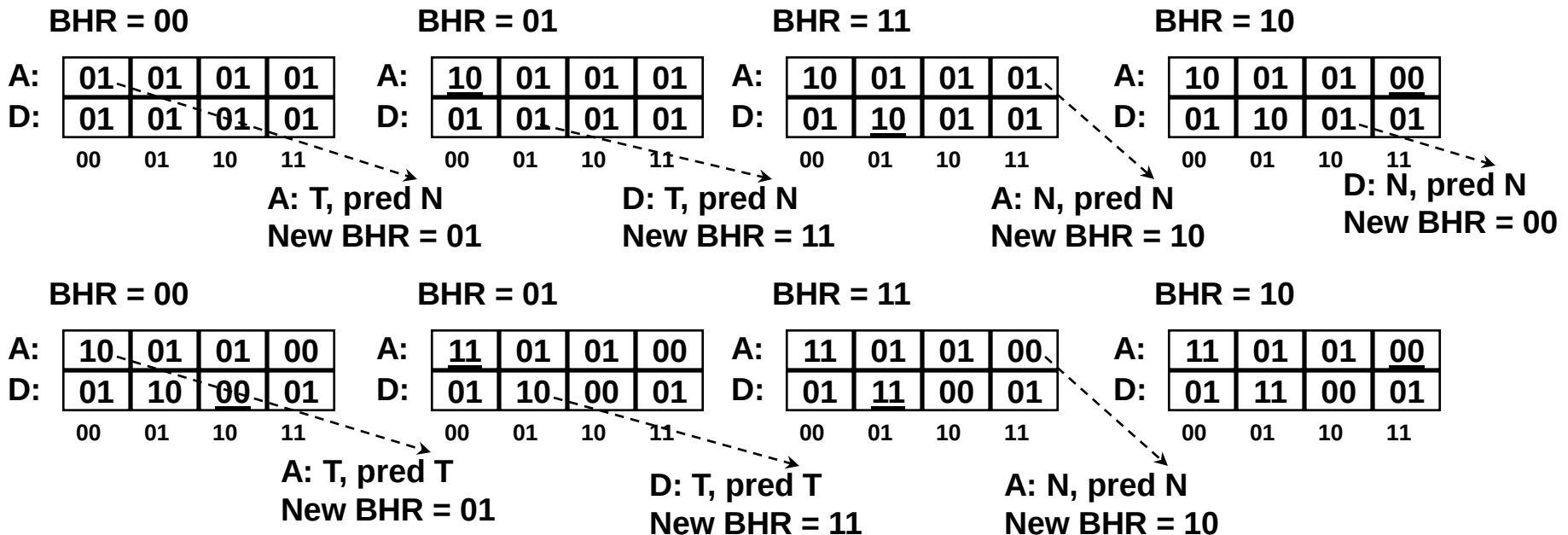
example *gselect* predictor

- Select row using branch PC
 - In simulation that follows, only the two rows indexed by A and D are shown for clarity
- Select column using a global branch history register (BHR) of length two bits (arbitrary design choice)
 - BHR contains the taken/not-taken outcomes of the two most-recent dynamic branches
 - Therefore, this *gselect* predictor has four columns (indexed by two bits of global branch history)

gselect example



underlined means entry was updated due to last branch execution



gselect example (1)

- Predict 1st instance of A:
 - Predict not-taken (MISPREDICTION)

BHR is 00

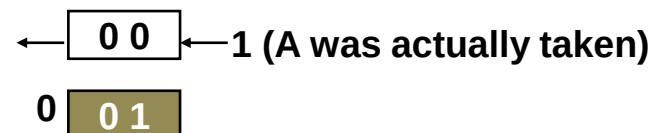
		00:	01:	10:	11:
PC is A	A:	01	01	01	01
	D:	01	01	01	01

- Update for 1st instance of A
 - Actually taken

BHR is 00

		00:	01:	10:	11:
PC is A	A:	10	01	01	01
	D:	01	01	01	01

- Update the BHR to set up next prediction



gselect example (2)

- Predict 1st instance of D:
 - Predict not-taken (MISPREDICTION)

BHR is 01

		00:	01:	10:	11:
PC is D	A:	10	01	01	01
	D:	01	01	01	01

- Update for 1st instance of D
 - Actually taken

BHR is 01

		00:	01:	10:	11:
PC is D	A:	10	01	01	01
	D:	01	10	01	01

- Update the BHR to set up next prediction



gselect example (3)

- Predict 2nd instance of A:
 - Predict not-taken (CORRECT)

BHR is 11

		00:	01:	10:	11:
PC is A	A:	10	01	01	01
	D:	01	10	01	01

- Update for 2nd instance of A
 - Actually not-taken

BHR is 11

		00:	01:	10:	11:
PC is A	A:	10	01	01	00
	D:	01	10	01	01

- Update the BHR to set up next prediction



gselect example (4)

- Predict 2nd instance of D:
 - Predict not-taken (CORRECT)

BHR is 10

		00:	01:	10:	11:
PC is D	A:	10	01	01	00
	D:	01	10	01	01

- Update for 2nd instance of D
 - Actually not-taken

BHR is 10

		00:	01:	10:	11:
PC is D	A:	10	01	01	00
	D:	01	10	00	01

- Update the BHR to set up next prediction



gselect example (5)

- Predict 3rd instance of A:
 - Predict taken (CORRECT)

BHR is 00

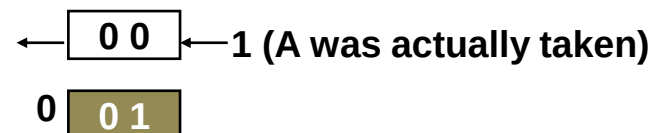
		00:	01:	10:	11:
PC is A	A:	10	01	01	00
	D:	01	10	00	01

- Update for 3rd instance of A
 - Actually taken

BHR is 00

		00:	01:	10:	11:
PC is A	A:	11	01	01	00
	D:	01	10	00	01

- Update the BHR to set up next prediction



gselect example (6)

- Predict 3rd instance of D:
 - Predict taken (CORRECT)

BHR is 01

		00:	01:	10:	11:
PC is D	A:	11	01	01	00
	D:	01	10	00	01

- Update for 3rd instance of D
 - Actually taken

BHR is 01

		00:	01:	10:	11:
PC is D	A:	11	01	01	00
	D:	01	11	00	01

- Update the BHR to set up next prediction



gselect example (7)

- Predict 4th instance of A:
 - Predict not-taken (CORRECT)

PC is A

		BHR is 11			
		00:	01:	10:	11:
A:		11	01	01	00
D:		01	11	00	01

- Update for 4th instance of A
 - Actually not-taken

PC is A

		BHR is 11			
		00:	01:	10:	11:
A:		11	01	01	00
D:		01	11	00	01

- Update the BHR to set up next prediction



gselect example (8)

- Predict 4th instance of D:
 - Predict not-taken (CORRECT)

BHR is 10

		00:	01:	10:	11:
PC is D	A:	11	01	01	00
	D:	01	11	00	01

- Update for 4th instance of D
 - Actually not-taken

BHR is 10

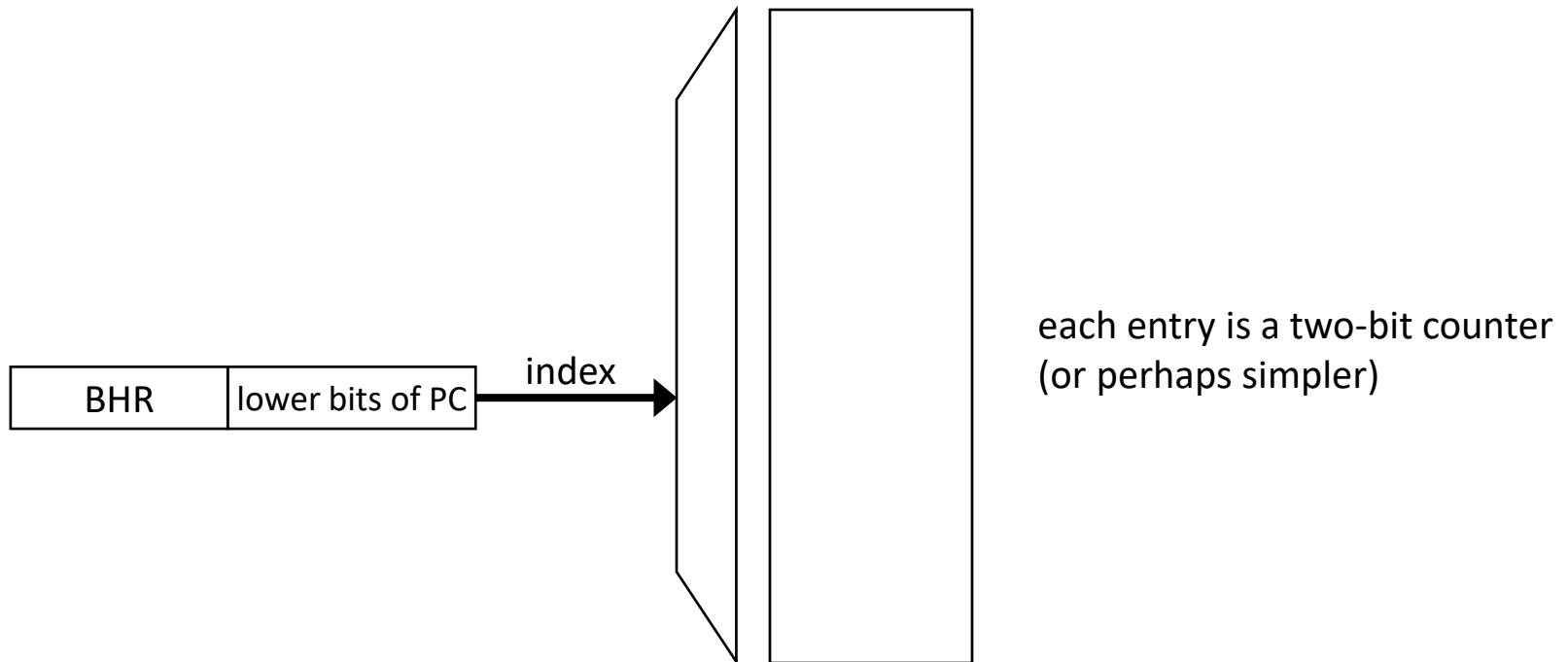
		00:	01:	10:	11:
PC is D	A:	11	01	01	00
	D:	01	11	00	01

- Update the BHR to set up next prediction

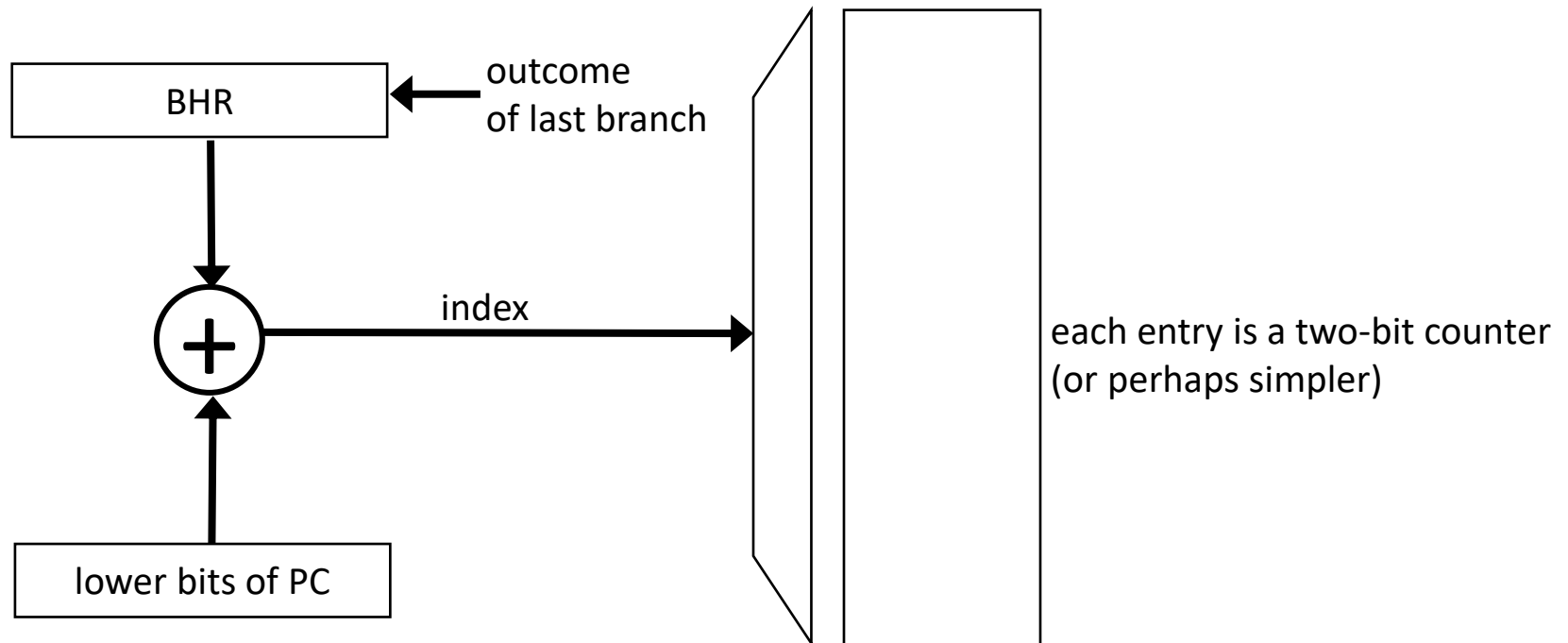


gselect implementation

- *gselect*'s indexing method is tantamount to concatenating PC and BHR



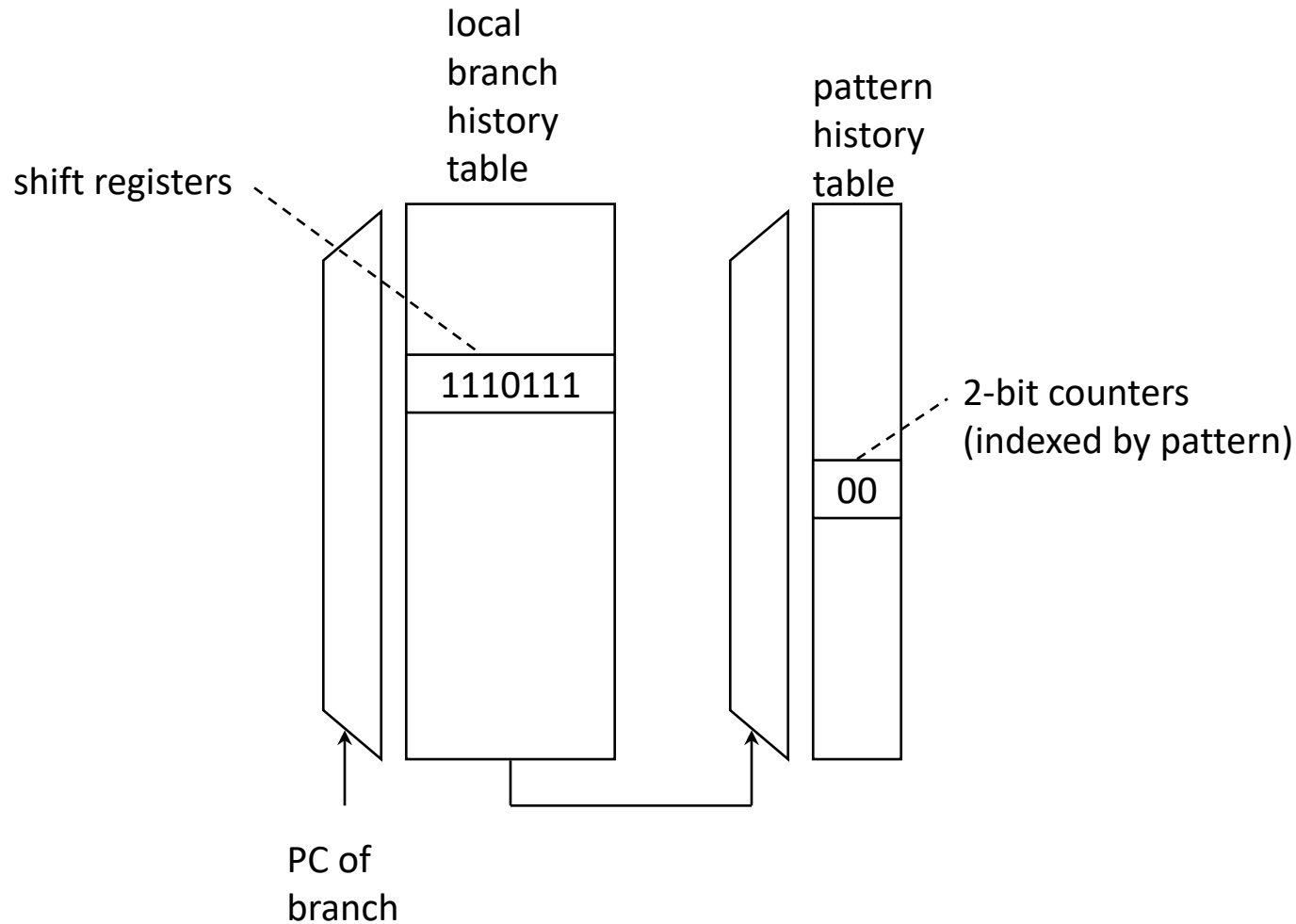
gshare branch predictor



gshare vs. *gselect*: rationale

- Compared to *gselect*, *gshare* enables using more PC bits and more BHR bits, for the same total number of index bits
- Hopefully combining bits with XOR preserves valuable information from both PC and BHR

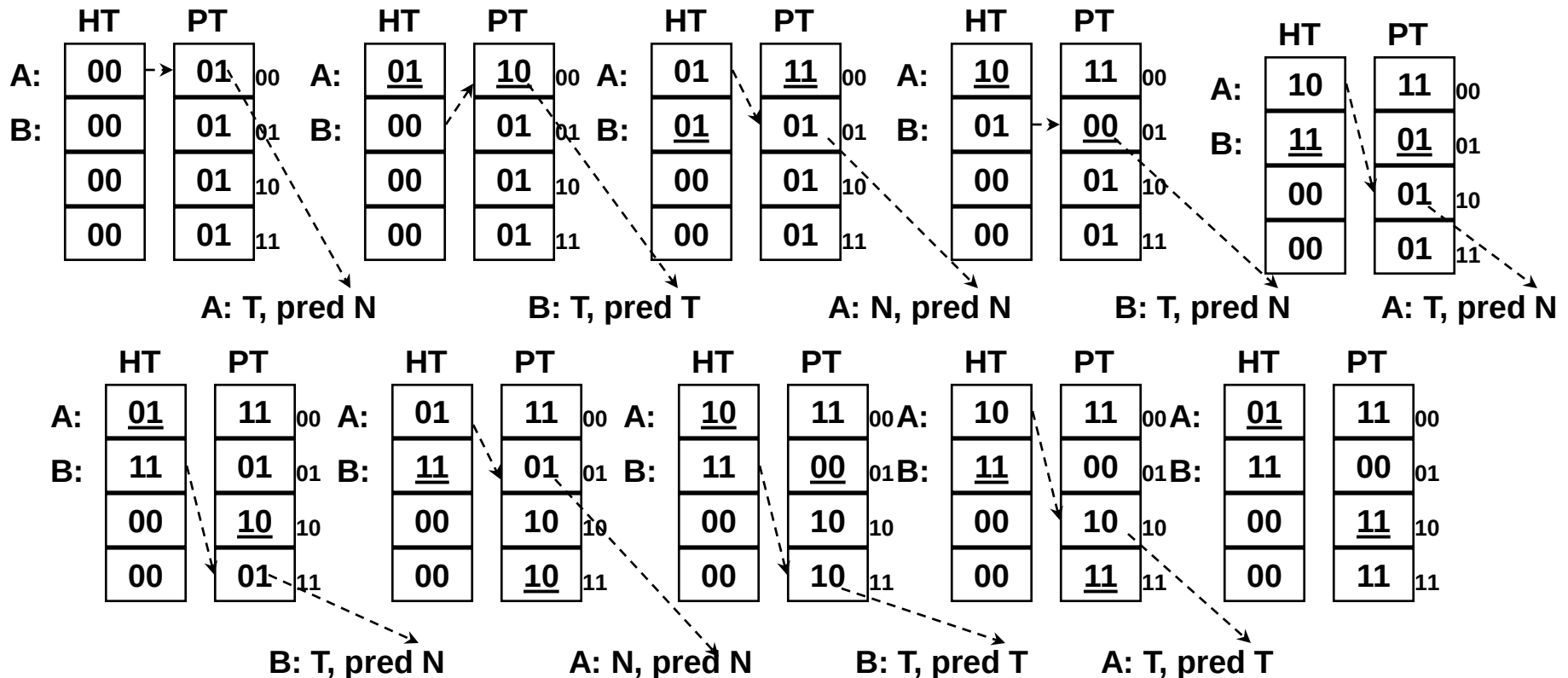
Yeh/Patt branch predictor



Yeh/Patt Example: toggle branch

A: TNTNTNTNTNTNTNTNTNTN

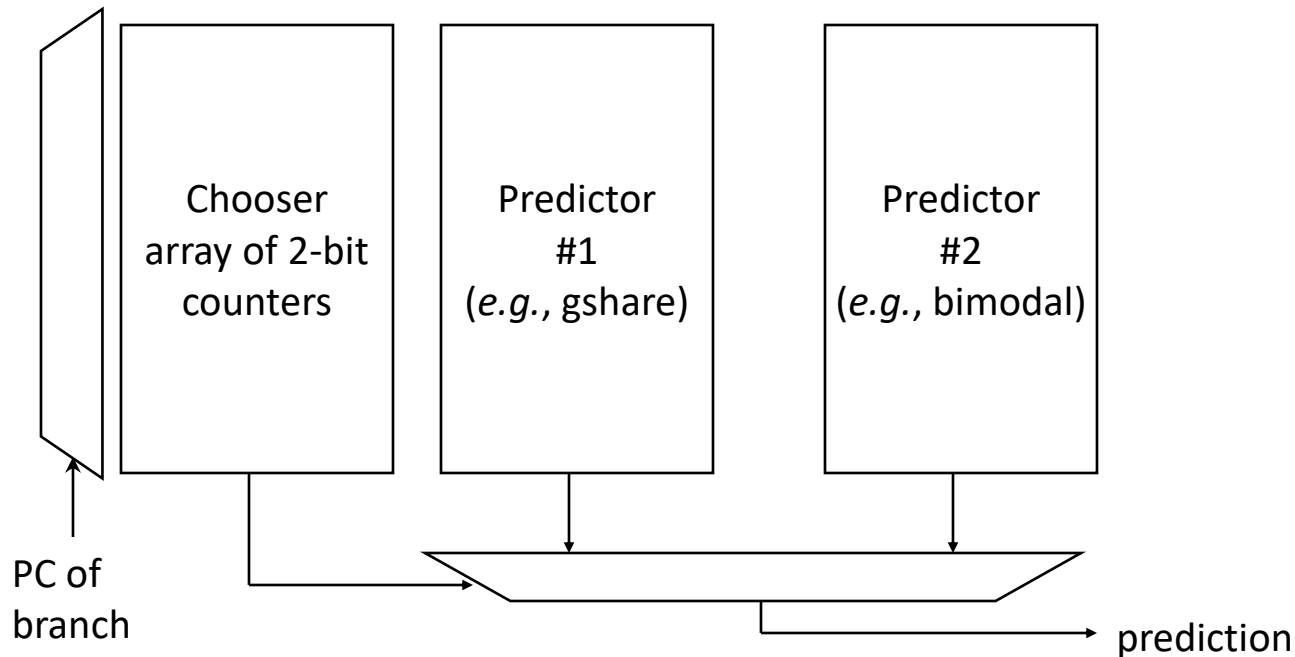
B: TTTTTTTTTTTTTTTTTTTTTT



In general: provides 96-98% accuracy for integer code

PT entries 01, 10 are "trained" for A and 11 is "trained" for B

hybrid predictors



- Both predictors supply a prediction. Fetch unit uses only one as selected by chooser.
- Chooser updated based on which predictor was correct
 - Increment chooser counter if #1 was correct, decrement if #2 was correct
 - For detailed implementation, see Project #2 spec.

ECE 463/563

Fall `22

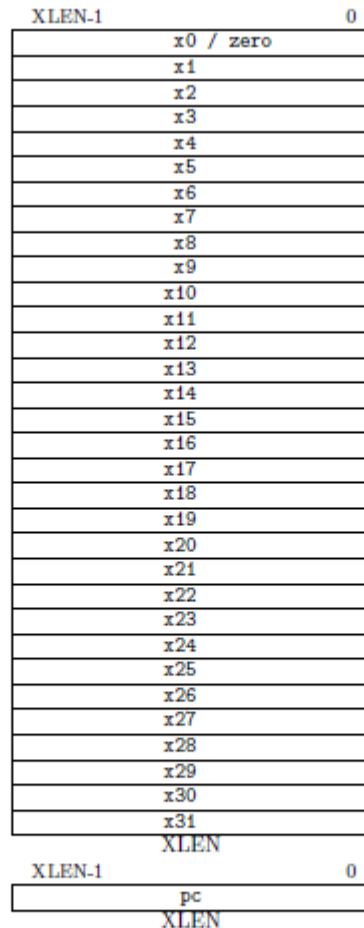
RISC-V instruction formats
Design RISC-V unpipelined datapath
Introduce RISC-V pipelined datapath

Prof. Eric Rotenberg

Designing a processor

- Classify instructions for RISC-V:
 - Memory references (loads and stores)
 - Register-Register ALU Operations
 - Register-Immediate ALU Operations
 - Branches
- Work out the execution for each instruction class
- Design appropriate hardware
- Look for opportunities to improve...
- ...while maintaining correct execution

RISC-V General-Purpose Integer Registers



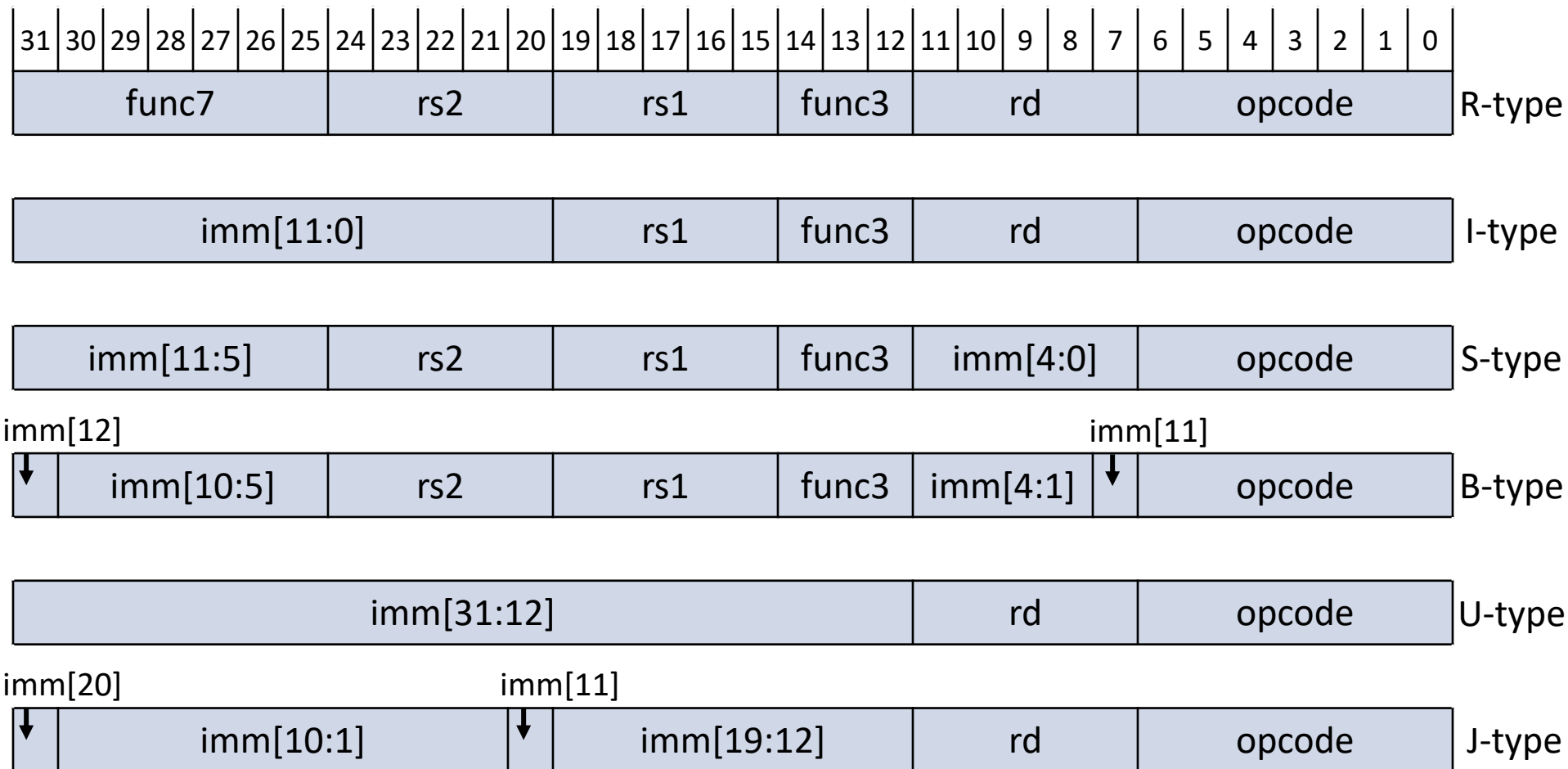
RISCV32: XLEN = 32

RISCV64: XLEN = 64

Note: RISC-V also has an optional floating-point extension that defines 32 general-purpose floating-point registers (f0-f31) and instructions that operate on them.

Figure 2.1: RISC-V user-level base integer register state.

RISC-V Instruction Formats



Some Observations

- The formats were designed with various pipeline design considerations in mind, for example...
- RISC-V architects wanted to keep rs1, rs2, and rd, in the same locations across all formats that use them, so that reading from the register file can proceed in parallel with decoding the opcode.*
- Across all formats with imm, the most-significant-bit (msb) of imm is always at bit 31, so that sign-extension of imm to a full 32-bit (RV32) or 64-bit (RV64) immediate operand can begin before decoding the opcode

* This mostly helps simple pipelines. My opinion: for superscalars, decode and register renaming will likely be serialized anyway to facilitate renaming a bundle of instructions in parallel.

format	instruction class	notation	effect
R-type	register-register ALU operations	<i>op</i> rd, rs1, rs2 (e.g., add)	$RF[rd] = RF[rs1] \text{ op } RF[rs2]$
I-type	register-immediate ALU operations	<i>op</i> rd, rs1, #imm (e.g., addi)	$RF[rd] = RF[rs1] \text{ op } \text{sign_extend}(\text{imm})$
	loads	l <i>size</i> rd, #imm(rs1) (sizes: lb, lh, lw, ld) (also: signed or unsigned)	$\text{addr} = RF[rs1] + \text{sign_extend}(\text{imm})$ $RF[rd] = \text{MEM}[\text{addr}]$
	JR, JALR		
S-type	stores	s <i>size</i> #imm(rs1), rs2 (sizes: sb, sh, sw, sd)	$\text{addr} = RF[rs1] + \text{sign_extend}(\text{imm})$ $\text{MEM}[\text{addr}] = RF[rs2]$
B-type	conditional branches	b <i>op</i> rs1, rs2, #imm (e.g., beq, bne, blt, etc.)	$\text{condition} = (RF[rs1] \text{ op } RF[rs2])$ $\text{target} = \text{PC} + 4 + \text{sign_extend}(\text{imm})$ $\text{PC} = (\text{condition} ? \text{target} : \text{PC} + 4)$
U-type	LUI, AUIPC		
J-type	J, JAL		

How to Execute an Instruction

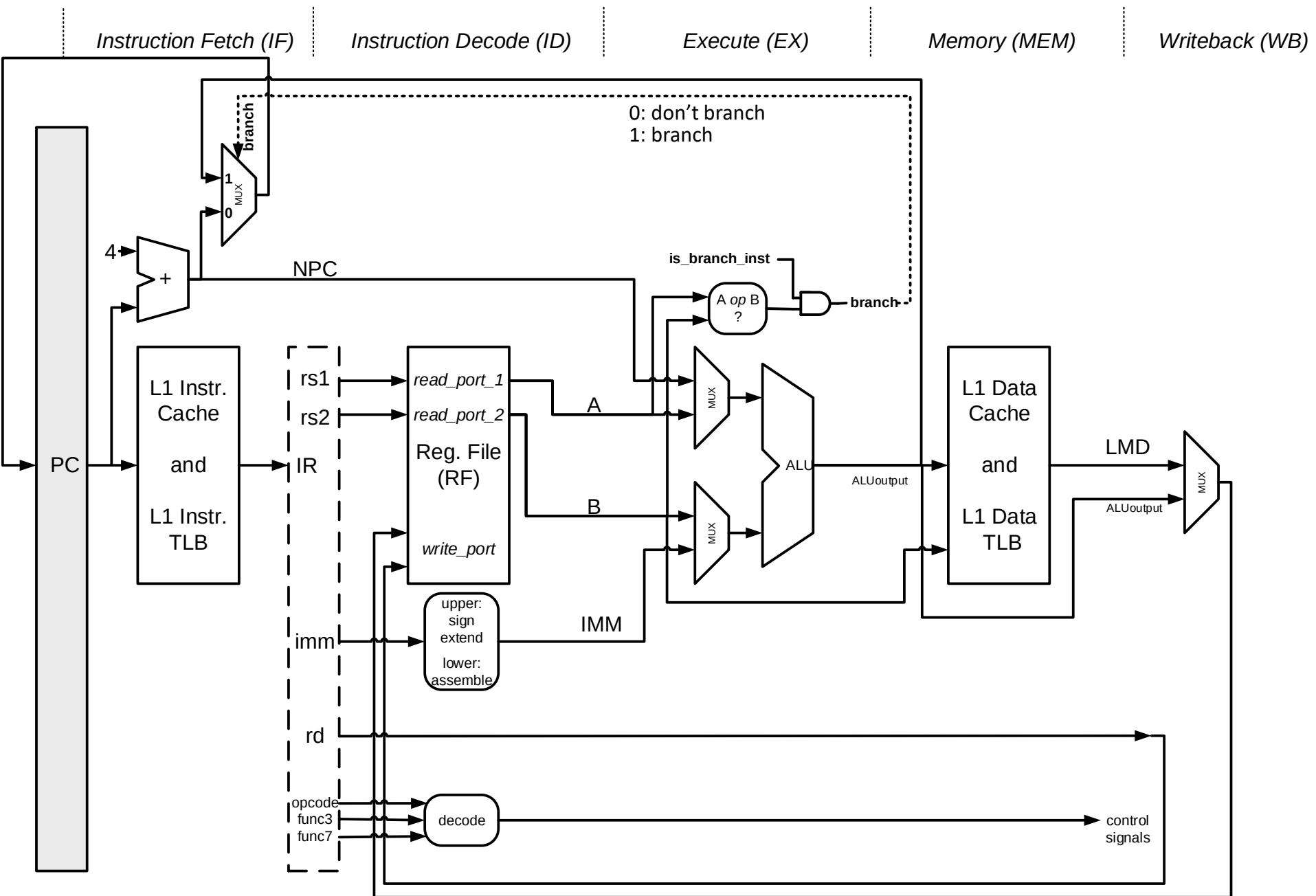
- Instruction fetch (“IF”)
 - $IR = MEM[PC]$
 - $NPC = PC + 4$
- Instruction decode/Register read (“ID”)
 - $A = RF[rs1]$
 - $B = RF[rs2]$
 - $IMM = \text{sign_extend}(imm)$
 - $\text{control} = \text{decode}(\text{opcode}, \text{func3}, \text{func7})$
 - Resulting decoded control signals flow with instr. to later units, to orchestrate the datapath

Executing an Instruction (cont.)

- Execute (“EX”)
 - load or store:
 - $ALUOutput = A + IMM$
 - register-register ALU operation:
 - $ALUOutput = A \text{ op } B$
 - register-immediate ALU operation:
 - $ALUOutput = A \text{ op } IMM$
 - branch:
 - $ALUOutput = NPC + IMM$
 - In addition, evaluate whether to branch or not:
 - $branch = (is_branch_inst) \ \&\& \ (A \text{ op } B)$

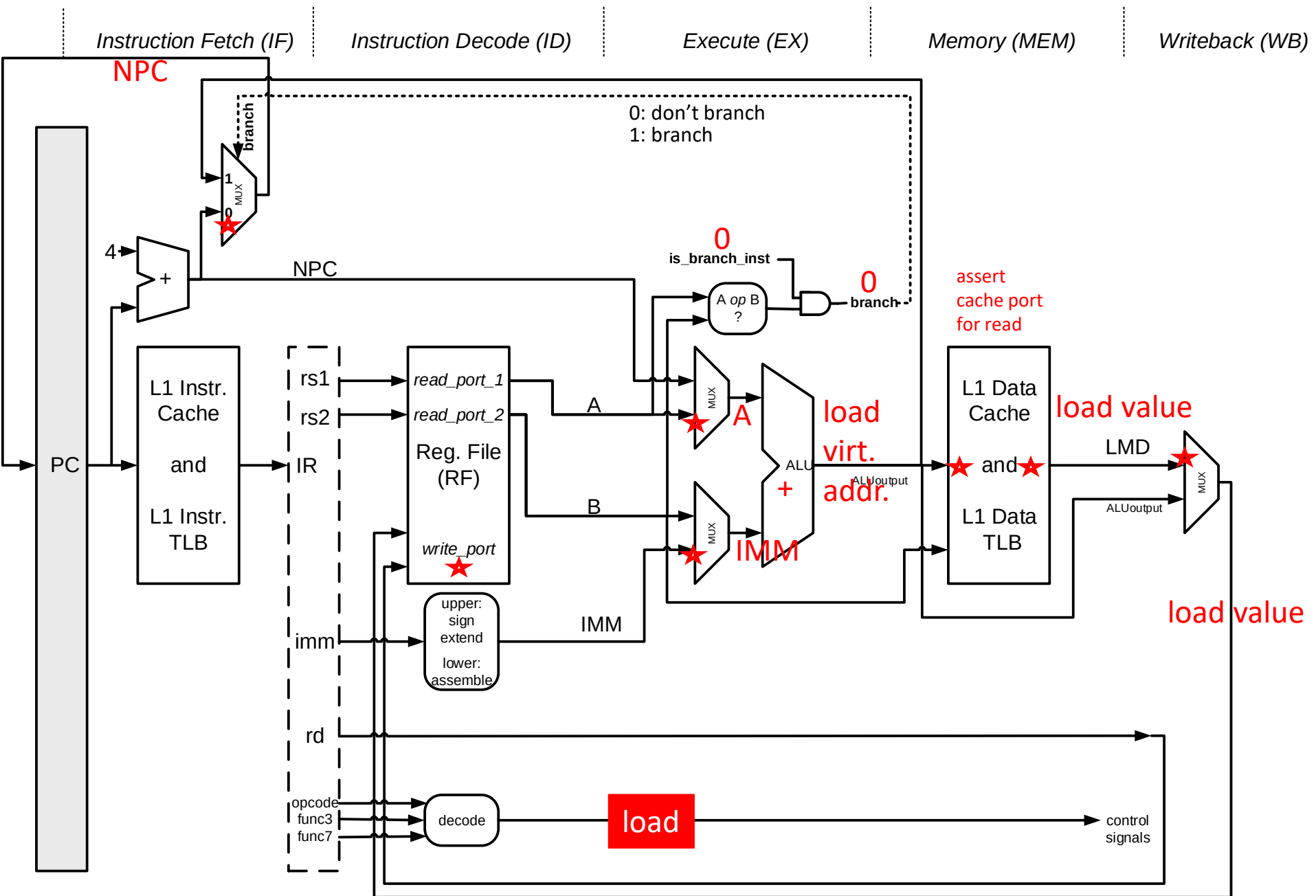
Executing an Instruction (cont.)

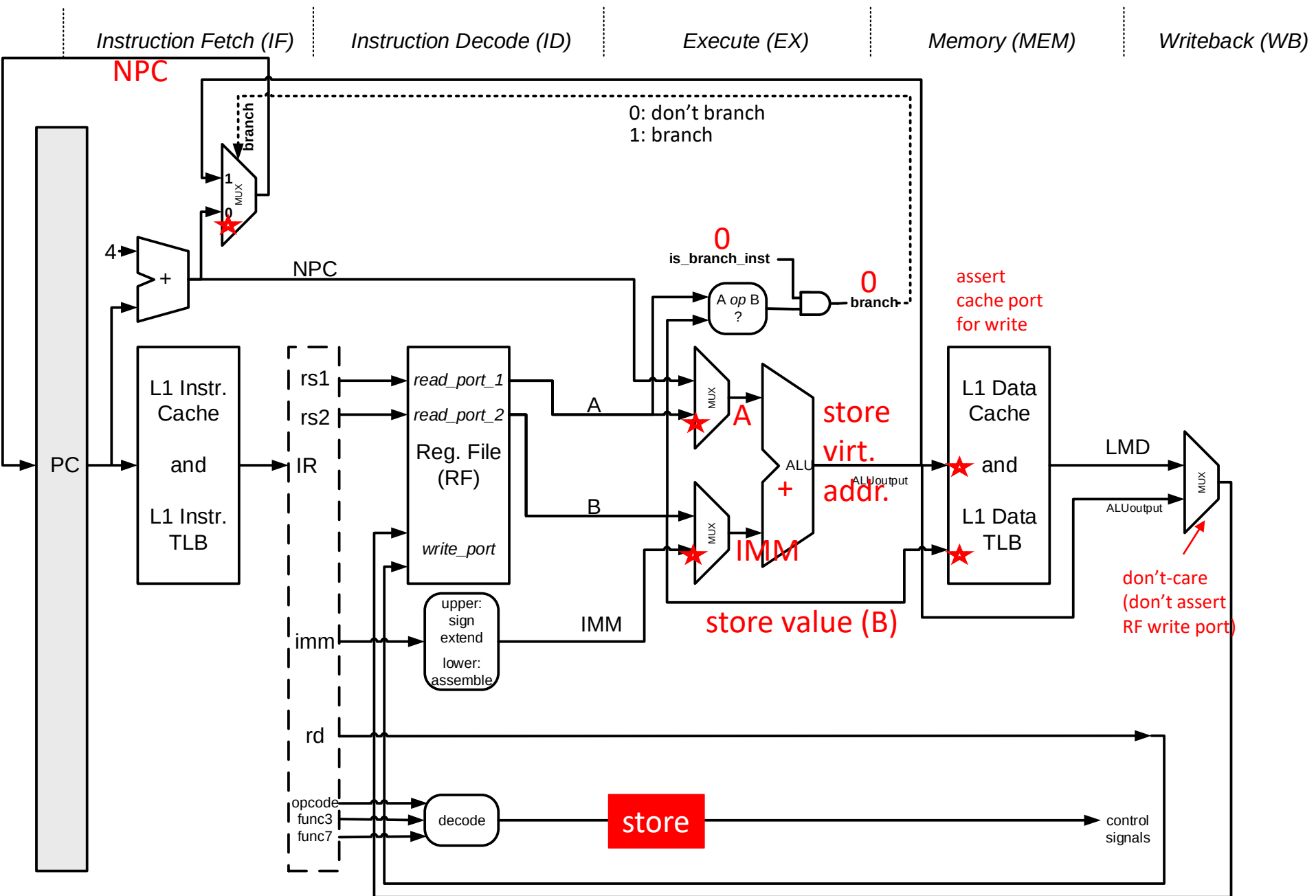
- Memory access/Branch completion (“MEM”)
 - Memory access:
 - $\text{Load_Mem_Data} = \text{MEM}[\text{ALUOutput}]$ */* load */*
 - $\text{MEM}[\text{ALUOutput}] = B$ */* store */*
 - Branch completion:
 - $\text{PC} = (\text{branch} ? \text{ALUOutput} : \text{NPC})$
- Write back (“WB”)
 - register-register or register-immediate ALU operation:
 - $\text{RF}[\text{rd}] = \text{ALUOutput}$
 - load instruction:
 - $\text{RF}[\text{rd}] = \text{Load_Mem_Data}$

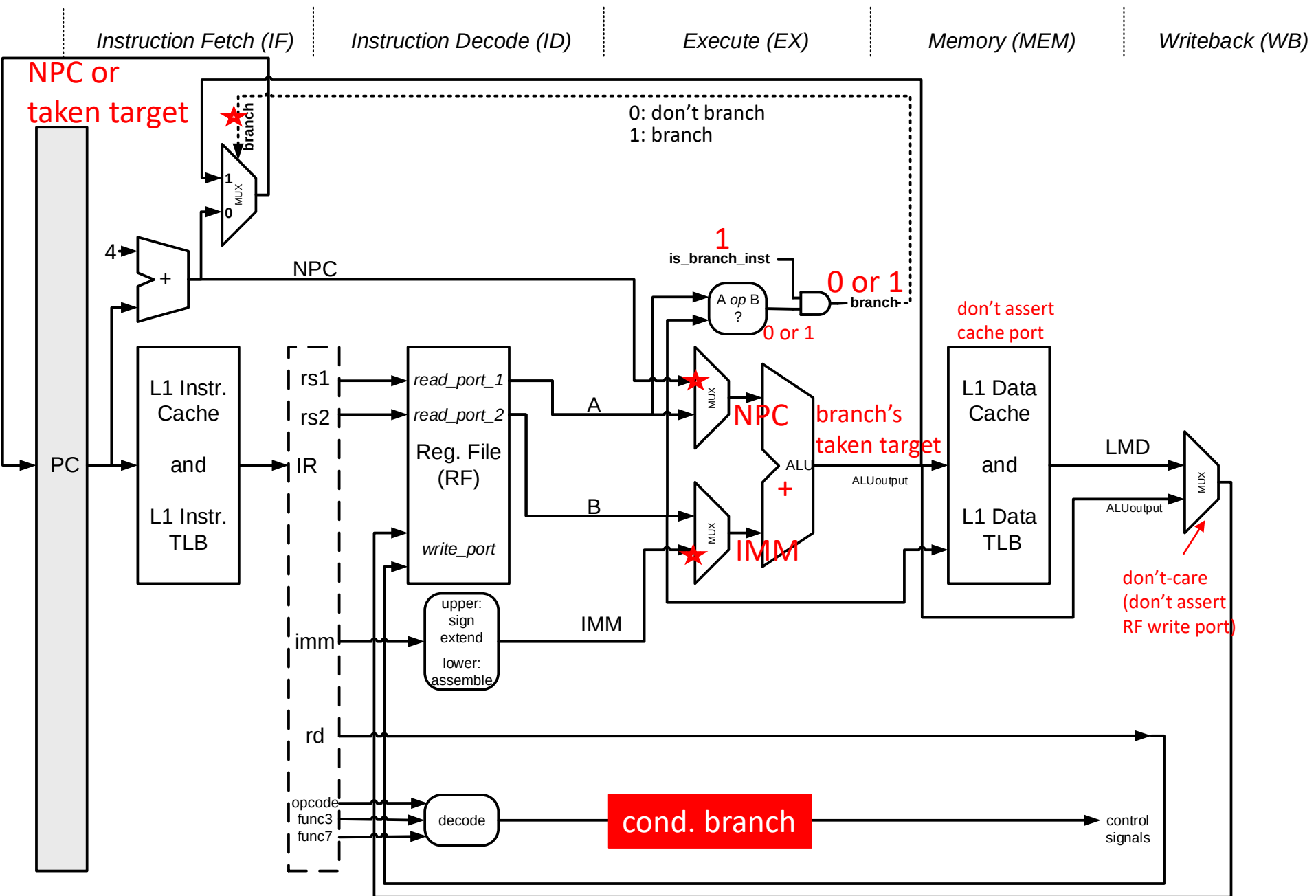


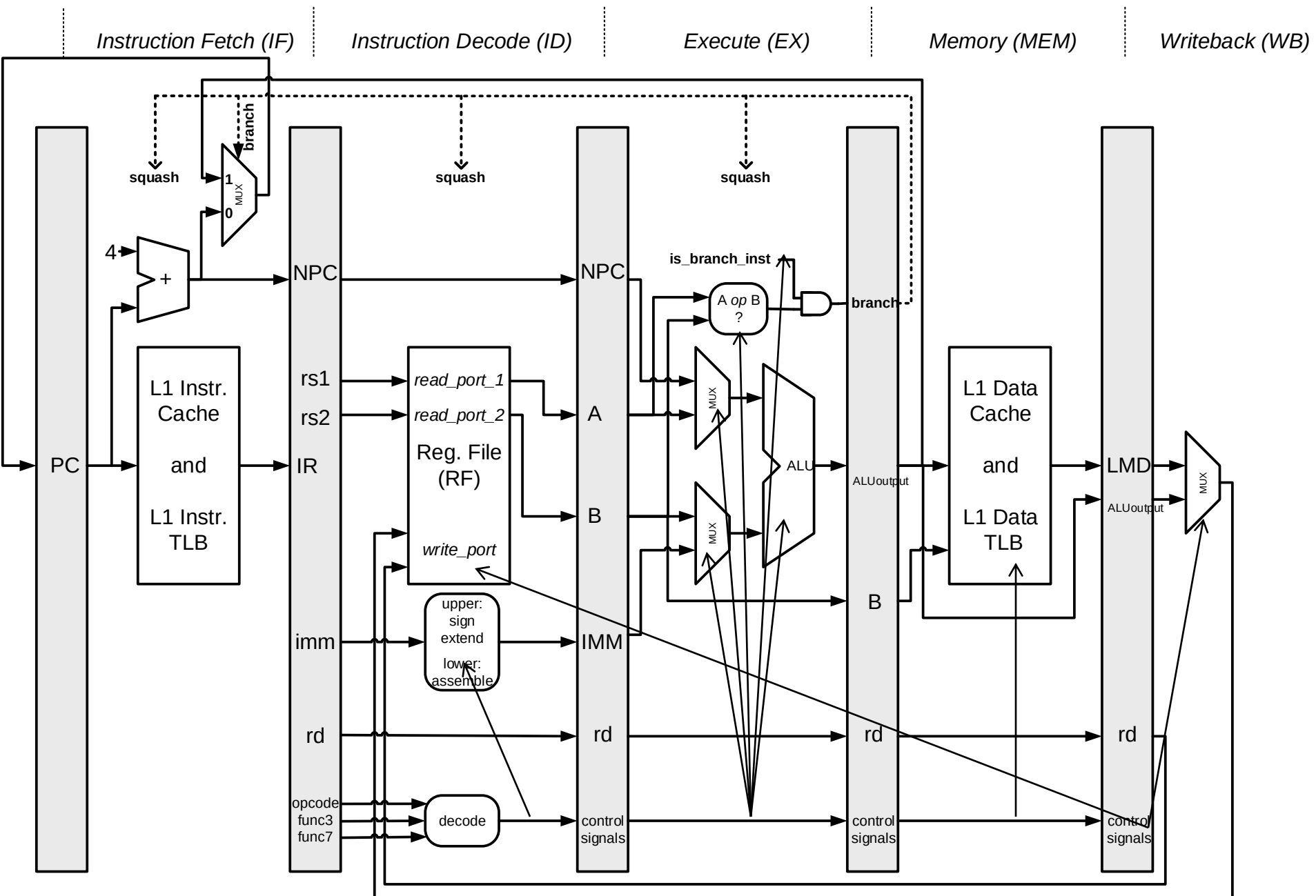


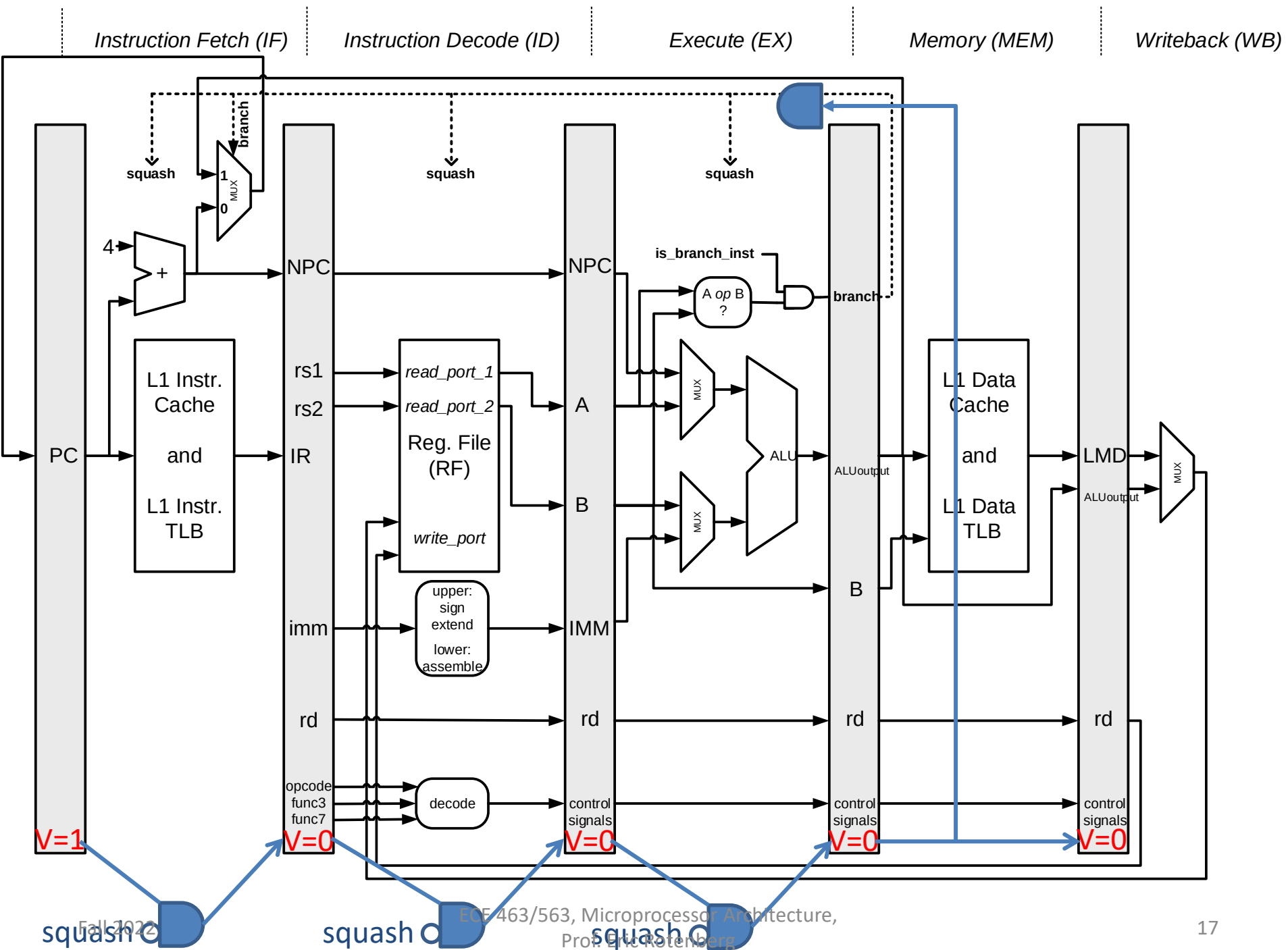


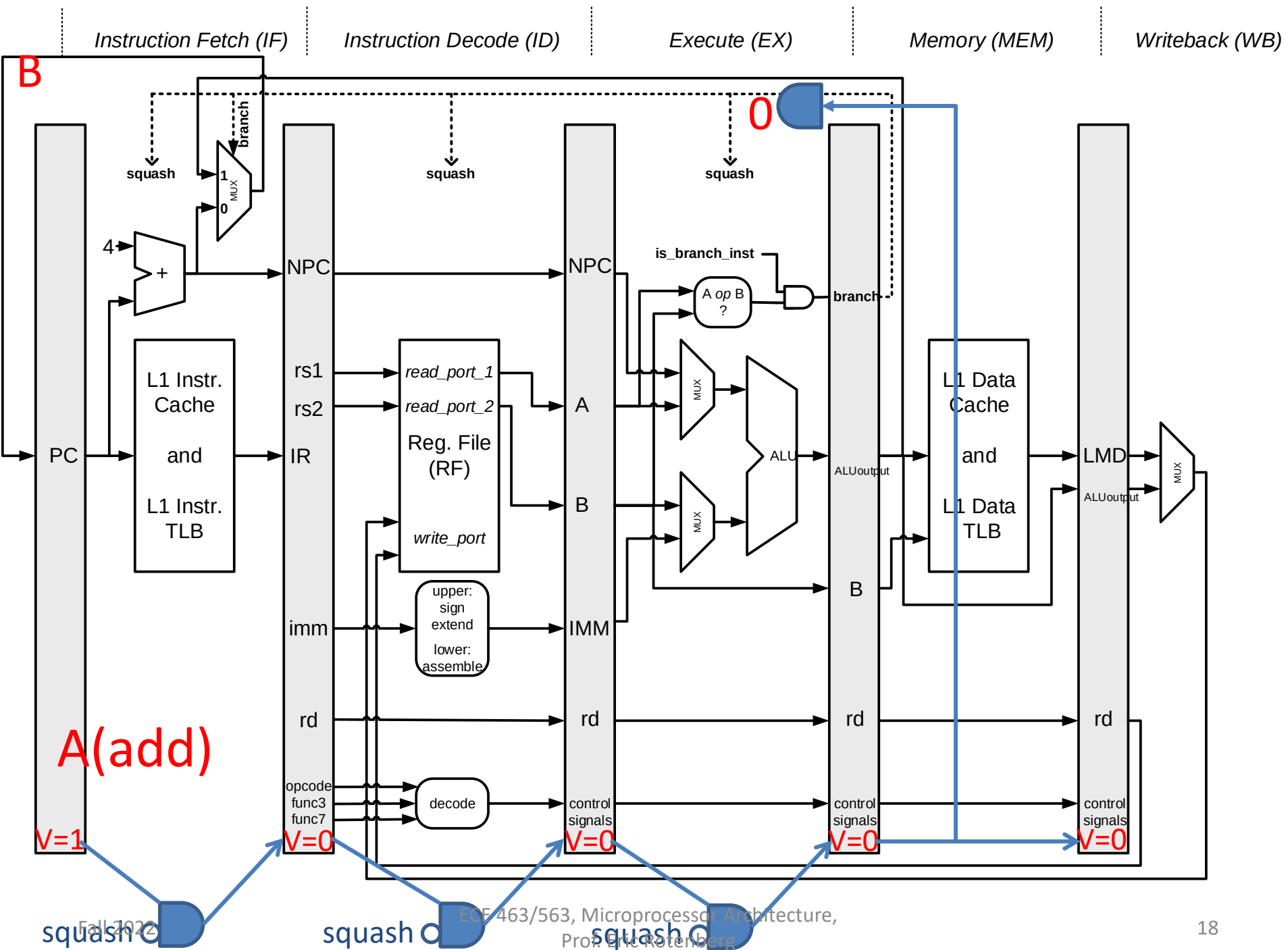


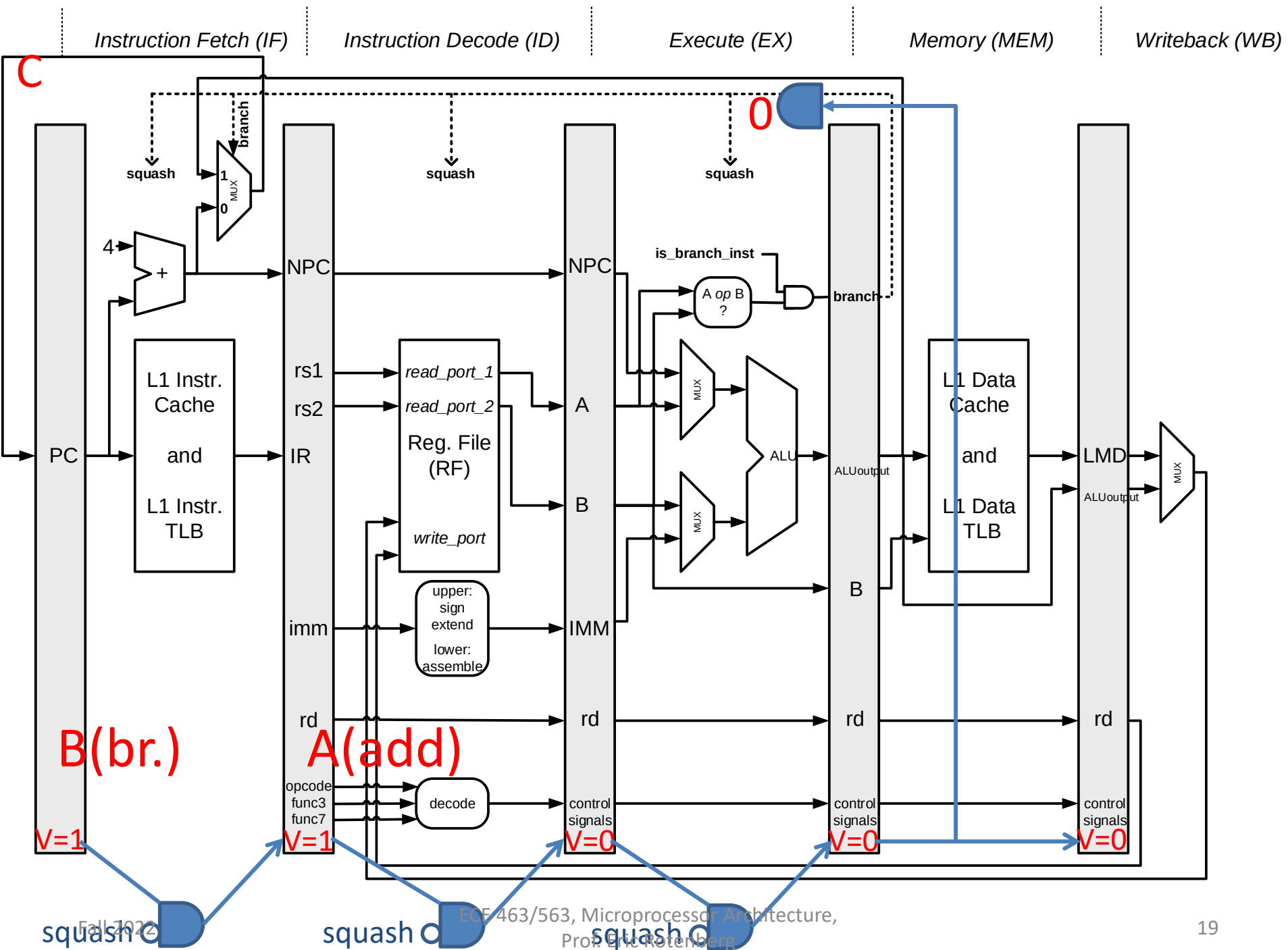


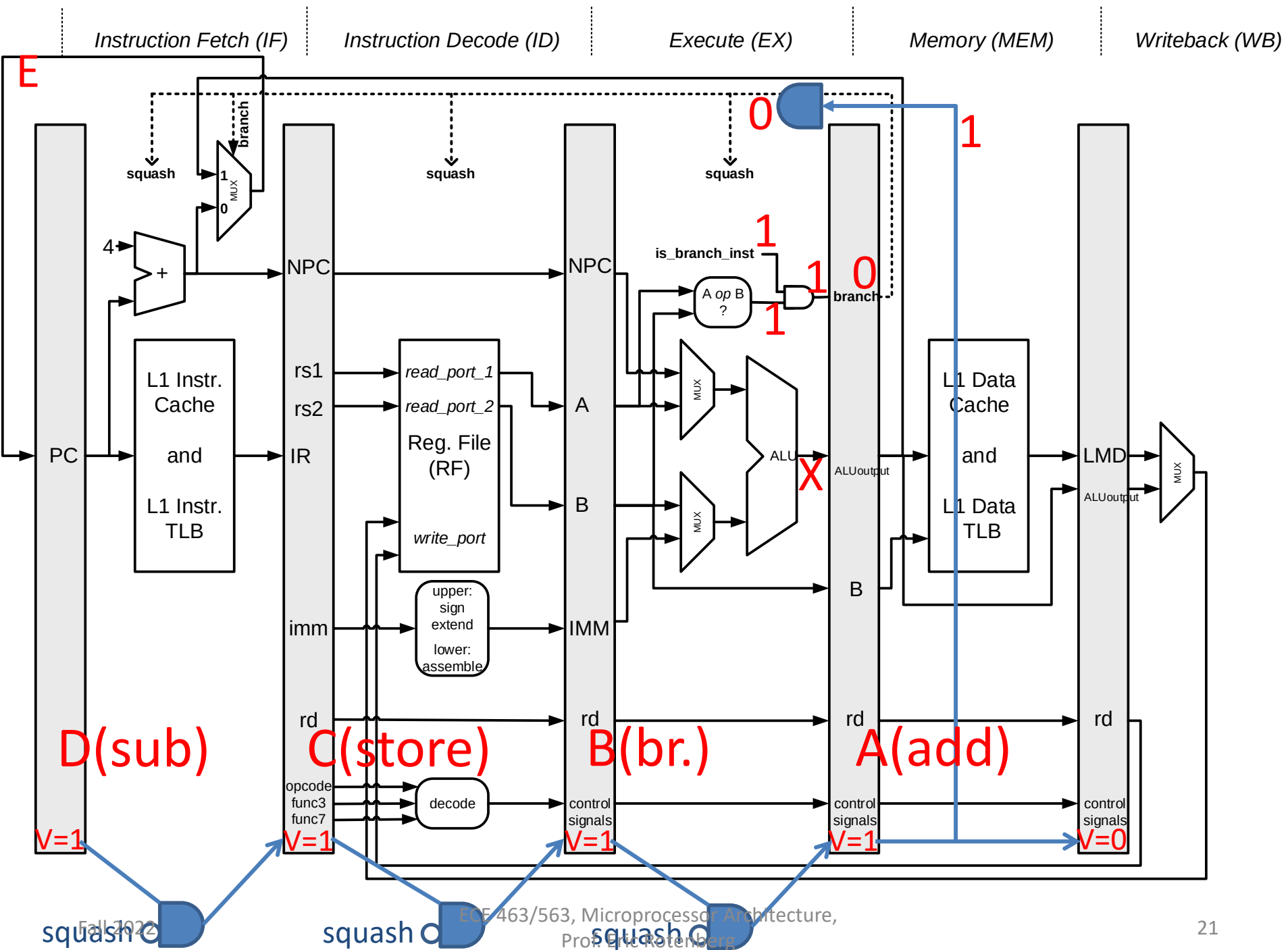


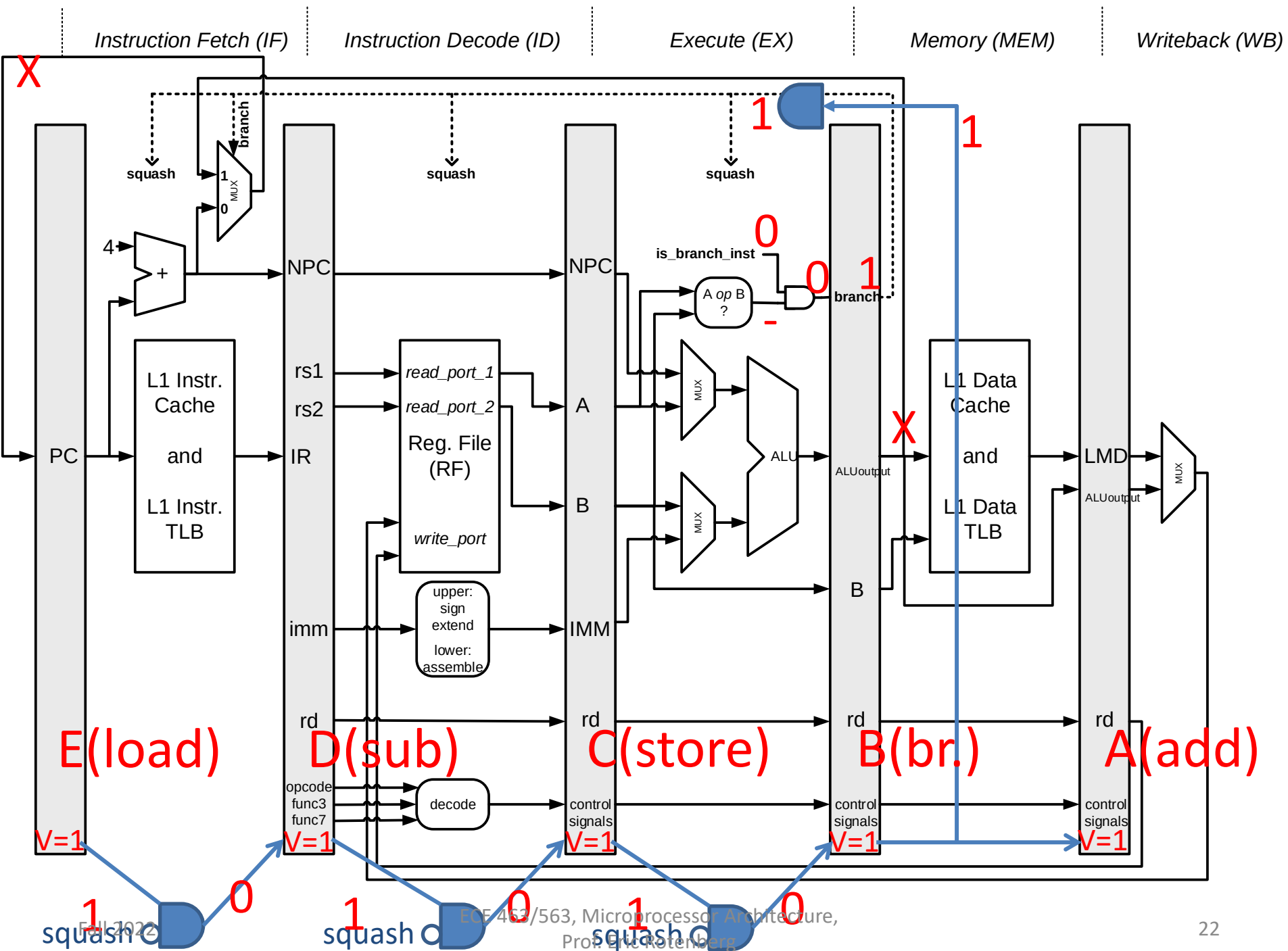


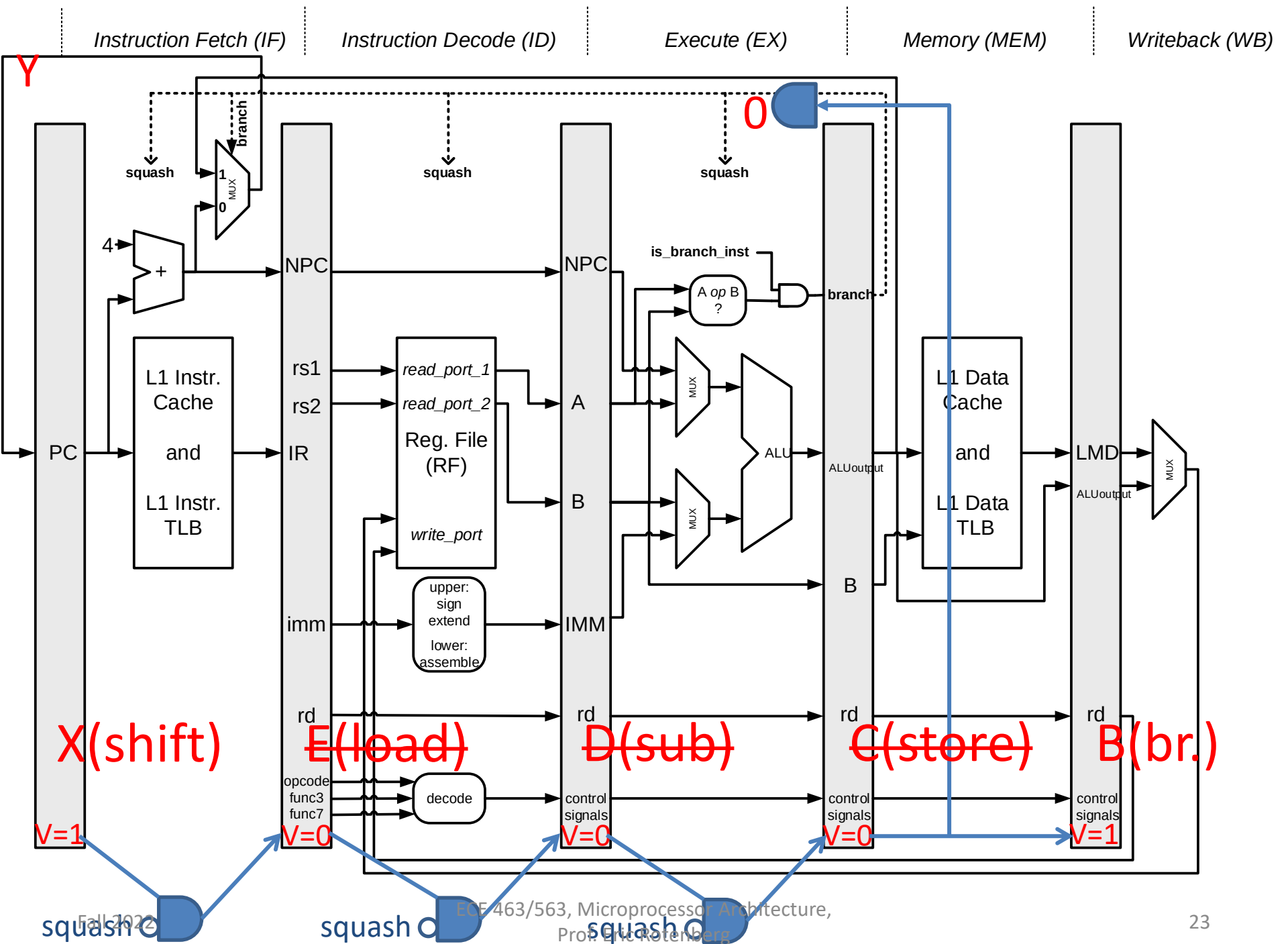












ECE 463/563

Fall `20

pipelining:
static branch prediction

Prof. Eric Rotenberg

Static Branch Prediction: Rationale

- Simple embedded processors
 - Low cost and low power
 - Die area and power are at a premium
 - May not have area and power budget for a dynamic branch predictor
- Static branch prediction
 - Compiler gives hint to hardware as to most likely direction of branch
 - Advantages:
 - More accurate than always predicting not-taken
 - No area or power cost for making predictions
 - Disadvantages:
 - Lower accuracy than dynamic branch prediction because branch always chooses one direction (cannot adapt to different behavior, exploit patterns, *etc.*)
 - Hidden power cost: fetch more incorrect instructions

Static Branch Prediction: ISA Support

- Change branch encoding to add a *likely* bit
- If compiler thinks branch is frequently taken
 - Compiler sets *likely* bit = 1
- If compiler thinks branch is frequently not-taken
 - Compiler sets *likely* bit = 0

Methods for setting likely bit

- Heuristics
- Profiling

Heuristics

- Example: BTFNT
 - “backward-taken, forward-not-taken”
 - Predict backward branches to be taken and forward branches to be not-taken
 - Anticipates that most backward branches are loop branches
 - Notes:
 - Since branch target is PC relative, sign bit of offset = the prediction
 - Jim Smith reports 70% accuracy for this scheme for scientific workloads

Profiling

- Three steps:
 - Run the program multiple times with diverse inputs
 - Record the average preferred direction (taken or not-taken) for each conditional branch
 - Recompile to set the *likely* bits of conditional branches
- Problems:
 - A branch's prediction is fixed for entire run
 - What if inputs used for profiling are not representative?

Started on	Tuesday, October 18, 2022, 5:18 PM
State	Finished
Completed on	Monday, October 24, 2022, 11:57 PM
Time taken	6 days 6 hours
Grade	75.00 out of 100.00

Question 1

Correct

5.00 points out
of 5.00

Provide your name as your electronic signature confirming that you abide by the Honor Pledge: "I have neither given nor received unauthorized aid on this quiz."

Answer: Jonathan N. Quance



The correct answer is:

Information

Please upload one or more files that show your work for this quiz, *e.g.*, photos of your work done on paper, Word or pptx documents, *etc.* Uploaded files will be used to confirm authenticity of work performed, provide context for manual re-grading for partial credit, *etc.* A separate file submission link will be announced.

Question 2

Correct

10.00 points out
of 10.00

Suppose you develop a technique that increases instructions-per-cycle (IPC) by a factor of 1.32. (Note: *IPC is simply the reciprocal of CPI.*) The extra circuit complexity causes a factor of 1.10 increase in processor cycle time. What is the overall speedup?

Answer: 

Solution:

$$T_{\text{old}} = IC * CPI * CT = (IC * CT)/(IPC)$$

$$T_{\text{new}} = (IC * (1.1 * CT)) / (1.32 * IPC)$$

$$T_{\text{new}} = (1.1 / 1.32) * T_{\text{old}}$$

$$\text{speedup} = T_{\text{old}} / T_{\text{new}} = T_{\text{old}} / ((1.1 / 1.32) * T_{\text{old}}) = (1.32 / 1.1) = 1.2$$

The correct answer is: 1.2

Question 3

Correct

10.00 points out
of 10.00

A processor running a certain Program P, spends 32% of the time waiting for main memory to supply instructions and data. Suppose the processor's frequency can be doubled simply by manufacturing it using transistors and wires that are twice as fast as before. Meanwhile, the processor's microarchitecture is not changed and main memory is not changed. That is, the only change is doubling the processor's frequency. What speedup is observed for Program P due to doubling the processor's frequency?

Round your answer to the nearest 1/100th position (*e.g.*, 3.456 gets rounded to 3.46).

Answer: 

Solution:

The program's total execution time (T) can be separated into two components:

(1) the time spent doing useful computation (T_c) and

(2) the time spent waiting for main memory to return needed instructions and data (T_m).

Increasing the processor's frequency only speeds up the computation component. Increasing the processor's frequency does not accelerate main memory: it still operates at the same speed as before, therefore, the memory component is not sped up. Moreover, since the microarchitecture is not changed, we can assume that the processor is no better and no worse at tolerating memory latency than before (*e.g.*, OOO execution can "hide" some memory latency...).

We can apply Amdahl's Law to determine the overall speedup due to speeding up only a fraction of the total execution time. In this case, the fraction that is sped up is 0.68 (the original computation component of time is 0.68 because the memory component is 0.32). The speedup of the enhancement is 2 because frequency is doubled, *i.e.*, cycle time is halved, and both instruction count (IC) and useful cycles-per-instruction (CPI_{useful}) are unchanged.

Amdahl's Law:

$$s_{\text{overall}} = 1 / ((1-f) + f/s)$$

$$s_{\text{overall}} = 1 / (0.32 + 0.68/2) = 1.52$$

In other words, doubling frequency yields a 1.52 speedup overall. The gap between ideal (2x) and overall speedup (1.52x) is due to memory stall time not being reduced.

The correct answer is: 1.52

Question 4

Correct

15.00 points out
of 15.00

Given the execution times of Programs 1, 2, 3, and 4, on each of Computers A, B, and C, which computer should you conclude is the fastest overall?

	Computer A time (s)	Computer B time (s)	Computer C time (s)
Program 1	100	200	300
Program 2	500	300	700
Program 3	400	800	300
Program 4	200	100	150

Select one:

- ☒ a. Computer A ✓
- ☐ b. Computer B
- ☐ c. Computer C

Your answer is correct.

For each computer, calculate either the sum of execution times or arithmetic mean of execution times of the four programs.

Computer A:

$$\text{sum} = 100 \text{ s} + 500 \text{ s} + 400 \text{ s} + 200 \text{ s} = 1200 \text{ s}$$

$$\text{arithmetic mean} = (\text{sum} / 4) = 300 \text{ s}$$

Computer B:

$$\text{sum} = 200 \text{ s} + 300 \text{ s} + 800 \text{ s} + 100 \text{ s} = 1400 \text{ s}$$

arithmetic mean = (sum / 4) = 350 s

Computer C:

sum = 300 s + 700 s + 300 s + 150 s = 1450 s

arithmetic mean = (sum / 4) = 362.5 s

Computer A has the lowest total execution time. Alternatively, it has the lowest arithmetic mean. So Computer A is the fastest overall.

The correct answer is: Computer A

Question 5

Correct

15.00 points out of 15.00

The execution times of Programs 1, 2, 3, and 4, on each of Computers A, B, and C, are given in the table, below. A computer user is trying to decide which computer to buy. She knows that she runs Programs 2 and 4 about three times as often as Programs 1 and 3. Which computer should she buy?

	Computer A time (s)	Computer B time (s)	Computer C time (s)
Program 1	100	200	300
Program 2	500	300	700
Program 3	400	800	300
Program 4	200	100	150

Select one:

- ☐ a. Computer A
- ☒ b. Computer B ✓
- ☐ c. Computer C

Your answer is correct.

For each computer, calculate either the weighted sum of execution times or weighted arithmetic mean of execution times of the four programs. If we assume Programs 1 and 3 have weights of 1 each, then Programs 2 and 4 have weights of 3 each (they are run three times as often as Programs 1 and 3).

Computer A:

$$\text{weighted sum} = 1(100 \text{ s}) + 3(500 \text{ s}) + 1(400 \text{ s}) + 3(200 \text{ s}) = 100 \text{ s} + 1500 \text{ s} + 400 \text{ s} + 600 \text{ s} = 2600 \text{ s}$$

$$\text{weighted arithmetic mean} = (\text{weighted sum} / \text{sum of weights}) = (2600 \text{ s} / (1+3+1+3)) = 325 \text{ s}$$

Computer B:

$$\text{weighted sum} = 1(200 \text{ s}) + 3(300 \text{ s}) + 1(800 \text{ s}) + 3(100 \text{ s}) = 200 \text{ s} + 900 \text{ s} + 800 \text{ s} + 300 \text{ s} = 2200 \text{ s}$$

$$\text{weighted arithmetic mean} = (\text{weighted sum} / \text{sum of weights}) = (2200 \text{ s} / (1+3+1+3)) = 275 \text{ s}$$

Computer C:

$$\text{weighted sum} = 1(300 \text{ s}) + 3(700 \text{ s}) + 1(300 \text{ s}) + 3(150 \text{ s}) = 300 \text{ s} + 2100 \text{ s} + 300 \text{ s} + 450 \text{ s} = 3150 \text{ s}$$

$$\text{weighted arithmetic mean} = (\text{weighted sum} / \text{sum of weights}) = (3150 \text{ s} / (1+3+1+3)) = 394 \text{ s}$$

Computer B has the lowest total weighted execution time. Alternatively, it has the lowest weighted arithmetic mean. So Computer B is the fastest overall and this is the computer she should buy.

The correct answer is: Computer B

Question 6

Incorrect

0.00 points out
of 15.00

The table, below, shows instructions-per-cycle (IPC) for three Computers A, B, and C, on each of programs 1, 2, 3, and 4. Which computer is fastest overall?

Assume that using only IPC to compare performance is valid in this case, because all three computers implement the same instruction set architecture, execute the same program binaries (hence, identical dynamic instruction count), and have the same processor frequency, hence, cycle time.

**Note: higher IPC (lower CPI)
means higher performance.**

	Computer A IPC	Computer B IPC	Computer C IPC
Program 1	1.00	8.00	4.00
Program 2	2.00	0.50	3.00
Program 3	3.00	0.25	2.50
Program 4	4.00	8.00	5.00

Select one:

- ☐ a. Computer A
- ☒ b. Computer B **✗**
- ☐ c. Computer C

Your answer is incorrect.

For each computer, calculate the harmonic mean of IPCs of the four programs. Harmonic mean is more appropriate than other means, because IPC is a rate; using harmonic mean essentially converts

IPCs to CPIs first (take reciprocals of IPCs), computes arithmetic mean of these CPIs (sum CPIs and divide by number of CPIs), and converts back to IPC (take reciprocal of arithmetic mean of CPIs).

Computer A:

$$\text{harmonic mean IPC} = (4 / (1/1.00 + 1/2.00 + 1/3.00 + 1/4.00)) = 1.92$$

Computer B:

$$\text{harmonic mean IPC} = (4 / (1/8.00 + 1/0.50 + 1/0.25 + 1/8.00)) = 0.64$$

Computer C:

$$\text{harmonic mean IPC} = (4 / (1/4.00 + 1/3.00 + 1/2.50 + 1/5.00)) = 3.38$$

Computer C has the highest harmonic mean IPC. So Computer C is the fastest overall.

The correct answer is: Computer C

Question 7

Incorrect

0.00 points out
of 10.00

Suppose you walk up a stairwell in 10 seconds. The next day you run up the same stairwell in 5 seconds.

(a) Compared to walking up the stairs, you expended  energy running up the stairs.

(b) Compared to walking up the stairs, it took  power to run up the stairs.

Your answer is incorrect.

Solution:

Assumption: Ignore the fact that walking and running may differ in several ways, *e.g.*, differences in frictional energy wasted, different energy conversion efficiencies in the human body when walking vs. running, *etc.*

(a) If we consider energy just in terms of the increase in potential energy of the system of moving your body to a higher point, then you expend the same amount of energy both walking and running up the stairs: $E_{\text{run}} = E_{\text{walk}}$.

(b) Power is the rate at which energy is expended. Therefore, running up the stairs increases power compared to walking up the stairs. In this case, since running up the stairs involves expending the same energy in half the time, average power is doubled.

$$P_{\text{walk}} = E_{\text{walk}}/T_{\text{walk}} = E_{\text{walk}}/(10 \text{ sec.})$$

$$P_{\text{run}} = E_{\text{run}}/T_{\text{run}} = E_{\text{walk}}/(5 \text{ sec.})$$

$$P_{\text{run}}/P_{\text{walk}} = (E_{\text{walk}}/(5 \text{ sec.})) / (E_{\text{walk}}/(10 \text{ sec.})) = 10/5 = 2$$

In this problem, it takes twice as much power to run up the stairs than to walk up the stairs. (This isn't necessarily a bad thing: this doubling is simply due to being faster, not due to consuming more energy.)

The correct answer is:

Suppose you walk up a stairwell in 10 seconds. The next day you run up the same stairwell in 5 seconds.

(a) Compared to walking up the stairs, you expended [the same] energy running up the stairs.

(b) Compared to walking up the stairs, it took [twice as much] power to run up the stairs.

Question 8

Correct

20.00 points out
of 20.00

Table 1 shows some physical parameters of a particular processor. Suppose a benchmark program is executed on the processor, and we measure the dynamic instruction count (IC), average cycles-per-instruction (CPI), and average switching activity factor (α); these measurements are provided in Table 2. Calculate the following: (i) execution time, (ii) dynamic power, static power, and total power, and (iii) dynamic, static, and total energy.

NOTE: $\mu\text{A} = 10^{-6} \text{ A}$, $\text{fF} = 10^{-15} \text{ F}$, $\text{GHz} = 10^9 \text{ Hz}$

**Table 1: Some physical
parameters of a particular
processor.**

Vdd	1.0 V
Average leakage current per gate: $i_{\text{gate_leakage}}$	0.1 $\mu\text{A/gate}$
Average capacitive load per gate: $c_{\text{gate_load}}$	0.1 fF/gate
Number of gates	100 million
Core frequency	2 GHz

**Table 2: Some
measurements of
benchmark program.**

dynamic instruction count (IC)	1 billion
average cycles-per-instr. (CPI)	0.87
average switching activity factor (α)	0.5

Table 3: Enter your solution here.

execution time	0.435	✓	s
dynamic power	10	✓	W
static power	10	✓	W
total power	20	✓	W
dynamic energy	4.35	✓	J
static energy	4.35	✓	J
total energy	8.7	✓	J

execution time = IC x CPI x CT = IC x CPI x (1 / freq) = (10**9 instr.) x (0.87 cycles/instr.) x (1 / (2x10**9 cycles/second)) = (0.5 x 0.87) seconds = 0.435 seconds

dynamic power = alpha x C x V**2 x freq = (0.5 of all devices switching / cycle) x (0.1x10**-15 F/gate x

$$100 \times 10^{**6} \text{ gates}) \times (1 \text{ volt} \times 1 \text{ volt}) \times (2 \times 10^{**9} \text{ cycles/second}) = 10 \text{ watts (W)}$$

(note the units: $F \times V \times V / s = C/V \times V \times V / s = C/s \times V = V \times A = W$)

$$\text{static power} = V \times \text{leakage_current} = (1 \text{ volt}) \times (0.1 \times 10^{**-6} \text{ A/gate} \times 100 \times 10^{**6} \text{ gates}) = 10 \text{ watts (W)}$$

$$\text{total power} = 20 \text{ watts}$$

Note: In this case, we can simply add the two rates (dynamic and static power) ONLY because these rates occur over the same time interval. Otherwise, if the two rates were over different time intervals, we'd have to convert the rates to energies (by multiplying each rate by its respective time interval), sum the energies (these can be added arbitrarily because they are quantities), and then divide the total energy by total time.

$$\text{energy} = \text{power} \times \text{time}$$

Therefore:

$$\text{dynamic energy} = \text{dynamic power} \times \text{execution time} = (10 \text{ W}) \times (0.435 \text{ s}) = 4.35 \text{ joules (J)}$$

$$\text{static energy} = \text{static power} \times \text{execution time} = (10 \text{ W}) \times (0.435 \text{ s}) = 4.35 \text{ joules (J)}$$

$$\text{total energy} = \text{dynamic energy} + \text{static energy} = 8.7 \text{ joules (J)}$$

Started on	Wednesday, November 2, 2022, 10:52 AM
State	Finished
Completed on	Monday, November 7, 2022, 10:09 AM
Time taken	5 days
Grade	94.00 out of 100.00

Question 1

Correct

8.00 points out
of 8.00

Provide your name as your electronic signature confirming that you abide by the Honor Pledge: "I have neither given nor received unauthorized aid on this quiz."

Answer: Jonathan N. Quance



The correct answer is:

Information

Please upload one or more files that show your work for this quiz, *e.g.*, photos of your work done on paper, Word or pptx documents, *etc.* Uploaded files will be used to confirm authenticity of work performed, provide context for manual re-grading for partial credit, *etc.* A separate file submission link will be announced.

Question 2

Correct

26.00 points out of 26.00

The first pipeline we designed in class, without a BTB and without register file bypasses, is shown in Figure 1. Various parts of the pipeline are labeled in the figure with numbers. Circular labels are used for wires, buses, *etc.*, and start at number 1. Triangular labels are used for components (adders, muxes, SRAMs, logic blocks, *etc.*) and start at number 80.

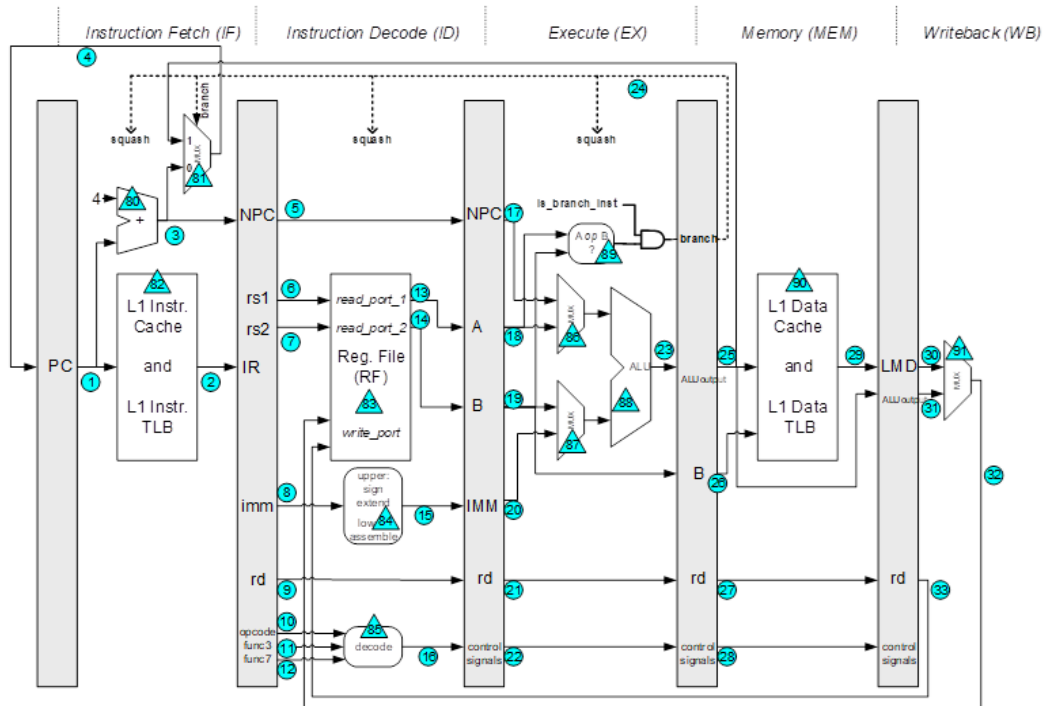


Figure 1. Pipeline diagram for the first pipeline we designed in class, without a BTB and without register file bypasses.

Fill in the table: next to each item, drag-and-drop the description (from the list below the table) that best describes the item.

ITEM	drag-and-drop the description (from the list below this table) that best describes the item (if applicable, a description may be used more than once)	
1	virtual address of fetched instruction	✓
2	fetched instruction	✓
13	a source register value	✓
18, for loads and stores	base address for virtual address calculation	✓
20, for loads and stores	offset for virtual address calculation	✓
25, for loads	load virtual address	✓

ITEM	drag-and-drop the description (from the list below this table) that best describes the item (if applicable, a description may be used more than once)	
25, for stores	store virtual address	✓
25, for conditional branches	branch's taken target	✓
24	signals detection of a taken conditional branch	✓
26	store value	✓
29	load value	✓
33, for stores	not used	✓
89	calculates branch condition	✓

store virtual address	store value	load virtual address
load value	computes branch's taken-target	computes load virtual address
fetches instruction	register file	a source register value
branch's taken target	branch's not-taken target	virtual address of fetched instruction
physical address of fetched instruction	base address for virtual address calculation	offset for virtual address calculation
calculates branch condition	signals detection of a taken conditional branch	not used

Your answer is correct.

The correct answer is:

The first pipeline we designed in class, without a BTB and without register file bypasses, is shown in Figure 1. Various parts of the pipeline are labeled in the figure with numbers. Circular labels are used for wires, buses, *etc.*, and start at number 1. Triangular labels are used for components (adders, muxes, SRAMs, logic blocks, *etc.*) and start at number 80.

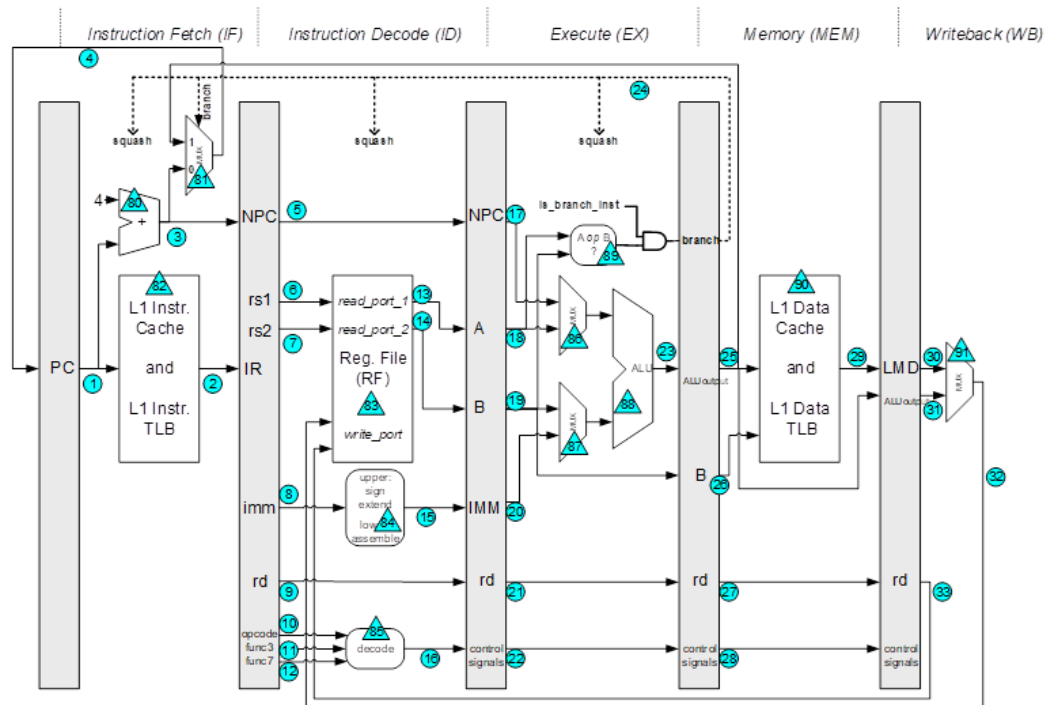


Figure 1. Pipeline diagram for the first pipeline we designed in class, without a BTB and without register file bypasses.

Fill in the table: next to each item, drag-and-drop the description (from the list below the table) that best describes the item.

ITEM	drag-and-drop the description (from the list below this table) that best describes the item (if applicable, a description may be used more than once)
1	[virtual address of fetched instruction]
2	[fetched instruction]
13	[a source register value]
18, for loads and stores	[base address for virtual address calculation]
20, for loads and stores	[offset for virtual address calculation]
25, for loads	[load virtual address]
25, for stores	[store virtual address]
25, for conditional branches	[branch's taken target]
24	[signals detection of a taken conditional branch]
26	[store value]
29	[load value]

ITEM	drag-and-drop the description (from the list below this table) that best describes the item (if applicable, a description may be used more than once)
33, for stores	[not used]
89	[calculates branch condition]

Question 3

Partially correct

18.00 points out of 24.00

Consider a pipeline with a BTB. A 1-bit prediction is included with each branch in the BTB. For each scenario in the table below: (1) indicate whether or not the scenario is possible and (2) if the scenario is possible, indicate whether the fetch unit will speculatively fetch the next sequential instruction ("sequential") or speculatively branch somewhere else ("branch"); if the scenario is not possible, then select "not applicable" for the column "where to speculatively fetch next?".

Is instruction in IF a branch?	Did instruction hit or miss in BTB?	1-bit prediction from BTB	Is this scenario possible?	Where to speculatively fetch next?
No	Miss	1-bit prediction not available	Yes	not applicable
No	Hit	not-taken	No	not applicable
No	Hit	taken	No	not applicable
Yes	Miss	1-bit prediction not available	No	not applicable
Yes	Hit	not-taken	Yes	sequential
Yes	Hit	taken	Yes	branch

Your answer is partially correct.

You have correctly selected 9.

The correct answer is:

Consider a pipeline with a BTB. A 1-bit prediction is included with each branch in the BTB. For each scenario in the table below: (1) indicate whether or not the scenario is possible and (2) if the scenario is possible, indicate whether the fetch unit will speculatively fetch the next sequential instruction ("sequential") or speculatively branch somewhere else ("branch"); if the scenario is not possible, then select "not applicable" for the column "where to speculatively fetch next?".

Is instruction in IF a branch?	Did instruction hit or miss in BTB?	1-bit prediction from BTB	Is this scenario possible?	Where to speculatively fetch next?
No	Miss	1-bit prediction not available	[Yes]	[sequential]
No	Hit	not-taken	[No]	[not applicable]
No	Hit	taken	[No]	[not applicable]
Yes	Miss	1-bit prediction not available	[Yes]	[sequential]
Yes	Hit	not-taken	[Yes]	[sequential]
Yes	Hit	taken	[Yes]	[branch]

Question 4

Correct

42.00 points out of 42.00

A particular branch executes 10 times and we observe the following outcomes. (Note that "T" means "taken" and "N" means "not-taken".)

T T N T N T T T N N

(a) What is the misprediction rate of the branch using a 1-bit counter (initialized to 1)? Show your work by filling in the table below: (i) show the state of the 1-bit counter after each instance of the branch, and (ii) show the prediction that was made for each instance of the branch.

Actual Outcome		T		T		N		T		N		T		T		T		N		N	
1-bit counter	1		1		1		0		1		0		1		1		1		0		0
Prediction		T		T		T		N		T		N		T		T		T		N	

misprediction rate = 50% ✓

(b) What is the misprediction rate of the branch using a 2-bit counter (initialized to 3)? Show your work by filling in the table below: (i) show the state of the 2-bit counter after each instance of the branch, and (ii) show the prediction that was made for each instance of the branch.

Actual Outcome		T		T		N		T		N		T		T		T		N		N	
2-bit counter	3		3		3		2		3		2		3		3		3		2		1
Prediction		T		T		T		T		T		T		T		T		T		T	

misprediction rate = 40% ✓

Your answer is correct.

The correct answer is:

A particular branch executes 10 times and we observe the following outcomes. (Note that "T" means "taken" and "N" means "not-taken".)

T T N T N T T T N N

(a) What is the misprediction rate of the branch using a 1-bit counter (initialized to 1)? Show your work by filling in the table below: (i) show the state of the 1-bit

counter after each instance of the branch, and (ii) show the prediction that was made for each instance of the branch.

Actual Outcome		T		T		N		T		N		T		T		T		N		N	
1-bit counter	1		[1]		[1]		[0]		[1]		[0]		[1]		[1]		[1]		[0]		[0]
Prediction		[T]		[T]		[T]		[N]		[T]		[N]		[T]		[T]		[T]		[N]	

misprediction rate = [50%]

(b) What is the misprediction rate of the branch using a 2-bit counter (initialized to 3)? Show your work by filling in the table below: (i) show the state of the 2-bit counter after each instance of the branch, and (ii) show the prediction that was made for each instance of the branch.

Actual Outcome		T		T		N		T		N		T		T		T		N		N	
2-bit counter	3		[3]		[3]		[2]		[3]		[2]		[3]		[3]		[3]		[2]		[1]
Prediction		[T]		[T]		[T]		[T]		[T]		[T]		[T]		[T]		[T]		[T]	

misprediction rate = [40%]

Started on	Saturday, November 12, 2022, 10:40 AM
State	Finished
Completed on	Tuesday, November 15, 2022, 7:49 PM
Time taken	3 days 9 hours
Grade	100.00 out of 100.00

Question 1

Correct

3.00 points out of 3.00

Provide your name as your electronic signature confirming that you abide by the Honor Pledge: "I have neither given nor received unauthorized aid on this quiz."

Answer: Jonathan N. Quance



The correct answer is:

Information

Please upload one or more files that show your work for this quiz, *e.g.*, photos of your work done on paper, Word or pptx documents, *etc.* Uploaded files will be used to confirm authenticity of work performed, provide context for manual re-grading for partial credit, *etc.* A separate file submission link will be announced.

Question 2

Correct

10.00 points out of 10.00

Certain scenarios cause certain pipeline hazards. Match up the scenarios on the left with the pipeline hazards on the right.

Two instructions that have a true dependence are in the pipeline at the same time.

RAW data hazard



Encounter a branch instruction.

Control hazard



Two instructions attempt to use the same resource at the same time.

Structural hazard



Two instructions that have an output dependence are in the pipeline at the same time.

WAW data hazard



Two instructions that have an anti-dependence are in the pipeline at the same time.

WAR data hazard



Your answer is correct.

The correct answer is:

- Two instructions that have a true dependence are in the pipeline at the same time. → RAW data hazard
- Encounter a branch instruction. → Control hazard
- Two instructions attempt to use the same resource at the same time. → Structural hazard
- Two instructions that have an output dependence are in the pipeline at the same time. → WAW data hazard
- Two instructions that have an anti-dependence are in the pipeline at the same time. → WAR data hazard

Question 3

Correct

87.00 points out of 87.00

Use the program shown in Table 1, below, for this problem. The pipeline for this problem is the original RISC-V in-order scalar pipeline with blocking EX and MEM stages for multi-cycle operations ([pipe-riscv-formats-and-datapath-NEW-WITH-VALID-BITS.pptx](#)). Recall in this pipeline, that a load instruction generates its address in EX and accesses the TLB+D\$ in MEM. Recall in this pipeline, that all instructions flow through five pipeline stages: IF, ID, EX, MEM, WB.

KEY:

```

register-register ALU instructions:  <INSTR> <dest. reg.>, <source reg. 1>, <source reg. 2>
register-immediate ALU instructions: <INSTR> <dest. reg.>, <source reg. 1>, #<immed. value>
load instructions:                  <INSTR> <dest. reg.>, #<displacement>(source reg. 1)

```

Table 1: program for this problem.

PC	Instruction	Comment
A:	add r3, r1, r2	// takes 1 cycle in EX
B:	andi r4, r3, #3	// takes 1 cycle in EX
C:	lw r1, #0(r3)	// load word: takes 1 cycle in EX (to compute address) and 1 cycle in MEM (TLB hit, D\$ hit)
D:	sub r5, r1, r6	// takes 1 cycle in EX

(a) Simulate the program assuming the pipeline DOES NOT support data forwarding (no register file bypasses): fill in the cycle-level timing diagram, below. You must fill in every cell: select "(blank)" from the pull-down menu to mark a cell in the schedule as blank, as needed. See Slide 3 of [pipe-data-hazards.pptx](#) for an example schedule without data forwarding (as in the lecture, assume that if a register is being written to by the WB stage in cycle X, the earliest cycle it can be successfully read by the ID stage is in cycle X+1).

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A:	IF	ID	EX	MEM	WB											
B:	(blank)	IF	ID	ID	ID	ID	EX	MEM	WB	(blank)	(blank)	(blank)	(blank)	(blank)	(blank)	(blank)
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
C:	(blank)	(blank)	IF	IF	IF	IF	ID	EX	MEM	WB	(blank)	(blank)	(blank)	(blank)	(blank)	(blank)
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
D:	(blank)	(blank)	(blank)	(blank)	(blank)	(blank)	IF	ID	ID	ID	ID	EX	MEM	WB	(blank)	(blank)
	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

(b) Simulate the program assuming the pipeline supports data forwarding (has register file bypasses): fill in the cycle-level timing diagram, below. You must fill in every cell: select "(blank)" from the pull-down menu to mark a cell in the schedule as blank, as needed. See Slides 9-10 of [pipe-data-hazards.pptx](#) for example schedules with data forwarding.

In addition, in the table below the schedule:

For each source register of each instruction, indicate whether it was obtained from the register file, bypass 1, bypass 2, or bypass 3.

	0	1	2	3	4	5	6	7	8	9	10
--	---	---	---	---	---	---	---	---	---	---	----

	0	1	2	3	4	5	6	7	8	9	10
A:	IF	ID	EX	MEM	WB						
B:	(blank) ✓	IF ✓	ID ✓	EX ✓	MEM ✓	WB ✓	(blank) ✓	(blank) ✓	(blank) ✓	(blank) ✓	(blank) ✓
C:	(blank) ✓	(blank) ✓	IF ✓	ID ✓	EX ✓	MEM ✓	WB ✓	(blank) ✓	(blank) ✓	(blank) ✓	(blank) ✓
D:	(blank) ✓	(blank) ✓	(blank) ✓	IF ✓	ID ✓	ID ✓	EX ✓	MEM ✓	WB ✓	(blank) ✓	(blank) ✓

	first source register	From where, was first source register obtained?	second source register	From where, was second source register obtained?
A:	r1	Reg. File ✓	r2	Reg. File ✓
B:	r3	Bypass 1 ✓		
C:	r3	Bypass 2 ✓		
D:	r1	Bypass 2 ✓	r6	Reg. File ✓

Your answer is correct.

The correct answer is:

Use the program shown in Table 1, below, for this problem. The pipeline for this problem is the original RISCv in-order scalar pipeline with blocking EX and MEM stages for multi-cycle operations ([pipe-riscv-formats-and-datapath-NEW-WITH-VALID-BITS.pptx](#)). Recall in this pipeline, that a load instruction generates its address in EX and accesses the TLB+D\$ in MEM. Recall in this pipeline, that all instructions flow through five pipeline stages: IF, ID, EX, MEM, WB.

KEY:

```

register-register ALU instructions:  <INSTR> <dest. reg.>, <source reg. 1>, <source reg. 2>
register-immediate ALU instructions: <INSTR> <dest. reg.>, <source reg. 1>, #<immed. value>
load instructions:                  <INSTR> <dest. reg.>, #<displacement>(source reg. 1)

```

Table 1: program for this problem.

PC	Instruction	Comment
A:	add r3, r1, r2	// takes 1 cycle in EX
B:	andi r4, r3, #3	// takes 1 cycle in EX
C:	lw r1, #0(r3)	// load word: takes 1 cycle in EX (to compute address) and 1 cycle in MEM (TLB hit, D\$ hit)
D:	sub r5, r1, r6	// takes 1 cycle in EX

(a) Simulate the program assuming the pipeline DOES NOT support data forwarding (no register file bypasses): fill in the cycle-level timing diagram, below. You must fill in every cell: select "(blank)" from the pull-down menu to mark a cell in the schedule as blank, as needed. See Slide 3 of [pipe-data-hazards.pptx](#) for an example schedule without data forwarding (as in the lecture, assume that if a register is being written to by the WB stage in cycle X, the earliest cycle it can be successfully read by the ID stage is in cycle X+1).

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A:	IF	ID	EX	MEM	WB											
B:	[(blank)]	[(IF)]	[(ID)]	[(ID)]	[(ID)]	[(ID)]	[(EX)]	[(MEM)]	[(WB)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
C:	[(blank)]	[(blank)]	[IF]	[IF]	[IF]	[IF]	[ID]	[EX]	[MEM]	[WB]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]
D:	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[IF]	[ID]	[ID]	[ID]	[ID]	[EX]	[MEM]	[WB]	[(blank)]	[(blank)]

(b) Simulate the program assuming the pipeline supports data forwarding (has register file bypasses): fill in the cycle-level timing diagram, below. You must fill in every cell: select "(blank)" from the pull-down menu to mark a cell in the schedule as blank, as needed. See Slides 9-10 of [pipe-data-hazards.pptx](#) for example schedules with data forwarding.

In addition, in the table below the schedule:

For each source register of each instruction, indicate whether it was obtained from the register file, bypass 1, bypass 2, or bypass 3.

	0	1	2	3	4	5	6	7	8	9	10
A:	[IF]	[ID]	[EX]	[MEM]	[WB]						
B:	[(blank)]	[IF]	[ID]	[EX]	[MEM]	[WB]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]
C:	[(blank)]	[(blank)]	[IF]	[ID]	[EX]	[MEM]	[WB]	[(blank)]	[(blank)]	[(blank)]	[(blank)]
D:	[(blank)]	[(blank)]	[(blank)]	[IF]	[ID]	[ID]	[EX]	[MEM]	[WB]	[(blank)]	[(blank)]

	first source register	From where, was first source register obtained?	second source register	From where, was second source register obtained?
A:	r1	[Reg. File]	r2	[Reg. File]
B:	r3	[Bypass 1]		
C:	r3	[Bypass 2]		
D:	r1	[Bypass 2]	r6	[Reg. File]

Started on	Wednesday, November 23, 2022, 1:36 PM
State	Finished
Completed on	Monday, November 28, 2022, 10:08 PM
Time taken	5 days 8 hours
Grade	100.00 out of 100.00

Question 1
Correct
10.00 points out
of 10.00

Provide your name as your electronic signature confirming that you abide by the Honor Pledge: "I have neither given nor received unauthorized aid on this quiz."

Answer: Jonathan N. Quance



The correct answer is:

Information

A [scratchwork powerpoint file](#) (← click the link) is available for you to use. It will greatly simplify entering information into tables in this quiz, if you first do your work within the powerpoint (or on paper).

Please upload one or more files that show your work for this quiz, *e.g.*, photos of your work done on paper, Word or pptx documents, *etc.* Uploaded files will be used to confirm authenticity of work performed, provide context for manual re-grading for partial credit, *etc.* A separate file submission link will be announced.

Question 2

Correct

30.00 points out of 30.00

KEY:

```
add <dest. reg.>, <source reg. 1>, <source reg. 2> // 1 cycle in EX
mult <dest. reg.>, <source reg. 1>, <source reg. 2> // 5 cycles in EX
```

Consider a scalar OOO pipeline with a 3-entry IQ and 32-entry ROB. The pipeline is comprised of 9 stages as considered in class:

FE, DE, RN, RR, DI, IS, EX, WB, RT.

a. Fill in the schedule for the following 6-instruction sequence. (i1 is fetched in cycle 1. i6 is fetched in cycle 10 due to an L1 IS miss.) All cells with drop-down menus must be filled: to mark a cell as blank, select "blank".

To help fit the entire schedule in one screen (and so that the instructions i1-i6 are less likely to be "wrapped" in their cells), please try the zoom-out feature of your web browser.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
i1: mul r3, r1, r2	FE	DE ✓	RN ✓	RR ✓	DI ✓	IS ✓	EX ✓	EX ✓	EX ✓	EX ✓	EX ✓	WB ✓	RT ✓	(blank) ✓	(blank) ✓	(blank) ✓	(blank) ✓	(blank) ✓	(blank) ✓
i2: add r3, r3, r4		FE	DE ✓	RN ✓	RR ✓	DI ✓	IS ✓	IS ✓	IS ✓	IS ✓	IS ✓	EX ✓	WB ✓	RT ✓	(blank) ✓	(blank) ✓	(blank) ✓	(blank) ✓	(blank) ✓
i3: add r6, r5, r3			FE	DE ✓	RN ✓	RR ✓	DI ✓	IS ✓	IS ✓	IS ✓	IS ✓	IS ✓	EX ✓	WB ✓	RT ✓	(blank) ✓	(blank) ✓	(blank) ✓	(blank) ✓
i4: add r3, r1, r4				FE	DE ✓	RN ✓	RR ✓	DI ✓	IS ✓	EX ✓	WB ✓	RT ✓	RT ✓	RT ✓	RT ✓	RT ✓	(blank) ✓	(blank) ✓	(blank) ✓
i5: add r2, r3, r7					FE	DE ✓	RN ✓	RR ✓	DI ✓	IS ✓	EX ✓	WB ✓	RT ✓	RT ✓	RT ✓	RT ✓	RT ✓	(blank) ✓	(blank) ✓
i6: add r3, r2, r3						(i6 missed in L1 IS, hence delayed fetch of i6)				FE	DE ✓	RN ✓	RR ✓	DI ✓	IS ✓	EX ✓	WB ✓	RT ✓	(blank) ✓

b. In the table below, for each source operand, indicate where the value was obtained from: select either "ARF", "ROB", or "Bypass/IQ".

	source reg. 1		source reg. 2	
i1: mul r3, r1, r2	ARF	✓	ARF	✓
i2: add r3, r3, r4	Bypass/IQ	✓	ARF	✓
i3: add r6, r5, r3	ARF	✓	Bypass/IQ	✓
i4: add r3, r1, r4	ARF	✓	ARF	✓
i5: add r2, r3, r7	Bypass/IQ	✓	ARF	✓
i6: add r3, r2, r3	ROB	✓	ROB	✓

Your answer is correct.

The correct answer is:

KEY:

```
add <dest. reg.>, <source reg. 1>, <source reg. 2> // 1_cycle in EX
mult <dest. reg.>, <source reg. 1>, <source reg. 2> // 5_cycles in EX
```

Consider a scalar OOO pipeline with a 3-entry IQ and 32-entry ROB. The pipeline is comprised of 9 stages as considered in class: FE, DE, RN, RR, DI, IS, EX, WB, RT.

a. Fill in the schedule for the following 6-instruction sequence. (i1 is fetched in cycle 1. i6 is fetched in cycle 10 due to an L1 IS miss.) All cells with drop-down menus must be filled: to mark a cell as blank, select "blank". To help fit the entire schedule in one screen (and so that the instructions i1-i6 are less likely to be "wrapped" in their cells), please try the zoom-out feature of your web browser.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
i1: mul r3, r1, r2	FE	[DE]	[RN]	[RR]	[DI]	[IS]	[EX]	[EX]	[EX]	[EX]	[WB]	[RT]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]
i2: add r3, r3, r4		FE	[DE]	[RN]	[RR]	[DI]	[IS]	[IS]	[IS]	[IS]	[EX]	[WB]	[RT]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]
i3: add r6, r5, r3			FE	[DE]	[RN]	[RR]	[DI]	[IS]	[IS]	[IS]	[IS]	[EX]	[WB]	[RT]	[(blank)]	[(blank)]	[(blank)]	[(blank)]	[(blank)]
i4: add r3, r1, r4				FE	[DE]	[RN]	[RR]	[DI]	[IS]	[EX]	[WB]	[RT]	[RT]	[RT]	[RT]	[(blank)]	[(blank)]	[(blank)]	[(blank)]
i5: add r2, r3, r7					FE	[DE]	[RN]	[RR]	[DI]	[IS]	[EX]	[WB]	[RT]	[RT]	[RT]	[RT]	[(blank)]	[(blank)]	[(blank)]
i6: add r3, r2, r3						(i6 missed in L1 IS, hence delayed fetch of i6)				FE	[DE]	[RN]	[RR]	[DI]	[IS]	[EX]	[WB]	[RT]	[(blank)]

b. In the table below, for each source operand, indicate where the value was obtained from: select either "ARF", "ROB", or "Bypass/IQ".

	source reg. 1	source reg. 2
i1: mul r3, r1, r2	[ARF]	[ARF]
i2: add r3, r3, r4	[Bypass/IQ]	[ARF]
i3: add r6, r5, r3	[ARF]	[Bypass/IQ]
i4: add r3, r1, r4	[ARF]	[ARF]
i5: add r2, r3, r7	[Bypass/IQ]	[ARF]
i6: add r3, r2, r3	[ROB]	[ROB]

Information

Based on your schedule in the previous question and the initial ARF contents below, for the next three questions in this quiz, show the pipeline state in cycle 10, cycle 13, and cycle 16.

- **Showing the state of the Issue Queue (IQ):** For each instruction, indicate whether or not the instruction is in the Issue Queue (IQ). If the instruction is in the IQ, indicate what its IQ entry contains. If it is NOT in the IQ, select "not applicable" for all other cells in the row of that instruction.
- **Showing the state of the Reorder Buffer (ROB):** For each instruction, indicate whether or not the instruction is in the ROB: it must be between the ROB head and tail to be considered "in the ROB". If it is in the ROB (between ROB head and tail), indicate at which ROB entry it resides and what its ROB entry contains. If it is NOT in the ROB (NOT between ROB head and tail), select "not applicable" for all other cells in the row of that instruction.
NOTE: If an instruction is in the ROB, but its value is not yet ready, specify "- (not avail.);" for its value field.
NOTE: As in the class notes, if the head instr. is in RT, it is being retired this cycle and assume the head pointer H has been incremented past it (no longer in ROB). For example: [ILP1.pptx](#) ↗, Slide 83, Cycle 14, our "convention" is to consider rob3/i1 to not be in the ROB anymore as it is being retired (new H == rob4). This is the convention you must use in the Moodle quiz, please, for auto-grading.
- **Showing the state of the Rename Map Table (RMT):** If an entry in the RMT has a valid bit of 0, indicate that fact and select "dont-care" for that entry's "ROB tag" field. If an entry in the RMT has a valid bit of 1, indicate that fact and make the correct selection for that entry's "ROB tag" field.
- **Showing the state of the Architectural Register File (ARF):** Select the correct value for each entry in the ARF.

Assume the initial pipeline state below:

- The pipeline is initially empty (no in-flight instructions).
- The ROB Head (H) and Tail (T) pointers are initially both at ROB entry 3 (H == T == rob3).
- Initial contents of the ARF:

	value
r0	#-7
r1	#10
r2	#20
r3	#30
r4	#40
r5	#50
r6	#60
r7	#70

Question 3

Correct

20.00 points out of 20.00

PIPELINE STATE AT CYCLE 10:

Issue Queue (IQ)

	Is this instruction in the IQ?	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
i1: mul r3, r1, r2	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i2: add r3, r3, r4	in IQ ✓	rob4 ✓	0 ✓	rob3 ✓	1 ✓	#40 ✓
i3: add r6, r5, r3	in IQ ✓	rob5 ✓	1 ✓	#50 ✓	0 ✓	rob4 ✓
i4: add r3, r1, r4	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i5: add r2, r3, r7	in IQ ✓	rob7 ✓	0>1 ✓	rob6 ✓	1 ✓	#70 ✓
i6: add r3, r2, r3	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓

Reorder Buffer (ROB)

	Is this instruction in the ROB? (i.e., is it between ROB head and tail?)	rob entry # (ROB tag)	value	dst	rdy	exc.	mis.	pc
i1: mul r3, r1, r2	in ROB (is between H and T) ✓	rob3 ✓	- (not avail.) ✓	r3	0 ✓	0	0	i1
i2: add r3, r3, r4	in ROB (is between H and T) ✓	rob4 ✓	- (not avail.) ✓	r3	0 ✓	0	0	i2
i3: add r6, r5, r3	in ROB (is between H and T) ✓	rob5 ✓	- (not avail.) ✓	r6	0 ✓	0	0	i3
i4: add r3, r1, r4	in ROB (is between H and T) ✓	rob6 ✓	- (not avail.) ✓	r3	0 ✓	0	0	i4
i5: add r2, r3, r7	in ROB (is between H and T) ✓	rob7 ✓	- (not avail.) ✓	r2	0 ✓	0	0	i5
i6: add r3, r2, r3	NOT in ROB (NOT between H and T) ✓	not applicable ✓	not applicable ✓	r3	not applicable ✓	0	0	i6

Rename Map Table

(RMT)

	v	ROB tag
r0	0 ✓	dont-care ✓
r1	0 ✓	dont-care ✓
r2	1 ✓	rob7 ✓
r3	1 ✓	rob6 ✓
r4	0 ✓	dont-care ✓
r5	0 ✓	dont-care ✓
r6	1 ✓	rob5 ✓
r7	0 ✓	dont-care ✓

Architectural

Register File

(ARF)

	value	
r0	#7	✓
r1	#10	✓
r2	#20	✓
r3	#30	✓
r4	#40	✓
r5	#50	✓
r6	#60	✓
r7	#70	✓

Your answer is correct.

The correct answer is:

PIPELINE STATE AT CYCLE 10:

Issue Queue (IQ)

	Is this instruction in the IQ?	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
i1: mul r3, r1, r2	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i2: add r3, r3, r4	[in IQ]	[rob4]	[0]	[rob3]	[1]	[#40]
i3: add r6, r5, r3	[in IQ]	[rob5]	[1]	[#50]	[0]	[rob4]
i4: add r3, r1, r4	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i5: add r2, r3, r7	[in IQ]	[rob7]	[0>1]	[rob6]	[1]	[#70]
i6: add r3, r2, r3	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]

Reorder Buffer (ROB)

	Is this instruction in the ROB? (i.e., is it between ROB head and tail?)	rob entry # (ROB tag)	value	dst	rdy	exc.	mis.	pc
i1: mul r3, r1, r2	[in ROB (is between H and T)]	[rob3]	[- (not avail.)]	r3	[0]	0	0	i1
i2: add r3, r3, r4	[in ROB (is between H and T)]	[rob4]	[- (not avail.)]	r3	[0]	0	0	i2
i3: add r6, r5, r3	[in ROB (is between H and T)]	[rob5]	[- (not avail.)]	r6	[0]	0	0	i3
i4: add r3, r1, r4	[in ROB (is between H and T)]	[rob6]	[- (not avail.)]	r3	[0]	0	0	i4
i5: add r2, r3, r7	[in ROB (is between H and T)]	[rob7]	[- (not avail.)]	r2	[0]	0	0	i5
i6: add r3, r2, r3	[NOT in ROB (NOT between H and T)]	[not applicable]	[not applicable]	r3	[not applicable]	0	0	i6

Rename

Map

Table

(RMT)

	v	ROB tag
r0	[0]	[dont-care]
r1	[0]	[dont-care]
r2	[1]	[rob7]
r3	[1]	[rob6]
r4	[0]	[dont-care]
r5	[0]	[dont-care]
r6	[1]	[rob5]
r7	[0]	[dont-care]

Architectural

Register File

(ARF)

	value
r0	[#-7]
r1	[#10]
r2	[#20]
r3	[#30]
r4	[#40]
r5	[#50]
r6	[#60]
r7	[#70]

Question 4

Correct

20.00 points out of 20.00

PIPELINE STATE AT CYCLE 13:

Issue Queue (IQ)

	Is this instruction in the IQ?	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
i1: mul r3, r1, r2	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i2: add r3, r3, r4	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i3: add r6, r5, r3	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i4: add r3, r1, r4	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i5: add r2, r3, r7	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i6: add r3, r2, r3	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓

Reorder Buffer (ROB)

NOTE: As in the class notes, if the head instr. is in RT, it is being retired this cycle and assume the head pointer H has been incremented past it (no longer in ROB).

	Is this instruction in the ROB? (i.e., is it between ROB head and tail?)	rob entry # (ROB tag)	value	dst	rdy	exc.	mis.	pc
i1: mul r3, r1, r2	NOT in ROB (NOT between H and T) ✓	not applicable ✓	not applicable ✓	r3	not applicable ✓	0	0	i1
i2: add r3, r3, r4	in ROB (is between H and T) ✓	rob4 ✓	#240 ✓	r3	1 ✓	0	0	i2
i3: add r6, r5, r3	in ROB (is between H and T) ✓	rob5 ✓	- (not avail.) ✓	r6	0 ✓	0	0	i3
i4: add r3, r1, r4	in ROB (is between H and T) ✓	rob6 ✓	#50 ✓	r3	1 ✓	0	0	i4
i5: add r2, r3, r7	in ROB (is between H and T) ✓	rob7 ✓	#120 ✓	r2	1 ✓	0	0	i5
i6: add r3, r2, r3	in ROB (is between H and T) ✓	rob8 ✓	- (not avail.) ✓	r3	0 ✓	0	0	i6

Rename Map Table

(RMT)

	v	ROB tag
r0	0 ✓	dont-care ✓
r1	0 ✓	dont-care ✓
r2	1 ✓	rob7 ✓
r3	1 ✓	rob8 ✓
r4	0 ✓	dont-care ✓

	v		ROB tag	
r5	0	✓	dont-care	✓
r6	1	✓	rob5	✓
r7	0	✓	dont-care	✓

Architectural

Register File

(ARF)

	value	
r0	#-7	✓
r1	#10	✓
r2	#20	✓
r3	#200	✓
r4	#40	✓
r5	#50	✓
r6	#60	✓
r7	#70	✓

Your answer is correct.

The correct answer is:

PIPELINE STATE AT CYCLE 13:

Issue Queue (IQ)

	Is this instruction in the IQ?	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
i1: mul r3, r1, r2	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i2: add r3, r3, r4	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i3: add r6, r5, r3	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i4: add r3, r1, r4	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i5: add r2, r3, r7	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i6: add r3, r2, r3	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]

Reorder Buffer (ROB)

NOTE: As in the class notes, if the head instr. is in RT, it is being retired this cycle and assume the head pointer H has been incremented past it (no longer in ROB).

	Is this instruction in the ROB? (i.e., is it between ROB head and tail?)	rob entry # (ROB tag)	value	dst	rdy	exc.	mis.	pc
i1: mul r3, r1, r2	[NOT in ROB (NOT between H and T)]	[not applicable]	[not applicable]	r3	[not applicable]	0	0	i1
i2: add r3, r3, r4	[in ROB (is between H and T)]	[rob4]	[#240]	r3	[1]	0	0	i2
i3: add r6, r5, r3	[in ROB (is between H and T)]	[rob5]	[- (not avail.)]	r6	[0]	0	0	i3
i4: add r3, r1, r4	[in ROB (is between H and T)]	[rob6]	[#50]	r3	[1]	0	0	i4
i5: add r2, r3, r7	[in ROB (is between H and T)]	[rob7]	[#120]	r2	[1]	0	0	i5
i6: add r3, r2, r3	[in ROB (is between H and T)]	[rob8]	[- (not avail.)]	r3	[0]	0	0	i6

Rename

Map

Table

(RMT)

	v	ROB tag
r0	[0]	[dont-care]
r1	[0]	[dont-care]
r2	[1]	[rob7]
r3	[1]	[rob8]
r4	[0]	[dont-care]
r5	[0]	[dont-care]
r6	[1]	[rob5]
r7	[0]	[dont-care]

Architectural

Register File

(ARF)

	value
r0	[#-7]
r1	[#10]
r2	[#20]
r3	[#200]
r4	[#40]
r5	[#50]
r6	[#60]
r7	[#70]

Question 5

Correct

20.00 points out of 20.00

PIPELINE STATE AT CYCLE 16:

Issue Queue (IQ)

	Is this instruction in the IQ?	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
i1: mul r3, r1, r2	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i2: add r3, r3, r4	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i3: add r6, r5, r3	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i4: add r3, r1, r4	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i5: add r2, r3, r7	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓
i6: add r3, r2, r3	NOT in IQ ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓	not applicable ✓

Reorder Buffer (ROB)

NOTE: As in the class notes, if the head instr. is in RT, it is being retired this cycle and assume the head pointer H has been incremented past it (no longer in ROB).

	Is this instruction in the ROB? (i.e., is it between ROB head and tail?)	rob entry # (ROB tag)	value	dst	rdy	exc.	mis.	pc
i1: mul r3, r1, r2	NOT in ROB (NOT between H and T) ✓	not applicable ✓	not applicable ✓	r3	not applicable ✓	0	0	i1
i2: add r3, r3, r4	NOT in ROB (NOT between H and T) ✓	not applicable ✓	not applicable ✓	r3	not applicable ✓	0	0	i2
i3: add r6, r5, r3	NOT in ROB (NOT between H and T) ✓	not applicable ✓	not applicable ✓	r6	not applicable ✓	0	0	i3
i4: add r3, r1, r4	NOT in ROB (NOT between H and T) ✓	not applicable ✓	not applicable ✓	r3	not applicable ✓	0	0	i4
i5: add r2, r3, r7	in ROB (is between H and T) ✓	rob7 ✓	#120 ✓	r2	1 ✓	0	0	i5
i6: add r3, r2, r3	in ROB (is between H and T) ✓	rob8 ✓	- (not avail.) ✓	r3	0 ✓	0	0	i6

Rename Map Table

(RMT)

	v	ROB tag
r0	0 ✓	dont-care ✓
r1	0 ✓	dont-care ✓
r2	1 ✓	rob7 ✓
r3	1 ✓	rob8 ✓
r4	0 ✓	dont-care ✓

	v		ROB tag	
r5	0	✓	dont-care	✓
r6	0	✓	dont-care	✓
r7	0	✓	dont-care	✓

Architectural

Register File

(ARF)

	value	
r0	#-7	✓
r1	#10	✓
r2	#20	✓
r3	#50	✓
r4	#40	✓
r5	#50	✓
r6	#290	✓
r7	#70	✓

Your answer is correct.

The correct answer is:

PIPELINE STATE AT CYCLE 16:

Issue Queue (IQ)

	Is this instruction in the IQ?	dst tag	rs1 rdy	rs1 tag/value	rs2 rdy	rs2 tag/value
i1: mul r3, r1, r2	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i2: add r3, r3, r4	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i3: add r6, r5, r3	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i4: add r3, r1, r4	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i5: add r2, r3, r7	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]
i6: add r3, r2, r3	[NOT in IQ]	[not applicable]	[not applicable]	[not applicable]	[not applicable]	[not applicable]

Reorder Buffer (ROB)

NOTE: As in the class notes, if the head instr. is in RT, it is being retired this cycle and assume the head pointer H has been incremented past it (no longer in ROB).

	Is this instruction in the ROB? (i.e., is it between ROB head and tail?)	rob entry # (ROB tag)	value	dst	rdy	exc.	mis.	pc
i1: mul r3, r1, r2	[NOT in ROB (NOT between H and T)]	[not applicable]	[not applicable]	r3	[not applicable]	0	0	i1
i2: add r3, r3, r4	[NOT in ROB (NOT between H and T)]	[not applicable]	[not applicable]	r3	[not applicable]	0	0	i2
i3: add r6, r5, r3	[NOT in ROB (NOT between H and T)]	[not applicable]	[not applicable]	r6	[not applicable]	0	0	i3
i4: add r3, r1, r4	[NOT in ROB (NOT between H and T)]	[not applicable]	[not applicable]	r3	[not applicable]	0	0	i4
i5: add r2, r3, r7	[in ROB (is between H and T)]	[rob7]	[#120]	r2	[1]	0	0	i5
i6: add r3, r2, r3	[in ROB (is between H and T)]	[rob8]	[- (not avail.)]	r3	[0]	0	0	i6

Rename

Map

Table

(RMT)

v	ROB tag
r0 [0]	[dont-care]
r1 [0]	[dont-care]
r2 [1]	[rob7]
r3 [1]	[rob8]
r4 [0]	[dont-care]
r5 [0]	[dont-care]
r6 [0]	[dont-care]
r7 [0]	[dont-care]

Architectural

Register File

(ARF)

	value
r0	[#-7]
r1	[#10]
r2	[#20]
r3	[#50]
r4	[#40]
r5	[#50]
r6	[#290]
r7	[#70]

